

—— // STRYKE // ——

stryke

v0.17.39

THE COMPANION

design · idioms · the machine underneath

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
Chapter 1 — The Pitch	4
Chapter 2 — Hello, stryke	11
Chapter 3 — The Topic and the Underscore	16
Chapter 4 — Threads and Pipes	21
Chapter 5 — Going Wide	27
Chapter 6 — Sigils, Strings, and Interpolation	34
Chapter 7 — Ranges and Slicing	39
Chapter 8 — Shaping Data	45
Chapter 9 — Control Flow and Functions	51
Chapter 10 — Text and the AWK Inheritance	55
Chapter 11 — Batteries Included: The Shape of Ten Thousand Builtins	59
Chapter 12 — Numbers, Math, and Sequences	64
Chapter 13 — Collections and Iteration	68
Chapter 14 — Strings, Encoding, and the Network	72
Chapter 15 — The Scientific Shelf	77
Chapter 16 — Probabilistic Structures and Sketch Algebra	82
Chapter 17 — Persistence: The rkyv KV Store and Friends	86
Chapter 18 — AI as a Primitive	90
Chapter 19 — From Source to Bytecode	94
Chapter 20 — The Virtual Machine and the JIT	99
Chapter 21 — Values and the Cache	106
Chapter 22 — Compatibility and Conviction	110
Chapter 23 — Worked Examples: Algorithms and Numerics	113
Chapter 24 — Worked Examples: Parallel and Streaming	124

Chapter 25 — Worked Examples: Text, Data, and Simulations	130
Chapter 26 — Worked Examples: AI and Agents	139
Chapter 27 — Worked Examples: Graph Algorithms	144
Chapter 28 — Worked Examples: Simulation and Physics	151
Chapter 29 — Cryptography, Hashing, and Encoding	157
Chapter 30 — Parsing, Text, and Bioinformatics	167
Chapter 31 — Optimization, Numerics, and Data Structures	178
Chapter 32 — Packages, FFI, and the Registry	187
Chapter 33 — Objects, Classes, and Aspects	192
Chapter 34 — Errors, Guards, and the Shape of Failure	195
Chapter 35 — The Interactive Loop: REPL, Debugger, and One-Liners	197
Chapter 36 — Reflection: The Language That Describes Itself	199
Chapter 37 — The Fleet: Controller, Agents, and the Congregation	201
Chapter 38 — Async, Generators, and Time	203
Appendix A — The Style Guide in Full	205
Appendix B — Operator and Sigil Reference	209
Appendix C — The Thread-Macro Cheat Sheet	213
Appendix D — A Glossary of stryke Terms	216

Before You Begin

This is not the reference manual. The reference enumerates every one of stryke’s builtins, keywords, and aliases — more than eleven thousand of them — and you reach for it when you need to know exactly what `cidr_aggregate` returns. This book is the other thing: the one you read in a chair, front to back, to understand not just *what* stryke does but *why* it is shaped the way it is.

stryke is a Perl-5-compatible scripting language written in Rust. That sentence is true and almost completely fails to describe it. A fuller description: stryke is what you get when one person spends decades writing Perl, Ruby, Clojure, Scala, Racket, and F#, decides each of them got exactly one thing right, and then builds a single language that keeps those things and throws away the ceremony. It compiles to bytecode, JITs the hot paths through Cranelift, and fans work across every core with rayon. It ships the largest standard library of any programming language in existence, in one static binary. And it treats a call to a large language model as a two-letter builtin, the way every other language treats `print`.

The book is in five movements. The first — Chapters 1 through 10 — is the language itself: how it reads, the topic and the underscore, the threading operators, parallelism, data, and text. The second — Chapters 11 through 18 — is the standard library, that improbable reference shelf of ten thousand functions. The third — Chapters 19 through 22 — is the machine underneath: the bytecode compiler, the two-tier JIT, nan-boxing, and the two flags that bracket the whole language. The fourth is a long tour through real, complete programs from the `examples` directory, walked through line by line. The fifth turns from the language to the language *in operation* — the object and aspect model, the shape of failure, the interactive loop, the runtime that describes itself, the distributed fleet, and time — the chapters that describe what it is like to live inside stryke rather than merely to write it. The last is reference appendices — the style guide, the operators, the threading cheat sheet, a glossary.

Everything here is grounded in the actual repository. Every code snippet is copied from a real file — there are nearly three thousand example programs to draw on — and every claim about the machine is traceable to a specific source file. Where a number or a phase is still provisional, this book marks it as such rather than rounding up. stryke is the work of MenkeTechnologies, and this book is grounded in his code.

Let’s start.

Chapter 1 – The Pitch

Most programming languages are designed for the average programmer. They optimize for the person who will read the code six months from now without remembering having written it, for the team that needs onboarding ramps, for the tutorial that has to fit on a slide. These are good goals, and they have produced good languages. `stryke` is not one of them. `stryke` is designed for the person who has already internalized the trade-offs, who types faster than they read, who lives in the shell, and who considers every keystroke that does not carry meaning to be a small tax levied against a lifetime of work. It is a power-user's language, written by a power user, and it makes no apology for that.

The name of the thing it optimizes is density — semantic density per character, per line, and per process. Where other languages ask “is this readable to a beginner?”, `stryke` asks “does this say exactly what I mean, in the fewest tokens that still parse unambiguously?” The two questions have different answers more often than the industry likes to admit, and `stryke` consistently picks the second one. This is a deliberate inversion of the prevailing taste, and understanding why it is deliberate — and what you get in exchange — is the whole point of this chapter.

What `stryke` is

`stryke` is a dynamically-typed scripting language with a Perl-compatible core, implemented in Rust as a bytecode virtual machine with a Cranelift JIT and rayon-backed parallel primitives across every CPU core. That sentence is doing a lot of work, so unpack it slowly. “Perl-compatible core” means that the sigils are there — `$scalar`, `@array`, `%hash` — that regular expressions are first-class literals, that `$_` is the topic variable, and that a large fraction of existing Perl 5 code will run unchanged under a `--compat` flag. But the word “core” is load-bearing: `stryke` is not a Perl rewrite. It is a superset that absorbs Perl's terse text-processing heritage as one layer among several, and then builds a modern language on top of it. The interpreter you are running is not a tree-walker that re-reads your source on every loop iteration; it compiles your script to bytecode once and runs that bytecode on a stack VM, with the hottest paths handed to a native-code JIT.

The distinction between a tree-walker and a bytecode VM is not academic, and it is the first place `stryke` parts company with the languages it most resembles in spirit. A tree-walking interpreter — the architecture of early Ruby, of most teaching languages, and of countless hand-rolled scripting engines — holds the parsed abstract syntax tree in memory and re-traverses it node by node every time control flows through a region of code. A loop that runs a million times walks the same subtree a million times, paying the cost of dispatching on node type, resolving variable references by name, and reconstructing intermediate state on each pass. A bytecode VM pays the parse and the compile exactly once, flattening the tree into a linear instruction stream that a tight dispatch loop executes directly. Variable references become slot indices, control flow becomes jumps, and the per-iteration cost collapses to the cost of the instructions themselves. `stryke` goes one tier further: the paths that run hottest are handed off to Cranelift, which emits native machine code, so the inner loop of a numeric kernel stops being interpreted bytecode and becomes the same kind of instructions the CPU would run for hand-written Rust. This three-tier shape — parse once, interpret bytecode, JIT the hot paths — is why a scripting language with Perl's surface can post the benchmark numbers it does, and it is the reason the speed claims later in this chapter are not in tension with the convenience claims. The terseness is a property of the surface syntax; the speed is a property of the machine underneath, and the two are decoupled.

The reference implementation lives in a single Rust binary. There is no separate compiler, no separate package manager entry point, no runtime to install on each machine. One file, `s`, contains the lexer, parser, compiler, VM, JIT, the entire standard library, the test runner, the fleet agent, and the controller. You can copy that one file to a server with nothing else on it and run `stryke` programs. This is not an incidental packaging decision; it is

a design commitment that shapes everything downstream, and we will return to it. Consider what it eliminates. For a program that leans only on the standard library — the common case, given how much that library covers — there is no dependency resolution step before it runs, because there are no dependencies to resolve: the standard library is statically linked into the binary, not fetched at install time the way Python's `pip` or Node's `npm` populate a per-project tree. (stryke *does* ship a full package manager for the cases the standard library does not cover — your own multi-file projects, third-party source libraries, and native FFI extensions — and Chapter 27 covers it; the point here is narrower, that reaching a standard-library function never routes through it.) There is no version skew between the language and its libraries, because they ship as one artifact built together. There is no “works on my machine” gap introduced by a differently-pinned transitive dependency, because the dependency graph for a stryke program is, in the common case, empty. Deployment becomes a file copy; a container image needs nothing in it but the binary and the script. The cost of this commitment is binary size — the whole library is always present whether a given program touches it or not — and we will see in the density section that the trade lands at roughly forty-four megabytes for more than ten thousand builtins, which is a bargain measured per function.

stryke is an original synthesis — not a clone of any one predecessor, but a deliberate convergence of syntactic ideas drawn from across the history of programming languages. Every example in this book is grounded in a real file in the stryke source tree; nothing here is invented for pedagogy. When you see a code block, it was copied from an example program, a test, or the style guide that ships with the language.

The power-user thesis

The thesis behind stryke is simple to state and uncomfortable to accept: optimizing for the lowest common denominator is a choice, and it has costs that fall on the expert. Every verbose keyword that helps a newcomer is a keyword the expert types ten thousand times. Every safety prompt is an interruption to someone with muscle memory measured in millions of keystrokes. Every “explicit is better than implicit” rule that a beginner appreciates is a place where the expert has to spell out something the compiler could have inferred.

stryke makes the opposite bet. It assumes you know what you mean. It gives you `p` instead of `print "...\\n"`. It gives you `_` instead of forcing you to name a closure parameter you will use exactly once on the next line. It gives you `->` so that a four-stage transformation reads left-to-right in the order it executes, instead of inside-out through three nested closing parens. None of these are conveniences bolted onto a conventional language; they are the language's organizing principle. The style guide that ships with stryke states it in one line: **min chars win**. Every design call defaults to the terser form, with no newbie concessions and no optional verbosity.

The trade-off here deserves to be named plainly rather than waved away, because it is real and it cuts both ways. The cost of density is a steeper initial climb: a reader encountering `-> " hello " tm uc rv |> p` for the first time has to learn what `tm`, `uc`, and `rv` mean, what `->` threads and what `|>` terminates, before the line reads as plainly as `print reverse(uc(trim(" hello ")))` reads to someone who already knows those four function names. The wager stryke makes is that this cost is paid once, by one person, early — and the saving is paid back on every keystroke for the rest of that person's career. A name you learn once is amortized across a million uses; a character you type a million times is not. Languages that optimize for the first reading optimize for the moment that happens least often in a working programmer's life. stryke optimizes for the moment that happens most. This is the same calculus that makes professional tools — a surgeon's instruments, a pilot's cockpit, a stenographer's keyboard — look hostile to a novice and feel like extensions of the body to an expert. The style guide's `--no-interop` flag is the enforcement mechanism: it rejects the verbose Perl spellings at parse time, so the terse form is not merely encouraged but mechanically required in new code, turning a style preference into a forcing function.

This is not minimalism for its own sake. The terseness is in service of a specific workflow: the shell. stryke is built to be embedded inside a shell, dispatched through `@`-prefix one-liners, used the way AWK and Perl one-liners have always been used — except faster, with a vastly larger standard library, and with modern parallel and async primitives that AWK never had. The design goal is that AWK should become redundant: text processing, field extraction, line-by-line transformation, all absorbed into stryke's one-liner role. When the cost of expressing an idea drops far enough, you stop reaching for a separate tool. AWK has held its niche for

nearly fifty years not because nothing is faster but because the activation energy of writing the equivalent in a general-purpose language has always been higher than the activation energy of an `awk '{print $1}'`. `stryke` attacks exactly that activation energy. `stryke -lane 'p $F[0]'` is no longer keystrokes than the AWK form, it starts in about three milliseconds so the per-invocation tax that would sink Python or Julia in this role never materializes, and behind the one-liner sits a standard library and a parallel runtime that AWK was never going to grow. The bet is that once the one-liner is this cheap, the reason to keep a separate text-processing tool in the toolchain quietly evaporates.

Three legs: fast, dense, new

A language earns the right to exist only if it does something no existing language does, and does it well. `stryke` stands on three legs, and all three are simultaneously true. Most language pitches manage one. The interesting ones manage two. The bar `stryke` sets for itself is all three at once: it must be fast, it must be dense, and it must be genuinely new.

The reason all three have to hold together — rather than any one being sufficient — is that each one alone describes a tool that already exists. A faster dynamic language that does nothing new is a re-implementation; the world has many of those and they tend to fade, because being twenty percent faster at a thing other tools already do is not worth a migration. A novel language that is slow is an academic prototype; the literature is full of beautiful ideas that never left the paper they were published on because nobody could run them at production speed. A dense language that is neither fast nor new is a code-golf curiosity. The intersection is the rare and defensible position: a capability that exists nowhere else, delivered at a speed that makes it usable for real work, in a syntax compact enough that the capability is actually convenient to reach. The three legs are not a marketing triad; they are the minimum set of independent properties that, held simultaneously, leave no existing tool occupying the same ground.

Fast

The benchmark harness in `bench/run_bench_all.sh` compares `stryke` against Perl 5.42.2, Python 3.14.4, Ruby 4.0.2, Julia 1.12.6, Raku, and LuaJIT on an Apple M5 18-core machine — mean of ten hyperfine runs with three warmups, and critically, **including process startup**, not steady-state. That last detail matters: these are scripting benchmarks, where you pay the startup cost on every invocation, which is exactly the workload a shell language faces.

Against the four mainstream dynamic languages, `stryke` wins every benchmark. Faster than Perl 5 on all eight (the `fib` benchmark by 27×, `loop` by 29×). Faster than Python 3 on all eight (`loop` by 60×). Faster than Ruby on all eight, faster than Julia on all eight, faster than Raku on all eight — in Raku's case by between 20× and nearly 2000×, where its regex benchmark runs in 25 seconds against `stryke`'s 12.6 milliseconds.

The harder comparison is against LuaJIT, Mike Pall's tracing JIT and the fastest dynamic-language runtime ever built. Single-threaded, `stryke` comes in second to LuaJIT overall — but it beats LuaJIT outright on three of the eight benchmarks: `loop` (0.74×), `array` (0.17×), and `regex` (0.07×). It reaches near-parity on `string` (1.21×) and `fib` (1.43×). LuaJIT wins on `hash` (3.4×) and `map_grep` (4.2×), where its tracing JIT eliminates dispatch overhead that `stryke` still pays. That is the precise shape of the result: second-fastest single-threaded dynamic-language runtime ever benchmarked, behind only LuaJIT, beating it on three of eight.

It is worth being honest about *why* LuaJIT wins where it wins, because the two places it pulls ahead are exactly the places its architecture is built to exploit. A tracing JIT works by recording the actual sequence of operations executed through a hot loop — a single linear trace of what really happened — and then compiling that trace to native code with every type already known and every dispatch already resolved. On `hash` and `map_grep`, the inner operation is a dictionary lookup or a higher-order call, and tracing lets LuaJIT inline the whole thing down to pointer arithmetic with no per-element dispatch left to pay. `stryke`'s compiler does not trace; it compiles ahead of execution and hands hot regions to Cranelift, which is a method-at-a-time JIT rather than a trace-at-a-time one. That architecture gives up some of the dispatch elimination a tracing JIT gets for free on tight hash-heavy loops. The flip side shows up on `regex`, where `stryke` is faster by more than ten times:

the LuaJIT benchmark uses Lua patterns rather than PCRE, and stryke's three-tier regex engine simply does more, faster, on real pattern-matching work. The comparison is not stryke losing to a better implementation of the same thing — it is two different JIT philosophies trading wins across workloads that suit each one. And on every multithreaded benchmark, stryke is fastest outright, for a reason that is not subtle: LuaJIT has no parallel primitives at all, single-threaded by construction, while stryke rides rayon work-stealing across every core. The moment a workload is embarrassingly parallel, the comparison stops being close, because one of the two runtimes cannot use more than one CPU.

Dense

stryke ships **10,453 standard-library primaries** — distinct callable builtins — with **11,229 total names** once aliases and keywords are folded in. This is the largest bareword library of any programming language. For scale, Wolfram Language v14.3's high-band estimate is roughly 7,300 built-in functions; stryke clears that by about 3,149. Every one of those primaries is present without an import or package-load step. There is no “first install the crypto library, then the stats library” dance — `sha256`, `tdigest_new`, `bloom_new`, `unit_energy`, `is_prime`, and ten thousand others are simply there, barewords you can call.

The Wolfram comparison is the right yardstick precisely because Wolfram is the only other system in the same league. Most languages keep their standard library deliberately small and push everything else into a package ecosystem — Python's “batteries included” tops out in the low hundreds of modules, and the real breadth lives in PyPI; Ruby leans on RubyGems; Node ships a thin core and defers to npm. Those ecosystems are larger than any single binary in aggregate, but they are not the *language* — they are a network you reach across, with versioning, resolution, and supply-chain surface that comes with it. Wolfram took the opposite path, putting an encyclopedic function set inside the system itself, and stryke takes that same path and goes further: every probabilistic sketch, every hashing primitive, every statistics function, every unit conversion is a bareword you call without an import, because it was compiled into the binary you are already running. The design consequence is that “is there a function for this?” almost never routes through a package search; it routes through autocompletion on names already loaded.

All of it lives in a single **44 MB static binary**. That works out to roughly 4.3 KB amortized per builtin — about 200× denser than Wolfram Engine measured per builtin-per-byte. And that binary starts cold in under ten milliseconds: the startup benchmark clocks 3.3 ms against Perl's 2.3 ms, Python's 14.3 ms, Ruby's 23.8 ms, and Julia's 68.3 ms. The density and the speed are not in tension; they reinforce each other, and the mechanism is worth spelling out. A 44 MB binary does not load 44 MB of work at startup. The operating system memory-maps the executable, and only the pages actually touched on a given run are faulted into physical memory — the crypto code does not cost you anything on a run that never calls a hash function. Startup is therefore dominated by the fixed cost of bringing up the VM and resolving the entry point, not by the size of the library sitting dormant on disk. This is why the encyclopedic standard library and the three-millisecond cold start are not a contradiction: the same single-binary, no-external-runtime design that makes the library exhaustive is exactly what makes deployment a file copy and startup a memory map. Compare the alternative: Julia's 68 ms startup is more than twenty times stryke's, and a meaningful share of that is the runtime's own initialization and JIT warm-up, paid on every single invocation — which is precisely the tax that makes Julia a poor fit for the one-shot scripting workload these benchmarks measure, however well it performs in a long-lived session.

New

Speed and density alone would make stryke a very good tool, but not a novel one. The third leg is the set of capabilities that exist nowhere else.

Sketch algebra. Probabilistic data structures are first-class operands for the arithmetic and set operators. You write `$bloom_a + $bloom_b` for a Bloom-filter union, `$hll_a + $hll_b` for a HyperLogLog union, `$cms_a + $cms_b` to sum two Count-Min sketches, `$td_a + $td_b` to merge two t-digests by centroid, and `$rb_a | $rb_b` (or `&`, `^`, `-`) for Roaring-bitmap set algebra. The operators are functional — operands are never mutated. No other language treats these as syntactic primitives; everywhere else they are library function calls.

The significance is easy to miss if you read it as mere syntactic sugar over a `merge()` function. It is more than that, because the algebraic spelling carries the semantics it should. A Bloom filter union genuinely *is* an addition in the monoid sense — it is associative, it has an identity, and merging two filters built over disjoint sets yields a filter that answers membership for the union of those sets. A HyperLogLog merge has the same algebraic shape; so does a Count-Min sum and a t-digest centroid merge. These structures were designed by their inventors to compose this way precisely so that you can shard a stream across machines, build a sketch per shard, and combine the partial sketches into a single answer with no loss of accuracy. That distributed-aggregation pattern is the whole reason probabilistic sketches matter at scale. Every other language forces you to express that combination as a named function call, `bloom_union($a, $b)`, which reads as an operation done *to* the operands rather than an algebra they *participate in*. Writing `$a + $b` and `$a | $b` puts the structure’s mathematical nature back into the syntax, and because the operators are functional rather than mutating, a fold over a list of sketches — `reduce { _ + _1 } $empty, @shards` — does exactly what the mathematics says it should. Here is the union, lifted verbatim from the README’s one-liner section:

```
my $a = bloom_new(1024); my $b = bloom_new(1024);
bloom_add($a, $_) for 1:50;
bloom_add($b, $_) for 30:80;
p bloom_count($a + $b)
```

An rkyv-backed key-value store. stryke ships a first-class CRUD store — `kv_open`, `kv_put`, `kv_get`, `kv_del`, `kv_scan`, `kv_commit`, `kv_batch`, and the rest — with rkyv as the on-disk codec. Reads are `mmap + validate + cast`: zero-copy, no parse, no per-row allocation, the same primitive the bytecode cache already uses. It has SQLite-shaped ergonomics and beats SQLite on reads for any store that fits comfortably in RAM, with atomic commit (tmp-write plus rename) and a versioned format header — the `STKV` magic plus a `format_version` field.

The “zero-copy read” claim is the part that earns the performance, and it rests on what rkyv does differently from an ordinary serialization format. A conventional store — SQLite, or anything that serializes rows with *serde* and *bincode* — has to *parse* on read: it walks the on-disk bytes, allocates fresh in-memory objects, and copies field values into them before your code can touch a single value. rkyv’s archived layout is designed so that the bytes on disk are already in the exact shape the in-memory type expects. Reading is therefore `mmap` the file, validate the archived root once to confirm it is well-formed, and then `cast` a pointer into the mapped bytes — the values are read in place, with no allocation per row and no parse step at all. For a store that fits comfortably in RAM, where the file is fully resident in the page cache, this collapses a read to a pointer chase, which is why it beats SQLite on the read path for that case. The same primitive backs the bytecode cache, so this is not a bespoke trick built for the KV store — it is the runtime’s general answer to “load a structured artifact from disk fast,” applied twice. The atomic-commit discipline (write a temporary file, then rename it into place) borrows the oldest reliable trick in filesystem durability: rename is atomic on POSIX filesystems, so a reader either sees the entire old store or the entire new one, never a torn half-written state.

Every zsh glob qualifier, in a scripting language. Every qualifier from zsh’s `zshexpn(1)` works wherever stryke takes a glob — file-type, permission, ownership, size and time numerics, sort and descending sort, `[N,M]` selection, null-glob, dotfiles, the lot — backed by the zshrs glob engine as a single source of truth with zero stryke-side reimplementation. The phrase “single source of truth” is the engineering substance here, not a slogan. zsh’s glob qualifiers are notoriously intricate — `*(.om[1])` means “the most recently modified plain file,” `*/` restricts to directories, `*(L+10240)` selects files larger than ten thousand kilobytes — and any language that wanted to offer them by re-implementing the parser would be signing up to track zsh’s behavior, bug for bug, edge case for edge case, forever. stryke does not re-implement them. It calls through to the zshrs glob engine, the same engine that powers the Rust-rewritten shell, so the qualifier semantics are inherited verbatim from one canonical implementation. There is exactly one place the behavior is defined, and stryke and the shell both consult it, which means a fix or an addition in the glob engine appears in the scripting language for free and the two can never drift.

An ai builtin. Large-language-model invocation is a language primitive, not a library. `ai($prompt)` makes a call; `ai($prompt, tools => [...])` runs a multi-turn agent loop speaking both the Anthropic tool-use and OpenAI function-calling protocols, with `max_turns` and `max_cost` ceilings. A `tool fn` keyword auto-derives the JSON schema from a function’s signature and auto-attaches it to bare `ai()` calls. Vector memory ships too,

via `ai_memory_save` and `ai_memory_recall`, backed by a SQLite embedding store with cosine retrieval. `stryke` is AI-native in the same sense that it is parallel-native: the capability is a bareword, like `print`.

What makes this different from the dozens of LLM client libraries that exist in every language is the level at which the capability sits. In Python or JavaScript, talking to a model means importing an SDK, instantiating a client, constructing a request object, and threading an agent loop by hand — the model is a service you call across a library boundary. In `stryke` the model is a verb, the same way `print` is a verb, and the agent loop is built into the runtime rather than hand-assembled per project. The `tool fn` keyword closes the loop that is most tedious to write by hand: it reads a function's signature, derives the JSON schema the model needs to call that function, registers it, and auto-attaches it to bare `ai()` calls, so the boilerplate of “describe my tools to the model in the format the model expects” disappears into a keyword. The two ceilings, `max_turns` and `max_cost`, are the operational guardrails an agent loop needs to be safe to embed in a script — they bound how long the loop can run and how much it can spend before it returns control. Speaking both the Anthropic tool-use and OpenAI function-calling protocols from one call site means a script does not bind itself to a single provider's wire format. That this is a *language* feature rather than a popular *library* is the novel claim: the design treats a model call as foundational infrastructure, on the same footing as the parallel primitives and the regex engine, not as an add-on a user bolts on.

The design lineage

`stryke` did not invent terseness, threading, or implicit parameters out of nothing. Its originality is in the synthesis — taking the best syntactic idea from each of several traditions and combining them into one coherent language where they reinforce rather than collide.

From **Clojure** comes the threading macro: the `->` and `->>` operators that take a value and thread it through a sequence of forms, first-argument or last-argument. `stryke` spells these `->` and `->>/->>` and makes them compile-time AST rewrites with zero runtime cost. From **Racket** comes the `->` arrow itself and the idea of a value that short-circuits on absence as it flows down a pipeline. From **Scala** comes the bare underscore `_` as an implicit parameter, and the `val/var` distinction — immutable-by-default bindings, a flip of Perl's mutable-by-default `my`. From **Perl** come the sigils, the `$_` topic, the regex-as-literal heritage, and the whole text-processing one-liner culture that `stryke` is built to inherit and surpass. From **Ruby** come `p` as the terse print verb and `#{expr}` string interpolation. From **F#** comes the pipe-forward `|>`, the single-stage operator that threads the left side into the first argument of the right.

No single one of these languages gives you all of it, and the gaps are instructive because they are not accidental — each language declined the other features for reasons that made sense in its own design context. Clojure has threading but no sigils and no terse output, because it is a Lisp and its uniformity comes from everything being a parenthesized form; sigils would violate that homogeneity. Scala has the underscore and immutability but no regex literals and a heavy JVM startup, because it was built for long-running services where a few hundred milliseconds of warm-up amortizes to nothing and where regex-as-first-class-literal would clash with its type discipline. Perl has sigils and regex and the richest text-processing heritage of any language, but it threads nothing and prints verbosely, because it predates the threading-macro idea by decades and its `print` was designed when an explicit record separator was the norm. Each ancestor optimized for its own era and its own audience, and each left the features it declined on the table.

`stryke`'s wager is that those declined features were left on different tables, and that nothing in their nature makes them mutually exclusive — the reasons each language skipped a given feature were local to that language, not universal. Sigils and threading do not conflict; immutability and regex literals do not conflict; a terse print verb and a heavy standard library do not conflict. The synthesis works because the conflicts were always contingent. So `-> " hello " tm uc rv |> p` reading as trim-then-uppercase-then-reverse-then-print is a line you could not have written in any one of its ancestors, not because any ancestor lacked the *idea*, but because no ancestor collected all four ideas in one place. Here is that exact line from the style guide:

```
-> " hello " tm uc rv |> p          # "OLLEH" (trim → uc → reverse)
```

Four transformations, one process, left-to-right, no named intermediate variables, no nested parens. That is

the language in miniature: the terse output verb from Ruby, the thread-first macro from Clojure and Racket, the pipe-forward boundary from F#, and the Perl heritage underneath that makes `uc` and `rv` barewords you can call without ceremony. The rest of this book is an unfolding of that one line.

Chapter 2 – Hello, stryke

The first program in most language books prints a greeting and explains nothing. The first program in stryke prints a greeting and, if you read it closely, tells you most of what makes the language what it is. So we will start small and read closely.

```
p "hello, world"
```

That is a complete, runnable stryke program. There is no `main`, no preamble, no import, no boilerplate. The verb is `p`. It prints its argument followed by a newline — the record separator is built into the verb, which is why you never write `"...\n"`. This is the single most-used line in stryke, and it costs exactly one character of verb where Perl would cost five (`print` plus a literal newline) and Python or Ruby would cost more still. The style guide makes `p` rule number two of its ten hard rules, right after “use `fn` not `sub`”: **`p` not `print`, never `say`**. The Perl `say` keyword is rejected outright by `--no-interop`, because `p` already does what `say` does, in one character instead of three.

It is worth dwelling for a moment on why the record separator belongs *in the verb* rather than in the argument, because the choice encodes a view about what the common case is. Perl’s `print` is honest about doing one thing — emit exactly these bytes — and leaves the newline to you, which means every line of output-bearing code carries a `"\n"` that is pure ceremony nineteen times out of twenty. Perl itself conceded the point and added `say` in 5.10, a `print`-with-newline that exists precisely because the no-newline default was the wrong default. stryke skips the historical detour: the default verb already includes the separator, so the common case is free and the rare case — output with no trailing newline — is the one you spell out specially. This is the density thesis applied at the smallest possible scale, a single character, and it is instructive because it shows the principle is not about clever operators but about getting the defaults right. The most common operation should cost the fewest keystrokes. Rejecting `say` under `--no-interop` is the same forcing function seen with `sub`: stryke will not let two spellings of the same idea coexist in new code, because a language with two ways to print is a language where a reader has to know both, and that cognitive cost is exactly what density is meant to eliminate.

No semicolons, statements end at newline

Look again at the hello program. There is no semicolon. stryke terminates a statement at the newline, the way the shell and AWK do — not at a `;` the way C and Perl do. A trailing semicolon is legal (so that Perl ports keep parsing) but it is noise, it adds a character to every line, and new stryke code that ships with trailing semicolons fails style review. The one place a semicolon legitimately appears is between two statements that genuinely share a line — most often inside a C-style `for` header, `for (my $i = 0; $i < 10; $i++)`, where the semicolons are separators, not terminators.

This has a practical consequence for multi-line expressions, and it is one of the few places where stryke’s terseness has a sharp edge worth respecting. Because the newline terminates, a binary operator left dangling at the end of a line does *not* silently continue onto the next — it terminates, and you get a wrong value with no error raised. This is the cost of a newline-terminated grammar, and it is the same trap shell programmers know: write `$x = $a +` on one line and `$b` on the next, and a C-or-Perl reader’s instinct says the expression continues, but in a newline-terminated language the first line is a complete statement assigning `$a` to `$x` and discarding nothing, while `$b` on the next line is a second, useless statement. No parser error fires because both lines are individually valid. The exception is pipelines: a line that *begins* with `|>` or `~>` auto-extends the previous statement, so you can break a long threading chain across lines naturally — the leading operator is an unambiguous signal that more is coming, and the parser treats it as a continuation. For any other

mid-expression break, you use a trailing backslash, exactly as the shell does, which makes the continuation explicit at the point where the line would otherwise have ended. The rule to internalize is simple: a line ends where its text ends unless you said otherwise with a leading pipe operator or a trailing backslash.

Running a program

The reference binary is `s` (it also answers to `stryke`). You run a script by handing it the path:

```
stryke script.stk arg1 arg2
```

Scripts conventionally open with a shebang line so they can be made executable directly:

```
#!/usr/bin/env stryke
```

You can also run code straight from the command line. The `-e` flag is optional — if the first argument is not a file on disk and looks like code, stryke runs it. `stryke 'p 42'` and `stryke -e 'p 42'` are equivalent. You reach for explicit `-e` mainly when combining it with the line-processing flags `-n`, `-p`, `-l`, and `-a`, which give stryke its AWK-and-Perl-one-liner role. These flags are lifted directly from Perl's command-line vocabulary, and they mean what they mean there: `-n` wraps your code in an implicit read-each-line loop without printing, `-p` does the same but prints `$_` after each iteration, `-l` handles line-ending chomp-and-re-add, and `-a` turns on autosplit, populating the field array `@F` from each line the way AWK populates `$1`, `$2`, and friends. Stack them and they compose: `stryke -lane 'p $F[0]'` chomps each input line, auto-splits it into `@F`, and prints the first field — the stryke spelling of `awk '{print $1}'`, no longer to type and faster to start. Because the binary starts cold in about three milliseconds, paying the startup cost on every one-liner invocation is not a problem the way it is for Python or Julia, where the per-invocation runtime warm-up is large enough to feel in an interactive shell pipeline. This is the concrete payoff of the startup numbers from the previous chapter: a fast cold start is not a benchmark bragging right, it is the difference between a language you can drop into a shell pipeline a hundred times an hour and one you reach for only when the job is big enough to justify the wait.

Immutable by default: `val` and `var`

Here is where stryke makes its sharpest break from Perl. Perl's `my` declares a mutable binding — everything is reassignable unless you go out of your way to freeze it. stryke flips this. The default binding keyword is `val`, and `val` means *frozen*. A `val` binding cannot be reassigned, cannot be pushed to or popped, cannot have an element or hash slot assigned, cannot have a key deleted — all of those error at *compile* time, not run time. When you actually need mutation, you write `var`.

The choice of compile-time enforcement rather than run-time is the part that makes this more than a documentation convention. A run-time immutability check would catch a stray reassignment only when execution happened to reach it, which on a rarely-taken branch could be never until it bites in production. Compile-time enforcement means a program that tries to mutate a `val` does not compile at all — the error surfaces the moment you build, on every path, regardless of whether that line ever runs. This is the same guarantee Rust's own `let` versus `let mut` gives, and it is no coincidence that a language implemented in Rust would adopt Rust's instinct here: mutability is a property worth tracking in the type-and-binding system rather than leaving to discipline. The default direction matters as much as the enforcement. By making the frozen form the *shorter, default* keyword and the mutable form the one you reach for deliberately, stryke aligns the path of least resistance with the safer choice. In Perl, immutability is the effortful case nobody bothers with; in stryke, mutability is the marked case you opt into, so the common binding is immutable not because the programmer was disciplined but because it was the easier thing to type.

```
val $t = tdigest_new()
tdigest_add $t, _ for 1:1000
val $q = tdigest_quantile $t, 0.5
val @s = reservoir_sample 5, 1:1e4
```

Every binding in that fragment — from a real statistics example in the source tree — is a `val`, because nothing in it is reassigned after creation. (The middle line is a statement, not a binding: it streams a thousand values into the t-digest, which mutates the sketch through its own API rather than through a reassignment, so no `var` is needed.) You reach for `var` only at the points where you genuinely mutate, and the asymmetry is the point: mutation should be visible, marked, and rare, the way it is in Scala and Kotlin. Here is the other side, from a base-converter:

```
fn Base::from($s, $base) {
  var $idx = 0
  var $neg = 0
  if ($s[0] eq "-") {
    $neg = 1
    $idx = 1
  }
  var $total = 0
  ...
}
```

`$idx`, `$neg`, and `$total` are all `var`, because the body reassigns each of them. The accumulator pattern is exactly where `var` earns its keep: a running index, a running flag, a running sum — each is a value that exists to be updated in place across a loop, and each is correctly marked. Both `val` and `var` desugar to `my` in the abstract syntax tree — same scope, same bytecode, same slot allocation — so the immutability check is purely a compile-time guard with zero runtime cost. This is the detail that makes the safety free: there is no runtime `val` object that wraps the value and intercepts writes, no extra indirection, no allocation. After the compiler has verified that no `val` is mutated, both keywords compile to the identical slot machinery, and the binding you chose for safety costs the same at execution as the one you would have written for speed. `val` even takes an optional type annotation, `val $port : Int = 8080`, which documents intent and gives the compiler a second thing to check. Bare `my` stays legal for CPAN ports — it is what Perl 5 spells, so leaving it valid is what lets old code parse — but in new stryke code it is wrong on style, and `--no-interop` rejects it, which is the same firewall pattern seen with `sub` and `say`: the compat spelling survives for ports and is forbidden in fresh code. Default to `val`; reach for `var` only when you mutate.

Functions: fn, two shapes, no return on the tail

Functions are declared with `fn`, never with Perl's `sub` — that is rule one of the ten hard rules, and `sub` is rejected by `--no-interop`. There are two shapes, and choosing between them is a matter of how much the body does.

The **expression-bodied** form is for anything that fits on one line. You write `fn name = <expr>`: no braces, no named parameters, implicit topic.

```
fn double = _ * 2
fn shout  = uc(_) . "!"
fn area   = _0 * _1
```

`double` takes one argument and reads it as the bare topic `_`. `area` takes two arguments and reads them as `_0` and `_1` — zero-indexed positional slots, which we will cover in full in the next chapter. There is no `return`, because there is nothing to return *from*; the expression *is* the value. From real code in the source tree, the expression-bodied form is everywhere:

```
fn B5::union      = _0 | _1
fn B5::intersect = _0 & _1
fn B5::symdiff    = _0 ^ _1
fn Star::heur     = abs(_ - _2) + abs(_1 - _3)
fn BF::is_inf     = _ >= 1e17
```

Notice the namespacing — `B5::union`, `Star::heur`, `BF::is_inf`. With 10,453 builtins, common short names are taken, and a hard rule forbids shadowing a builtin. Naming a function `union` or `log` or `close` will

abort. The convention is to namespace under `Module::Sub::name`, which is why these examples wear `BS::` and `Star::` prefixes.

This is the unavoidable tax of an encyclopedic standard library, and it is the one place where stryke's density at the library level pushes back on density at the call site. When ten thousand barewords are already defined, the short, obvious names — `union`, `log`, `close`, `cos`, `open` — are spoken for, and a user-defined function that quietly reused one of them would either shadow a builtin or be shadowed by it, in either case producing a subtle and silent bug. stryke refuses to let that happen: shadowing aborts at definition, loudly, rather than allowing the ambiguity. The namespacing convention is the discipline that resolves the tension — `BS::union` is unambiguous, will never collide with the builtin `union`, and costs only a short prefix. The trade is that a few extra characters of namespace buy a guarantee that no user function is ever silently confused with a library function, which in a library this size is a guarantee worth its cost. It is the same reason large codebases in any language eventually adopt module prefixes; stryke simply makes the prefix mandatory the moment a name would collide.

The **block-bodied** form, `fn name { ... }` or `fn name($params) { ... }`, is for multi-statement bodies, for functions where named parameters genuinely document their role, and for anything needing explicit `var` declarations or control flow:

```
fn Demo::Fibonacci::fib_n($n) {
  return $n if $n <= 1
  $memo{$n} //= Demo::Fibonacci::fib_n($n - 1) + Demo::Fibonacci::fib_n($n - 2)
}
```

This function shows the one legitimate use of `return`. The line `return $n if $n <= 1` is an *early return* — a guard clause that bails out before the rest of the body runs. That is correct, and you should use early returns freely. What you must never write is `return` on the *tail* expression — the last expression in a body is automatically the return value. Here, the final line `$memo{$n} //= ...` is the tail, and it produces the function's value with no `return` keyword in sight. Writing `return` there would be noise that visually hides the actual value the function computes. The rule applies inside arrow-blocks, `if/else` arms, and `match` arms too: the last expression in any expression-position block is its value.

This is the expression-oriented stance shared by the ML family, Rust, Scala, and Ruby, and the reason it is worth the rule is that it changes what a function body *looks like* it is doing. When the last expression is implicitly the result, the eye reads the bottom of a function as “this is the value,” and a stray `return` there reads as a contradiction — a control-flow keyword applied to something that was not branching. Reserving `return` strictly for early exits restores the keyword's signal: every `return` in a stryke body marks a place where the function bails out *before* its natural end, which is exactly the information a reader scanning for edge cases wants to find. The `//=` on the tail line is itself worth noting — it is the defined-or-assign operator, which assigns to `$memo{$n}` only if that slot is currently undefined, and otherwise leaves the cached value in place. That single operator is the entire memoization mechanism: the first call for a given `$n` computes the two recursive subproblems and stores the sum; every later call finds the slot already defined and returns it untouched, turning an exponential naive Fibonacci into a linear one. The function reads as one line of base case and one line of memoized recurrence, with the caching folded into an operator rather than spelled out as an if-present-return-else-compute block.

Block-bodied functions also support default parameter values, in the Perl-derived signature syntax:

```
fn BaseConv::enc($n, $base = 10) {
  ...
}
```

Call `BaseConv::enc(255)` and `$base` is 10; call `BaseConv::enc(255, 16)` and it is 16.

Comments and the shape of a program

Comments are introduced with `#` and run to end of line — the Perl and shell convention. There is no block-comment syntax; you prefix each line. The shebang `#!/usr/bin/env stryke` is itself just a comment from the language's point of view.

Putting it together, here is a complete, runnable program from the source tree — the parallel prime finder — which exercises almost everything in this chapter:

```
#!/usr/bin/env stryke
# Run: stryke --no-interop examples/parallel_prime_finder.stk

p "--- Primes pgreps ---"
val @res = ~> (1:100) pgreps { is_prime _ } collect sort { _ <=> _1 }

p "Found #{len @res} primes: #{ join ' ', @res }"

len(@res) |> aeq 25, "1:100"
@res[0]   |> aeq 2,  "first"
test_run
```

Read it top to bottom. The shebang and a run-hint comment. A `p` greeting, no semicolon, no escaped newline. A `val` binding — immutable, because `@res` is computed once. The range `1:100` uses stryke's colon-range operator, not Perl's `...`. The `~>` thread macro filters with `pgreps { is_prime _ }` — a parallel streaming filter — then collects and sorts, with `_` and `_1` standing in for the sort comparands. The output line uses `#{...}` Ruby-style interpolation to drop `len @res` and a `join` directly into the string, with no concatenation. And the last three lines are assertions and a test runner — the file is simultaneously a demo and a test. Every piece of that program is a topic this book will expand, but you can already read it, and that is the point: stryke front-loads its density so that even a first program says something real.

Chapter 3 — The Topic and the Underscore

If you take away one thing from this book about what makes stryke code *look* like stryke code, it is this: you almost never name a closure parameter. Where another language forces you to invent a name — `x`, `item`, `element` — for a value you will use once on the very next token, stryke gives you a slot you can reference directly. That slot is the underscore, and the system built around it is the language's calling card. The style guide elevates it to a hard rule: **implicit closure parameters always**. Named parameters are correct only when a closure is a real top-level `fn` whose arity is greater than two *and* the names actually carry meaning. Everywhere else — every `map`, `grep`, `sort`, `reduce`, every pipeline stage, every short lambda — you use the underscore.

The topic and its slots

The bare `_` is the **topic** — the current value flowing through a closure or pipeline stage. It is sigil-free. In a `map`, it is the element being mapped; in a `grep`, the element being tested; in a one-argument function, the single argument. It is the same idea as Perl's `$_`, but spelled without the sigil, because the sigil is a Perl-ism needed only inside string interpolation. Bare `_` is the stryke spelling, and it is the calling card.

Dropping the sigil is a smaller change than it looks and a larger one than it sounds. Smaller, because the topic was always a single distinguished value and a one-character spelling serves it perfectly well — `$_` carried a dollar sign in Perl mostly because Perl's grammar leans on sigils to tell variables from barewords, and the topic is special enough to earn an exception. Larger, because once the topic is `_` rather than `$_`, it stops reading like a variable at all and starts reading like punctuation — a hole in the expression where the current value goes. That shift in how the eye parses it is what lets `_ * 2` read as “double the thing” rather than “multiply the variable named underscore by two.” The sigil survives in exactly one place, string interpolation, where `"$_"` still needs the dollar to mark where the variable begins inside literal text; everywhere outside a string, bare `_` is correct and the sigil form is the Perl compatibility spelling.

When a closure takes more than one argument, you index the slots: `_0` is slot zero, `_1` is slot one, `_2` is slot two, and so on. They are **zero-indexed** and they are **barewords** — no sigil required. `_` and `_0` refer to the same thing, slot zero; `_` is just the shorter spelling for the common case. This is a small but consequential decision, and it is the one place stryke diverges from the language whose numbered-block-parameter idea it most resembles. Ruby added `_1`, `_2` block parameters that are *one-indexed* — `_1` is the first argument. stryke makes them zero-indexed, so `_0` is the first argument and `_` is an alias for `_0`. The reason is consistency with the rest of the language: array indexing is zero-based, slice positions are zero-based, and `_[0]` is the first character of the topic, so making positional slots one-based would create exactly one counting system that ran against all the others. By starting slots at zero, stryke ties the topic `_` to slot zero and counts up from there, and a reader never has to hold two different base offsets in their head depending on whether they are indexing a slot or an array. The cost is that a programmer arriving from Ruby has to unlearn `_1`-means-first; the benefit is that everyone, Ruby background or not, gets one counting rule for the whole language. Here are the slots replacing Perl's sort-and-reduce magic variables, from the source tree:

```
val @nums = {3, 1, 4, 1, 5, 9, 2, 6, 5}
val @asc  = sort { _ <=> _1 } @nums
val @desc = sort { _1 <=> _ } @nums
```

In Perl you would write `sort { $a <=> $b }`. The `$a` and `$b` are package-global magic variables, valid only inside `sort` and `reduce` blocks, and they pollute the enclosing package. That last point is a genuine wart, not pedantry: because `$a` and `$b` are package globals, a Perl programmer who happens to use `$a` as an ordinary variable name elsewhere in the same package is sharing storage with the `sort` machinery, and the two uses can interfere in ways that are maddening to debug. The magic is also strictly limited to `sort` and `reduce` — there is no `$c` for a third argument, so the moment a callback needs three values, the `$a/$b` convention runs out and you fall back to unpacking `@_`. `stryke's` `_` and `_1` are the universal positional protocol — the *same* spelling you use everywhere — and they are lexical to the block, so they cannot collide with anything in an enclosing scope. They also extend naturally: a comparator needs `_` and `_1`, but a three-argument callback gets `_2`, a four-argument one gets `_3`, with no new syntax to learn and no point where the convention runs out. The next example, again from the source, shows that extension:

```
fn Demo::compute = _ + _1 * _2
val $sum = Demo::compute(1, 2, 3)
```

`Demo::compute` is an expression-bodied function of three arguments. It never names them; it reads slot zero, slot one, and slot two directly. Called with `(1, 2, 3)`, it computes `1 + 2 * 3`, which is `7`.

map, grep, reduce: the underscore in daily use

The places you reach for the underscore most are the list builtins. A `map` transforms each element, a `grep` keeps the elements that pass a test, a `reduce` folds the list down to a single value. All three take a `{ ... }` block, and inside that block the topic is `_`:

```
val @doubled = map { _ * 2 } (1:5)
val @evens   = grep { _ % 2 == 0 } (1:10)
```

`@doubled` is `(2, 4, 6, 8, 10)`; `@evens` is `(2, 4, 6, 8, 10)` as well, but for a different reason — those are the even numbers in one through ten. Read the blocks: `_ * 2` doubles the topic, `_ % 2 == 0` tests it. There is no `|element|` to declare, no `$x ->` to write. The block *is* the body, and `_` *is* the element.

`reduce` is where the two-slot form pays off. A fold takes an accumulator and the next element, and combines them:

```
val @vals = (1, 2, 3, 4, 5)
val $sum  = reduce { _ + _1 } 0, @vals
val $prod = reduce { _ * _1 } 1, @vals
```

`reduce { _ + _1 } 0, @vals` threads an accumulator starting at zero through the list, adding each element — slot zero is the accumulator, slot one is the next element. `$sum` is 15; `$prod` is 120. This is why `reduce` is where the two-slot form earns its place: a fold is inherently binary, taking what you have so far and the next thing, and the `_/_1` pair maps onto that shape with no naming at all. The accumulator is slot zero precisely because it is the value that threads through every step, and reading `_` as the accumulator everywhere in a fold becomes second nature quickly. The reason this beats `$a/$b` is the same reason it beats it in `sort`: Perl's magic variables are package globals scoped to those two builtins, while `_/_1` are lexical to the block, cannot collide with an outer variable, and extend uniformly to `_2`, `_3`, and beyond when a callback needs more than two values — a place Perl's two-letter convention simply has no answer.

The underscore is just as natural when the elements are structures. Sorting a list of hashrefs by a field needs no named parameters at all:

```
val @items = (
  { name => "carol", age => 30 },
  { name => "alice", age => 25 },
  { name => "bob",   age => 28 },
)
val @by_age = sort { _->{age} <=> _1->{age} } @items
```

`_->{age}` reaches the `age` field of the first comparand, `_1->{age}` the second. The comparator reads exactly like the question it answers — compare these two by age — with no scaffolding between intent and expression.

The outer chain: reaching enclosing topics

Nesting is where most languages' implicit-parameter schemes fall apart, and it is the strongest argument for why stryke's underscore system is a designed protocol rather than a bag of shorthands. The moment you put a `map` inside a `map`, the inner topic shadows the outer one — and if the inner closure needs the outer element, you are forced to capture it into a named variable *before* the inner closure opens. This is the failure mode shared by every implicit-parameter scheme that came before: Clojure's `%`, Scala's `_`, Ruby's `_1`, Swift's `$0`, Raku's `$_a` all bind to the *current* closure's argument and stop there. None of them can name a value one frame out. The result is that every implicit-parameter shorthand silently has a depth limit of one — it works beautifully until you nest, and then it forces you back to named capture exactly when the code is getting complicated enough that you most wanted the brevity. The shorthand evaporates at the moment of greatest need.

stryke's outer-topic chain removes that limit, and it is the piece of the underscore system with no prior art: a positional implicit parameter that can reach a named depth of enclosing scope.

The outer-topic chain is `_<`. Each `<` climbs one level of closure nesting outward. `_` is the current topic, `_<` is the enclosing closure's topic, `_<<` is two levels out, `_<<<` three, up to `_<<<<<` for five. The `<` glyph is iconic — “back, before, earlier” is universal in math and ASCII, from `<-` to `<<` to version comparison — and `_` is “the topic,” so the composition tells you its meaning on sight. Here is the canonical nested-map, from the source tree:

```
val @result = map {
  # outer topic here = 10, 20, 30
  (1:3) |> map { _ + _< }
  # inner topic = 1..3, _< = current outer
} (10, 20, 30)
# Expected: [[11,12,13],[21,22,23],[31,32,33]]
```

Read the inner block `_ + _<`. Inside it, `_` is the inner element — one, two, or three — and `_<` reaches out one level to the *outer* element — ten, twenty, or thirty. So for outer element ten, the inner map produces `11`, `12`, `13`; for twenty, `21`, `22`, `23`; and so on. Without the outer chain you would have to write a capture line, `var $outer = _`, before the inner map and then reference `$outer` inside it. The style guide shows both, and prefers the chain:

```
@xs |> maps { _ + $outer }    # explicit lexical capture
@xs |> maps { _ + _< }       # `_<` reads cleaner once internalized
```

The chain composes with arbitrary depth. The multiplication table is the cleanest demonstration — a nested `map` where the inner cell multiplies its column by the outer row:

```
var @table = map { join(" ", map { sprintf("%4d", _ * _<) } 1:5) } 1:5
```

`_ * _<` is column times row. And it keeps going. A three-level nest can reach two frames out with `_<<`:

```
val @scores = map {
  map {
    map {
      bonus
      _ + _< + _<<
    } 1, 2
  } 10, 20
} 100, 200
# _ = bonus ∈ {100, 200}
# _ = row ∈ {10, 20}, _< = bonus
# _ = col ∈ {1, 2}, _< = row, _<< =
```

In the innermost block, `_` is the column, `_<` is the row one frame out, and `_<<` is the bonus two frames out. The expression `_ + _< + _<<` sums all three across the entire cross-product, producing cells from `111` to

222. The depth marker is orthogonal — it composes uniformly with any slot, so `_1<<` is slot one two frames out, `_2<<<<` is slot two five frames out. One concept, applied uniformly.

There is a subtle correctness guarantee underneath this, and it is the kind of detail that separates a feature that works in the demo from one that holds up in real code. The outer chain always reads the *lexical* outer-scope topic of the current closure — the value the outer iteration is on right now — never an in-flight mutation from a sibling iteration of the outer loop. This matters because the naive way to implement “reach the outer element,” capturing it through the same cell machinery that named outer-scope variables use, would expose the inner closure to whatever the outer loop’s variable currently holds, which can drift if the outer body mutates as it goes. The implementation avoids this by routing topic-variant writes through a frame-local path that bypasses the capture-cell write-through used by named outer-scope variables, so `_<` resolves to a stable snapshot of the outer topic for the duration of the inner iterations even while the outer loop is mid-flight. The practical upshot is that you get the readability of an implicit capture with the safety of a frozen value: `_<` cannot be accidentally clobbered by the outer loop advancing, because it reads the topic frame, not a shared mutable cell. The style guide is explicit that if `_<<` resolution ever shows up hot in a profile, switching that one spot to an explicit lexical capture is fine — but it also says this is rarely needed, and to profile before reaching for it, which is the correct order: write the readable form first, and only trade it away with measurement in hand.

The two underscores: `[N]` versus `$(N)`

There is one place the underscore notation forks, and it is worth being precise about because the two forms look almost identical and mean entirely different things.

`_[N]` — bare underscore, square brackets — is the **char-of-topic**. It indexes into the *current topic* as if it were a string, returning the character at position N. `_[3]` is the fourth character of `_`. It is an alias for the bang-index form `_[!N]`, and it exists so you never have to write `substr(_, 3, 1)`. The style guide lists it among the topic spellings: `_[3]` is preferred over the `substr` form.

`$_[N]` — dollar-underscore, square brackets — is something else entirely. It is the **legacy Perl `@_` positional argument**: the sigil-and-brackets form that, in Perl, indexes the current subroutine’s argument array. `$_[0]` is the first argument passed to the enclosing function. `stryke` keeps it for compatibility so that Perl ports parse and run, but it is not how you write new `stryke` code. For positional arguments, you use the slot barewords `_0`, `_1`, `_2`; for character access into the topic, you use `_[N]`. The dollar-bracket form is a compat artifact, nothing more.

The distinction matters because the failure is silent. If you mean “the third character of the topic” and you write `$_[3]`, you will not get a parse error — you will get the fourth element of the enclosing function’s argument list, which is almost certainly not what you wanted. Both forms are syntactically valid, both produce a value, and nothing flags the mix-up at compile time, so a single stray dollar sign turns a character lookup into an argument lookup with no diagnostic to catch it. This is the inevitable cost of keeping the Perl compat spelling alive: `$_[N]` *has* to mean Perl’s `@_` indexing for ports to run, and `_[N]` *should* mean char-of-topic because that is the `stryke` idiom, and the two cannot help looking nearly identical. The discipline that keeps them apart is simple and worth making a habit: in new `stryke` code you never write `$_[N]` at all. For a character of the topic you write `_[N]`; for a positional argument you write the slot bareword `_N`. The `$_[N]` form is a compat artifact you read in ported code and never type in fresh code, and once that rule is internalized the silent failure has no opportunity to occur, because you are never reaching for the dollar-bracket form in the first place. Keep the two coordinate systems straight: bare `_[N]` reads *into* the topic by character; `$_[N]` reads *the arguments* by position, and in new code you reach for `_N` instead.

Why this matters

It would be easy to read all of this as a collection of shortcuts — clever spellings that save a few characters. That undersells it. The underscore system is a *protocol*, and its value is that it is uniform. The same `_` is the topic in a `map`, the topic in a `grep`, the first comparand in a `sort`, the accumulator in a `reduce`, and the single

argument of an expression-bodied function. The same `_N` indexes positional slots everywhere, zero-indexed, sigil-free, with no special cases. The same `<` depth marker climbs frames everywhere, composing with any slot. You learn one idea and it applies in every context where a closure receives values.

That uniformity is what lets stryke code stay dense without becoming cryptic. A named parameter carries information only the first time you read it; after that it is overhead — a name to track, a binding to scope, a line of declaration to skip past. The underscore carries the *same* information positionally, with no name to invent and no declaration to write, and it does so identically across the whole language. Combined with the threading operators of the previous chapters, it is why a stryke pipeline reads as a single continuous thought rather than a stack of named intermediate results. You do not name what you do not need to name, and in stryke, you rarely need to.

Chapter 4 – Threads and Pipes

There is a particular kind of code that every working programmer has written a thousand times and read a million: the deeply nested function call. You start with a value, and you want to clean it, transform it, and present it, so you write `present(transform(clean(value)))` and move on. The expression is correct. It is also backwards. The first thing that happens to `value` — `clean` — is written closest to the value, but the last thing the reader’s eye reaches before the value is `present`, the operation that happens *last*. To understand what the code does, you read it inside-out: find the innermost call, work outward, and reverse the execution order in your head as you go. Mainstream languages have trained us to do this reflexively, but it is a tax, and stryke does not collect it.

This chapter is about the two operators that abolish inside-out reading: `|>`, the pipe-forward, and `->`, the thread-first macro. They are the spine of idiomatic stryke. Style guide rule 3 states the principle: `f(g(h(x)))` reads inside-out; `x |> h |> g |> f` reads left-to-right and shows up at the call site without diving the cursor through three closing parens. Always pipeline.

The idea is older than stryke and worth placing in its lineage, because stryke is a deliberate synthesis rather than an invention from nothing. Clojure’s threading macros `->` and `->>` (1996-era Lisp made mainstream) take a value and weave it through a sequence of forms, injecting at the first or last argument slot. Racket’s `->` from the `threading` library does the same with a more flexible placeholder. F# and Elixir made the single-argument pipe `|>` a load-bearing part of everyday code — in Elixir especially, `|>` is so pervasive that the language’s whole idiom is “thread the subject through transformations.” stryke takes all three traditions and unifies them under one roof: the explicit-separator pipe `|>` from the ML/Elixir line, the space-separated thread `->/->>` from the Lisp line, and then extends past every one of them with parallel and streaming members no prior threading construct has. The sequential operators are familiar territory imported faithfully; the parallel ones in the next chapter are where stryke leaves the prior art behind.

The shape of a pipeline

A pipe-forward chain is read exactly the way the data flows — top to bottom, left to right, in execution order. Here is the canonical example from `examples/pipe_forward.stk`:

```
val $r1 = 5 |> sqrt |> int;           # int(sqrt(5)) = 2
```

The value `5` enters on the left. `sqrt` runs first because it is written first. `int` runs second because it is written second. The result, `2`, is what `int(sqrt(5))` would have produced — but you never had to count parentheses or read the expression in reverse. The comment makes the equivalence explicit: `int(sqrt(5))`. The same computation, two notations, and the pipeline is the one your eye already reads in the right order.

The mechanics are simple and worth stating precisely: each `|>` takes the value on its left and makes it the **first argument** of the callable on its right. When the right-hand stage is a bare function name with no other arguments, the left value is its only argument. When the stage already carries arguments, the piped value slots in ahead of them. This second case is the one that trips up newcomers, so the same example file demonstrates it directly:

```
val $r2 = "hello world" |> split(/ /)
```

`split` ordinarily takes a pattern and a string. Here the string `"hello world"` arrives via the pipe and lands in front of the explicitly-written `/ /` pattern — so the stage is `split("hello world", / /)`... except that `split`’s natural signature is pattern-first, which is exactly the situation thread-last exists to fix, and we will

come back to it. The point for now is that the piped value is always inserted at argument position one, and any arguments you write in the stage follow it.

This first-argument injection rule is not arbitrary; it is the convention that makes a chain composable. Because every stage receives the threaded value in the same known position, you can read a pipeline as a strict left-to-right reduction: the output of stage n is the first argument of stage $n+1$, and nothing about a stage's behavior depends on where it sits in the chain. A stage written as `uc` means “uppercase whatever arrives”; it means the same thing first, last, or in the middle. That positional stability is what lets you cut a chain anywhere, splice a stage in or out, and reason about the result without re-deriving the whole expression. The cost of the convention is the occasional pattern-first callee like `split`, where the threaded value wants to land somewhere other than position one — and stryke answers that with the placeholder `_` and the thread-last operators, both covered below, rather than abandoning the rule.

Stages that do real work

A pipeline is only as useful as the stages you can put in it, and stryke accepts more than bare function names. The most common richer stage is the block-arg form — `map { ... }, grep { ... }, sort { ... }` — where the block uses the implicit topic `_` for the current element. From `pipe_forward.stk`, a complete top-five-longest-words pipeline:

```
val $text = "The quick brown fox jumps over the lazy dog repeatedly"
val @top = $text |> split(/\s+/) |> sort { len(_1) <=> len(_) }
val @top5 = @top[0:4]
p " top 5: ", join(" ", @top5)
```

Read it straight down: take `$text`, split it on whitespace, sort the resulting words so the longest come first. The sort block uses `_` and `_1` for the two elements being compared — the universal positional topic vars, not the `$a/$b` magic that Perl's `sort` imposes — and `len(_1) <=> len(_)` reads “longer one first” because the second operand sorts ahead of the first. There is no temporary array for the split result, no intermediate variable for the sorted list until you choose to bind one. Each `|>` hands its output straight to the next stage.

The same file shows a longer chain that mixes `map`, `grep`, and a terminal `sum`, broken across lines for readability:

```
val $total = (1:100)
  |> map { _ * _ }
  |> grep { _ % 2 == 0 }
  |> sum
p " sum of even squares 1:100 = $total"
```

Note the range literal `1:100` — stryke's polymorphic colon-range, not Perl's `...`. The pipeline squares every number from 1 to 100, keeps the even results, and sums them, and it does so in four lines that each name exactly one operation. When a pipeline starts a fresh line with `|>` the parser auto-continues the previous statement, so no trailing backslashes are needed; the leading `|>` on each line is itself the signal that the expression isn't finished. This is one of the few places stryke's newline-terminates-statement rule bends, and it bends in the pipeline's favor.

The reason this rule deserves stating precisely is that stryke, unlike Perl, terminates statements at the newline — there is no implicit continuation across lines for most expressions, and a multi-line arithmetic expression that ran past a newline without a `\` would silently produce a truncated value. The pipeline is a deliberate exception carved out by the parser: an operator at the *start* of a line is unambiguous proof that the previous line could not have been complete, so the parser pulls the two together. This is the same mechanism that makes the broken-across-lines chain above readable. Each `|>`-led line is a single named operation, vertically stacked, and the result is a transformation you scan top-to-bottom the way you would read a recipe — split, then square, then filter, then sum. Compare that to the equivalent `sum(grep { ... } map { ... } 1:100)` written on one line: same bytecode, but the reader has to unwind three nested calls to recover the same four steps the pipeline lays out flat.

The inline-lambda stage

Sometimes a stage needs a small computation that isn't a named function and isn't a list operation. For that, `pipe_forward.stk` uses the arrow-block `>{ ... }` as an immediately-invoked stage:

```
val $info = (1:5) |> >{ "n=" . len(_) . "; first=" . _->[0] }  
p " IIFE stage : $info"
```

The piped value — the list `1:5` — becomes the topic `_` inside the block, so `len(_)` is its length and `_->[0]` is its first element. The arrow-block lets you wedge an anonymous computation into the chain without defining a function for it. There is a subtlety here worth stating precisely: in a `|>` pipeline you normally use plain `{ ... }` blocks for `map/grep/sort`, and `>{ ... }` is reserved for the thread-macro family — but as a *standalone* IIFE stage in a `|>` chain, the arrow-block is the right tool, because a bare `{ ... }` in stage position would be ambiguous with a hash literal. Use `>{ }` when the whole piped value is the topic and you want to run an inline lambda over it.

From pipe to thread: ~>

The pipe-forward is the day-to-day default, but at three or more stages a second operator earns its keep. `~>` is the thread-first macro — stryke's spelling of Clojure's `->` and Racket's `~>` — and its payoff is one fewer operator character per stage. Where `|>` separates stages with an explicit `|>` between each, `~>` writes the source once, immediately after the arrow, and then lists the stages **space-separated**:

```
val $upper = ~> "hello world" uc rev  
p " ~> uc rev: $upper"
```

That line, from `examples/threading_macros.stk`, threads `"hello world"` through `uc` and then `rev`. No `|>` between `uc` and `rev` — just whitespace. The threaded value is injected as the first argument of each stage, exactly as with pipe-forward; the difference is purely surface syntax, and it is purely a parse-time rewrite. Style guide section 7.6 quantifies the trade-off: use `~>` when you have three or more stages and want to drop one operator-character per stage; the macro form pays for itself at three calls. Below three stages, `|>` is clearer because the explicit separators read as deliberate steps; at three and above, the space-separated thread is denser without losing the left-to-right ordering.

The space-separated form has a second, quieter virtue beyond saving keystrokes: it removes a visual repetition that, past a few stages, starts to read as noise. In a long `|>` chain the eye sees the separator as often as it sees the operations, and the operator stutters down the line. The thread form writes the source once, right after the arrow, and then lets the stages stand on their own — the operations are the only tokens, and the chain reads as a list of verbs applied in order. This is exactly the aesthetic Clojure's `->` was built around, and stryke inherits it directly. The mechanism that makes it work is the same parse-time rewrite: the parser sees the arrow, takes the immediately-following token as the seed value, and then folds each subsequent space-separated stage into a nested call, building the identical AST that the equivalent `|>` chain or hand-written nested calls would build.

A `~>` chain can mix every stage form. The same file shows a multi-stage data pipeline with block-arg stages:

```
val @squared = ~> 1:5 map { _ * _ } collect
```

and the style guide's section 7.7 lists the four notations a thread stage can take:

```
~> $x uc                # bare fn: x → uc(x)  
~> $x div(_, 2)         # placeholder: x → div(x, 2)  
~> $x >{ _ * 2 + 1 }    # arrow-block: x → (x*2+1)  
~> @xs map { _ * 10 } sum # block-arg form: map { ... } @xs → sum(...)
```

A stage is a bare function name, a `name(args)` call with an explicit `_` placeholder marking where the threaded value goes, an anonymous `>{ ... }` arrow-block, or a `name { ... }` block-arg call like `map { ... }`. The

placeholder form deserves attention. When a function's natural first argument is *not* your threaded data, you write `_` where the data should land, and the macro injects there instead:

```
-> 10 div(2) p           # bare-fn stage: div(10, 2)
-> 10 div(_, 2) p        # explicit `_` placeholder: div(10, 2)
```

Both produce `div(10, 2)` because `div`'s first argument is the dividend either way — but the explicit `_` makes the injection point unambiguous and overrides the implicit position when present.

The exact desugaring, and why it costs nothing

It is worth being precise about what `|>` and `->` actually *are*, because the answer governs when you can reach for them. They are not runtime constructs. They are not function calls into a pipeline library. They are parse-time AST rewrites — the parser sees `5 |> sqrt |> int` and builds exactly the same syntax tree it would have built for `int(sqrt(5))`, before a single byte of bytecode is emitted. Zero runtime cost: parse-time only, reading exactly the way the data flows. The sequential `->` and `|>` are the same semantics as Clojure's `->`, Racket's `->`, and Elixir's `|>`.

Walk the rewrite through concretely. The parser reads `5 |> sqrt |> int` left to right. It takes `5` as the seed. At the first `|>` it folds the seed into the right-hand stage `sqrt`, producing the call node `sqrt(5)`. At the second `|>` it folds *that whole node* into `int`, producing `int(sqrt(5))`. The output is one nested-call AST, indistinguishable from what the programmer would have typed by hand. The bytecode compiler that runs next never knows a pipe was involved — it sees only calls. The thread form `-> 5 sqrt int` runs the identical fold; the only difference is that the seed sits adjacent to the arrow and the stages are space-separated rather than `|>`-separated. Both spellings collapse to the same tree before compilation.

This matters for two reasons. First, performance: there is no closure allocation, no iterator object, no indirection, no pipeline-runtime dispatch. A pipeline compiles to identical bytecode as the nested-call form it replaces, so choosing the readable spelling never costs you a cycle. This is the strict sense of a “zero-cost abstraction” — the abstraction exists only in the source text and is gone by the time anything executes. It is the same property Rust claims for its iterators and C++ for its templates, except achieved more simply, by AST rewriting rather than by trusting an optimizer to fuse and inline away the overhead. There is nothing to optimize away because nothing was ever built. Second, semantics: because the rewrite happens before execution, a pipeline is exactly as debuggable as the calls it desugars to. Each stage is standalone debuggable — you can lift any single stage out, run it on a literal, and confirm its behavior, because it is a real call, not a fused mega-operation. A stack trace from inside a pipeline points at the real function the stage names, not at some pipeline-driver frame.

The parallel members of the thread family — the streaming `~s>` and the chunk-parallel `~p>`, covered in the next chapter — are different in kind. Those bake execution semantics directly into the operator: `~s>` spawns worker threads and channels, `~p>` fans chunks across rayon. They are runtime constructs, and the operator is doing real work at execution time. But the two sequential operators of this chapter are free abstractions in the strict sense: pure notation, zero cost. Knowing which family an operator belongs to tells you immediately whether it has a runtime footprint — the sequential pair never does, the parallel ones always do.

Thread-last: when the data belongs at the end

The thread-first operators inject the piped value at argument position one. That is the right place most of the time, but a whole tradition of list-processing functions takes the *block first and the list last* — Perl's `map BLOCK LIST`, `grep BLOCK LIST`, `reduce BLOCK LIST`. For these, you want the threaded value at the *last* argument position, and stryke provides `->>` and `~>>` — thread-last, Clojure's `->>` — for exactly that. The syntax is identical to `->`; only the injection position changes. From the style guide's section 7.3:

```

fn div = _0 / _1
->> 10 div(2) p           # div(2, 10) = 0.2 (10 → last arg)
->> 10 div(_, 2) p        # explicit `_` overrides: div(10, 2) = 5

->> (1,2,3,4,5) take(3) p  # take(3, list) = (1,2,3)

```

Compare the first line here with the thread-first `~> 10 div(2)` from earlier. Thread-first made `10` the first argument: `div(10, 2)`. Thread-last makes `10` the *last* argument: `div(2, 10)`, which is $2 / 10 = 0.2$. The `take(3)` example shows the natural use: `take`'s signature is count-then-list, so threading `(1,2,3,4,5)` into the last position gives `take(3, (1,2,3,4,5))`, exactly the shape `take` wants. And as with thread-first, an explicit `_` placeholder overrides the default position — `->> 10 div(_, 2)` puts `10` back at the front and yields `5`.

The rule of thumb: reach for `->>/~>>` when the callee's natural last argument is your data — it saves wrapping `_` placeholders. If you find yourself writing `func(_, fixed_arg)` in every stage of a thread-first chain, you wanted thread-last all along.

This first-versus-last split is the exact distinction Clojure draws between `->` and `->>`, and the reason both exist there and here is that the function ecosystems split cleanly into two camps. Functions that operate *on a primary subject* — string methods, accessor calls, transformations of a single value — want their subject first; thread-first serves them. Functions that operate *over a collection with a fixed operation* — `map`, `grep`, `reduce`, `take`, `filter` in the Perl/Lisp tradition where the block or count comes first and the list comes last — want the collection last; thread-last serves them. Having both operators means you never have to fight a function's signature. You pick the thread direction that matches the callee's natural shape and the threaded value lands where it belongs with no placeholder ceremony. The `_` placeholder remains the escape hatch for the rare stage whose injection point matches neither default — it overrides whichever operator you're using and pins the value exactly where you write the `_`.

Precedence and the parens you don't need

In a pipeline, parentheses are almost always noise. The style guide's section on paren-less calls puts it directly: “In a pipeline, parens are almost never needed because `|> / ~>` already supply the staging — write `$x |> uc |> rev` not `rev(uc($x))` and not `($x) |> uc() |> rev()`.” Each stage is a list-operator call; the pipeline's staging does the work that parens would otherwise do.

The one place precedence forces parens is when a binary operator follows a call *inside a stage* and would re-bind. The classic trap is length-then-compare: `len @a > 1` parses as `len(@a > 1)` — `len` of a boolean — because `>` binds tighter than the implicit call boundary. The fix is `len(@a) > 1`, sealing the call before the comparison. But that is a property of bare list-operator calls, not of pipelines; inside a chain, the staging already seals each stage, so you rarely hit it. When you do need a binary result mid-chain, the arrow-block stage is the clean answer — wrap the comparison in `>{ ... }` and let the block be the stage.

Why `~>` over `|>`, everywhere

The decision rule for the operators in isolation is `|>` as the day-to-day default and `~>` for three-plus stages. The working convention goes further: default to the thread macro `~>` for *any* chain, not only the long ones. The reasoning is consistency more than character-count. A codebase where every chain is a `~>` thread reads uniformly — the eye learns one shape — and the space-separated stage list is the denser, more scannable form once internalized. The single-character saving per stage is real but secondary; the primary win is that you stop deciding, per-chain, which of two near-identical operators to use, and simply thread.

The `|>`-for-short, `~>`-for-long split is the right default for code that will be read by people still learning the operators, because the explicit `|>` separator makes the staging visible and self-explanatory. For fluent stryke — the power-user register the whole language is tuned for — `~>` is the spine, and `|>` is what you drop to only when a single stage carries arguments awkward enough that the explicit separator reads better. Both desugar

to the same tree; both cost nothing; the choice is entirely about how the source reads. The thread macro reads as the language's native gesture, and standardizing on it removes a recurring micro-decision from every chain you write — which matters in a language where chains are the dominant control structure, not an occasional flourish.

Threads and pipes are where stryke's whole stance becomes concrete. The language is built so that the order you write a transformation is the order it runs and the order you read it — and so that choosing that readable form never costs a cycle, because the readability lives entirely in the parser. Once your hands learn `~>`, the inside-out nested call starts to look like what it always was: a notation that made the machine's convenience the reader's burden. The next chapter takes the same arrow and adds cores.

Chapter 5 – Going Wide

Most languages treat parallelism as a different language. You start with a clean sequential loop, decide it's too slow, and then rewrite it — thread pools, work queues, futures, join handles, a mutex or two, and a careful audit of which variables are now shared. The rewrite is large, error-prone, and bears little resemblance to the loop you started with. The thesis of this chapter is the opposite stance, and it is stryke's whole argument about concurrency: **parallel should cost a prefix, not a rewrite**. The sequential `map` becomes a `pmap`. The sequential thread `->` becomes the chunk-parallel `-p>`. The shape of the code does not change; a letter or a character does, and the work fans across every core.

The pool you never configure

Underneath every parallel builtin in stryke is a single rayon work-stealing thread pool, and you never see it. The contract: rayon work-stealing under the hood, every `p*` builtin runs across all cores, results stay deterministic, no setup, no thread-pool boilerplate. Work-stealing is the property that makes this practical without tuning — idle threads pull work from busy ones, so uneven workloads balance themselves and you never size a queue or pick a chunk count for the common case.

It is worth dwelling on why work-stealing, specifically, is what makes “just add a prefix” honest rather than a leaky promise. The naive way to parallelize a loop is to split the input into one chunk per thread up front and let each thread grind its chunk — static partitioning. That works only when every item costs the same. The moment work is uneven — some items trigger a slow path, some inputs are larger, some hit a cache and some don't — static partitioning leaves you with one thread still churning through its unlucky chunk while every other core sits idle, and your speedup collapses toward serial. Work-stealing fixes this structurally: the pool keeps a deque of pending work per thread, and a thread that empties its own deque steals from the tail of a busy thread's deque. The slow items get spread out dynamically as they reveal themselves, so the cores stay saturated regardless of how lumpy the workload turns out to be. This is the same scheduler design behind Cilk, Java's ForkJoinPool, and Go's goroutine runtime — a well-trodden, decades-proven approach — and rayon brings it to Rust with the additional guarantee that the data-race classes those other runtimes leave to discipline are caught by Rust's ownership system at compile time. stryke inherits both properties for free: the dynamic load balancing and the data-race safety, neither of which the programmer has to think about.

The single shared pool is also a deliberate choice rather than an accident. Spawning a fresh thread pool per parallel call would mean paying thread-creation cost on every `pmap`, and nested parallel calls would oversubscribe the machine — more OS threads than cores, each fighting the others for the CPU. One process-wide pool sized to the core count means a `pmap` inside a `pmap` (as the web crawler later in this chapter does) reuses the same workers rather than multiplying them, and the work-stealing scheduler flattens the nested parallelism onto the available cores without explosion. You get composable parallelism — parallel calls inside parallel calls — without the oversubscription that naive thread-per-call designs suffer.

The simplest demonstration is `pmap`, the order-preserving parallel map. From `parallel_primitives.stk`:

```
fn Demo::Par::slow_square($n) { sleep 0.02; $n * $n }
val $t1 = now_ns()
val @squares = pmap {Demo::Par::slow_square} (1:20)
val $dt = (now_ns() - $t1) / 1e9
printf "  pmap(1:20, sleep 0.02ea) : %.2fs    (serial = 0.40s)\n", $dt
```

Twenty items, each sleeping twenty milliseconds — serially that is four hundred milliseconds. Run across cores it collapses to a fraction of that, and the result `@squares` comes back in input order, so `pmap` is a true drop-in for

`map`. The body uses the implicit topic; the named helper `Demo::Par::slow_square` is namespaced because shadowing builtins is forbidden, and `Demo::Par::` keeps it clear of the runtime's own names. Right beside it, `pgrep` is the parallel filter, also order-preserving:

```
val @primes = pgrep {
  return 0 if _ < 2
  for val $d (2:(~> _ sqrt int)) { return 0 if _ % $d == 0 }
  1
} (1:50)
```

Note the nested `~> _ sqrt int` inside the loop header — a thread-first chain computing the integer square root of the current topic. Parallel and threaded code compose without ceremony.

Streaming versus materializing

There are two flavors of every parallel list operation, and the distinction is the single most important thing to understand in this chapter. The plain forms — `pmap`, `pgrep`, `pflat_map` — are eager and order-preserving: they run across all cores, collect every result, and return a real array in input order. The trailing-`s` forms — `pmaps`, `pgreps`, `pflat_maps` — are streaming: lazy iterators backed by background worker threads that yield results in *completion* order, not input order, the instant each is ready. The line:

eager variants return in INPUT order (collect + re-sort) — streaming variants return in COMPLETION order (lower latency, non-deterministic order)

The mechanism behind the order difference explains the latency difference. An eager `pmap` must hold back every result until it can present the whole array in input order, which means it cannot hand you element three until elements one and two are also done — even if three finished first. To restore input order from out-of-order completion, it collects everything and re-sorts by original index. That final collect-and-sort is a barrier: nothing downstream sees a single result until the slowest item has finished and the array is assembled. The streaming form removes the barrier entirely. Each worker thread, as it finishes an item, pushes the result onto a channel that the consumer drains immediately. There is no array to assemble and no index to sort by, so the first result reaches the consumer the instant any worker produces it, and the consumer can begin work while later items are still in flight. You trade deterministic order for the elimination of the tail-latency barrier — and the trade is free whenever order doesn't matter.

When is each correct? Streaming is the right choice when the downstream stage is order-insensitive — `sum`, `count`, `uniq`, `freq`, any sketch — or when you want to start consuming results before all inputs finish. If the next thing you do is `sum`, `count`, `deduplicate`, or build a histogram, order does not matter, and the streaming form gives you lower latency because you never wait for the slowest item to take its place in a sorted output. There is a second, subtler win: the streaming form has a bounded memory footprint. The eager form must hold every result in the array at once; the streaming form holds only the items currently in the channel between producer and consumer, so a `pmaps` feeding a running `sum` never materializes the full result set — it processes a hundred-million-item stream in constant memory. The canonical pattern is stream-then-sum:

```
val $sum_squares = pmaps { _ * _ } 1:100 |> sum
die "sum-squares" unless $sum_squares == 338350 # n(n+1)(2n+1)/6
```

`pmaps` emits squares in whatever order the workers finish; `sum` doesn't care; the answer is exact. The same file shows `pgreps` as a streaming filter feeding a count, and `pflat_maps` for one-to-N streaming expansion feeding a `uniq`:

```
val $total_items = len pflat_maps { (_, _ * 10, _ * 100) } 1:10
die "expand" unless $total_items == 30

val @u = uniq(pflat_maps { (_, _ % 3, _ % 5, _ % 7) } 1:50)
```

Each input becomes three outputs, streamed as they're produced, then `len` or `uniq` consumes the stream. The guide's iteration-shorthand section makes the default explicit: streaming variants “are the default when

chaining; the array-returning forms (`pmap`, `pgrep`, `flat_map`) are only correct when you genuinely need a `vec`.” Reach for the `s` forms first; drop the `s` only when you actually need an indexable array back in input order.

Two ways to thread across cores

The streaming-versus-eager split lives at the level of individual builtins. One level up, at the level of whole pipelines, stryke offers two parallel members of the thread-macro family, and they parallelize along different axes. Understanding the axis each one parallelizes is the key to choosing between them, because they are not interchangeable — they exploit different kinds of available parallelism.

`~s>` is the per-item streaming pipeline; it parallelizes *across stages*. Each stage runs as its own worker thread, connected to the next by a bounded channel, and items flow through all stages concurrently — intermediate arrays are never materialized. Section 7.4 describes it as the choice for uneven latencies, an I/O-plus-CPU mix, where fast stages can pull ahead while slow ones catch up. This is pipeline parallelism in the classic sense: think of a CPU instruction pipeline or an assembly line, where every stage is busy on a different item at the same moment. If stage one is I/O-bound (fetching) and stage two is CPU-bound (parsing), `~s>` lets the fetcher pull ahead and queue work for the parser, so the I/O latency of item $n+1$ overlaps the compute of item n . The throughput approaches the slowest single stage rather than the sum of all stages. From the showcase:

```
val $items = ~s> [1, 2, 3, 4, 5] map { _ * 10 } map { _ + 1 }
p "  basic chain: $items items"

val $matched = ~s> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] grep { _ % 2 == 0 } map { _ * _ }
```

The mental model is a factory line: stage one multiplies by ten and hands each result down the channel, stage two adds one as soon as it has an item to add to. No stage waits for the previous stage to finish the whole batch. Because output is non-deterministic in order, the macro's *value* is the count of items the last stage emitted — useful precisely when you don't care about order. The bounded channel between stages provides automatic backpressure: if a downstream stage is slow, the channel fills, and the upstream stage blocks rather than racing ahead and buffering unbounded work in memory. The showcase also shows `~s>` carrying shared state through a `varsync` counter, which we come to shortly.

`~p>` is the chunk-parallel pipeline, and it parallelizes the orthogonal axis — *across data* rather than across stages. It splits the input into chunks and runs the *whole pipeline on each chunk* in parallel via rayon, then auto-merges the per-chunk results. Section 7.5 gives its identity as `par_reduce { stage1 |> stage2 |> ... } SRC`. Where `~s>` runs one item through many concurrent stages, `~p>` runs many items through one copy of the whole pipeline per core. This is data parallelism — the map-reduce shape — and it is the right axis when the stages themselves are uniform and cheap but the input is large: every chunk does the identical sequential work, and the only coordination is the merge at the end. The two axes are genuinely orthogonal, which is why stryke ships both rather than picking one: `~s>` wins when stages have unequal cost and you want them overlapped; `~p>` wins when the input is large and the per-item work is uniform. A pipeline that is both stage-heterogeneous and data-heavy can even compose them, though that is rare. The showcase opens its `~p>` section with a letter-frequency histogram fanned across cores:

```
val $h = ~p> "the quick brown fox jumps over the lazy dog" letters freq
```

The auto-merger is the clever part. It inspects the first chunk's result type and picks a combine strategy: hash-of-numbers merges key-wise with `+` to give the canonical histogram, numeric scalars sum, arrays concat, strings concat. So `letters freq` per chunk produces a hash of letter counts, and the merger adds those hashes key-wise into one histogram — exactly what you'd want, with no merge code written. This is the part that makes the prefix honest. In a hand-rolled map-reduce, the merge step is where the bugs live: you write `pmap` over chunks easily enough, but then you have to write the combine — and getting “add two histograms key-wise” right, handling keys present in one chunk but not another, is exactly the fiddly per-result-type logic that a one-character prefix has no room to express. The auto-merger absorbs it by reading the result type and applying the algebraically-correct combine for that type. Crucially, all four of these combine operations — integer `+`, hash key-wise `+`, array concat, string concat — are associative, which is what makes chunking *legal*

at all: the result must not depend on how the input was split into chunks or in what order the partial results merge, and associativity is precisely the property that guarantees it. The merger does not invent a combine; it applies the monoid the result type already carries. The same file proves it at corpus scale, threading every `.stk` file in the project through a frequency count:

```
val $corpus = ~p> c("**.stk") letters freq
val @top5 = (sort { $corpus->{_1} <=> $corpus->{_2} } keys %$corpus)[0:4]
```

And it ends with a timing block that runs the same histogram first sequentially with `~>` and then chunk-parallel with `~p>`, printing the speedup — the clearest possible demonstration that the prefix is the only difference between the two:

```
val $h_seq = ~> c("**.rs") letters freq
...
val $h_par = ~p> c("**.rs") letters freq
...
printf " ~p> chunk-par:  %.0f ms (%.2fx speedup)\n", $par_ms, $seq_ms / $par_ms
```

`~>` to `~p>` — one character — and the work goes wide.

The parallel-to-sequential boundary

Some pipelines fan out, do parallel work, and then need the *whole merged result* for a final step. You can't parallelize “sum all the values in the one merged histogram” — there's only one histogram. `stryke` marks that transition explicitly with the `||>` boundary, or its English-spelled twin `|then|`. From the threading showcase:

```
val $alpha = ~p> "the quick brown fox jumps over the lazy dog" letters freq ||> values |>
  ~ sum
val $alpha2 = ~p> "the quick brown fox jumps over the lazy dog" letters freq |then|
  ~ values |> sum
```

Everything left of `||>` runs chunk-parallel and auto-merges; everything right of it runs sequentially on the merged whole. Here the parallel side builds the frequency histogram across chunks; the `||>` boundary then drops to sequential `~>` mode to pull the `values` and `sum` them into a single alphabetic-character count. The two spellings are identical in meaning — `||>` for the symbol-minded, `|then|` for code meant to read aloud. The rule: use `||>` or `|then|` to switch to sequential `~>` mode for stages that need the merged whole.

The boundary is not just convenience syntax; it marks the one place where parallelism genuinely cannot continue. A chunk-parallel pipeline works because each chunk is independent — but the moment you need a value computed *from the merged result* of all chunks, there is only one merged result and nothing left to fan out. Summing the values of a single histogram is inherently serial: there is one histogram. Without an explicit boundary, a chunk-parallel pipeline would have to guess where the data-parallel phase ends and the serial reduction begins, and guessing wrong silently corrupts results. The `||>` makes the phase transition a syntactic fact the reader and the runtime both see. It is the analogue of a fork-join barrier in a classic parallel program — the line where the workers rejoin and a single thread takes the combined result forward. Making it a visible operator rather than an implicit inference is the same design instinct as the rest of the chapter: the cost and the structure of parallelism are written into the source, never hidden.

Channels, spawns, and shared state

Below the macros sit the explicit primitives for hand-built concurrency, and they are deliberately small. `pchannel` is the bounded channel; `spawn` launches a detached background thread; together they give you the producer-consumer pattern with no setup. From `examples/multi_threaded_channels.stk`:

```

val ($tx, $rx) = pchannel(2)

spawn {
  for (1:5) {
    p " [Producer] Sending: $_"
    $tx->send(_)
    sleep 0.01
  }
}

var @results = ()
for (1:5) {
  var $val = $rx->recv()
  p " [Consumer] Received: $val"
  push @results, $val
}

```

`pchannel(2)` is a bounded channel of capacity two, returning a sender/receiver pair. The `spawn` block runs the producer on its own thread, pushing five values; the main thread receives them. Backpressure is automatic — the bound means the producer blocks once two items are unconsumed, so a fast producer can't outrun a slow consumer and exhaust memory.

This is the same communicate-don't-share model that Go built its concurrency story on with channels and goroutines, and that Erlang and its descendant Elixir pushed even further with message-passing processes. The principle is that two threads coordinate by handing values across a channel rather than by both reaching into the same mutable memory and racing. `stryke` gives you the channel primitive directly — `pchannel` plus `spawn` is producer-consumer with no setup — so the patterns you'd reach for in Go translate almost verbatim. The bounded capacity is the load-bearing detail: an unbounded channel turns backpressure into a memory leak, because a producer that is faster than its consumer just keeps queuing, and the queue grows until the process dies. Bounding the channel makes the producer's speed self-regulate to the consumer's — the exact behavior you want and the reason `pchannel` takes a capacity argument rather than defaulting to infinite.

For shared mutable state — the cases where the communicate-don't-share model is genuinely awkward and a shared structure is the natural fit — `stryke`'s primitive is `varsync` (and its package-scoped sibling `oursync`). It is the `stryke` shared-state primitive with lockless reads and lock-on-write; under the hood it is an `Arc<Mutex>` wrapping the declared variable, so you mutate it from many threads without rolling a mutex by hand. The lockless-read, lock-on-write split is the standard reader-writer optimization: reads are by far the common case in patterns like a visited-set or a memo table, so making them cheap and reserving the lock for the rarer writes is where the performance comes from. The clearest use is in `examples/parallel_web_crawler.stk`, where a thread-safe visited-set guards a parallel recursive crawl:

```

varsync %visited

fn Crawler::crawl($url, $depth) {
  return if $depth <= 0
  return if $visited{$url}
  $visited{$url} = 1
  ...
  @links |> pmap { Crawler::crawl(_, $depth - 1) }
}

```

`$visited{$url} = 1` is an atomic write to shared state; the `return if $visited{$url}` guard reads it locklessly. The recursion fans out through `pmap`, so many crawler threads touch `%visited` at once, and `varsync` keeps it correct without a single explicit lock. This is also the clearest demonstration of why the single shared pool composes: the `pmap` inside `Crawler::crawl` is itself called from inside a `pmap`, recursively, so the parallelism nests arbitrarily deep as the crawl descends — yet the work-stealing pool flattens all of it onto the same fixed set of workers without spawning a thread explosion. The streaming showcase uses the same primitive for a shared accumulator threaded through a `~s>` pipeline — `varsync $accum = 0` mutated inside

a streaming map stage — proving the primitive composes with the macros, not just the bare builtins. A program can mix bare parallel builtins, the parallel macros, hand-built channels, and shared `varsync` state, and they all coordinate through the one pool and the one ownership model underneath.

The wider toolbox

The two streaming flavors and two parallel macros cover most of the ground, but the parallel family is broader, and `parallel_primitives.stk` walks the rest. `pfor` is fire-and-forget parallel iteration for side-effecting work — writing to a key-value store, firing metrics — where you want the iteration parallel but have no return value to collect:

```
pfor { kv_put($db, "key-$_", _ * _) } (1:100)
```

`psort` is the parallel sort, taking the same `_/_1` comparator block as sequential `sort`:

```
val @sorted = psort { _ <=> _1 } @random
```

`preduce` is the parallel tree-fold — valid only for associative operations, since the tree shape means the combine order isn't left-to-right — and `pmap_reduce` fuses a parallel map and reduce with no intermediate vector:

```
val $sum = preduce { _ + _1 } (1:1000)
val $sum_sq = pmap_reduce { _ * _ } { _ + _1 } (1:100)
```

The associativity requirement on `preduce` is the same constraint that governed `~p>`'s auto-merger, and it bites for the same reason: a parallel reduce does not fold left-to-right one element at a time the way sequential `reduce` does. It splits the input, reduces each piece independently on its own core, and then combines the partial results in a tree. Addition and multiplication survive this because $(a+b)+c$ equals $a+(b+c)$ — the tree can group the operands however it likes and the answer is invariant. Subtraction and division do not survive it; reassociating them changes the result, so handing `preduce` a non-associative combine yields a fast, parallel, *wrong* answer with no error raised. This is the one place where the parallel prefix is not a free swap: you must confirm the operation is associative before you parallelize the fold. The fused `pmap_reduce` matters for a different reason — it never materializes the mapped vector. A `pmap` followed by a `preduce` would build the full intermediate array of mapped values and then reduce it; fusing the two means each worker maps its chunk and folds it in one pass, so a billion-element map-then-sum runs in the memory of the partial accumulators rather than a billion-element array. It is the parallel analogue of the loop-fusion an optimizing compiler does to a sequential `map().sum()`.

Two more named members round out the established set. `puniq` is the parallel deduplicate — `-> @input puniq |> sort` in the test corpus — and `pany` is the short-circuiting parallel existence check: it stops the moment any worker finds a match. The short-circuit is what distinguishes `pany` from a `pgrep` followed by a length test: `pgrep` evaluates every item, while `pany` signals all workers to stop as soon as one finds a match, so a hit early in a large input returns almost immediately instead of paying for the whole scan. `(1:100) |> pany { _ > 90 }` returns true and stops early; `pany { _ > 200 } (1:100)` returns false after exhausting the input, because proving *no* item matches requires checking every item. Beyond the common forms, `pmap_chunked` groups input into contiguous batches before distributing — the right tool when profiling shows rayon overhead dominates the actual computation, i.e. when per-item work is so cheap that scheduling each item separately costs more than the work itself. The default per-item scheduling has a small fixed cost per task; for trivial bodies that cost can exceed the body itself, and at that point you want to amortize the scheduling overhead across a batch by handing each worker a chunk of items rather than one. It is the manual override for the rare workload where the automatic work-stealing granularity is too fine. And `pmap_on` is the distributed cousin: a parallel map that fans work across a *cluster* of remote machines rather than local cores, serializing each closure once and caching it on every worker. Its one constraint follows directly from running in a separate process on a remote host — the block must be self-contained and cannot close over local file handles or database connections, because those resources do not exist on the remote machine. The closure ships, but the surrounding process does not, so anything the closure needs must travel inside it.

The prefix, proven

Step back and look at what changed between the sequential and parallel versions of each pattern in this chapter. `map` became `pmap`. `map { }` chaining became `maps`, and `pmaps` to go wide and streaming. `~>` became `~p>`. `sort { }` became `psort { }`. `reduce` became `preduce`. In every case the body of the work — the block, the comparator, the stages — is byte-for-byte identical to its sequential twin. The web crawler's parallel recursion is the same recursion you'd write single-threaded, plus a `varsync` on one declaration and a `pmap` on one call. The corpus histogram is `~> c("**.stk") letters freq` going parallel by gaining one letter. That is the whole claim made concrete: parallelism in `stryke` is a prefix you add to working sequential code, not a second program you write to replace it.

Chapter 6 — Sigils, Strings, and Interpolation

stryke wears Perl's sigils — `$`, `@`, `%` — and to a reader coming from a sigil-free language they can look like decoration, three flavors of dollar sign cluttering every variable. They are not decoration. A sigil is type information carried at every use site, and it tells you, at the exact moment you read a variable, what shape of thing you are touching. This chapter is about why that per-use-site shape annotation pays for itself, and about the string interpolation that sigils make possible — including the one interpolation form, Ruby's `#{ ... }`, that you will reach for constantly.

Three sigils, three shapes

The three sigils mark the three fundamental container shapes:

- `$` marks a **scalar** — a single value: a number, a string, a reference, a boolean. `$port`, `$name`, `$res` are all one-of-something.
- `@` marks an **array** — an ordered list of values. `@endpoints`, `@results`, `@links` are many-of-something, indexed by position.
- `%` marks a **hash** — an unordered map from keys to values. `%failure_rate`, `%visited`, `%latency` are lookups, indexed by key.

The declarations in `examples/api_health_check.stk` show all three side by side, each sigil announcing the shape before you read a single value:

```
var @endpoints = (  
  "/api/users",  
  "/api/orders",  
  ...  
)  
  
val %failure_rate = (  
  "/api/users" => 0.05,  
  "/api/orders" => 0.10,  
  ...  
)  
  
var %latency  
var %total  
var %ok
```

`@endpoints` is plainly a list; `%failure_rate` is plainly a map from endpoint to a number; `%latency` is a map you can see is being initialized empty before it's filled. You learn the shape of each from its first character. Note also the `val/var` distinction layered on top of the sigil — `val %failure_rate` is the *immutable* hash of rates, frozen at compile time, while `var %latency` is mutable because the loop below fills it. The sigil tells you the shape; `val/var` tells you whether it moves.

These two annotations answer the two questions you most often have about a variable, and they answer them at the declaration without you reading the right-hand side or scrolling to find later assignments. A sigil-free language carries the same information, but it carries it implicitly: you infer that something is a list because you

saw it constructed from a list literal, or because a later line indexes it, and you infer mutability from whether you can spot a reassignment. Both inferences require reading more code than the declaration itself. The sigil and the `val/var` keyword front-load that work into the two characters at the start of the line. The cost is one sigil character per variable; the return is that shape and mutability are never something you have to reconstruct from context, because they are stated where the variable is introduced and, as the next section shows, restated at every place the variable is used.

Why shape-at-use-site helps

The deeper payoff of sigils is that the shape annotation travels to every use, not just the declaration — and crucially, the sigil at a use site reflects what you’re extracting, not just what was declared. This is the part that surprises newcomers and the part that earns the system its keep.

Consider element access. In the health-check dashboard, `%total` is a hash, declared with `%`. But when you read a single endpoint’s total, you write `$total{$ep}` — with a `$`, because you are pulling *one scalar value* out of the hash:

```
for val $ep (@endpoints) {  
  val $n = $total{$ep}  
  val $err = $client_err{$ep} + $server_err{$ep}  
  ...  
}
```

The `$` on `$total{$ep}` is not a contradiction of the `%` on the declaration — it is information. It says: this expression yields a scalar. Likewise `$endpoints[0]` indexes the `@endpoints` array and the `$` announces that the result is one element, a scalar. The sigil tracks the *result shape of this access*, so reading `$h{$k}` you know you’re getting a value, reading `@h{@keys}` (a hash slice) you know you’re getting a list. The shape is never ambiguous and never requires you to remember the declaration — it’s restated, correctly, at the point of use. That is what “per-use-site shape info helps” means: you read the local expression and you know what it produces, without scrolling up.

The same logic governs interpolation, which is where the practical payoff is densest, because the parser uses the sigil to know how to expand a variable inside a string.

Interpolation: the four forms

stryke interpolates variables directly into double-quoted strings, and the rule is unambiguous: always interpolate, never build strings with `.` concatenation when interpolation reaches. There are four forms, ordered here from cheapest to most powerful.

This is where the per-use-site shape information from the sigil pays a concrete, mechanical dividend rather than a merely readability one. The interpolation parser, scanning the inside of a double-quoted string, uses the leading sigil to decide *how* to expand what follows. A `$` means “read one scalar and splice its value”; an `@` means “read an array and splice its elements joined by the separator.” Without the sigil, the parser inside a string would have no way to know whether `name` is meant as the literal text `name` or a variable to expand, and no way to know whether a variable should expand as one value or as a spread list. The sigil resolves both questions at the point of use, which is exactly why a sigil-free language needs heavier machinery — `${...}` blocks, format placeholders, explicit interpolation calls — to do what stryke does by reading one character. The four forms below are four answers the parser gives once the sigil has told it what kind of thing it’s expanding.

Bare scalar — `$var`. The default, the cheapest form. The variable’s value drops into the string at its position:

```
val $tmp = "/tmp/stryke_adv_io.txt"  
spurt $tmp, "line 1: hello\nline 2: world\nline 3: stryke\n"  
p "Wrote data to $tmp"
```

"\$tmp" interpolates the path. No quotes-and-dots, no `. $tmp .` dance — the variable reads inline as part of the literal.

Disambiguated scalar — `${name}`. When the character following a variable would extend its name, braces seal it. `$name_v2` looks up a variable literally called `name_v2`; `${name}_v2` interpolates `$name` and then appends the literal `_v2`. The canonical example is `"file_${name}.txt"`. Use `${ }` whenever an adjacent character — a letter, digit, or underscore — would otherwise be read as part of the variable name. It is the same brace-disambiguation Perl and shell readers already know, and it exists for exactly the cases where bare `$name` would grab too much. The need for it is a direct consequence of how greedy the bare form is: the parser reads `$` and then consumes the longest run of name characters it can, because that is the only sensible default when most variable names are bare words with no trailing literal. The braces are the manual boundary you draw when the longest-run default would overshoot, telling the parser precisely where the name ends and the literal text resumes.

Arbitrary expression — `#{ EXPR }`. This is the Ruby-borrowed form, and it is the one that changes how you write strings. The other three forms interpolate *variables*; `#{ }` interpolates an arbitrary **expression** — arithmetic, a function call, a method call, a whole pipeline if you like. From `examples/advanced_file_ops.stk`:

```
val $content = c"$tmp"
p "Slurped content (length: #{len $content})"
```

`#{len $content}` runs the `len` builtin inside the string and drops its result in. There is no concatenation, no temporary variable, no breaking out of the string literal — the call sits inline where its result belongs. The health-check and link-checker examples lean on it constantly:

```
p "Checking status codes for #{len @urls} sites..."
...
p " [OK]  #{res->{status}} #{res->{url}}"
```

`#{len @urls}` calls `len` on the array; `#{res->{status}}` reaches into a hash reference; `#{res->{url}}` does the same — each an expression too complex for bare `$var` interpolation, each dropped inline with `#{ }`. The web crawler reports its result the same way:

```
p "Crawl complete. Visited #{len %visited} unique URLs."
```

`#{len %visited}` counts the keys of the shared hash inside the string. This is the only interpolation form that is not just-a-variable — the other three splice in the value of a name, while `#{ }` runs an arbitrary expression — and once it's in your fingers, the old pattern of breaking a string into pieces with `.` to splice in a computed value disappears. The contrast is direct: `"sum: " . sum(@a)` becomes `"sum: #{sum @a}"`, and the second reads as one literal where the first reads as three fragments stitched together. The deeper point is that `#{ }` collapses the distinction between “string with holes” and “expression that builds a string.” In a concatenation-based language those are two different modes — you write literal text in quotes and you escape to expression mode with `.` or `+` every time you need a computed value, so a string with three computed pieces is six fragments alternating between the two modes. The `#{ }` form lets the whole thing stay a single literal with computed islands inside it, which is both fewer tokens and a flatter structure: one string, not a tree of concatenations. This is precisely the win Ruby's `#{ }` and the template-literal forms in other languages provide, and stryke takes the Ruby spelling directly because it is the most compact of the lot.

Array — `@arr`. An array interpolated bare expands to its elements joined by the list separator `$`, which defaults to a single space. From the threading showcase earlier in the book, `" squares: @squared"` prints the squared values space-separated. The contrast is `"items: @arr"` against the verbose `"items: " . join(" ", @arr)` — array interpolation is the join, built in, and it's the sigil that tells the parser to expand `@arr` as a list rather than look for a scalar named `arr`. This is the clearest single illustration of why the sigil earns its character. The same name `arr` means two different expansions depending on the leading character: `$arr` splices one value, `@arr` splices the joined elements. A sigil-free language cannot encode that distinction inside a string at all — it has to fall back to an explicit `join` call, breaking out of the literal — because the variable name alone does not say whether to spread. The separator `$` being a settable variable is the Perl

inheritance: change it once and every bare-array interpolation downstream joins with the new separator, which is occasionally exactly the lever you want for, say, comma-separated output without touching any individual interpolation site.

The zsh form: `${ ... }` parameter expansion

Beyond the four sigil forms is a grammar of a different kind, and it is the one no other general-purpose language carries. Inside a double-quoted string, stryke understands the entire zsh **parameter-expansion** micro-language — the `${ ... }` operators shell programmers use to slice, trim, default, and transform a value without ever calling a function. `"${file##*.}"` is the extension of a path; `"${file%*.}"` is its stem; `"${name:-staging}"` is `$name`, or the literal `staging` if `$name` is empty; `"${(U)x}"` is `$x` upper-cased; `"${path/old/new}"` rewrites the first `old` to `new`; `"${#s}"` is a length; `"${arr[2,4]}"` is an array slice, and `"${arr[(i)needle]}"` is the index at which `needle` first appears. It runs the array-wide flags too — `"${(o)arr}"` sorts, `"${(u)arr}"` dedups, `"${(j:/:)arr}"` joins on a slash, `"${arr:#glob}"` drops every element the glob matches — and reaches into associative arrays: `"${hash[key]}"` is one value, `"${(k)hash}"` the keys, `"${(v)hash}"` the values. The whole of `man zshexpn`'s parameter chapter lives inside stryke's strings.

stryke does not reimplement any of it. The language embeds zshrs — a complete zsh, rewritten in Rust — and when the string parser meets a `${ ... }` it cannot read as a plain variable, it hands the raw expression to zshrs's `paramsubst`, the very engine the real shell runs, after bridging the referenced stryke variables into the parameter table. This is the identical move stryke makes for filename globbing, where every qualifier works because `glob` is `zsh:glob` rather than a stryke imitation of it. Single source of truth, zero reimplementation, every obscure flag correct by construction — including the ones nobody remembers exist.

It sits a little oddly against the argument the rest of this chapter makes, and the tension is worth naming. The four sigil forms exist precisely so that stryke does *not* need the shell's `${ ... }` heavy machinery to disambiguate a variable inside a string: one sigil character answers what a sigil-free language needs braces and placeholders for. The zsh form is not that. It is not disambiguation, it is text manipulation — the shell programmer's reflex of reshaping a value in place, in the string, without escaping to a function. stryke adopts it for the one population that reaches for it without thinking, the people arriving from the shell, and because, having already embedded an entire zsh to power the transpiler and the glob engine, the expansion grammar was sitting there to be exposed. You pay nothing for a form you can ignore, and shell veterans get muscle memory that simply works.

Two forms cannot be read statically: `${#x}` is a string length when `x` is a scalar and an element count when it is an array, and a positional like `${1:-d}` could mean the script argument or a regex capture. The string sugar takes the scalar reading and leaves the rest to an explicit valve — the `zpexpand` builtin. You call it with each variable supplied at its real type, arrays by reference (`\@arr`, so the argument list does not flatten into the name/value pairs), and zshrs returns exactly what the shell would: `zpexpand('${#arr}', "arr", \@arr)` is the element count, unambiguously. The layering is deliberate — native `"${ ... }"` for the common, readable case; `zpexpand` for the corner where only runtime type information can decide.

And like every stryke extension, it yields to compatibility. Under `--compat` the zsh routing is off entirely and `${name}` recovers its plain Perl meaning, so `${main:cfg}` is a package variable and nothing more. The zsh form is a stryke-mode luxury, never a tax on the Perl programmer.

When concatenation is still right

Interpolation is the default, not an absolute. The narrow case where `.` concatenation remains correct: only when both sides are non-string-literals — `$a . $b` joining two variables — or when computing a string lazily across statements. Joining two existing scalars with `.` — no surrounding literal to interpolate into — is fine, because there's no string literal context for interpolation to live in; forcing a literal there just to interpolate would mean writing `"$a$b"`, which is harder to read than `$a . $b`, not easier. And building a string up piece by piece across several statements, where each piece is computed separately, is a legitimate use of `.` because there's no single literal that could hold the whole expression — the pieces don't coexist at one point in the

source for a single literal to wrap them. The dividing line is whether a surrounding literal exists or naturally could: inside a single double-quoted literal, always interpolate; the moment you find yourself writing `"text $thing . " more"`, you wanted `"text $thing more"` or `"text #{ $thing } more"`, because the literal text on both sides is the giveaway that you broke a string apart to splice rather than concatenating two genuinely separate values.

Sigils in interpolation, and the `$_` exception

One small wrinkle ties the chapter together. The topic variable is bare `_` in code — that is the stryke calling card, and `$_` is rejected by `--no-interop`. But inside string interpolation the sigil-prefixed `$_` is needed, because the interpolation parser keys off the `$` to know it's looking at a variable. The topic-variable rule records the exception precisely: the sigil-prefixed `$_` is needed inside string interpolation, while bare `_` is the stryke calling card everywhere else. So you write `_` in a `map` block and `"$_"` inside a string in that same block, and both are correct. The apparent inconsistency is actually the rule holding firm: interpolation expansion is driven entirely by the leading sigil, so a bare `_` inside a string would be indistinguishable from the literal underscore character — there would be nothing to tell the parser to expand it. The `$` is not an exception to the bare-topic convention so much as the convention meeting the one context where dropping the sigil would make expansion impossible. Everywhere the parser can identify the topic from position — a `map` body, a `grep` test, a thread stage — bare `_` is right; inside a string literal, where position carries no such signal, the `$` is what flags the token as a variable at all. The sigil is required in the string for the same reason it's required everywhere: it's how the parser knows shape and knows it's reading a variable rather than text.

This is the throughline of the whole chapter. The sigil is not ornament inherited from Perl for nostalgia's sake. It is the mechanism that lets you read shape at every use site, the mechanism that lets the parser expand variables inside strings without ambiguity, and the mechanism that makes the four interpolation forms — bare, braced, expression, and array — each unambiguous about what they splice in. You pay one character per variable. In return, every variable announces what it is, every string expands without a concatenation, and the most powerful interpolation form, `#{ }`, lets an arbitrary expression sit exactly where its result belongs.

Chapter 7 – Ranges and Slicing

Most languages bolt ranges and slicing on as two unrelated features. You get a range type — `range(0, 10)` in Python, `0..10` in Rust, `1..10` in Perl — and then a wholly separate substring API: `substr`, `slice`, `[start:end]`, `.substring()`. The two share no syntax, no mental model, and frequently no convention about whether the upper bound is included. Worse, the conventions actively disagree across the boundary: Python's `range(0, 10)` is half-open and excludes 10, but its slice `s[0:10]` is also half-open, so the two at least agree with each other while disagreeing with everyone else; Rust splits the difference with `0..10` exclusive and `0..=10` inclusive depending on a punctuation mark you have to remember; Perl's `1..10` is inclusive but its `substr` is length-based, not endpoint-based, so the moment you stop counting by hand the off-by-one returns. Three languages, three answers, and inside each language two or more sub-conventions that don't line up. `stryke` collapses all of it into a single operator, the colon, and applies it everywhere a span of values appears: in a `for` header, between two array indices, inside a string bracket, with an optional step. Learn the colon once and you have learned ranges, slices, stepped iteration, and substring extraction all at the same time.

This chapter is about that operator and the indexing conventions that surround it. The payoff is consistency — once `1:10` means “one through ten inclusive” in a loop, it means exactly the same thing inside `$s[1:10]`, and you never have to ask whether the end is off-by-one. That single design decision — make every appearance of the colon mean the identical thing — is what eliminates the entire category of bug that the comparison above describes. There is no second convention to confuse it with, because there is no second convention.

The colon range, not ..

Perl spells a range `1..10`. `stryke` spells it `1:10`, and in new code the colon is the only correct form:

```
1:10      # DO    - the stryke polymorphic range
1..10     # DON'T - Perl 5 baggage, rejected by --no-interop
```

The `..` form still parses, because `stryke` keeps the door open for line-by-line CPAN ports, but `--no-interop` rejects it and any review will flag it. The reason to switch isn't aesthetic. The colon range is *polymorphic*: the same operator produces integer sequences, character sequences, and — per the style-guide range table — dates, IPs, and Roman numerals, with the type inferred from the literals on either side. `1:10` is an integer span; `'a':'z'` is a character span:

```
1:10      # ints
'a':'z'   # chars, same operator, type-inferred from the literals
```

The type inference is the part that matters. In Perl, `1..10` and `'a'..'z'` are both the `..` operator, but the second one only works because of a specific magic-string-increment rule bolted onto the operator for the string case; reach for dates or IP ranges and you leave the operator behind and call a module. `stryke` makes the polymorphism a property of the operator itself: the colon dispatches on the *type* of its endpoints and produces the right kind of sequence, so the same six-character expression spans integers, characters, dates, addresses, or numerals without ever changing the syntax. You write the endpoints; the operator figures out what kind of sequence connects them, and there is no second spelling for the “harder” cases. This is the same consolidation logic as the slice operator, applied to generation instead of extraction — one surface, inferred behavior, no special cases promoted to their own syntax.

The third colon is the step

A range can carry a step as a third colon-separated term. This is where the consolidation really pays off, because Perl has no native stepped range at all — you reach for `grep`, a C-style `for`, or a helper. In `stryke` the step is just another colon:

```
1:100:5      # eager - builds 1, 6, 11, ... 96 right now
range(1, 100, 5) # lazy - same sequence, produced on demand
```

The colon form and the `range()` builtin are not a right/wrong pair; they are two range forms with different evaluation semantics, and which one you want depends entirely on whether you need the whole list to exist. `1:100:5` is *eager* — it materializes the whole list at once, in memory, before the next stage runs. `range(start, end, step)` is *lazy* — it yields values on demand, one at a time, so a downstream `take` or early `last` can stop the generator before it has produced more than a handful of elements.

The difference is not academic; it is the difference between a few megabytes and a few gigabytes. Run `(1:1e8) |> take 3` and the eager colon builds a hundred-million-element list first, then throws away all but three of them — measured peak memory footprint is about **2.4 GB**. Run `range(1, 1e8) |> take 3` instead and the lazy generator emits three values and stops — peak memory footprint about **9 MB**. Same sequence, same three results, roughly a 260× memory difference, because the lazy form never builds the body it isn't asked for. Push the upper bound to `1e9` and the eager form either exhausts memory or spends seconds allocating ten gigabytes it will discard, while the lazy form is unchanged: still three values, still instant, still constant-memory.

The decision rule follows directly. Reach for `range()` whenever the span is large, streamed, capped by an early `take`, or feeding a pipeline that might short-circuit — anywhere the full list is wasteful or impossible to hold. Reach for `:` whenever the span is small and bounded and you genuinely want the materialized list — a loop over screen rows, a slice of a known window, a handful of integers fed to `join`. For a `for val $i (0:$H - 1)` over a few hundred rows the eager list is trivially cheap and the colon is the right, readable choice; for `range(0, 1e9) |> pmaps { ... } |> take 10` the laziness is load-bearing and the colon would be a memory bomb. Both are correct; they are tools for different sizes of the same job.

The step can be negative, which gives you a descending sequence with no special syntax. The slicing pin test exercises all three cases directly, and the assertions tell you exactly what each one produces:

```
join(",", (0:10:2)) |> aeq "0,2,4,6,8,10", "range step 2"
join(",", (1:10:3)) |> aeq "1,4,7,10", "range step 3"
join(",", (10:0:-2)) |> aeq "10,8,6,4,2,0", "range step -2"
join(",", (5:5)) |> aeq "5", "range single element"
```

Read those four lines as the definition of the operator. `0:10:2` walks from 0 to 10 inclusive in steps of two and stops on 10 because 10 is reachable. `10:0:-2` counts *down* — the negative step reverses direction, and again the endpoint is included. And `5:5`, a degenerate range where both ends coincide, yields a single element rather than the empty sequence you might expect from a half-open convention. That last assertion is the tell: `stryke` ranges are inclusive on both ends, and `5:5` is the smallest proof of it.

Stepped ranges show up constantly in real code as the natural way to stride through a sequence. The RNA codon translator walks an RNA string three letters at a time using a single stepped range as the loop driver:

```
for val $i (0:len($s)-3:3) {
  val $codon = $s[$i:$i+2]
  val $aa = Bio::translate_codon($codon)
  last if $aa eq "STOP"
  push @amino_acids, $aa
}
```

`0:len($s)-3:3` is the entire chunking strategy. Start at index 0, advance by 3 each iteration, stop once there are fewer than three characters left — and inside the body, `$s[$i:$i+2]` slices out exactly the three-character codon. One range to step the loop, one slice to extract the window. There is no separate “chunk” library call

because the colon already expresses both halves.

Inclusive slicing, and why

Here is the single most important convention in this chapter, and the one most likely to trip up someone arriving from Perl or Python: **stryke's `[N:M]` slice is inclusive on both ends.** `$s[1:3]` returns the characters at indices 1, 2, and 3 — three characters — not the two-character half-open span that `[1:3]` produces in Python, where `"hello"[1:3]` is `"el"`.

This is a deliberate break from the dominant convention, and it is worth being explicit about what is being traded. Half-open slicing — Python's, Rust's, and the C-derived mainstream — has one genuine virtue: adjacent slices tile cleanly, since `s[0:3]` and `s[3:6]` partition without overlap, and the length of a slice is the simple subtraction $M - N$. stryke gives that up on purpose, because it optimizes for a different property: agreement between the range operator and the slice operator. If `1:3` generates `(1, 2, 3)` as a sequence, then `$s[1:3]` *must* address positions 1, 2, and 3 or the colon would mean two different things in two places — and eliminating exactly that kind of cross-context disagreement is the entire point of the chapter. The cost is that slice length is $M - N + 1$ and adjacent windows share an endpoint; the benefit is that you never carry two mental models. For a language whose colon appears in loops, ranges, steps, string slices, and array slices, the consistency is worth more than the clean tiling. stryke never calls `substr`; bracket slicing replaces it entirely:

```
$s[1]      # DO - single-char index / DON'T: substr($s, 1, 1)
$s[1:3]    # DO - chars 1,2,3 inclusive / DON'T: substr($s, 1, 3)
$s[-1]     # DO - last char / DON'T: substr($s, -1, 1)
$s[-3:-1]  # DO - last three chars / DON'T: substr($s, -3, 3)
@arr[2:5]  # DO - array slice, same `:` / DON'T: @arr[2..5]
```

Why inclusive? Because it makes the slice notation agree with the range notation, as just argued — `1:3` produces `(1, 2, 3)` as a range, so `$s[1:3]` returning the characters at positions 1, 2, and 3 is the only choice that keeps the colon honest. A half-open slice would mean the colon meant one thing in a `for` loop and a subtly different thing inside brackets — exactly the kind of off-by-one trap stryke is built to eliminate. The divergence from Perl is sharper still, because Perl's `substr` isn't endpoint-based at all: its third argument is a *length*, not an end index, so `substr($s, 1, 3)` means “three characters starting at 1,” a completely different parameterization that happens to coincide with `[1:3]` only because both yield three characters here. The guide spells the trap out:

stryke's `[N:M]` is **inclusive on both ends**. `$s[1:3]` returns 3 chars (indices 1, 2, 3). Perl's `substr($s, 1, 3)` returns 3 chars starting at index 1 (length-based). Same result for that example, but the mental model is different and they diverge for any range that crosses an end.

The “same result for that example” is the trap. For `$s[1:3]` both conventions happen to give three characters, so a casual reader assumes they're equivalent and gets burned the first time the ranges differ. Internalize “inclusive both ends” and the ambiguity disappears.

The pin test nails this down against the string `"hello"`:

```
val $s = "hello"

$s[1:3] |> aeq "ell", "s[1:3] mid slice"
$s[0:2] |> aeq "hel", "s[0:2] from start"
$s[0:4] |> aeq "hello", "s[0:4] full string"
```

`$s[0:4]` returning the *whole* five-character string is the clincher. Index 4 is the last character of `"hello"`, and an inclusive slice reaches it. Under a half-open convention you'd need `[0:5]` and would index past the end. Here, `0:4` spans the full string because both endpoints are real positions and both are included.

Negative indices

Indices count from the end when negative, with `-1` denoting the last element. This works for single access and for both ends of a slice, so “the last three characters” is `[-3:-1]` and reads almost like English:

```
$s[-1] |> aeq "o", "s[-1] last"
$s[-2] |> aeq "l", "s[-2]"
$s[-5] |> aeq "h", "s[-5] wraps to first"

$s[-3:-1] |> aeq "llo", "s[-3:-1] last 3"
```

`$s[-5]` on a five-character string wraps all the way back to the first character — `-5` is “five from the end,” which is the front. Negative bounds compose into slices the same way positive ones do; `-3:-1` is “third-from-last through last, inclusive,” and on “hello” that’s “llo”.

This makes a common real-world operation trivial. Pulling a file extension is just a negative-bounded slice:

```
val $ext = "report.pdf"
$ext[-3:-1] |> aeq "pdf", "file extension last 3"
```

And slicing a known span out of a URL is the same operation with positive bounds:

```
val $url = "https://example.com/path"
$url[8:18] |> aeq "example.com", "url slice [8:18]"
```

Both are the same `[N:M]` form. There is no separate “from the end” API and no separate “from the start” API — one bracket, one colon, signed indices.

Out-of-bounds clamps instead of failing

A slice whose upper bound runs past the end of the value clamps to the end rather than warning or throwing. Perl warns when `substr` runs off the end, Rust *panics* on an out-of-range slice index, and Python silently clamps — stryke takes Python’s forgiving choice and stops at the last real character:

```
$s[1:99] |> aeq "ello", "s[1:99] clamps gracefully"
$s[0:99] |> aeq "hello", "s[0:99] clamps to full"
```

`$s[1:99]` on “hello” returns “ello” — everything from index 1 to the end, with the absurd upper bound 99 truncated to the last valid index. This is the no-defensive-code principle from the style guide applied to slicing: you don’t bounds-check before slicing, because the slice itself does the sane thing. The win is concrete in text processing, where bounds are routinely uncertain — “give me the next ten characters” shouldn’t fault when only six remain, it should hand back the six. A language that panics here forces a length check before every slice; a language that clamps lets you write the slice you mean and trust it to stop. `$s[2:]` open-ended “to the end” forms exist for exactly this reason, but even when you over-specify the bound, the result is the same, which means you can reach for either spelling without thinking about which one is “safe.”

Open-ended and stepped slices — the missing end is inferred

The clamping story has a natural companion: if a slice that overshoots its upper bound quietly stops at the end, then you should not have to name the end at all. Leave it off. `$s[2:]` is “from index 2 to whatever the last index turns out to be,” `$s[:2]` is “from the start through index 2 inclusive,” and `$s[:]` — both ends omitted — is the whole value. The colon is doing the same job it always does; the endpoints on either side of it are simply optional, and when one is missing the slice supplies the obvious default: the front of the value for a missing start, the back for a missing end. This is the same open-ended slice grammar that arrays have always

accepted, now honored verbatim on scalars and on the topic, so there is one slice notation and it reads the same wherever a bracket appears:

```
val $s = "hello"

$s[2:]    |> aeq "llo",    "s[2:] open-ended to the end"
$s[:2]    |> aeq "hel",    "s[:2] open-ended from the start, inclusive"
$s[:]     |> aeq "hello",  "s[:] the whole string"
```

`$s[:2]` returning `"hel"` — three characters — is the inclusive convention showing through one more time: the omitted start defaults to index 0, the stated end is index 2, and both are included, so the span is 0, 1, 2. If you carry the half-open reflex from Python, where `s[:2]` is two characters, this is the place it will bite; internalize “inclusive both ends” and the open-ended forms follow the exact same rule as the closed ones, because they *are* the same rule with a default filled in.

A slice can also carry the step in the third position while leaving the endpoints open, and this is where two idioms fall out for free. `$s[::2]` steps by two across the whole string — every second character — and `$s[::-1]`, the negative step over both open ends, walks the string backwards and hands you the reverse. There is no separate `reverse`-a-string call to remember and no argument order to get wrong; the reversal is just a slice whose step happens to point the other way, which is exactly what `[::-1]` means in the languages that popularized it:

```
$s[::2] |> aeq "hlo",    "s[::2] every second char"
$s[::-1] |> aeq "olleh", "s[::-1] the whole string reversed"
```

The reason this is worth its own section is that until recently these forms only parsed on array *variables* — `@arr[2:]` worked, but `$s[2:]` and the topic form `_[2:]` went through the generic expression parser, which demanded an endpoint after the colon and rejected the bare `]` with an “unexpected token” error. That gap is closed: scalar subscripts and the topic char-index now run the full slice grammar, so `_[::-1]` inside a `map` block reverses each element with nothing but the topic and a slice.

```
map { _[::-1] } ("abc", "xyz")  # ("cba", "zyx") - reverse each, topic + slice
```

The payoff is the one the whole chapter has been building toward, extended one notch further: the colon means the identical thing in a range, a closed slice, and now an open-ended or stepped slice, on a string, an array, or the topic, with no spelling reserved for the “harder” cases. Omit an endpoint and it defaults to the obvious edge; flip the step and the slice runs backward. There is nothing new to learn here, which is precisely the point — the open-ended forms are the closed forms with a blank left where a default belongs.

Array slices are the same operator

Everything above is true of arrays with no change in syntax. The colon ranges, the inclusive ends, the negative bounds, and the clamping all carry over verbatim — arrays and strings share one slicing form:

```
val @arr = (10, 20, 30, 40, 50)

$arr[0] |> aeq 10, "arr[0]"
$arr[-1] |> aeq 50, "arr[-1]"

join(",", @arr[1:3]) |> aeq "20,30,40", "@arr[1:3]"
join(",", @arr[2:4]) |> aeq "30,40,50", "@arr[2:4]"
join(",", @arr[-2:-1]) |> aeq "40,50", "@arr[-2:-1] last 2"
```

`@arr[1:3]` returns three elements — indices 1, 2, 3 inclusive — exactly mirroring `$s[1:3]` returning three characters. An array slice composes straight into a pipeline or an aggregate, so summing a window is one expression:

```
val @big = (10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
sum(@big[2:5]) |> aeq 180, "sum of @big[2:5]"
sum(@big[-3:-1]) |> aeq 270, "sum of @big[-3:-1]"
```

`@big[2:5]` is indices 2 through 5 inclusive — (30, 40, 50, 60) — and `sum` reduces it to 180. The slice feeds the reducer with no intermediate variable, which is the whole point of having slices and aggregates speak the same notation.

In the advanced file-ops example, a slice grabs the first few matched files for display, where the comment confirms the inclusive count:

```
var @files = glob("$examples_dir/*.stk(.)")
for val $f (@files[0:2]) { # Slice: first 3
  p "  - $f"
}
```

`@files[0:2]` is indices 0, 1, 2 — three files — and the inline comment says “first 3” because the author counted inclusively, which is the correct mental model.

Byte versus codepoint: a deliberate two-coordinate system

There is one subtlety worth surfacing, because it bites people who assume all string indexing operates in the same unit. `stryke` maintains two coordinate systems on strings, and they are intentionally *not* interchangeable. Codepoint-based operations — the `[N:M]` slice and the `len` count — work in Unicode scalar values, so `len` and a `[N:M]` slice are a matched pair: count codepoints with `len`, index codepoints with `[N:M]`. Byte-based operations form the other pair. The rule is to pair byte with byte and codepoint with codepoint, never to cross them. Mixing a byte offset into a codepoint slice (or vice versa) is the mismatch to avoid.

The reason both systems exist, rather than one, is that they serve genuinely different jobs and collapsing them would force every program to pay for the one it doesn't need. Codepoint coordinates are what you want for human-facing text: “the third character,” “the last three characters,” a slice that respects character boundaries regardless of how many bytes each character occupies — these are codepoint operations, and addressing them in bytes would split a multi-byte character down the middle and corrupt the string. Byte coordinates are what you want for the machine-facing side: a binary protocol with fixed-width fields, a file offset, a length prefix, anything where the unit of measurement is the octet and Unicode is irrelevant. A single coordinate system can serve one of these cleanly but not both — a codepoint-only language makes byte-exact protocol work clumsy, and a byte-only language makes character work dangerous on anything past ASCII. `stryke` ships both and asks you to keep your units consistent within an operation, which is the only discipline that is actually required.

On pure-ASCII text the two coordinate systems coincide — every character is exactly one byte — so the distinction is invisible, which is exactly why it's dangerous to ignore on non-ASCII input: code that pairs a byte length with a codepoint slice will pass every ASCII test and corrupt the first emoji or accented character it meets. When your text is all ASCII — as in every example in this chapter — `len`, `[N:M]`, and any byte offset agree, and you never have to think about it. The moment multi-byte characters enter, keep your units consistent: `len` pairs with `[N:M]`, byte length pairs with byte indexing, and neither pairing ever crosses into the other.

Chapter 8 – Shaping Data

A program is its data structures plus the operations over them. `stryke` gives you the full Perl substrate — arrays, hashes, references — and layers modern shapes on top: a hashref constructor that reads like a literal, structs with methods and an implicit `$self`, and data-carrying enums dispatched by `match`. The design here is a deliberate gradient, from loosest to tightest: a bare `+{ }` hashref when you want an anonymous bag of named values, a `struct` when the shape is fixed and carries behavior, an `enum` plus `match` when a value is one of a closed set of alternatives. You climb the gradient as your data acquires more structure and more invariants, and each rung buys you something the one below it couldn't enforce. This chapter is about choosing the right rung and wiring it up correctly, with one hard rule threaded through all of it: never name a function after a builtin.

Arrays, hashes, and the `val/var` split

The base containers are Perl's `@array` and `%hash`, sigil-typed, flat by default. What changes is the binding keyword. `stryke` flips Perl's mutable-by-default `my` into immutable-by-default: `val` freezes a binding at compile time, `var` is the mutable counterpart, and bare `my` is reserved for CPAN ports. Default to `val`; reach for `var` only when you actually mutate. The Adam optimizer shows the split in one factory function — moment vectors are built once and frozen, the returned hashref is the assembled result:

```
fn Adam::new($n, $lr, $beta1, $beta2, $eps) {  
  val @m = (0.0) x $n  
  val @v = (0.0) x $n  
  +{  
    n    => $n,  
    lr   => $lr,  
    beta1 => $beta1,  
    beta2 => $beta2,  
    eps  => $eps,  
    m    => \@m,  
    v    => \@v,  
    t    => 0,  
  }  
}
```

`(0.0) x $n` is the list-repeat operator: a list of `$n` zeros, the idiom for a zero-initialized vector of known length. The `val @m` binding says these arrays are constructed once and never reassigned — which is true, because the actual moment updates happen through the *reference* `\@m` stored in the hashref, not through the local binding.

When the value genuinely mutates in place, `var` is correct. The same file accumulates a gradient vector with `push` inside a loop, so the binding has to be mutable:

```
var @grads  
for val $j (0:$n - 1) {  
  push @grads, 2.0 * ($params[$j] - $targets->[$j])  
}
```

This pairing — `val` for assembled-once, `var` for grown-in-place — is the everyday discipline. It costs nothing at runtime (both desugar to `my`) and documents intent at the binding site.

The `+{ }` hashref constructor

Perl's `{ ... }` is ambiguous between a code block and an anonymous hash, which is why Perl programmers learn the `+{ ... }` and `{; ... }` disambiguation tricks. `stryke` promotes `+{ }` to the canonical hashref constructor — it always builds a hash reference, never a block, so there's no guessing. You saw it return an optimizer above; it's equally the shape for ad-hoc records. The Adam test cases are a list of `+{ }` records, one per line per the hash-declaration style rule:

```
var @cases = (
  +{ name => "1-D quadratic, a=3, x0=0",      target => 3.0,  start => 0.0,  steps =>
    500, lr => 0.1 },
  +{ name => "1-D quadratic, a=-7, x0=5",      target => -7.0, start => 5.0,  steps =>
    500, lr => 0.1 },
  +{ name => "1-D quadratic, a=π, x0=10",      target => 3.14, start => 10.0, steps =>
    1000, lr => 0.05 },
  +{ name => "1-D quadratic, a=0, x0=100",     target => 0.0,  start => 100.0, steps =>
    5000, lr => 0.5 },
)

for val $c (@cases) {
  val $final = Adam::run_quadratic($c->{target}, $c->{start}, $c->{steps}, $c->{lr})
  printf " %-32s %8.4f %10.4f\n", $c->{name}, $c->{target}, $final
}
```

Each `+{ }` is a record; `$c->{target}` accesses a field through the arrow. This is the lightweight shape for “a bag of named values” when you don’t want the ceremony of a struct — closer to a Python dict or a JSON object than to a class.

The parallel link checker uses `+{ }` to build result records inside a parallel map, including the topic variable `_` as a value:

```
var @results = pmap {
  val $res = eval { http_request(_, +{ timeout => 5 }) }

  if ($?) {
    val $err = $_ =~ s/ at .*/r
    +{ url => _, ok => 0, status => 0, err => $err }
  } else {
    +{ url => _, ok => ($res->{status} < 400), status => $res->{status} }
  }
} @urls
```

Note `+{ timeout => 5 }` as an inline options hashref passed straight into a call, and `+{ url => _, ... }` as the return value of each branch. The constructor works anywhere a hash reference is wanted — argument position, return position, list element.

References

References are Perl's, spelled the same way. `\@array` takes a reference to an array, `\%hash` to a hash, and you dereference with the arrow or sigil-deref. The Adam stepper mutates the caller's parameter array through a reference passed in:

```
fn Adam::step($opt, $params, $grads) {
  $opt->{t}++
  val $n = $opt->{n}
  for val $i (0:$n - 1) {
    $opt->{m}->[$i] = $opt->{beta1} * $opt->{m}->[$i] + (1.0 - $opt->{beta1}) *
      $grads->[$i]
    ...
    $params->[$i] = $params->[$i] - $opt->{lr} * $m_hat / (sqrt($v_hat) + $opt->{eps})
  }
}
```

The caller passes `\@params` and `\@grads`; `step` writes through `$params->[$i]`, so the update lands in the caller's array. `$opt->{m}->[$i]` reaches through the hashref into the stored array-ref — references nest exactly as in Perl. When you need to read back a whole array from a ref, `@$x0_arr` and `len(@$targets)` deref-and-count:

```
fn Adam::run_md_quadratic($targets, $x0_arr, $steps, $lr) {
  val $n = len(@$targets)
  val $opt = Adam::new($n, $lr, 0.9, 0.999, 1e-8)
  val @params = @$x0_arr
  ...
  \@params
}
```

`len(@$targets)` is the canonical count idiom — `len`, never `scalar` — applied to a dereferenced array. The function returns `\@params`, a reference to its result, which is how stryke hands a vector back without flattening it into the caller's list.

Structs with methods and implicit `$self`

When a record has behavior, not just fields, promote it to a `struct`. A struct declares typed fields and methods together, and methods receive the instance as an implicit `$self` — you never write `$self` in the parameter list. The ASCII ray tracer's 3D vector is the canonical example: six methods, every one operating on `$self`, all packed into the type:

```
struct Vec3 {
  x => Float, y => Float, z => Float
  fn add($other) = Vec3(x => $self->x + $other->x, y => $self->y + $other->y, z =>
    $self->z + $other->z)
  fn sub($other) = Vec3(x => $self->x - $other->x, y => $self->y - $other->y, z =>
    $self->z - $other->z)
  fn mul($s)      = Vec3(x => $self->x * $s, y => $self->y * $s, z => $self->z * $s)
  fn dot($other) = $self->x * $other->x + $self->y * $other->y + $self->z * $other->z
  fn mag()       = sqrt($self->dot($self))
  fn unit()      = $self->mul(1.0 / $self->mag)
}
```

Several things to read off this. The fields are typed — `x => Float` — so a `Vec3` is a fixed shape, not an open hash. This is the line that separates a struct from a `+{ }` record: a hashref will silently accept a typo'd key or a missing field and hand you `undef` three call-frames later, while a struct's field set is closed at declaration. You trade the hashref's anything-goes flexibility for a shape the type system knows the full extent of — the same trade Rust makes between a `HashMap` and a `struct`, and made for the same reason: when the keys are known and fixed, naming them in the type catches the mistakes a map defers to runtime. The methods are *expression-bodied*: `fn add($other) = ...`, no braces, no `return`, the expression is the result. `$other` is the only declared parameter; `$self` appears in the body without ever being declared, because it's implicit — the same implicit-receiver convention as Python's `self` or Rust's `&self`, except stryke drops it from the signature entirely rather than making you write it as the first parameter. And methods call each other freely —

`mag` calls `dot`, `unit` calls `mul` and `mag` — building a small algebra entirely inside the type. Construction is `Vec3(x => ..., y => ..., z => ...)`, a named-field call, which is why the fields are addressable by name at the construction site and not just positionally.

At the call site, methods chain with the arrow and read left-to-right like a pipeline:

```
val $LIGHT_DIR = Vec3(x => 1.0, y => -1.0, z => -1.0)->unit()

val $normal = $hit_pt->sub($SPHERE_CENTER)->unit()
```

`$hit_pt->sub($SPHERE_CENTER)->unit()` subtracts then normalizes, each method returning a fresh `Vec3` for the next to consume. This is the “shape with behavior” case: when your data has operations that belong to it, a struct keeps them together and the implicit `$self` keeps the method bodies terse.

stryke also has `struct Name { x => Float; fn m($o) = ... }` with semicolon-separated members on one line, and a `class` form for richer types — but the field-plus-method struct above covers the common case and is what you reach for first.

Enums and match

When a value is one of a fixed set of alternatives, model it as an `enum` and dispatch with `match`. The variants are referenced everywhere as `EnumName::Variant` — the bare variant name appears only on the declaration line:

```
enum TrafficLight { Red, Yellow, Green }

val $sig = TrafficLight::Red
```

Dispatch is a `match` expression whose arms map each variant to a value. Because `match` is an expression, the whole thing is the function’s return — no `return`, no intermediate variable:

```
fn TrafficLight::action($c) {
  match ($c) {
    TrafficLight::Red    => "stop",
    TrafficLight::Yellow => "caution",
    TrafficLight::Green  => "go",
  }
}

for val $v (TrafficLight::Red, TrafficLight::Yellow, TrafficLight::Green) {
  p " ", $v, " → ", TrafficLight::action($v)
}
```

The arms cover every variant, which is what makes the dispatch exhaustive: add a fourth light and the match has an unhandled case staring at you. This is the structural advantage an `enum` plus `match` holds over the obvious alternative — a string constant and an `if/elsif` chain. A string `"red"` is one of infinitely many strings; nothing tells you when you’ve handled all the lights, and a typo’d `"redd"` is a value the compiler is perfectly happy with and a branch that silently never fires. An `enum` closes the set to exactly three variants, so the question “have I handled every case?” has a definite answer, and `match` is where that answer is checked. This is the same closed-set-plus-pattern-match design as Rust’s `enum` and `match`, ML’s variants and `case`, or Swift’s `enum` and `switch` — the lineage that made exhaustiveness a language-enforced property rather than a thing you hope you remembered. Each arm’s right-hand side is a tail expression, so `"stop"` is the value the function yields when `$c` is `Red`.

Variants can carry data. A variant is declared with `=> Type` to attach a payload — here a shape that holds its dimension:

```

enum Shape {
  Circle    => Float,      # radius
  Square    => Float,      # side
  Rect,      # bare variant, no data
}

fn Shape::area($s) {
  match ($s) {
    Shape::Circle => 3.14 * 1.0 * 1.0,
    Shape::Square => 5.0 * 5.0,
    Shape::Rect   => 0.0,
  }
}

```

`Circle => Float` and `Square => Float` carry a float payload; `Rect` is a bare variant with none. Mixing data-carrying and bare variants in one enum is fine — `match` dispatches on the variant regardless. This is what lifts the enum from a plain set of named constants (a C `enum`, a Python `Enum`) to a *tagged union*: each variant can carry its own differently-typed payload, so a single `Shape` value is either “a circle with this radius” or “a square with this side” or “a rectangle,” and the tag tells you which payload is present. The combination is the thing — the tag makes the dispatch exhaustive and the payload makes each variant carry the data that case needs, so `match` both selects the branch and, in the full pattern-matching form, binds the payload in one step. The enum is the shape for “this value is one of N structured alternatives,” and `match` is how you take it apart without a chain of `if/elsif` that no compiler can confirm is complete.

Namespacing, and never shadowing a builtin

You may have noticed every function above is namespaced: `Adam::new`, `Vec3::add` (via the struct), `TrafficLight::action`, `Bio::translate_codon`. This is not decoration. The hard rule is **never name a function after a builtin** — `fn close`, `fn log`, `fn cos`, `fn open` all abort at definition time because they collide with the runtime. The fix is always a namespace:

```

fn Rosetta::Foo::helper = ... # DO    - namespaced
fn helper = ...             # DON'T when `helper` collides with a builtin

```

Namespacing buys two things. It keeps your function out of the builtin’s way, and it groups related functions under a module prefix — `Adam::new`, `Adam::step`, `Adam::run_quadratic` read as one cohesive unit. The convention from the naming guide is `Module::Sub::name` for namespaced functions, `CamelCase` for the struct/enum/class names themselves, and `snake_case` for plain locals. When in doubt about whether a name is taken, namespace it; the cost is a few characters and the alternative is a runtime abort.

The token-bucket example uses the older `package TB; /package main;` form to scope its functions, which is the same idea expressed Perl-style — `TB::new`, `TB::try_consume` are reachable from the `main` package by their qualified names. New code prefers the `fn Module::name` spelling directly, but both land functions in a namespace and out of the builtins’ path.

Sets: dedup-by-value as a native container

Arrays and hashes are the two aggregates Perl gives you, and for decades the set — a collection that holds each value at most once, with fast membership and the union/intersection/difference algebra — was something you *simulated* by using a hash’s keys and ignoring its values. That idiom works, but it leaks: you write `%seen` and `$seen{$x}++` and `keys %seen` and carry the mental overhead of remembering that the values are meaningless. `stryke` promotes the set to a first-class container with its own constructor and its own operators, so the intent is in the type rather than in a convention layered on top of a hash.

The constructor is the `set` builtin, or the object form `Set->new`, and either accepts a list — including a list piped in, which is the idiomatic feed:

```

val $s = set(1, 2, 2, 3)           # 3 members – the duplicate 2 collapses
val $t = (1, 1, 2, 4) |> set       # piped-in list; 3 members
val $u = $s | $t                  # union      → {1,2,3,4}
val $i = $s & $t                  # intersection → {2}
$s->has(2)                        # 1 (aliases: ->contains, ->member)
$s->size                          # 3 (aliases: ->len, ->count)
val @v = $s->values               # array, insertion order preserved

```

Two design decisions make the set feel native rather than bolted on. The first is that deduplication is by *value*, not by identity or by stringification alone — the set computes an internal canonical key for each member, so two structurally-equal values collapse to one entry, while `->values` still hands them back in insertion order rather than in some hash-scrambled sequence. The insertion-order guarantee matters more than it looks: it means a set can stand in for “the distinct elements, first-seen order” — the `uniq` of a stream — without a separate ordered-set type, and a round-trip through `set` then `->values` is a stable dedup you can drop into a pipeline.

The second decision is operator overloading with a fallback. `|` and `&` are union and intersection *when either operand is a set*, and they revert to their ordinary bitwise-integer meaning when neither is — the runtime dispatches on the operand type, so the same two symbols carry set algebra and bit algebra without ambiguity, because a value is never both a set and an integer at once. This is the same functional-operator philosophy the sketch algebra of Chapter 16 rests on: the operators read as the mathematics they implement, and they do not mutate their operands, so `$s | $t` is a fresh set and both inputs survive. The one place mutation is explicit is under parallelism — Chapter 5’s `varsync` supports the compound `|=` and `&=` forms so a shared set can accumulate a union or narrow to an intersection across worker threads without a hand-rolled lock, which is the set-shaped counterpart to `varsync $count = 0` for a shared counter.

Chapter 9 – Control Flow and Functions

stryke's control flow is Perl's, sharpened. You get `if/unless`, their postfix forms, `for` and `while`, `last/next`, the ternary, `eval/$@` for errors, and `match` as an expression — and on top of those, a function-definition family with a strong default toward the terse, implicit-return form. The skill here is choosing the lightest construct that says what you mean: a postfix `if` over a three-line block, an expression-bodied `fn` over a braced one, an early-return guard over nested conditionals.

`if`, `unless`, and their postfix forms

The block forms are conventional. What you'll reach for constantly is the *postfix* form, where the condition trails the statement and reads as a single clause. The Game of Life inner loop skips the center cell with a postfix `next`:

```
for val $dr (-1:1) {
  for val $dc (-1:1) {
    next if $dr == 0 && $dc == 0
    ...
    $count++ if $nr >= 0 && $nr < 5 && $nc >= 0 && $nc < 5 && $grid->[$nr][$nc]
  }
}
```

`next if ...` and `$count++ if ...` are each one line that would be three with a braced block. The postfix form is correct whenever the body is a single statement — and most guard conditions are. `unless` is the negated twin, and the Adam optimizer's invariant checks lean on it heavily for assertions:

```
die "1-D quadratic err #{$err} > 1e-3 on \"#{ $c->{name}}\"" unless $err < 1e-3
```

`die ... unless $cond` reads as “this had better be true, or fail loudly.” It's the assertion idiom: the happy path is the condition, the message is the failure. Writing it as `if (!($err < 1e-3)) { die ... }` would be longer and harder to read — `unless` exists precisely so you don't negate by hand.

`for` and `while`

`for val $x (LIST)` is the iteration workhorse, and the list is usually a colon range. The ray tracer's render loop is two nested ranges over the screen dimensions:

```
for val $y (0:$H - 1) {
  var @row
  for val $x (0:$W - 1) {
    ...
    push @row, $RAMP[$idx]
  }
  p join(" ", @row)
}
```

`0:$H - 1` is the inclusive range of row indices. Note `for val` — the iterator binding is immutable by default, which is right because you almost never reassign the loop variable. The rule is to use `for val` unless the body reassigns the iterator, which is almost never. When you *do* mutate the loop variable inside the body, `for var` is available, but it's the exception you should be able to justify.

`while` is the conditional loop and appears where the bound isn't a simple count — though much of what would be a `while` in C is a `for` over a range in stryke. The token-bucket refill is conditional logic guarded by a plain `if`, advancing a clock the caller controls rather than looping internally:

```
fn _refill($bk, $now) {
  val $elapsed = $now - $bk->{last}
  if ($elapsed > 0) {
    val $add = $elapsed * $bk->{rate}
    val $new = $bk->{tokens} + $add
    $bk->{tokens} = $new > $bk->{cap} ? $bk->{cap} : $new
    $bk->{last} = $now
  }
}
```

That `$bk->{tokens} = $new > $bk->{cap} ? $bk->{cap} : $new` is the ternary doing a clamp inline — “cap it if it overflowed, else take the new value.” The ternary is the right tool when both arms are values feeding one assignment; reaching for an `if/else` block here would split one expression into four lines.

last and next

`last` breaks out of the enclosing loop, `next` skips to the next iteration. You saw `next` in the Game of Life. `last` shines as an early termination on a sentinel — the RNA translator stops the instant it hits a STOP codon:

```
for val $i (0:len($s)-3:3) {
  val $codon = $s[$i:$i+2]
  val $aa = Bio::translate_codon($codon)
  last if $aa eq "STOP"
  push @amino_acids, $aa
}
```

`last if $aa eq "STOP"` is the loop's exit condition expressed where it happens, mid-body, rather than hoisted into the loop header. This is the natural shape for “process until a marker”: the range drives the stride, `last` handles the stop. The translator's own tests confirm a STOP codon mid-sequence truncates the output — a long sequence with an internal STOP yields exactly seven amino acids, because `last` fired.

The ternary

The ternary `cond ? a : b` is a value-producing expression, and stryke uses it freely wherever both branches are values. The Game of Life cell-update rule is a *nested* ternary — the alive/dead transition and the survival/birth counts all in one expression fed straight to `push`:

```
push @row, $alive ? ($n == 2 || $n == 3 ? 1 : 0)
                : ($n == 3 ? 1 : 0)
```

Read it as: if the cell is alive, it survives with 2 or 3 neighbors; if dead, it's born with exactly 3. Each branch is itself a ternary. This is dense, but it's *correct* density — the whole Conway rule is one expression, and splitting it into nested `if/else` blocks would scatter a single decision across a dozen lines. The ternary earns its keep when the alternatives are values, not statements.

eval / \$@ for error handling

stryke catches failures the Perl way: wrap the risky code in `eval { }`, and check `$@` afterward — non-empty means the block died. The parallel link checker uses this to survive network failures without aborting the whole run:

```
val $res = eval { http_request(_, +{ timeout => 5 }) }

if ($@) {
  val $err = $@ =~ s/ at .*//r
  +{ url => _, ok => 0, status => 0, err => $err }
} else {
  +{ url => _, ok => ($res->{status} < 400), status => $res->{status} }
}
```

`eval { http_request(...) }` returns the result on success or sets `$@` on failure. The `if ($@)` branch builds an error record; the `else` branch builds a success record. Crucially, this is per-URL inside a parallel map — one site going down produces a failure record, not a crashed program. `$@ =~ s/ at .*//r` trims Perl's " at FILE line N" suffix off the error message non-destructively (more on `s///r` in the next chapter). The whole `eval` is the boundary-validation pattern: trust internal code, but wrap genuinely fallible operations — I/O, network, parsing — in `eval` and branch on `$@`.

match as an expression

`match` was introduced in Chapter 8 as the enum dispatcher, but the broader point belongs here: it is an *expression*, so it produces a value and slots into a function tail without `return`. That's why `Shape::area` and `TrafficLight::action` work as one-expression bodies — the `match` is the value:

```
fn TrafficLight::action($c) {
  match ($c) {
    TrafficLight::Red    => "stop",
    TrafficLight::Yellow => "caution",
    TrafficLight::Green  => "go",
  }
}
```

Where a chain of `if/elsif` would lean on early `returns` or a mutable accumulator, `match` collapses the whole decision into one expression that is itself the answer. This is the deeper reason expression-orientation matters: a statement-oriented `switch` has to *do* something in each branch — assign a variable, call `return` — because a statement doesn't produce a value. An expression-oriented `match` is a value, so it drops straight into a function tail, a `push`, a pipeline stage, or a variable binding with no plumbing in between. The same property is why the ternary, `if/else` arms, and `match` arms are all interchangeable in tail position throughout this chapter — they are all expressions, and stryke treats "the value of this construct" as a first-class thing rather than a side effect you have to capture. Prefer `match` whenever you're dispatching on a value's shape or a fixed set of cases.

The fn family in full

stryke has one keyword for functions — `fn`, never `sub` — and several body forms. Choosing among them is mostly about how much the body needs.

Expression-bodied, the default. Anything that fits on one line uses `fn name = expr`: no braces, no `return`, implicit topic. The `Vec3` methods are all this form, and so are small helpers like the codon lookup:

```
fn Bio::translate_codon($codon) = $CODONS{uc $codon} // "Unknown"
```

One expression, hash-lookup with a `//` default, no braces. This is the form to reach for first; `fn double = _ * 2` outranks every braced alternative, and a braced body that contains nothing but a single expression is a sign you should drop the braces and the `return` with them.

Block-bodied, for multi-statement work. When the body needs locals, loops, or several statements, use braces. The body's *last expression* is still the return value — you don't write `return` for it:

```
fn Bio::translate_sequence($seq) {
  var @amino_acids
  val $s = uc $seq
  for val $i (0:len($s)-3:3) {
    val $codon = $s[$i:$i+2]
    val $aa = Bio::translate_codon($codon)
    last if $aa eq "STOP"
    push @amino_acids, $aa
  }
  \@amino_acids
}
```

The function ends on `\@amino_acids` as a bare tail expression — that *is* the return. No `return \@amino_acids`. The implicit-return rule is firm: the last expression in any body is the value, and writing `return` on it is noise that hides the actual result.

Named parameters, only when they earn it. The default is `_` for slot 0 and `_N` for the later positional args; you name parameters only when the function is a real top-level `fn` whose arity exceeds two *and* the names carry meaning. The examples in this book mostly use named params because their arities are genuinely above two and the names document the role — `fn Adam::step($opt, $params, $grads)` would be unreadable as `_0, _1, _2`. But a one- or two-argument helper should take the topic, not invent names. `fn double = _ * 2`, not `fn double($x) = $x * 2`.

Guards: the only legitimate return. `return` is correct exactly once — as an early-exit guard near the top of a function, before the main body runs. The token bucket's `time_until` returns zero immediately when there's no wait:

```
fn time_until($bk, $now, $n) {
  TB::_refill($bk, $now)
  return 0 if $bk->{tokens} >= $n
  val $deficit = $n - $bk->{tokens}
  $deficit / $bk->{rate}
}
```

`return 0 if $bk->{tokens} >= $n` is the guard: if there are enough tokens, the answer is 0 and we bail. The *tail* — `$deficit / $bk->{rate}` — returns implicitly, no keyword. This is the discipline in one function: `return` for the early guard, nothing for the real answer. `try_consume` in the same file follows the identical shape, with a `return 1` guard on success and a bare `0` tail on failure.

So the decision tree is short. Fits on a line? `fn name = expr`. Needs statements? Braces, with the last expression as the implicit return. More than two meaningful arguments? Name them. Need to bail early? `return X if cond` — and only there.

Chapter 10 — Text and the AWK Inheritance

awk’s enduring role is the one-off text job: slurp a file, split it into fields, transform line by line, print the result. stryke is built to absorb that role outright — text processing, field extraction, line-by-line transformation are all first-class, so the standalone awk becomes redundant in a stryke-centered toolkit. What awk does in a terse domain-specific language, stryke does in a few builtins and pipelines, and then keeps going into structured data, parallelism, and a regex engine far past awk’s reach.

The reason awk survived fifty years is worth naming, because it explains exactly what stryke has to match before it can replace it. awk’s model is implicit and total: it wraps your program in an invisible loop over input lines, splits each line into `$1`, `$2`, `$3` on whitespace for free, and runs your pattern-action pairs against every line — so `awk '/error/ {print $2}'` is a complete program because the loop, the split, and the iteration are all assumed. That implicit machinery is awk’s whole appeal and also its ceiling: the moment you want a hash that outlives the line, a second pass, structured output, or anything concurrent, awk’s model fights you and you reach for a real language. stryke’s bet is that you can keep awk’s terseness — the file-to-fields-to-output pipeline as a single readable line — while removing the ceiling, because the pipeline stages are ordinary builtins composing into ordinary stryke, so a text-extraction one-liner continues straight into a hash aggregation, a struct, or a parallel map without a language boundary to cross. awk owns the first ten characters; stryke owns those *and* everything that comes after them. This chapter walks the text primitives — slurp, lines, split, the filter-map-print pipeline — and the three-tier regex engine underneath.

Slurp with `c "..."`

The shortest path to a file’s contents is the `c "..."` slurp prefix — `c` immediately before a quoted path reads the whole file as a string. The advanced file-ops example writes a file and slurps it back:

```
spurt $tmp, "line 1: hello\nline 2: world\nline 3: stryke\n"
val $content = c"$tmp"
p "Slurped content (length: #{len $content})"
```

`spurt $path, $data` is the write counterpart — one call, no filehandle, no `open/print/close` dance. `c"$tmp"` reads it all back. Note the path interpolates: `c"$tmp"` slurps the file named by `$tmp`. This is the “read the whole thing” mode, for when you want the file as one string to regex over or hand to a parser.

Line iteration with `lines`

When you want the file a line at a time — the awk default — `lines` gives you a lazy line iterator that you pipe through filters. The file-ops example greps a slurped string for matching lines and prints them:

```
c"$tmp" |> lines |> greps { /stryke/ } |> e p
```

Read the pipeline left to right: slurp the file, split it into lines, keep the lines matching `/stryke/`, print each. That single line is a complete awk-style filter — `awk '/stryke/'` — expressed as a stryke pipeline, and the correspondence is close enough to be instructive. awk’s `/stryke/` is a bare pattern with an implicit “print the line” action; stryke’s `greps { /stryke/ }` is an explicit filter stage whose body is the same bare match against the topic. What’s implicit in awk is explicit-but-terse in stryke, and the trade is that the explicit

form composes: `greps { /stryke/ }` is a stage you can drop more stages before or after, where `awk`'s pattern-action is a fixed top-level form. `greps { /stryke/ }` is the streaming filter (the `s`-suffixed forms are lazy, no intermediate array, so the file is processed line-by-line rather than slurped into one giant list), the bare `/stryke/` is a match against the topic `_`, and `e p` calls `p` on each surviving line. The whole filter-print loop collapses to one readable line — and unlike the `awk` version, the next thing in the pipeline can be a `maps`, a `pmaps`, or a reduction into a hash, with no exit from the language.

Fields and `split`

`awk`'s killer feature is automatic field splitting — `$1`, `$2`, `$3` are the whitespace-separated columns. `stryke` covers this with `split` for explicit delimiters and `fields` for the whitespace-default case, both returning a list you index or pipe. `split` is the general form: `split /pattern/, $string` breaks on a regex or literal. The RNA translator does its chunking with a range-slice rather than `split` because the chunk width is fixed, but for delimited text `split` is the tool — and `lines` itself is conceptually `split` on newlines.

The pattern across all of these is the same as `awk`'s: get the file, break it into units (lines, fields, chunks), transform, emit. The difference is that `stryke`'s units feed into the full pipeline and structured-data machinery, so a field-extraction job can continue straight into a hash aggregation or a parallel map without leaving the language.

The filter-map-print pipeline

The canonical text shape is filter, transform, print — and `stryke` spells it as a pipeline whose stages are `greps`, `maps`, and a terminal `ep` or `e p`. The shape is exactly the same over a list of lines:

```
fn shout = uc(_) . "!"
@lines |> maps { trim(_) |> shout } |> greps { len(_) > 0 } |> ep
```

Trim and shout each line, drop the empties, print the rest. That is the entire structure of a line-oriented text program: `maps` transforms, `greps` filters, `ep` prints. The `s`-suffixed `maps/greps` are the streaming variants — lazy, no intermediate vector — which is what you want when chaining. `ep` is each-print shorthand, equivalent to `e p`, and it goes to stdout. Swap the closures and you have any `awk` one-liner.

The three-tier regex engine

Regular expressions are first-class, with a three-tier engine underneath: the Rust `regex` crate for the common case, `fancy-regex` when you need backreferences, and `pcre2` for the full Perl-compatible feature set.

The three tiers exist because there is no single regex engine that is both fast and fully featured — the two properties are in genuine tension, and most languages pick one end and live with the consequences. Rust's `regex` crate is built on finite automata: it guarantees linear-time matching, so it cannot be made to backtrack catastrophically on a pathological pattern, but the price of that guarantee is that it *cannot* support backreferences or lookahead at all, because those features are mathematically incompatible with the automaton model. PCRE2, at the other extreme, supports every Perl construct — backreferences, lookahead, recursion, conditionals — but it is a backtracking engine, which means a poorly-formed pattern against adversarial input can take exponential time. Most languages force the choice on you: pick the fast linear engine and lose half of Perl's syntax, or pick the full backtracking engine and accept the worst-case blowup on everything, even the simple patterns that didn't need it.

`stryke` routes instead of choosing. You never select a tier; the engine picks the cheapest one that can express your pattern. A plain pattern — anchors, classes, quantifiers, alternation — runs on Rust's `regex` and gets the linear-time guarantee for free. The moment your pattern uses a backreference, it transparently escalates to `fancy-regex`, which adds backreferences and lookahead on top of the Rust engine. Reach for a PCRE-only

construct and it lands on `pcr2`, the full Perl surface. The routing is `regex` → `fancy-regex` → `pcr2`, lowest tier that suffices wins, and the practical consequence is that the common case — the overwhelming majority of patterns, which use no backreferences and no PCRE verbs — pays nothing for the power it isn't using, while the rare pattern that needs the heavy engine gets it without you having to ask. You write Perl-flavored regex and the runtime hands each pattern to the fastest engine that supports what you wrote, with no annotation.

The full Perl surface is present: `=~` to match, `s///` to substitute, `tr///` to transliterate, the `g/i/m/s/x/e/r` flags, numbered captures `$1...$n`, named groups via `%+/{name}`, `\Q... \E`, and `qr//`. There is one documented gap worth knowing: named-backref interpolation in an `s///` replacement (`${name}`) is broken, so use numbered backrefs `$1`, `$2` in replacements even when the pattern uses named groups. Everything else behaves as in Perl.

`=~ s///r` — non-destructive substitution

The `/r` flag makes a substitution *return* the modified string instead of mutating the original and returning a count. This is the pipeline-friendly form, because it produces a value you can pass onward. The link checker uses it twice. First to trim the " at FILE line N" suffix off an error message:

```
val $err = $@ =~ s/ at .*//r
```

`$@ =~ s/ at .*//r` reads `$@`, strips everything from " at " onward, and yields the cleaned string into `$err` — `$@` itself is untouched. Without `/r` you'd have to copy `$@` first, then mutate the copy. The `/r` flag is also auto-injected when `s///` or `tr///` appears as a pipe-forward right-hand side, so `$str |> s/world/perl/` works as a non-destructive transform stage without writing the flag — the pipeline context demands a returned value, and the runtime supplies the `/r` semantics that produce one.

The second use extracts a capture by rewriting the whole string down to a backreference:

```
val $url = $links[$i] =~ s/.*href="([^\"]+)" .*/$1/r
```

This replaces the entire matched line with just its first capture group `$1` — a one-line "pull the URL out of the `href`" — and returns the result non-destructively. It's the substitution-as-extraction idiom: match everything, replace with the part you want.

`match_all` — every match at once

When you want *all* matches of a pattern rather than the first, `match_all` returns the lot. The link checker harvests every `href` from a fetched HTML page:

```
val @links = match_all('(?i)href="([^\"]+)"', $html)
p "Found #{len @links} links."
```

`match_all('(?i)href="([^\"]+)"', $html)` runs the pattern globally over `$html` and returns the list of matches into `@links` — the pattern is the *first* argument, the subject the second. `(?i)` is the inline case-insensitive flag. This is the structured-extraction counterpart to a global match: where `awk` would loop and accumulate, `match_all` hands you the whole list in one call, ready to count with `len` or pipe onward.

Putting it together

A complete stryke text program reads top to bottom as data flowing through stages. Take the link checker's fetch-and-extract block, which is a full small program:

```

val $html = eval { fetch("https://wikipedia.org") }
if ($?) {
  p "  Failed to fetch wikipedia: $@"
} else {
  val @links = match_all('(?!i)href="([^\"]+)"', $html)
  p "Found #{len @links} links."
  p "Sample of first 5 links:"
  for val $i (0:4) {
    if ($links[$i]) {
      val $url = $links[$i] =~ s/.*href="([^\"]+)"$/\1/r
      p "  - $url"
    }
  }
}

```

Fetch the page inside an `eval` so a network failure is a branch, not a crash. On success, `match_all` pulls every link, `len` counts them, the `0:4` range slices off the first five, and `s///r` cleans each one for display. Every line is text in, text out. That is `awk`'s job — fetch, extract, transform, print — done in a language that doesn't stop at text: the same `@links` could feed a parallel map, a hash histogram, or a JSON serializer without ever leaving stryke. The one-liner role is absorbed; the ceiling is gone.

Chapter 11 — Batteries Included: The Shape of Ten Thousand Builtins

There is a moment, somewhere in the first hour with most languages, when you reach for something obvious — a Levenshtein distance, a CIDR aggregation, a Poisson PMF, a BLAKE3 digest — and discover it isn't there. The language ships a syntax and a promise; the substance lives in a registry somewhere, behind an install step, a version pin, a transitive dependency tree, and a small act of faith that the package you found at 2am is the one everybody actually uses. The standard library is a lobby. The real building is somewhere else, and you have to go get the keys.

stryke makes the opposite bet. The battery *is* the language. When you type `s`, you are holding 10,453 standard-library primaries in `%b`, and 11,229 total callable spellings in `%all` once aliases and keywords are folded in. Nothing to install, nothing to import, nothing to pin. `blake3` is as present as `+`. `levenshtein` is as present as `print`. The whole point of this chapter — and the three that follow it — is to make that number concrete: not “ten thousand functions” as a figure of speech, but ten thousand *named, categorized, documented, dispatchable* operations that are in the binary on your `$PATH` right now.

The size is not the only thing worth noticing — the *flatness* is. There is no `import math`, no `use List::Util`, no `from itertools import combinations`. `combinations`, `gamma`, `cidr_aggregate`, `blake3`, and `levenshtein` all sit at the same namespace depth, reached the same way, at the same cost. A standard library that you have to navigate by module is a filing cabinet; one where every operation is a bareword is a vocabulary. The difference shows up in how code reads. A stryke pipeline is a sentence built from a shared dictionary, and the four chapters in this part are a tour of that dictionary's largest neighborhoods — numbers, collections, and text-to-wire — chosen because they are the ones every program eventually walks through.

The number, and what it is measured against

It is worth being precise about what 10,453 counts, because round numbers invite suspicion. The figure in `%b` is *primaries* — distinct callable operations, each counted once, with no double-counting of aliases. `blake3` and its short form `b3` are one primary, not two. `base64_encode` and `b64e` are one primary. The alias inflation lives in a separate hash (`%a`) and is only added back when you ask for the total in `%all`, where it comes to 11,229 keys. So the headline count is the *clean unique-op count* — the number you would get if you asked “how many genuinely different things can this language do without me writing any of them.”

The natural comparison is Wolfram Language, the long-standing high-water mark for “everything is a builtin.” Wolfram's published primary count has historically sat in the several-thousand range — the high-band estimate for v14.3 lands around 7,300 symbols. stryke clears that by roughly 3,149 primaries; it holds the largest bareword library of any language, and the gap over Wolfram's high-band estimate is on the order of three thousand functions. That is not a marginal lead. It is a different category of completeness, and it is the reason the chapters ahead can pull real names — `hungarian_assignment`, `bellman_ford`, `ou_path`, `zeta_riemann`, `cidr_aggregate`, `stun_classify` — from domains that most languages would consider out of scope entirely.

The comparison sharpens against the two ecosystems most engineers actually reach for. Python's standard library is broad but deliberately conservative; the heavy lifting — `numpy`, `scipy`, `cryptography`, `networkx`, `python-Levenshtein` — lives in PyPI, which means every one of the functions this part names would be an install, a version pin, and a wheel that may or may not have a binary for your platform. Perl's CPAN is wider still and older, but the model is identical: `List::Util`, `Digest::SHA`, `Net::CIDR`, `Text::Levenshtein` are

all separate distributions with their own dependency graphs. In both worlds the language ships the syntax and the community ships the substance, and the substance is a supply chain. stryke's bet is that for the operations in these four chapters — arithmetic and number theory, list transformation, text, encoding, crypto, and the network — the supply chain *is* the binary, and there is nothing downstream of it to break.

One binary, ~4.3 KB per builtin

The second number that matters is physical. All of this ships in a *single 44 MB static binary* — `~/ .cargo/bin/s` — with every builtin compiled in. Divide the binary size by the primary count and you get roughly 4.3 KB of amortized footprint per builtin. That is dense. The README puts the comparison at roughly 200× denser than the Wolfram Engine on a per-builtin-per-byte basis, and the practical consequence is a cold start measured in single-digit milliseconds.

The density is not an accident of compression; it is an architectural choice that the introduction already touched on. Builtins are Rust functions reachable through a match-dispatch table in the bytecode VM, not heap-allocated interpreter objects constructed at startup. There is no library load, no dynamic linking of a numerics package, no spin-up of a runtime that reads thousands of definitions from disk. The whole corpus is *already resident* the instant the binary maps into memory. Sub-10ms cold start is the same fact viewed from the user's stopwatch: a language with ten thousand builtins that launches faster than most languages launch with none of theirs loaded.

This is the single-binary discipline that makes “batteries included” survivable. A library ecosystem grows by addition — every new capability is another package, another version, another thing that can break on a fresh machine in 2031. stryke grows by compilation. A new builtin is a new arm in the dispatcher; it ships in the next binary or it doesn't exist. There is exactly one artifact to copy, one thing to put on `$PATH`, and one cold-start cost that does not scale with how much of the standard library you happen to use that day.

The deployment story falls out of this directly. `scp` the binary, or drop it in a container **FROM scratch**, and the entire standard library went with it — no `pip install`, no `cpan`, no lockfile resolution against a registry that may have rotted or been yanked. A program that hashes a file with `blake3`, signs it with `ed25519_sign`, and ships the result over a `udp_open` socket has, on the target machine, exactly one dependency: the file you copied. The comparison table in the README puts the single-binary size at 44 MB against “system pkg” for Python, Perl, Ruby, PHP, and Node — those runtimes are not one artifact at all, they are a package plus whatever the program `imported`, resolved at deploy time against a network. stryke's 44 MB is the *whole* answer, resolved at compile time, frozen.

The introspection hashes

Ten thousand builtins are only useful if you can find them, and stryke's answer is to make the standard library *queryable from inside the standard library*. The language exposes its own catalog as a set of ordinary Perl-shaped hashes — nine of them in the core set: `%b`, `%k`, `%all`, `%pc`, `%e`, `%a`, `%d`, `%c`, `%p`. Each is a real hash with O(1) lookup, and together they let a program reason about the language the same way the language reasons about itself.

The forward maps answer “what is this name?”:

- `%b` — **primary builtin name** → **category**. The clean primaries-only map. `$b{pmap}` is “parallel”; `$b{to_json}` is “pipeline / string helpers”. Counting `scalar keys %b` gives you the 10,453.
- `%k` — **keyword** → **category**. stryke's language keywords (`if`, `class`, `match`, `varsync`, ...) tagged by role (“control”, “decl”, “concurrency”, “oo”, ...). Disjoint from `%b` — keywords are not builtins.
- `%all` — **every recognized name** → **category**. `%a` + `%b` + `%k` merged; aliases inherit their primary's tag. This is the 11,229-key superset, the one you count for total callable spellings.
- `%pc` — **Perl 5 core subset of %b**. The names that exist for `--compat`, so you can ask “is this name a Perl carryover or a stryke original?”

- **%e** — **stryke-only extension subset of %b**. The complement of **%pc**. **\$e{pmap}** is "parallel"; **pmap** has no Perl ancestor.
- **%a** — **alias** → **canonical primary**. **\$a{tj}** resolves to "to_json"; **\$a{b3}** to "blake3". The map that lets **%all** exist.
- **%d** — **name** → **one-line description**. Sparse — only the labels with hover docs appear — but it is the source the LSP and **docs/help/h** builtin draw on.

Two inverted maps answer the reverse question, “what names live under this idea?”:

- **%c** — **category** → **arrayref of names**. **\$c{parallel}** is [**pmap**, **pgrep**, **preduce**, ...]. This is how you enumerate a whole domain at once.
- **%p** — **primary** → **arrayref of its aliases**. **\$p{to_json}** is [**tj**]. The mirror of **%a**.

(The full reflection surface in the reference is broader still — it adds **%o** for symbol operators and **%v** for special variables, among others — but the nine above are the load-bearing introspection set for the standard library itself.)

The reason this matters is that it collapses discovery and use into one language. You do not leave stryke to read documentation about stryke. You ask:

```
# How many parallel builtins exist?
p scalar @{$c{parallel}}

# Is `levenshtein` a Perl carryover or a stryke original?
p exists $pc{levenshtein} ? "perl5 core" : "stryke extension"

# What category does the dispatcher file `cidr_aggregate` under?
p $b{cidr_aggregate}          # "network / ip / cidr"

# Resolve an alias to its real name, then list every spelling of it
p $a{b3}                      # "blake3"
$p{blake3} |> e p              # every alias of blake3
```

Because these are hashes and not a special API, every collection operation in Chapter 13 works on them. You can **grep** the keys of **%b** for a substring, **sort** a category’s members, **map** descriptions out of **%d**, **frequencies** over the values of **%b** to count how many primaries each category holds. The catalog is data, and stryke is good at data. This is what closes the discovery loop: the same **grep/map/sort** vocabulary you use on a CSV, you use on the language’s own table of contents, so “what can this language do in the network domain” is **\$c{"network / ip / cidr"}** and “how many of those have hover docs” is a **grep exists \$d{_}** away. No external manpage, no doc site, no **--help** that scrolls off the screen — the reference is queryable from the prompt that the reference describes.

The builtins-over-UDFs hierarchy

The size of the standard library is not just a convenience; it is a *design instruction* about how to write stryke. The guidance is explicit and it runs against the grain of how most engineers are trained: **prefer a builtin to a user-defined function, every time one exists**. If the operation you need is in **%b**, you do not reimplement it. You do not write a five-line **fn** for a Levenshtein distance, a CIDR membership test, a frequency count, a Soundex code — because **levenshtein**, **ip_in_cidr**, **frequencies**, and **soundex** are already there, already in Rust, already faster than anything you would write in the language, and already tested.

This hierarchy has teeth precisely because the library is so large. In a language with a thin standard library, “use the builtin” is advice that runs out quickly — there simply isn’t a builtin for most of what you need, so UDFs are the norm and the question rarely comes up. In stryke, the library is wide enough that the *default assumption flips*: when you are about to write a helper function, the first move is to check whether the helper already exists as a primary. The introspection hashes exist partly to make that check a one-liner — **exists \$b{thing_i_was_about_to_write}**.

The payoff is threefold. Builtins are dispatched through the VM's match table, so they avoid the call-frame overhead of a stryke `fn`. They are written in Rust against the NaN-boxed value representation, so they avoid boxing round-trips. And they carry their own documentation and category, so a program built out of builtins is self-describing in a way a program built out of bespoke helpers is not. A pipeline of named primaries reads as a sentence in a known vocabulary; a pipeline of local `fns` reads as a private dialect only its author speaks.

There is a correctness argument layered under the performance one. A hand-rolled Levenshtein is a job interview question precisely because it is easy to get *almost* right — off-by-one on the matrix, wrong transposition handling, silent failure on Unicode. The library ships `levenshtein` alongside `damerau_levenshtein` (which counts adjacent transpositions as a single edit) and `hamming_distance` for the equal-length case, each implemented once and exercised against the same test suite that gates every release. The same holds across domains: `gcd` is variadic and correct on negatives, `cidr_aggregate` handles the adjacency and containment edge cases a one-off would miss, `argon2` picks Argon2id with sane parameters rather than the insecure defaults a hurried developer reaches for. “Prefer the builtin” is, in practice, “prefer the version that has already survived the bugs you are about to reintroduce.”

How the registry is generated

None of these numbers are typed by hand, and that is the point that keeps them honest. The reflection tables are *generated at build time* by `build.rs`, which reads the actual source of the dispatcher and the parser and emits `reflection.rs` into the build's `OUT_DIR`. The generator's own header states the contract: the tables “never drift from the real parser / dispatcher / LSP docs,” because they are derived from those files rather than maintained alongside them.

Concretely, `build.rs` does four extractions every time the relevant sources change:

1. **BUILTIN_ARMS** — it scans the `try_builtin` match arms in `builtins.rs` to recover every per-arm set of names. This is where aliases come from: an arm like `&["blake3", "b3"]` becomes a primary plus its alias, feeding the `%a` map.
2. **CATEGORY_MAP** — it parses the `// — category —` section comments inside `is_perl5_core` (which uses a `matches!` block) and `stryke_extension_name` (which uses a `match name` block) in `parser.rs`. The category strings — `"parallel"`, `"network / ip / cidr"`, `"integer sequences"`, `"crypto / encoding"` — are literally the human-readable section headers in the source.
3. **List builtins** — because operations like `sum`, `min`, `max`, and `pairs` are dispatched through `list_builtins::dispatch_by_name` rather than the main `try_builtin` arms, `build.rs` scans `list_builtins.rs` separately and folds those names in, so they show up in `%b` and `%all` rather than vanishing.
4. **DESCRIPTIONS** — it harvests the first line of each `///` doc comment in `builtins.rs` (the primary source for over a thousand functions) and merges in the hand-written entries from `lsp.rs` for keywords, operators, and the reflection hashes themselves. The result is sorted and de-duplicated, and it is deliberately *sparse*: a name only appears in `%d` if someone wrote a doc line for it.

The discipline this enforces is the reason you can trust the count in this book. You cannot inflate `%b` by editing a number; the number is `scalar keys %b`, computed from arms the dispatcher actually has. You cannot claim a builtin exists by documenting it; documentation is harvested *from* the function, not the other way around. If you want a new primary in the registry, you add a dispatcher arm, give it a category comment, and write a doc line — and only then does it count. The registry is downstream of the code, never upstream of it, which is exactly the property that makes “10,453” a fact about the binary rather than a claim about the README.

The category strings are not cosmetic, either — they are the spine of the introspection layer. Because `%b` maps every primary to the exact section header it was filed under in the source, the per-domain counts the next three chapters quote are recoverable by counting category members, not by anyone tallying them by hand. The `network / ip / cidr` neighborhood holds 167 entries; `crypto / encoding` holds 178; `integer sequences` holds 79. Those are not aspirational figures rounded for a slide — each is `scalar @{$c{"..."}}`, the length of the arrayref that `build.rs` assembled by parsing the section comments in `parser.rs`. Change the source, rebuild, and the number moves on its own. The generator is, in effect, a

continuously-recomputed census of the language, and the book's job in the chapters ahead is only to walk the neighborhoods that census already counted.

Chapter 12 – Numbers, Math, and Sequences

A language that wants to retire the calculator, the spreadsheet, the statistics package, and the number-theory scratchpad has to be unembarrassing about math from the first character. `stryke`'s numeric tower starts where Perl's did — `+` `-` `*` `/` `**`, with `**` for exponentiation and `%` for modulo — and then keeps going, well past the point where most languages hand you off to a library. This chapter walks the territory: ordinary arithmetic and math functions, the number-theoretic and integer-sequence builtins, and the statistics-and-distributions layer. Every name below is a real primary in the registry; the descriptions are the ones the language ships in `%d`.

Arithmetic and the elementary functions

The unary basics are exactly where your fingers expect them. `abs` returns the absolute value of a number, stripping any negative sign. `sqrt` returns the square root of a non-negative number, and `cbrt` the cube root. These are the kind of operations so fundamental that their presence is unremarkable — but the point of the `stryke` library is that the *unremarkable* and the *exotic* live in the same namespace at the same call cost. `sqrt(2)` and `zeta_riemann(2)` are both single tokens away.

Rounding is a full family rather than a single function, because rounding is where naive code silently diverges from intent. `floor` rounds down, `ceil` rounds up, `round` to the nearest integer, and `trunc` toward zero — four distinct behaviors that collapse to one buggy `int()` in languages that ship only one of them. `sign` extracts the signum, `clamp` constrains a value to a range, and `lerp` does linear interpolation between two endpoints; `hypot` computes $\sqrt{x^2+y^2}$ without the intermediate overflow a hand-written version risks.

The transcendental layer is complete. The exponential and logarithmic functions are `exp`, `ln` (natural log), `log2`, `log10`, and `log_base` for an arbitrary base, with `pow` for general exponentiation alongside the `**` operator. Trigonometry ships the full circle — `sin`, `cos`, `tan` and their inverses `asin`, `acos`, `atan`, the two-argument `atan2` for correct quadrant resolution, and the hyperbolic trio `sinh`, `cosh`, `tanh`. None of these is behind an `import`; the same flatness that makes `blake3` a bareword makes `atan2` one.

The bridge from elementary arithmetic to analysis runs through the special functions, and `stryke` ships the ones a numerics package would. `gamma` (aliased `tgamma`) computes the gamma function $\Gamma(x)$ — the extension of factorial to the reals — and its companion `lgamma` (aliased `ln_gamma`) computes the natural log of the gamma function, $\ln(\Gamma(x))$, which is what you actually want whenever a factorial would overflow. The error function `erf` arises in probability, statistics, and the heat equation; its complement `erfc` computes $\text{erfc}(x) = 1 - \text{erf}(x)$ directly, which matters numerically in the tail where $1 - \text{erf}(x)$ loses precision if you subtract by hand. The `digamma` function — the logarithmic derivative of gamma — rounds out the set for anyone working in Bayesian inference, where it appears constantly. These — `gamma`, `lgamma`, `digamma`, `erf`, `erfc` — are the quiet workhorses underneath half of the distribution layer below.

```
p gamma(5)           # 24    -  $\Gamma(5) = 4!$ 
p lgamma(1000)       # log-gamma where 999! would overflow
p erf(1)             # ~0.8427
p erfc(1)            # ~0.1573  (computed directly, not 1 - erf)
```

Number theory and integer sequences

This is where `stryke` pulls decisively ahead of “math built into a scripting language.” The registry carries a dedicated *integer sequences* category — 79 primaries on its own — and a number-theory layer that most languages relegate to a third-party package or a recursive one-liner you write yourself.

The combinatorics are direct. `factorial` is the factorial; `fibonacci` is the Fibonacci sequence; `binomial N, K` is the binomial coefficient $C(N, K)$; `catalan N` returns the Nth Catalan number; `multinomial` generalizes the binomial to several groups. The companion counting functions live here too: `derangements N` counts permutations with no fixed point, `subfactorial` is its other name, `double_factorial` computes $n!!$ (every other term), and the two Stirling-number families — `stirling_first` and `stirling_second` — count permutation cycles and set partitions respectively, the constants behind most combinatorial identities.

The divisibility primitives are variadic where it makes sense: `gcd` is a variadic GCD and `lcm` a variadic LCM, so `gcd(48, 60, 36)` is a single call rather than a fold. Factorization is first-class — `prime_factors` returns the multiset of prime factors, `divisors` lists all divisors, and the primality side carries `is_prime` for the test plus `next_prime`, `prev_prime`, and `nth_prime` for navigating the primes directly. `euler_totient N` computes Euler’s totient $\phi(N)$ — the count of integers up to N coprime with it, the backbone of a great deal of modular arithmetic — and the registry goes further into analytic number theory than any scripting language has a right to: `mobius` is the Möbius function, `jacobi_symbol` and `legendre_symbol` are the quadratic-residue symbols, `is_perfect` tests for perfect numbers, and `is_square` for perfect squares.

The named integer sequences are a genuine catalog. `lucas` is the Lucas numbers, `tribonacci` the three-term generalization of Fibonacci, `pell` the Pell numbers, and `perrin` and `padovan` the two classic third-order recurrences. The Collatz machinery is here as `collatz` (the trajectory) and `collatz_length` (the stopping time), `harmonic_number` gives the partial sums of the harmonic series, and `primorial` is the product of the first primes — the prime analogue of the factorial.

Two of the sequence builtins are worth singling out because they show the registry’s reach into genuinely specialized territory. `pascal_row N` returns the Nth row of Pascal’s triangle as a list — the binomial coefficients for a fixed N in one call. And `stern_brocot N` returns node N of the Stern-Brocot tree (1-indexed) as a pair `[a, b]` in Calkin-Wilf order: a builtin that enumerates the positive rationals without repetition, the kind of thing that is a small research project in most languages and a function name here.

The most pointed example of the library’s attitude toward its own scope is `ackermann`. The Ackermann function is the textbook example of a function that grows too fast to be primitive-recursive, and `stryke` ships it as `ackermann_limited M, N` — a *depth-limited* Ackermann that refuses to recurse past $M=3$, $N=14$. The refusal is not a missing feature; it is the responsible implementation of a function that would otherwise consume the machine. The library would rather give you a bounded, documented Ackermann than pretend the unbounded one is safe to call.

```
p gcd(48, 60, 36)      # 12    - variadic
p binomial(52, 5)      # 2598960 - poker hands
p catalan(10)          # 16796
p euler_totient(36)    # 12
p pascal_row(6)        # (1, 6, 15, 20, 15, 6, 1)
p stern_brocot(9)      # [a, b] - 9th positive rational, no repeats
```

Statistics

The statistics builtins are the ones that, in practice, displace a spreadsheet for the first ten thousand rows of any real analysis. The central-tendency set is `mean`, `median`, and `mode`, plus the two specialized means — `geometric_mean` for rates and ratios, `harmonic_mean` for averaging speeds — and the aggregate `sum`. Spread is covered by `variance`, `stddev` (population standard deviation), `percentile` for arbitrary quantiles, `quartiles` for the canonical three, and `iqr` for the interquartile range. The shape of a distribution gets its own pair: `skewness` measures asymmetry and `kurtosis` the tail weight, the two moments past variance that

tell you whether a dataset is Gaussian-shaped before you assume it is.

The data-prep verbs that always accompany statistics are present rather than improvised. `zscore` (aliased `z_score`) standardizes a value against a mean and standard deviation; `rank` assigns ordinal ranks; `histogram` bins a list; `cumsum` produces the running total and `diff` the successive differences — the two operations a time series needs before anything else. `sample` and `shuffle` cover random subselection and permutation, so a bootstrap or a train/test split is a builtin call rather than a loop.

The bivariate layer is where stryke starts replacing a stats package outright. `correlation LIST1, LIST2` computes the Pearson correlation coefficient between two equal-length numeric lists, and `covariance LIST1, LIST2` computes their sample covariance. When the relationship is the question rather than its strength, `linear_regression` fits the ordinary-least-squares line directly. These are the numbers you reach for the instant you have paired data and a hypothesis, and having them as primaries means an exploratory pass over a dataset is a pipeline, not a project.

```
@returns = (0.012, -0.004, 0.031, 0.008, -0.011, 0.022);
p mean(@returns)           # arithmetic mean
p stddev(@returns)         # population std dev
p percentile(@returns, 90) # 90th percentile

@x = (1, 2, 3, 4, 5);
@y = (2, 4, 5, 4, 5);
p correlation(@x, @y)      # Pearson r
p covariance(@x, @y)       # sample covariance
```

Distributions

Above the descriptive statistics sits the distribution layer — the probability density and mass functions that turn a dataset into a model. `normal_pdf($x, $mu, $sigma)` (aliased `normpdf`) is the Gaussian probability density function, and `normal_cdf($x, $mu, $sigma)` (aliased `normcdf`) its cumulative distribution function — the area under the Gaussian up to x , the number behind every z-score and confidence interval. For count data, `poisson_pmf($k, $lambda)` (aliased `poisson`) is the Poisson probability mass function, giving the probability of exactly k events when the rate is λ .

The continuous distributions extend past the Gaussian. `exponential_pdf` covers waiting times, `gamma_pdf` the gamma family, `beta_pdf` the beta distribution on $[0,1]$, and `chi_squared_pdf` the chi-squared distribution that underlies goodness-of-fit testing. Each of these is the density a domain reaches for first — reliability engineering lives in the exponential and gamma, Bayesian priors live in the beta, hypothesis tests live in chi-squared.

These are not toys bolted on for completeness; they are wired to the special functions above. The normal CDF is an error-function evaluation in disguise, which is why `erf` and `erfc` earn their place in the elementary layer, and the gamma and beta densities call straight through to `gamma` and `digamma` — the distribution builtins stand on the special functions, exactly as a numerics package would arrange them internally.

```
p normal_pdf(0, 0, 1)      # ~0.3989 - peak of the standard normal
p normal_cdf(1.96, 0, 1)   # ~0.975  - the 95% boundary
p poisson_pmf(3, 2.5)      # P(exactly 3 events | rate 2.5)
```

The shape of the numeric library

What the registry's category list makes clear is that “math” in stryke is not one section but a dozen overlapping ones — the 79-strong integer-sequence category sits next to a much larger numeric-extras layer, the distributions next to econometrics, signal processing, and the cosmology and quantum-mechanics corpora that show up elsewhere in this book. The builtins named in this chapter are the *general-purpose floor*: the arithmetic, number theory, statistics, and distributions that any program might touch. They are chosen here

because they are the ones you will reach for first, and because every one of them — `abs`, `sqrt`, `cbrt`, `floor`, `ceil`, `round`, `trunc`, `clamp`, `lerp`, `hypot`, `exp`, `ln`, `log2`, `log10`, `sin`, `cos`, `atan2`, `gamma`, `lgamma`, `digamma`, `erf`, `erfc`, `gcd`, `lcm`, `binomial`, `catalan`, `multinomial`, `derangements`, `prime_factors`, `divisors`, `next_prime`, `nth_prime`, `euler_totient`, `mobius`, `jacobi_symbol`, `lucas`, `tribonacci`, `collatz`, `primorial`, `pascal_row`, `stern_brocot`, `ackermann`, `mean`, `median`, `mode`, `geometric_mean`, `stddev`, `variance`, `percentile`, `quartiles`, `iqr`, `skewness`, `kurtosis`, `zscore`, `histogram`, `cumsum`, `correlation`, `covariance`, `linear_regression`, `normal_pdf`, `normal_cdf`, `poisson_pmf`, `exponential_pdf`, `gamma_pdf`, `beta_pdf`, `chi_squared_pdf` — is a verified primary you can call right now, with no import, against the same NaN-boxed numbers the VM uses internally.

Chapter 13 – Collections and Iteration

If math is where stryke proves its *reach*, collections are where it proves its *temperament*. A scripting language lives or dies on how naturally it transforms lists, and stryke inherits the right instinct from Perl — lists are everywhere, context decides their shape — and then sharpens it with two things Perl never had: a lazy iterator type and a parallel twin for nearly every operation. This chapter is the working vocabulary of list transformation, every name verified in the registry.

The eager core: map, grep, reduce, sort

The four operations that anchor every functional pipeline are present with their familiar names and their familiar semantics, refined by stryke's block conventions. `map BLOCK LIST` evaluates the block for each element — setting the topic `_` to the current element — and returns the list of all the block's return values. `grep { BLOCK } LIST` filters, returning only the elements for which the block evaluates true. The pairing of `map` and `grep` covers transform and select, the two halves of nearly every data pass.

Folding is where stryke's block grammar shows its hand. `reduce { _ OP _1 } @list` performs a sequential left fold, where `_` is the accumulator and `_1` the current element — the implicit-parameter convention that runs through the whole language, no named arguments required. When you need an explicit starting value, `fold { _ OP _1 } INIT, @list` is the same fold seeded with an initial accumulator. And when you want to see *the work* rather than just the final answer, `scanl FN, INIT, LIST` is Haskell's `scanl` — a fold that returns every intermediate accumulator, so a running total or running maximum falls out of one call.

```
@nums = (3, 1, 4, 1, 5, 9, 2, 6);

p [map { _ * _ } @nums]      # squares
p [grep { _ % 2 == 0 } @nums] # evens
p reduce { _ + _1 } @nums    # 31 - sum via fold
p fold { _ + _1 } 100, @nums  # 131 - seeded fold
p [scanl sub { _ + _1 }, 0, @nums] # running totals
```

Ordering and de-duplication round out the eager core. `sort [BLOCK] LIST` returns a new list in ascending order, taking an optional comparator block, and `sort_by` keys the sort on a derived value so you do not write a comparator at all. `uniq @list` removes duplicates while preserving the order of first occurrence — the order-stable de-dup that is what people almost always mean by “unique.” When uniqueness is by a derived key rather than the value itself, `uniq_by FN, LIST` returns unique elements by a key function — Ruby's `uniq` with a block, so you can dedup a list of records by one field.

Two verbs generalize `map` and `grep` into the shapes data work actually needs. `flat_map` maps each element to a list and flattens the results in one pass — the operation behind “expand each row into several.” `group_by FN, LIST` partitions a list into a hashref keyed by a function's result, the single most common reshaping in any analysis: bucket the records, then aggregate the buckets. Its boolean special case is `partition`, which splits a list into the elements that satisfy a predicate and those that don't, returning both halves so you never run the same filter twice with opposite signs.

Searching and testing: find, all, any, contains

Before you transform a list you often only need to interrogate it, and that family is complete. `find { COND } LIST` returns the first element satisfying a predicate, `find_index` returns its position, and `find_map FN, LIST` returns the first `Some` result from mapping — Rust’s `find_map`, which fuses “search then transform” into one pass. `first` and `last` take the ends, `nth` takes an arbitrary position, and `index_of` locates a value. The boolean reducers `all`, `any`, and `none` answer the universal and existential questions and short-circuit the moment the answer is decided, while `contains` tests membership and `count` totals matches. The keyed aggregates `sum_by`, `max_by`, and `min_by` reduce by a derived value — `max_by { length } @words` is the longest word without an explicit comparator — and `minmax` returns both extremes in a single traversal where two separate calls would cost two.

Reshaping: take, drop, flatten, zip, frequencies

Beyond transform-and-select sits the family that *reshapes* a list without touching its elements. `take N, LIST` (aliases `head`, `hd`) returns at most the first N elements; `drop N, LIST` (aliases `skip`, `drp`) skips the first N and returns the rest. Together they slice a list from either end without index arithmetic. Their conditional cousins are predicate-driven: `take_while { COND } LIST` emits leading elements while the predicate holds and stops at the first failure — the right tool for “everything up to the first bad row.”

Structural reshaping has its own verbs. `flatten` (alias `fl`) does a one-level flatten — plain arrays and arrayrefs expand, nested structure one layer down comes up to the surface — and `flatten_deep` recurses all the way down when the nesting is arbitrary. `compact LIST` removes the `undef` and empty-string holes, the cleanup you reach for after a sparse parse. `interleave @a, @b, ...` (alias `il`) is a round-robin merge that takes one element from each list in turn, and `intersperse` inserts a separator between every pair of elements. `zip(\@a, \@b, ...)` combines multiple arrays element-wise into a list of arrayrefs, the canonical “pair up parallel columns” operation, and `zip_with` is its variant that applies a combiner instead of just pairing. `transpose` flips an array-of-arrays so rows become columns — exactly the row/column swap a table needs — and `rotate N, LIST` rotates the list left by N (negative rotates right). The list-to-pairs adapter `pairs @list` takes a flat list and groups consecutive elements into two-element arrayrefs — exactly the shape you need to walk a flattened key/value list — while `with_index` (alias `wi`) tags each element with its 0-based position as `[item, index]`.

The windowing verbs slice a list into overlapping or adjacent runs. `each_slice N, LIST` walks consecutive non-overlapping slices of size N; `each_cons N, LIST` walks consecutive *overlapping* N-element windows — Ruby’s pair, and the right tools for “process three rows at a time” versus “compare each row to its neighbors.” `chunk_by` groups runs of adjacent elements that share a key, the streaming run-length operation behind log collapsing.

The combinatorial verbs turn a list into its derived sets. `combinations N, LIST` yields all N-element combinations and `permutations N, LIST` all N-element orderings — Python’s `itertools.combinations` and `permutations` as barewords. `cartesian_product LIST1, LIST2, ...` is the cross product (`itertools.product`), and `powerset` enumerates every subset. These are the operations that turn a brute-force search from a triple-nested loop into one named call.

Counting is a first-class reshaping operation, not an afterthought. `frequencies LIST` (aliases `freq`, `frq`) counts occurrences and returns a hashref of `{ value => count }` — the histogram-in-one-call that turns a raw list into a distribution — and `tally` is its synonym for the same operation. `count_by` is the keyed sibling, counting by a derived key rather than the raw value.

```

@xs = (5, 3, 8, 1, 9, 2, 7);
p [take 3, @xs]           # (5, 3, 8)
p [drop 3, @xs]           # (1, 9, 2, 7)
p [take_while { _ < 8 } @xs] # (5, 3) - stops at 8

@a = (1, 2, 3); @b = ('a', 'b', 'c');
p [zip(@a, @b)]           # ([1,a], [2,b], [3,c])
p [interleave @a, @b]     # (1, a, 2, b, 3, c)

@words = qw(red blue red green blue red);
p frequencies(@words)     # { red => 3, blue => 2, green => 1 }

```

The aggregates: sum, min, max

Three operations collapse a list to a single number often enough to deserve their own names. `sum @list` returns the numeric sum of all elements; `min @list` the numerically smallest value; `max @list` the numerically largest. These dispatch through the `list_builtins` path rather than the main builtin arms — the same path `pairs` takes — which is why `build.rs` scans that file separately so they still appear in `%b` and `%all`. From the caller's side that detail is invisible: they are primaries like any other, and they are what makes the tail of a pipeline a reduction to one value.

```

@latencies = (12, 45, 7, 88, 23, 4, 61);
p sum(@latencies)        # 240
p min(@latencies)        # 4
p max(@latencies)        # 88

```

The lazy layer: fi and the streaming iterators

Everything above is eager — it materializes a new list. `stryke`'s distinguishing collection feature is the *lazy* layer that doesn't. The naming convention is mechanical: the eager verb pluralized is its streaming twin. `maps` is the lazy, streaming counterpart of `map`, and `greps` the lazy counterpart of `grep` — each returns a pull iterator instead of a materialized list. `filter` (alias `fi`) `{ BLOCK } LIST` is the `stryke`-native lazy filter, functionally identical to `greps` but named for the Rust/Ruby/JS audience that expects `filter`; `flat_maps` is the lazy flat-map, evaluating the block per element and flattening the results into a single iterator. The difference is consumption. An eager `grep` over a million-element list allocates a result the moment it runs; `fi` over the same list allocates nothing until something pulls from it, and stops the instant the consumer stops asking.

The lazy layer has its own generators, which is what makes “unbounded” a usable word. `iterate FN, X` returns the infinite stream `x, f(x), f(f(x)), ...` — the canonical way to express a sequence by its recurrence rather than its length. `cycle LIST` returns an iterator that repeats the list forever. Neither is dangerous, because nothing materializes until a bounded consumer pulls a finite prefix: `iterate { _ * 2 } 1` is harmless until you `take 10` from it.

This is what makes `chunk N, ITERATOR` and the streaming family pay off. `chunk` yields `N`-element arrayrefs *streaming* — it reads from an iterator and emits fixed-size windows without ever holding the whole input in memory. Paired with a lazy `fi`, you can describe a transformation over an unbounded or very large source as a pipeline of small, composable stages, and only the elements that survive to the end are ever fully realized. A 50-GB log file becomes `lines |> fi { ... } |> chunk 1000 |> ...` — constant memory, because the file is a faucet, not a bucket.

There is one sharp edge worth stating plainly, because it follows directly from laziness: `fi` is a lazy filter, and a lazy filter wrapped in an array literal — `[fi { ... } LIST]` — does not necessarily drain the way an eager `grep` would. When you need the materialized list right now, eager `grep` is the correct tool; when you are building a streaming pipeline, `fi` is. The two are not blind drop-in replacements for each other, and choosing between them is choosing between “give me the whole list” and “give me a faucet.”

```
# Lazy: nothing happens until consumed
my $stream = fi { _ % 2 == 0 } (1..1_000_000);
$stream |> take 5 |> p           # first 5 evens, rest never touched

# Streaming windows over a large source
(1..20) |> fi { _ % 3 == 0 } |> chunk 2 |> p
```

The parallel twins

The collection vocabulary has a second copy of itself that runs on every core. This is covered in depth in the parallel chapter, but it belongs here as a recap because the *naming* is the whole ergonomic argument: the parallel form of an operation is the operation with a `p` in front of it.

`pmap` is parallel `map`, powered by rayon's work-stealing thread pool. `pgrep` is parallel `grep`, evaluating the filter predicate concurrently across all cores. `preduce` is a parallel tree-fold — it splits the list into chunks, reduces each independently with rayon's `reduce`, then merges the partials. `psort` is a parallel merge-sort. `pfor` is a parallel `foreach` for side-effecting blocks with no return value. `pfrequencies` (aliases `pfreq`, `pfrq`) is the rayon-parallel frequency count. The boolean reducers parallelize and short-circuit: `pany` is the parallel short-circuit `any` and `pall` its universal twin, `puniq` is parallel deduplication, and `pflat_map` is the parallel flat-map. The pattern holds — wherever the eager verb collapses or expands a list element-independently, a `p`-prefixed sibling does the same work across every core.

And the lazy layer has a *parallel-lazy* tier, which is the rarest combination of the three: `pmaps` is a streaming parallel map — a lazy iterator that processes items across all cores using a persistent worker-thread pool — and `pgreps` and `pflat_maps` are its filter and flat-map siblings. These give you the property that almost nothing else offers: bounded memory *and* full core utilization at the same time, a faucet that runs on every CPU.

```
# Same shape, parallel execution
my @hashed = pmap { blake3(_) } @files;
my @big    = pgrep { -s _ > 1_000_000 } @files;
my $total  = preduce { _ + _1 } @sizes;

# Parallel AND lazy
(1..10_000_000) |> pmaps { _ * _ } |> take 10 |> p
```

The lesson of the collection layer is consistency. There is one vocabulary — `map`, `grep`, `reduce`, `fold`, `sort`, `sort_by`, `uniq`, `uniq_by`, `take`, `drop`, `take_while`, `flatten`, `flat_map`, `group_by`, `partition`, `zip`, `transpose`, `combinations`, `permutations`, `frequencies`, `find`, `all`, `any`, `sum`, `min`, `max`, `minmax` — and it appears in three registers: eager, lazy (`maps`, `greps`, `fi`, `flat_maps`, `chunk`, `iterate`, `cycle`), and parallel (`pmap`, `pgrep`, `preduce`, `psort`, `pany`, `pall`, `puniq`, `pflat_map`, `pmaps`). Learning the noun teaches you all three forms, and choosing among them is choosing a *strategy*, not relearning an *API*.

Chapter 14 – Strings, Encoding, and the Network

The last stop in the standard-library tour is the part most languages treat as three separate worlds: text, bytes, and the wire. `stryke` treats them as one continuum. A string is a sequence of bytes you might hash, encode, hash again, percent-escape, and ship across a socket — and every step of that sentence is a builtin. This chapter walks the text operations, the encoding-and-crypto family, and the network/CIDR/IP algebra that is, by the registry’s own count, one of the largest single domains in the language.

The string core

The Perl-shaped string operations are all present and behave as you’d expect, with one important caveat the registry is explicit about: `stryke` distinguishes *byte* coordinates from *codepoint* coordinates, and the docs say which is which. `uc STRING` returns a fully uppercased copy; `lc STRING` a fully lowercased one; `ucfirst STRING` uppercases only the first character and `lcfirst STRING` lowercases it, leaving the rest untouched. `trim STRING` (also `trim LIST`) strips leading and trailing whitespace. The Perl line-editing pair `chomp` and `chop` are here — the first removes a trailing newline, the second the final character — alongside the character/codepoint bridges `chr` and `ord`, which convert between a character and its codepoint number.

The character-aware accessors matter once text stops being ASCII. `chars` splits a string into its codepoints and `graphemes` into user-perceived characters — the distinction between “é as e-plus-combining-accent” counted as one or two — so length-by-character and length-by-grapheme are different operations because the underlying questions are different. The unglamorous layout helpers live here too: `pad_left` and `pad_right` pad to a width, `center` centers within one, and `wrap_text STR, WIDTH` (alias `word_wrap`) reflows a paragraph to a column. Even the linguistic helpers are builtins — `pluralize WORD, COUNT`, `singularize`, and `ordinalize NUM` (which turns 1 into “1st”, 2 into “2nd”) — the functions every reporting script reinvents.

Splitting and joining are the two halves of every text pipeline. `split /PATTERN/, STRING [, LIMIT]` divides a string into a list of substrings on each occurrence of the pattern; `join SEPARATOR, LIST` is the inverse, concatenating a list with a separator between adjacent elements. Formatting is `sprintf`, which returns a formatted string using the same C-style specifiers as `printf` without printing it.

The coordinate distinction matters most for the position-and-slice operations, and getting it wrong is a real bug, not a style nit. `index STRING, SUBSTRING [, POSITION]` returns the zero-based **byte** position of the first occurrence, or `-1` if absent. `length STRING` returns the **byte** length. `substr STRING, OFFSET [, LENGTH [, REPLACEMENT]]` extracts or replaces a substring by **byte** offset. These three are a *byte-coordinate* family and they pair with each other — feed `index`’s byte offset to `substr`, measure with `length`. The codepoint-coordinate world (counting characters, slicing by character index) is a separate set of operations; the rule the language enforces is to never cross the two. Pair byte with byte, codepoint with codepoint. `reverse LIST` reverses element order in list context and characters in scalar context; `repeat STRING, N` (or `STRING |> repeat N`) repeats a string.

```

my $s = "Hello, World";
p uc($s)                # HELLO, WORLD
p ucfirst("rust")       # Rust
p [split /\s*/, "a, b, c"] # (a, b, c)
p join("-", 1, 2, 3)    # 1-2-3
p index($s, "World")    # 7 (byte offset)
p substr($s, 7)         # World
p sprintf("%08.3f", 3.14159) # 0003.142

```

Fuzzy text: distance, similarity, phonetics

stryke's text layer goes well past slicing into the territory of *fuzzy* matching — the operations behind spell-check, record linkage, and search. The edit-distance family is plural rather than singular, because “how different are these two strings” has several correct answers. `levenshtein STR1, STR2` is the Levenshtein edit distance, the minimum single-character insertions, deletions, and substitutions. `damerau_levenshtein` adds transposition of adjacent characters as a single edit — the right metric for typo correction, where “teh” → “the” is one mistake, not two. `hamming_distance` counts differing positions for equal-length strings, the metric for fixed-width codes. The similarity scores are a matching set: `jaro_winkler_similarity` weights matching prefixes more heavily and is the standard choice for short strings like names, `jaro_similarity` is its unweighted base, `dice_coefficient` measures set overlap, and `cosine_similarity` the angle between term-frequency vectors. For substring relationships there is `longest_common_subsequence` and `longest_common_substring`, the two operations behind diff and plagiarism detection.

The phonetic encoders are a full family, not a token `soundex`. `soundex STR` maps a word to a code so that words that *sound* alike collide — the classic “find me the Smiths and the Smythes” tool — and `metaphone`, `double_metaphone`, `nysiis`, and `caverphone` are the successively more sophisticated algorithms that handle the cases Soundex gets wrong, each tuned for a different language or era of names. `ngrams` slices a string into its n-grams for the fuzzy-search index those algorithms feed.

These exist as primaries for the same reason the math library carries `gamma`: they are operations everyone eventually needs and nobody should have to reimplement. A fuzzy join across two datasets is a `pmap` of `jaro_winkler_similarity` calls, not a project — and choosing `damerau_levenshtein` over `levenshtein` because your data is keyboard typos is a one-token decision rather than a library swap.

```

p levenshtein("kitten", "sitting") # 3
p jaro_winkler_similarity("Dwayne", "Duane") # ~0.84
p soundex("Robert") # R163
p soundex("Rupert") # R163 - collides, by design

```

Case-style conversion lives here too, because slugs and identifiers are a text problem. The full set of case conventions is present: `camel_case STRING` for camelCase, `pascal_case` for PascalCase, `snake_case` for snake_case, `kebab_case` for kebab-case, `constant_case` for CONSTANT_CASE, and `title_case` for Title Case. `slugify STR` produces a URL slug. These are the unglamorous functions that every web codebase reinvents — and reinvents subtly differently each time, which is exactly the inconsistency a single named primary removes.

Encoding and crypto

This is where text becomes bytes and bytes become digests, and it is one of the densest corners of the registry — the *crypto / encoding* category holds 178 entries. The base-N encoders are a complete set, not a token few. `base64_encode` (alias `b64e`) and `base64_decode` (alias `b64d`) cover standard Base64. `base32_encode` (alias `b32e`) does RFC 4648 Base32. `base58_encode` (alias `b58e`) uses Bitcoin’s Base58 alphabet — the one that drops the visually ambiguous `0`, `O`, `I`, and `l`. `base85_encode` (aliases `b85e`, `a85e`, also reachable as `ascii85_encode`) does Ascii85/Base85, `z85_encode` the ZeroMQ Z85 variant, and `base91_encode` the denser Base91. The transport encoders go further: `quoted_printable_encode` for MIME bodies and

`punycode_encode` for internationalized domain names. The raw-hex pair is `hex_encode` (alias `hxe`), which turns bytes into lowercase two-chars-per-byte hex, and `hex_decode` (alias `hxd`), its inverse. `url_encode` percent-escapes a string for safe embedding in a URL, and `rot13` is the classic letter rotation.

The hash functions span the spectrum from “fast checksum” to “password-grade,” and the spectrum is densely populated. For integrity there is `crc32`, the CRC-32 checksum returning an unsigned 32-bit integer, and `adler32`, the Adler-32 checksum — both fast, neither cryptographic. The non-cryptographic fast hashes for hash tables and content fingerprinting are a family of their own: `xxh3` (xxHash3-64, fastest on modern CPUs), `fnv1a` (the FNV-1a hash), `murmur3` (MurmurHash3 32-bit), and `siphash` (SipHash-2-4, the keyed hash that defends hash tables against collision attacks). For cryptographic digests the lineup runs `md5` and `sha1` (legacy, present for compatibility), through `sha224`, `sha256` (alias `s256`), `sha384`, `sha512` (alias `s512`), into the SHA-3 family `sha3_256` and `sha3_512` (NIST FIPS 202), and the BLAKE line — `blake2b`, `blake2s`, and `blake3` (alias `b3`), the modern default, faster than SHA-2 and parallelizable by construction.

The keyed and password-grade constructions are where a wrong choice is a security bug, so the right answers are all builtins. `hmac_sha256` (alias `hmac`), `hmac_sha512`, and `hmac_sha1` compute HMACs over data and a secret. For passwords specifically there is `argon2` (the `argon2_hash` builtin, Argon2id — the Password Hashing Competition winner), plus `bcrypt` and `scrypt`, the two other defensible password hashes, each returning its standard PHC-format string. Key derivation is `pbkdf2` (PBKDF2-HMAC-SHA256) and `hkdf` (HKDF-SHA256, RFC 5869). The authenticated-encryption and signature primitives round it out: `aes_encrypt` is AES-256-GCM, `ed25519_sign` and `ed25519_verify` are the Ed25519 signature pair, and the one-time-password generators `totp` (RFC 6238) and `hotp` (RFC 4226) cover 2FA. `jwt_encode` and `jwt_decode` handle JSON Web Tokens directly. And for the cases where a legacy protocol demands it, `des3_encrypt` (alias `tdes_enc`) does Triple DES in CBC mode.

The breadth here is the point, and it is a *security* point as much as a convenience one. In Python or Perl, “hash this password correctly” means choosing a PyPI/CPAN distribution, trusting its maintenance, and pinning its version — a chain where the weakest link is a developer who reaches for `hashlib.md5` because it was already imported. In stryke, `argon2` is as present as `print`, so the correct primitive is never further away than the wrong one. A program that ingests a file, checksums it for integrity, hashes it for a content address, signs the result, and Base64s the signature for transport touches `crc32`, `blake3`, `ed25519_sign`, and `base64_encode` — four builtins, no dependencies, one binary, no supply chain to audit.

```
my $data = "the quick brown fox";
p blake3($data)           # 64-char BLAKE3 hex
p sha256($data)           # 64-char SHA-256 hex
p crc32($data)            # u32 checksum
p adler32($data)          # Adler-32 checksum
p xxh3($data)             # fast non-crypto fingerprint
p sha3_256($data)         # SHA3-256 (FIPS 202)
p base64_encode($data)    # dGh1IHFlYWNR...
p z85_encode($data)       # Z85 / ZeroMQ encoding
p hmac_sha256($data, "secret-key") # authenticated MAC
p argon2("user-password") # Argon2id hash for storage
p bcrypt("user-password") # $2b$... password hash
```

The network: IP and CIDR as an algebra

The largest single block in this chapter is the *network / ip / cidr* category — 167 primaries by the registry’s count — and it is where stryke does something genuinely unusual for a general-purpose language: it treats IP addresses and CIDR blocks as a *first-class algebra*, not a string-formatting exercise.

Start with parsing and representation. `ip_parse` reads an address; `ip_version` reports whether it’s v4 or v6; `ip_family` reports the address family; `ip_canonical` returns the canonical form (the compressed, normalized spelling). The numeric bridges are `ip_to_int` and its inverse `int_to_ip`, plus `ip_to_bytes`, `ip_to_bits`, and the reverse `bits_to_ip` — so an address can move freely between dotted-quad, integer, byte array, and bit string. The v4-specific and v6-specific families are deep on their own: `ipv4_parse`, `ipv4_`

`to_int`, `ipv4_to_ipv6_mapped`, and `ipv4_to_ipv6_6to4` on one side; `ipv6_parse`, `ipv6_compress`, `ipv6_expand`, `ipv6_link_local_from_mac`, and `ipv6_solicited_node` on the other.

The classification predicates are a complete RFC-grade set rather than a token `is_private`. `ip_is_private`, `ip_is_loopback`, `ip_is_multicast`, `ip_is_link_local`, `ip_is_global`, `ip_is_reserved`, `ip_is_documentation`, `ip_is_benchmarking`, `ip_is_broadcast`, `ip_is_shared`, and `ip_is_unspecified` each answer one RFC question about an address. Sorting and comparison are first-class too: `ip_sort`, `ip_compare`, and the reverse-DNS helpers `ip_reverse` and `ip_arpa`.

The CIDR layer is where “algebra” stops being a metaphor. `cidr_parse` reads a block; `cidr_contains` tests membership; `cidr_network`, `cidr_broadcast`, `cidr_netmask`, `cidr_hostmask`, and `cidr_wildcard` derive the block’s structure; `cidr_num_hosts` and `cidr_hosts` enumerate it. The set operations are the unusual part: `cidr_aggregate` collapses a list of adjacent blocks into the minimal covering set, `cidr_summarize` and `cidr_minimum_covering` do the related summarizations, and `cidr_union`, `cidr_intersection`, and `cidr_difference` treat CIDR blocks as sets you can combine. `cidr_subnets` and `cidr_split` divide a block downward; `cidr_supernet` walks it upward; `cidr_overlaps` tests two blocks for intersection. This is a routing-table calculator built into the language — the operations a network engineer would otherwise script against a Python library, available as barewords.

The link layer and name resolution are covered without leaving the language. `mac_parse` reads a MAC address and `mac_normalize` canonicalizes its formatting, the two operations every switch-config parser needs first. `dns_resolve` (the `net_dns` builtin, also reachable as `resolve`) turns a hostname into its IP addresses, and `port_is_well_known` answers whether a port number falls in the reserved range. Together with the addressing and CIDR families, that 167-primary category spans the whole stack from a MAC string to a routing decision to a DNS lookup — every layer a network tool touches, none of it imported.

```
# Address algebra
p ip_to_int("192.168.1.1")      # 3232235777
p int_to_ip(3232235777)         # 192.168.1.1
p ip_is_private("10.0.0.5")     # true
p ip_is_global("8.8.8.8")       # true
p ip_canonical("2001:0db8::0001") # 2001:db8::1

# CIDR set operations
p cidr_contains("10.0.0.0/8", "10.1.2.3") # true
p cidr_num_hosts("192.168.1.0/24")        # 254
# collapse adjacent blocks into the minimal covering set
p cidr_aggregate("192.168.0.0/24", "192.168.1.0/24") # 192.168.0.0/23
```

NAT traversal at the edge

The network domain does not stop at addressing; it reaches all the way to the modern problem of getting two machines to talk *through* NATs. `stun` queries a STUN server through a socket and returns the public (`ip`, `port`) the server observed — RFC 8489, the first step of any peer-to-peer connection. `stun_classify` goes further: it queries *multiple* STUN servers over the same socket and classifies the local NAT based on whether the reported public ports agree across servers, which is how you detect a symmetric NAT before you waste time trying to hole-punch through one.

That a general-purpose scripting language ships STUN-based NAT classification as a builtin is the clearest single illustration of what “batteries included” means at the scale of ten thousand primaries. The library does not stop at the operations a textbook calls “standard.” It keeps going until the thing you actually needed — the Damerau-Levenshtein distance, the Argon2 hash, the Ed25519 signature, the CIDR aggregation, the NAT classifier — is already there, already in the binary, already one bareword away.

```
my $sock = udp_open();
my ($pub_ip, $pub_port) = stun($sock);      # what the internet sees
p $pub_ip, $pub_port
p stun_classify($sock)                      # NAT type – symmetric or not
```

The three layers of this chapter — text, encoding, and network — are one story told at three scales. A byte is a character is a hash input is a packet payload. `stryke` refuses to put walls between them, and the standard library is the proof. The 178-entry crypto/encoding category and the 167-entry network category are not bolt-ons; they sit at the same namespace depth as `uc` and `split`, so a pipeline can move from text to digest to wire without changing register. `uc`, `damerau_levenshtein`, `metaphone`, `blake3`, `ed25519_sign`, `base64_encode`, `cidr_aggregate`, and `stun_classify` are the same kind of thing, callable the same way, shipped in the same 44 MB binary, with nothing to install between you and any of them.

Chapter 15 – The Scientific Shelf

Most languages give you a standard library. Stryke gives you a reference shelf.

The distinction matters more than it sounds. A standard library is a kit of general-purpose parts — strings, lists, maps, a regex engine, some math — out of which you build the thing you actually want. Everything domain-specific is somebody else's package, a `pip install`, a `gem`, a dependency line in a manifest, a transitive supply-chain risk. The library is a foundation; the building is your problem.

Stryke inverts that. When you reach into `stryke` for the Nernst equation, the moist adiabatic lapse rate, the reverse complement of a DNA strand, or the Schwarzschild free-fall velocity, the function is already there — a native builtin, compiled into the same 44 MB static binary that holds `print` and `map`. There is no install. There is no import. `nernst_equation` is as available as `len`. That is what “reference shelf” means: not a toolbox you assemble domain knowledge from, but a pre-stocked shelf where the domain knowledge is already on it, labeled and ready.

To see the scale of that claim concretely: the build-generated reflection table (`target/debug/build/strykelang-*/out/reflection.rs`) declares 8,914 canonical builtin arms, before aliases. That number is not a slogan — each arm is one `&["name", "alias", ...]` group in the generated dispatch table, and a plain count of those groups returns exactly 8,914. The whole point of this chapter is that a large fraction of those arms are not plumbing — they are working scientific formulas across five distinct disciplines, every one of which would otherwise be a third-party package with its own version, its own maintainer, and its own way of going stale.

The aliasing matters too, because the shelf is generous about how you reach a formula. Most arms carry synonyms: `nernst_equation` answers to `nernst`, `gibbs_free_energy` to `delta_g`, `fft_magnitude` to `fftmag`, `stefan_boltzmann_constant` to `sigma_sb`. The canonical name is the long, self-documenting one; the alias is the one you reach for at the REPL. Both resolve to the same compiled function, so there is no “real name” you have to memorize against a friendlier one you'd rather type — the shelf meets you at whichever name you already know the formula by.

Bioinformatics: sequences as first-class data

Start with DNA, because it's the cleanest demonstration. A genome is a string over a four-letter alphabet, and the operations a biologist performs on it — complement, transcription, translation, GC content, k-mer counting, melting temperature — are exactly the operations that, in Python or Perl, send you to BioPython or BioPerl. In `stryke` they're builtins.

```
val $s = "ATGGCCTAC"
val $rc = $s |> dna_reverse_complement # "GTAGGCCAT"
val $pt = $s |> dna_translate           # "MAY" (Met-Ala-Tyr)
val $gc = $s |> dna_gc_content          # 0.5555...
```

That's the `bioinformatics_primer.stk` example (its assertions trimmed to comments here). `dna_reverse_complement` walks the strand 3'→5' swapping each base for its Watson-Crick partner; `dna_translate` reads codons three at a time through the standard genetic code and emits single-letter amino acids; `dna_gc_content` returns the guanine-cytosine fraction that predicts duplex stability. Three pipe stages, three domain operations, zero dependencies — and because the operations are pipe stages, they thread: a strand flows through complement, then translation, then a property calculation as naturally as a list flows through `map` and `filter`.

The shelf goes much deeper than this primer. The registry carries `dna_complement` (the single-strand swap without the reversal), `dna_transcribe` (DNA → RNA, the first step of the central dogma), `dna_at_content` as the AT-fraction complement to GC, `dna_kmer_count` and `dna_kmer_index` for the substring statistics that underpin de-novo assembly and read alignment — k-mer spectra are how modern assemblers like an overlap graph in the first place — and `dna_melting_temp` for the primer-design calculation every PCR run depends on. On the RNA side there's `rna_reverse_complement`, `rna_to_dna`, and `reverse_transcribe`, closing the loop back to DNA the way a retrovirus or an RT-PCR protocol does. The codon layer is its own cluster: `codon_to_aa` and `codon_to_amino_acid` for the genetic-code lookup, `codon_usage_table` for the organism-specific frequency table, `codon_adaptation_index` for scoring how well a sequence matches a host's preferences, and `codon_optimize` — the workhorse of synthetic-biology gene design, rewriting a coding sequence to a host organism's preferred codons without changing the protein it encodes. That last one is not a textbook formula; it is the operation a company ordering a synthetic gene actually pays for.

Proteins get the same treatment one tier up the central dogma. `protein_molecular_weight`, `protein_charge_at_ph`, `protein_hydrophobicity`, `protein_pI`, and `isoelectric_point_protein` cover the physicochemical properties you compute when you're deciding how a peptide will behave in a gel, on a column, or in a mass spectrometer. The charge-at-pH and isoelectric-point pair are the same calculation viewed two ways — the pH at which net charge is zero is exactly where a protein stops migrating in isoelectric focusing — and shipping both spellings means the book's formula and the bench protocol's vocabulary both resolve. Because sequence comparison is the daily bread of the field, the Hamming family is specialized by alphabet: `hamming_dna`, `hamming_protein`, and `rna_hamming` sit alongside the generic `hamming_distance` so a mutational distance is measured against the right substitution model — a DNA mismatch and an amino-acid substitution are not the same biological event, and the specialized builtins keep that distinction at the call site instead of in a comment.

None of these is a toy. `codon_adaptation_index` and `isoelectric_point_protein` are formulas a working bioinformatician would otherwise pull from BioPython or BioPerl and validate against a known sequence before trusting in a pipeline. Here they ship in the binary, versioned with the language, with no dependency that can drift out from under a five-year-old script.

Cosmology and general relativity: the universe in builtins

The second shelf is general relativity and cosmology — the domain where “just use a library” usually means a research codebase in C++ or a Jupyter notebook stitched together from `astropy` and hand-rolled tensor math. Stryke puts the same calculations behind plain function names.

The FLRW cosmology — the homogeneous, isotropic, expanding-universe model that frames modern observational cosmology — is represented directly. `hubble_lcdm` evaluates the Hubble parameter under Λ CDM; `hubble_constant`, `hubble_time`, and `hubble_distance` (with `hubble_distance_mpc` and `hubble_distance_si` unit variants so you never have to remember which scale the bare call returns), plus `comoving_distance`, give you the distance-and-age scaffolding the whole field reports observations against. `redshift_from_a` and `redshift_z` convert between scale factor and redshift — the two coordinates cosmologists actually argue in — and `luminosity_distance` gives you the distance that maps observed flux to intrinsic luminosity, the quantity behind every standard-candle measurement of the expansion history. `critical_density` and `critical_density_si` return the knife-edge density that separates an open universe from a closed one, and `cosmological_constant`, `cosmological_constant_term`, and `friedmann_density_total` close the loop back to the Friedmann equations that govern the whole model. Read together, these are not scattered conveniences — they are the full chain from a Hubble constant to a predicted distance-redshift relation, which is the calculation a cosmology problem set asks you to derive from scratch.

The general-relativity machinery beneath cosmology is here too, not abstracted away. `lorentz_factor`, `lorentz_gamma`, and `lorentz_factor_v` give you special-relativistic time dilation from velocity — three spellings of the γ -factor because it appears under all three names across textbooks. `grav_redshift` and

`doppler_relativistic` handle the two physically distinct ways a spectral line shifts: one from climbing out of a gravitational well, one from relative motion, and keeping them as separate builtins keeps the physics honest. The genuinely hard differential-geometry pieces — the ones that send people back to a library or a notebook — are present as stepwise builtins: `christoffel_first_kind_step` and `christoffel_second_kind_step` for the connection coefficients that encode how coordinates curve, `geodesic_curvature_step` and `geodesic_equation_step_zero` for the equation of motion a free-falling particle obeys, `metric_schwarzschild_step` and `kretschmann_schwarzschild` for the canonical black-hole metric and the curvature invariant that distinguishes real curvature from a coordinate artifact, and `freefall_velocity_schwarzschild` for the speed a test particle reaches falling in. The “step” suffix is the tell: these are the per-iteration kernels you compose into a numerical integrator, which is exactly the granularity a relativist needs and a high-level “compute the geodesic” black box would hide.

A language that ships the Kretschmann scalar of the Schwarzschild metric as a builtin is making a statement about what it considers in-scope. Most standard libraries stop at `sqrt`. This one keeps going until it reaches the curvature of spacetime.

Chemistry and electrochemistry

The third shelf is chemistry, with a deep electrochemistry specialization.

The general chemistry layer covers the conversions and energetics you do constantly: `chem_molar_mass`, `chem_concentration_to_molarity`, `chem_molarity_to_normality`, `chem_ph_from_h` and `chem_h_from_ph` for the acid-base bookkeeping that turns a hydrogen-ion concentration into a pH and back, plus the thermodynamic core — `gibbs_free_energy` (alias `delta_g`) for whether a reaction proceeds, `enthalpy_reaction` for its heat, and the Arrhenius family (`arrhenius_k`, `arrhenius_rate`, `arrhenius_temp_q10`, `chem_arrhenius_k`) that governs how reaction rate climbs with temperature. The `q10` variant is worth singling out: it’s the rate-doubling-per-ten-degrees rule of thumb that biochemists and food scientists use directly, shipped as its own builtin rather than left as an exercise on top of the raw exponential.

Electrochemistry is where the shelf gets unusually deep. The Nernst equation — which relates electrode potential to ion concentration, the equation behind every pH meter, battery, and ion-selective sensor — appears not once but as a family: `nernst_equation` (alias `nernst`), `nernst_potential_full`, `nernst_diffusion_layer`, `nernst_einstein_diffusivity`, and `nernst_planck_flux`. The Nernst-Einstein relation links a species’ diffusivity to its electrical mobility; the Nernst-Planck flux is the transport equation for charged species moving down both a concentration gradient and a potential gradient at once — it’s the math inside membrane physiology, ion-channel modeling, and fuel-cell design. Faraday’s laws of electrolysis come as `faraday_constant` (alias `faraday`), `faraday_mass_deposited` (how much metal plates out for a given charge), `faraday_efficiency_h2` (the fraction of current that actually makes hydrogen instead of side reactions), and `faraday_emf`. And the cell-level builtins — `emf_from_half_cells`, `electrode_potential_step`, `electrochem_impedance_z` — let you go from tabulated half-cell potentials to a full-cell EMF to its frequency-domain impedance, which is the measurement electrochemists run to characterize a battery or a corrosion interface.

You do not build out `nernst_planck_flux`, `faraday_efficiency_h2`, and `electrochem_impedance_z` for a language meant to do the occasional pH conversion. This is a shelf stocked by someone who expects electrochemistry to get computed in it.

Signal processing

The fourth shelf is digital signal processing, stocked the way a DSP practitioner would want it.

Transforms first: `dft` and `fft_magnitude` (alias `fftmag`) get you from time domain to frequency domain. Convolution comes in the three modes any signal engineer expects — `convolve_full`, `convolve_valid`, and `convolve_2d` for images — alongside `fftconvolve_step` for the FFT-accelerated path that any long filter needs to stay tractable, and `deconvolve_step` for the inverse, the operation that un-smears a blurred

measurement. Correlation is covered by `autocorrelation` (alias `acorr`) and `cross_correlation` (alias `xcorr`), the basis of every matched filter, pitch detector, and time-of-arrival estimator. `goertzel` gives you the single-bin DFT — the efficient way to detect one specific frequency, which is exactly why it's the algorithm inside DTMF touch-tone decoding. `hilbert_env` and `hilbert_envelope` extract the analytic-signal envelope for amplitude demodulation and instantaneous-amplitude tracking.

Then the window functions — the coefficients that control spectral leakage, the thing every FFT-based measurement gets wrong if you skip them, because a raw FFT of a finite chunk of signal implicitly multiplies it by a rectangular window and smears every tone across neighboring bins. Stryke ships the standard catalog: `hann_window`, `hamming_window`, `blackman_window`, `blackman_harris_window`, `bartlett_window`, `gaussian_window`, `dpss_window` (the discrete prolate-spheroidal / Slepian window for multitaper spectral estimation), and `apply_window` (alias `applywin`) to multiply a window against a frame. Having the whole catalog matters because the choice of window is the central tradeoff of spectral estimation — main-lobe width against side-lobe height — and a stdlib that ships only `hann_window` has quietly made that decision for you. For the wavelet practitioner, the discrete wavelet transform steps are present — `db4_wavelet_step` (Daubechies-4), `coiflet_wavelet_step`, `beylkin_wavelet_step` — for the multiresolution analysis that's displaced the windowed FFT across a lot of modern compression and denoising work.

A language that ships `dpss_window` has thought about multitaper spectral estimation specifically. That is the difference between a checkbox stdlib and a working DSP shelf.

Climate and atmospheric physics

The fifth shelf is atmospheric science, and the canonical example is in the registry by name: `adiabatic_lapse_rate_moist`. The moist adiabatic lapse rate is the rate at which a saturated rising air parcel cools — the calculation that determines cloud base height, the energy available to a thunderstorm, and the entire thermodynamic envelope of the troposphere. It is nontrivial precisely because it depends on the latent heat released as water vapor condenses on the way up, which warms the parcel and slows its cooling; that coupling is why it gets a builtin distinct from its dry-air sibling `adiabatic_lapse_rate_dry`, where no condensation happens and the parcel cools at the steeper, constant dry rate.

Around it sits the rest of the atmospheric toolkit: `saturation_vapor_pressure` and `dew_point_magnus / dew_point` for the humidity bookkeeping — the Magnus approximation is the one a forecaster actually uses to get dew point from temperature and relative humidity — `equivalent_potential_temp` and `potential_temperature_step` for the conserved quantities meteorologists track to follow an air mass across pressure levels without being fooled by adiabatic temperature changes, `geopotential_height_full` for the pressure-to-altitude conversion every weather balloon report depends on, and `relative_humidity_step`. The radiative-balance layer — the engine of climate itself — carries `stefan_boltzmann_radiation`, `stefan_boltzmann_luminosity` (alias `stellar_luminosity`), `stefan_boltzmann_constant` (alias `sigma_sb`), `emissivity_grey_body`, and `albedo_blackbody_balance`. Together these are the full energy-in-equals-energy-out calculation: incoming sunlight against outgoing thermal radiation, moderated by albedo and emissivity, which is how you compute a planet's equilibrium temperature from first principles and the starting point for understanding why Earth is warmer than its blackbody temperature.

Machine-learning primitives

A sixth cluster threads through the others: the ML primitive layer. Optimizers ship as single steps — `gradient_descent_step` and `adagrad_step` at the base, plus an `ml_`-prefixed family that covers the modern adaptive optimizers (`ml_adam_step`, `ml_adamw_step`, `ml_adamax_step`, `ml_nadam_step`, `ml_radam_step`, `ml_adagrad_step`) and `ml_batch_norm_step` for the normalization layer. Each is one update step you call inside your own loop, which is the right granularity for a teaching example or a hand-rolled optimizer where you want to see every parameter move. Activations are present — `gelu`, `leaky_relu` (alias `lrelu`), `log_sigmoid`, `hard_sigmoid` (alias `hardsigmoid`), `log_softmax`, `attention_softmax` — including the numerically-stable log-domain variants that matter when a naive softmax would overflow. Loss functions come as `cross_`

`entropy` (alias `xentropy`), `categorical_cross_entropy` (alias `cce`), `ml_binary_cross_entropy`, and `ml_cross_entropy_loss`. These are not a neural-network framework — there's no autograd tape stitching them into a backward pass — but they are the exact numerical kernels you'd otherwise import NumPy to get, available as builtins for the from-scratch optimization a one-liner or a teaching example needs without dragging in a tensor library.

Why a shelf and not a stdlib

The argument for this design is partly about deployment and partly about intent.

The deployment argument is the static binary. Every formula in this chapter is compiled into one ~44 MB executable. There is no version-skew between your DNA-translation function and someone else's; no day where a transitive dependency of your cosmology package breaks your build; no supply-chain question about who maintains the chemistry math. The Nernst equation does not have a release cycle. When you write `nernst_equation`, you are calling code that ships and versions with the language itself, which — for a language whose stated design horizon is decades — is the only honest way to make a domain formula a primitive.

The intent argument is harder to state and more important. A stdlib's domain coverage tells you what its authors thought was generic. Stryke's domain coverage tells you what its author thought was worth making first-class: that text processing and DNA translation and frequency-domain analysis and electrode potentials all belong at the same level of the language as `map` and `print`. That is the reference-shelf thesis. You don't go get the domain knowledge; it's already on the shelf, at arm's reach, in the same binary, callable by name. The breadth isn't decoration — it's the claim that for the work this language is built for, the formula you need is already inside it.

Chapter 16 – Probabilistic Structures and Sketch Algebra

Stryke lets you add two Bloom filters together with `+`.

```
val $both = $alice + $bob
```

Not `bloom_union($alice, $bob)`. Not `$alice.merge($bob)`. Not `$alice |> union($bob)`. The plus sign. The same operator that adds two integers, when its operands are Bloom filters, computes their set union. This is sketch algebra: probabilistic data structures whose merge operation is spelled as the arithmetic operator the merge already behaves like.

What a sketch is, and why the algebra matters

A probabilistic data structure — a *sketch* — trades exactness for size. A Bloom filter answers “have I seen this key?” using a fixed, tiny bit array instead of storing the keys; it can return a false positive but never a false negative. A HyperLogLog estimates how many *distinct* items a stream contained using a few kilobytes regardless of whether the stream had a thousand items or a billion. A Count-Min Sketch estimates per-key frequencies in sublinear space. A t-digest estimates quantiles — the p50, the p99 — of a numeric stream you never fully store. These are the structures that make streaming analytics, deduplication, cardinality estimation, and latency monitoring tractable at scale, and every serious data system uses them internally.

Their defining mathematical property is *mergeability*. Two Bloom filters of the same geometry union by a bitwise OR. Two HyperLogLogs union by taking the per-register maximum. Two Count-Min Sketches combine by pointwise counter addition. Two t-digests merge by combining their centroids. This is what lets you compute a sketch per shard, per hour, per machine, and then fold the partials into a global answer — the entire reason sketches scale horizontally.

The merge is the most important operation a sketch supports — it is the entire reason you reach for a sketch instead of an exact set in the first place. And in every other language it’s a function call with a verbose name buried in a library. The merge is also algebraic in the strict sense: Bloom and HLL union is commutative and associative with an identity (the empty sketch), so a fold of sketches obeys the same laws a fold of numbers does. When an operation satisfies the axioms of a monoid, spelling it as `+` is not a cute analogy — it is naming the operation after the algebra it already has. That is the move stryke makes: mergeability is algebraic, so it *becomes* an operator.

The operator table

Here is the actual sketch-algebra example (`sketch_algebra.stk`), which exercises the feature end to end. Bloom union:

```

val $alice = bloom_filter(1000, 0.01)
val $bob   = bloom_filter(1000, 0.01)
bloom_add($alice, _) for ("alice", "anna", "alex")
bloom_add($bob,   _) for ("bob", "ben", "beth")

val $both = $alice + $bob           # union: who's in either set?
p bloom_contains($both, "alice")    # 1 – in the union
p bloom_contains($bob,  "alice")    # 0 – operands UNCHANGED

```

That last line is the load-bearing detail. The operators are *functional*: `$alice + $bob` produces a new sketch and mutates neither operand. `$bob` still doesn't contain “alice” after the union. This is the difference between sketch algebra and an in-place `merge` call — `+` is an expression that returns a value, so it composes in pipelines and threads without surprise aliasing.

HyperLogLog uses the same `+` for cardinality union:

```

val $hll_a = hll(14)
val $hll_b = hll(14)
hll_add($hll_a, "user:$") for (1:50_000)
hll_add($hll_b, "user:$") for (40_000:90_000)    # 10k overlap

val $union = $hll_a + $hll_b
printf "|A ∪ B| ≈ %d (truth ≈ 90,000)\n", hll_count($union)

```

The HLL union estimates the cardinality of the combined set — about 90,000 distinct users — without ever materializing the union of the two membership sets. That's the whole power of the structure, and `+` is the natural notation for it.

Count-Min counters sum with `+`:

```

val $cms_day = $cms_morning + $cms_evening
printf "/home all day: ≥ %d hits\n", cms_count($cms_day, "GET /home")

```

t-digest centroids merge with `+`, giving you global quantiles from per-region streams:

```

val $global = $td_us + $td_eu
printf "global p99: %.0fms\n", td_quantile($global, 0.99)

```

And the SpaceSaving top-K heavy-hitters sketch merges with `+` to combine per-source heavy hitters into a global ranking.

Where stryke goes furthest is the roaring bitmap, which gets the *full* set algebra — all four operators:

```

val $reds   = roaring()
val $blues  = roaring()
rb_add($reds, _) for (1:10)
rb_add($blues, _) for (5:15)

rb_len($reds | $blues)  # union           (|)
rb_len($reds & $blues)  # intersection  (&)
rb_len($reds ^ $blues)  # symmetric diff (^)
rb_len($reds - $blues)  # difference / andnot (-)

```

A roaring bitmap is a compressed integer set, and integer sets have a genuine Boolean algebra — union, intersection, XOR, and set-minus. Stryke maps each to the operator a mathematician would reach for: `|` and `+` for union, `&` for intersection, `^` for symmetric difference, `-` for andnot. The code reads like set notation because it *is* set notation.

How the dispatch actually works

This is not syntactic sugar bolted onto a special parser. It's operator overloading wired into the VM's arithmetic path, and the implementation is small enough to read in full. In `sketches.rs`, the function `try_sketch_binop` is consulted whenever the VM evaluates `+`, `|`, `&`, `^`, or `-`:

```
pub fn try_sketch_binop(op: SketchOp, a: &StrykeValue, b: &StrykeValue) ->
    Option<StrykeValue> {
    // Bloom + Bloom → union. Also valid for `|`.
    if matches!(op, SketchOp::Add | SketchOp::Or) {
        if let (Some(la), Some(lb)) = (a.as_bloom_filter(), b.as_bloom_filter()) {
            let mut out = la.lock().clone();
            let other = lb.lock().clone();
            if !out.merge(&other) {
                return None; // shape mismatch – fall through to numeric add
            }
            return Some(StrykeValue::bloom_filter(Arc::new(Mutex::new(out))));
        }
    }
    // ... HLL, CMS, TopK, TDigest, Roaring follow the same pattern ...
}
```

The VM (`vm.rs`) calls `try_sketch_binop` *before* it commits to numeric arithmetic on the relevant opcodes — once each for `+` and `-` on the add/subtract path, and again on the bitwise opcodes where `&` maps to `SketchOp::And`, `|` to `SketchOp::Or`, and `^` to `SketchOp::Xor`. The op is passed in as a discriminator so a single function services all five operators. If both operands are the same sketch type and the merge succeeds, the result is a freshly-cloned, merged sketch. If they aren't sketches — or if they're two Bloom filters of incompatible geometry, where `out.merge(&other)` returns false — `try_sketch_binop` returns `None` and the VM falls through to ordinary numeric or bitwise semantics. That fall-through is the safety property: overloading `+` for sketches costs nothing for the overwhelming majority of `+` operations that add two numbers, because the very first thing each branch does is an `as_bloom_filter()` / `as_hll_sketch()` downcast that fails immediately on a non-sketch value and returns `None` before any clone or lock happens.

Three design points fall out of reading this. First, the operators are *functional by construction* — every branch does `la.lock().clone()` before merging, so the operands are never touched and the lock is held only long enough to copy the inner sketch out. Second, the operator-to-semantics mapping is deliberately per-type, not uniform: the Bloom and HLL branches gate on `SketchOp::Add | SketchOp::Or` (both spell union), the CMS, TopK, and TDigest branches gate on `SketchOp::Add` alone (counter-sum, heavy-hitter merge, and centroid merge respectively — there is no sensible `|` or `&` for them), and only the roaring branch `matches` all four set operators. Third, that asymmetry is correct, not a gap: a Count-Min Sketch can't be meaningfully intersected — its counters are an over-estimate, and intersecting two of them produces neither a valid lower nor upper bound — so `&` on two CMS sketches falls through to a numeric error rather than inventing a nonsense answer. The dispatch refuses to define an operation the mathematics doesn't support.

Why no other language does this

Every mainstream language *has* these data structures — as libraries. Python has `pybloom`, `datasketch`, `hyperloglog`. Java has the Apache DataSketches library and Guava's `BloomFilter`. Go has `roaring` and various HLL packages. Redis ships Bloom, Count-Min, t-digest, and top-K as server-side modules. The structures are everywhere.

What is missing everywhere else is the *algebra*. In all of those ecosystems, merging is a method or a function: `bf1.union(bf2)`, `hll1.merge(hll2)`, `RoaringBitmap.or(a, b)`. The merge is spelled as a procedure call for two reasons. First, the language's operators are reserved for its built-in numeric and string types, and in Java and Go there is no operator overloading at all, so `+` on a Bloom filter is a compile error no matter how natural it would read. Second, even in languages that *do* allow overloading — Python with `__add__`, C++ with

`operator+`, Rust with the `Add` trait — the sketches live in third-party libraries, and a library overloading `+` for its own type is possible but rare, because the type it overloads on is one it defined, never the language's own primitive. The result is the same everywhere: the merge is a named call, and the elegant arithmetic spelling is left on the table.

Stryke takes it because the sketches aren't a library. They're builtins, with VM-level value types: `BloomFilter`, `HllSketch`, `CmsSketch`, `TopKSketch`, `TDigestSketch`, and `RoaringBitmap` are `HeapObject` variants behind `StrykeValue` in `value.rs`, each reachable through an `as_*` accessor that the dispatch downcasts on. Because the VM owns both the operators and the sketch types, it can define what `+` means when both operands are sketches — and define it once, at the arithmetic dispatch site every `+` in the language already flows through, so the behavior is uniform rather than per-library. A library can't reach into the language's arithmetic dispatch. The language can.

That is the structural difference, not an unusual API choice. It isn't that nobody thought of writing `$a + $b` for two Bloom filters — it's that in a library-based world the type and the operator belong to different owners, and only the language owns both. The merge being an operator requires the sketch being a primitive. The payoff is code that expresses streaming analytics — fold these per-shard sketches into a global answer — in the notation of arithmetic: `reduce(@shards, |$acc, $s| { $acc + $s })`. A sum of sketches, written as a sum, folding through the same `reduce` you'd use on a list of numbers because to the VM it is the same operation.

Chapter 17 – Persistence: The rkyv KV Store and Friends

Every dynamic language eventually grows a way to put a hash on disk and get it back. Python has `shelve`, which pickles. Ruby has `PStore`, which marshals. Perl has `DBM_File`, bound to BerkeleyDB. Node reaches for `level`, which is FFI bindings to LevelDB. They all work, and they all share one cost: every read parses and allocates. You open the file, you find the bytes for a key, and then you reconstruct a live object — deserialize the pickle, un-marshal the dump, decode the record. The bytes on disk are a *representation* of your data, and turning them back into your data is work you pay on every `get`.

Stryke's KV store does not do that. Its read path is `mmap + validate + cast`. There is no parse. There is no per-row allocation. The bytes on disk are already in the shape the program reads, so a `get` is a pointer into mmap'd memory, not a reconstruction. No other scripting language ships a zero-copy archive store as a core builtin, and the reason it's possible connects the persistence layer to the rest of the runtime.

rkyv as the disk codec

The trick is that the on-disk format *is* an in-memory format. Stryke uses rkyv — a zero-copy deserialization framework — as the codec for the store. An rkyv archive is laid out so that the bytes on disk are already in the shape the program will read; relative pointers and field offsets are baked into the serialized bytes, so you don't deserialize them into a new structure, you cast a pointer into the mmap'd bytes and read fields directly off it. Validation (`check_bytes`, which the store opts into via `#[archive(check_bytes)]` on the wire types) confirms the archive is well-formed and every offset lands in bounds before any access, so a corrupt or truncated file fails fast instead of dereferencing a bad pointer. After that one validation pass, access is pointer arithmetic — no allocation, no copy, no decode.

This is not a novel format invented for the KV store. It's the *same* primitive the stryke runtime already uses for its bytecode cache. The `kvstore.rs` header makes the lineage explicit: `kv_get` is “`mmap + validate + cast` — same code path as `script_cache.rs:454` already uses for cached bytecode.” Stryke compiles your script to bytecode, archives that bytecode with rkyv, mmmaps it on the next run, and executes without re-parsing — that is how a cold start lands in single-digit milliseconds. The KV store is that exact mechanism pointed at user data instead of compiled code. When the runtime's authoritative store for its *own* bytecode is already a zero-copy rkyv archive, exposing the same archive to user scripts as a key-value store costs almost nothing new — the hard parts (the codec, the mmap path, the validation, the atomic rewrite) were already built and exercised on every script run the runtime has ever served.

The storage model is deliberately simple. Per the source, it's “Option 1 — pure rkyv file”: one `KvRoot` archive per store, mmap'd on open, with an in-memory `HashMap` mirror for mutations, and an atomic rewrite on commit. The mirror is what makes a read-mostly store a read-and-write store without abandoning the zero-copy read: clean keys are served straight from the mmap'd archive, while writes accumulate in the in-memory map and shadow the archived values until the next commit. `kv_commit` serializes the whole root to a temp file and renames it into place — the same tmp-plus-rename atomicity (`write_shard_atomic`) the bytecode cache uses. Rename is atomic on POSIX filesystems, so a crash mid-commit leaves the old store fully intact rather than a half-written file; the reader on the next open either sees the complete previous version or the complete new one, never a torn state.

The CRUD surface

The builtins read like SQLite's API ergonomics with none of SQLite's parse cost. The `kvstore_basics.stk` example, lightly condensed:

```
val $db = kv_open("/tmp/users.rkyv")

kv_put($db, "alice", { age => 30, role => "admin" })
kv_put($db, "bob",   { age => 28, role => "user"   })
kv_put($db, "carol", { age => 41, role => "user"   })
p "stored ", kv_len($db), " users"

val $alice = kv_get($db, "alice")      # → hashref, no parse step
p "alice -> ", to_json($alice)

kv_del($db, "bob")
p "still here: ", kv_exists($db, "bob") # 0

p "keys: ", join(" ", kv_keys($db))    # sorted

kv_commit($db)                        # atomic rewrite to disk
val $db2 = kv_open("/tmp/users.rkyv")
p "reopened, found ", kv_len($db2), " users"
```

Note that `kv_put` stores a compound value — a hashref — and `kv_get` returns it as a live stryke hashref ready for field access. Compound values round-trip through a `WireValue` enum whose variants are `Undef`, `Bool`, `Int`, `Float`, `Str`, `Array`, and `Hash` — the subset of `StrykeValue` that makes sense to persist. The runtime's in-memory values are NaN-boxed `Arc<HeapObject>` references, which can't be archived directly, so the store projects them down to `WireValue` at the disk boundary and back to live stryke values on read. `Array` and `Hash` are recursive variants, so a deeply nested structure — a hash of arrays of hashes — persists and reconstructs intact, each level rebuilt as the corresponding stryke ref. Anything outside the representable set is the documented loss boundary, not silent corruption: it either stringifies on the way out or comes back as `Undef`.

The full registry surface, confirmed in the reflection table, covers the operations you'd expect plus the aliases stryke is generous with: `kv_open` (alias `kv_new`); `kv_put` (`kv_set`); `kv_get`; `kv_del` (`kv_delete`, `kv_remove`); `kv_exists` / `kv_has`; `kv_keys`; `kv_scan` for prefix or range iteration; `kv_len` / `kv_count` / `kv_size`; `kv_commit` / `kv_flush` for durability; `kv_batch` for all-or-nothing multi-key writes that land in the mirror together or not at all; `kv_stats` / `kv_info` for the store's metadata (path, entry count, dirty flag, format version, commit count, and the stryke version that wrote it); and `kv_close`. The alias triples aren't padding — they're the same recognition the scientific shelf makes, that `kv_count` and `kv_size` and `kv_len` are the three names different programmers reach for to ask one question, and all three should answer.

Why it beats SQLite on reads

The claim in the source is specific and bounded: the rkyv store “beats SQLite on reads at any store size that fits comfortably in RAM.” That qualifier is doing honest work — it's not a claim that an mmap'd file beats a B-tree at arbitrary scale or under heavy concurrent write load. It's a claim about the read path for a store that fits in memory, and the mechanism makes it credible.

SQLite, on a read, parses SQL (or steps a prepared statement), walks a B-tree, reads a page, decodes a row's serialized column format into values, and hands them back. Even with the page cached in memory, there's a decode step per row. Stryke's `kv_get` mmaps the archive once on open — after which the OS page cache holds the bytes — and then a read is a HashMap-mirror lookup plus a cast into already-laid-out archived bytes. No SQL parse, no row decode, no allocation for the scalar case. For the workload the store is built for — a process that opens a store and reads from it many times, the cache pattern in `kvstore_cache.stk` — the

per-read cost is dominated by a pointer dereference rather than a deserialize.

The source is also honest about the ceiling: an LSM-backed backend (sled or redb) is named as “a v2 swap-in behind the same builtins” for stores too large to hold in RAM, and a `stryke kvd` server is the Phase 2 plan — speaking the same archive bytes over TCP, because the archived `KvRoot` is the wire frame body too. The format is versioned for that future: a `STKV` magic header (`0x53544b56`) and a `KV_FORMAT_VERSION` that bumps on incompatible schema changes, so old stores either upgrade or fail fast rather than mis-cast.

Provenance: value lineage as a primitive

The second persistence-adjacent primitive is `provenance` — automatic value-lineage tracking, which the `provenance_basics.stk` example notes no major scripting language (Perl, Python, Ruby, JavaScript, Lua, PHP) ships. The closest analogs are research dataflow languages (LIO, Adapton) that surface lineage as a type-system feature; stryke ships it as a `mark` / `provenance` / `unmark` builtin trio that any script can call without a type annotation.

```
val $config = mark({ host => "prod", retries => 3 })
val $p = provenance($config)
printf "origin      : %s\n", $p->{origin}
printf "origin_line : %d\n", $p->{origin_line}
```

`mark` tags a heap value, `provenance` retrieves its lineage record — origin call site, origin line, and an ops chain — and `unmark` clears it. Two refs to the same hash share lineage, because tracking is keyed on the value’s `Arc` identity. The cost model is the part that makes it shippable rather than a tax: a process-global `LEDGER_ACTIVE` atomic flag stays false until the first `mark` call, and until then the post-dispatch hook elides through a single inlined `AtomicBool::load` branch. A script that never calls `mark` pays nothing. The use cases are audit trails, “where did this bad value come from” debugging, and asserting a value was produced by an expected sequence of ops — with the honest v1 limit, documented in the example, that builtins returning bare strings lose `Arc` identity, so the idiom is to wrap a tracked string in a container whose stable `Arc` carries the lineage.

Teleport: SHM IPC for the multi-process case

The third primitive is `teleport` / `arrive` — a multi-target shared-memory broadcast. The parent builds one payload and `teleports` it to N forked children at once; each child calls `arrive` to receive it. The mechanism, per `teleport_broadcast.stk`, is one POSIX shared-memory segment that all N readers map, plus a per-receiver UDS (Unix domain socket) datagram notification carrying the shm name and the payload size — the datagram is the wake-up and the address-of-record, the shm segment is the data.

```
val $delivered = teleport($payload, @kids, { hold_ms => 800 })
# in each child:
val $msg = arrive(5000)           # ms timeout; returns the value or undef
val $shard = $msg->{shards}->{ pid() }
```

The win is in the copy count. A naive fan-out serializes the payload and pushes it down N pipes — $N \times \text{bincode-over-pipe}$, N copies into kernel buffers. `teleport` writes the payload into one SHM segment; each receiver mmap’s the same backing pages. For payloads in the 100KB-and-up range, only one process pays the copy-into-kernel-memory cost and the notification is a 40-byte datagram per receiver — cheap even for hundreds of workers. The top-level container arrives as a ref (`$msg->{key}`, `$msg->[i]`), matching `decode_json`’s convention, and nested structures come back as refs too, so the received value navigates exactly the way it was constructed on the sending side.

Taken together — the rkyv KV store, provenance lineage, and teleport SHM IPC — the persistence layer reflects the same thesis as the scientific shelf and the sketch algebra: capabilities that live in third-party libraries everywhere else are primitives here, and being primitives lets them do things a library can’t. A library can’t

reuse the runtime's own zero-copy bytecode codec for user data. A library can't add a near-zero-cost lineage hook to the VM's dispatch loop. A library can't broadcast a stryke value across forked stryke processes through shared memory. The runtime can, because it owns the pieces.

Chapter 18 — AI as a Primitive

The design document for stryke's AI layer opens with one sentence that contains the entire thesis: “`ai` is to stryke what `print` is to every other language: a builtin, two letters, ubiquitous, unlimited power.”

That is the bet. Every other language ships AI as a library — `import openai, from anthropic import Anthropic, npm install langchain`, then a page of client construction, schema definition, and tool-wiring before you can ask a model a question. Stryke ships AI as a primitive. `ai` is a builtin, always present, no import, two letters because — like `print` — it gets typed thousands of times. This chapter walks the surface that's actually shipped, grounded in `docs/AI_PRIMITIVES.md` and the runtime it documents.

Three forms, one primitive

The `ai` builtin has three syntactic forms, all compiling to the same underlying call:

```
ai "summarize this", $document      # function call
~> $document ai "summarize this"    # thread macro
$document |> ai "summarize this"     # pipe
```

The thread-macro form is the one that fits stryke's design lineage — the `~>` operator that threads a value through a chain of stages, here threading a document into an `ai` stage, so an AI step composes into a pipeline exactly like any other transform rather than interrupting it with a client-construction block. All three forms produce the same result: a prompt string plus optional context, run through the configured provider, returned as a `Str` in scalar context or a `Stream<Str>` in iter context. Configuration rides along as named options on the same call — `model`, `system`, `max_turns`, `cache`, `timeout` — so the simple form and the complex form are the same primitive with more arguments, not two different APIs you graduate between. The design doc states this as a rule: the simple form is `ai $prompt`, and the complex form composes from the same primitive with no separate “advanced API.” That matters because the usual library shape forces an early decision — quick one-shot call versus the “real” client object — and stryke refuses to make you choose, because the two-letter call and the fully-configured agent are one builtin.

What `ai` does by default is the important part: it auto-registers every in-scope `tool fn` as a callable tool, auto-attaches every connected MCP server, and runs an agent loop to completion — tool call, tool result, next call, repeat, until the model returns a final answer or `max_turns` is reached. Bare `ai $prompt` is single-shot; the moment tools are in scope, the same `ai $prompt` becomes an agent without changing a character of the call. The bare form is `ai_agent.run($prompt)` with sensible defaults — same primitive, the ergonomics shift based on what's in scope, not on which API you reached for.

`tool fn`: schema from signature

The ceremony that AI frameworks impose is the tool definition. In the current TypeScript or Python practice you define a function, then separately re-define its JSON schema by hand, then re-state the description inside that schema, then wire it into a `tools=[...]` array. Three restatements of one fact. Stryke collapses that to a single declaration:

```

tool fn weather($city: string) "Get current weather for a city" {
    "sunny in $city"
}

tool fn add_nums($a: int, $b: int) "Add two integers" {
    $a + $b
}

# bare ai() auto-attaches every tool fn defined in scope:
val $r = ai("what's the weather in Tokyo?")

```

At build time, the signature becomes the JSON schema (each `$param: type` annotation becomes a typed schema property), the docstring becomes the model-visible description, and the function body becomes the tool's implementation. All `tool fn` definitions in scope are auto-registered. The doc is explicit that this is **shipped**: a source-level pre-pass in `ai_sugar.rs` rewrites `tool fn NAME(args) "doc" { body }` into an ordinary `fn NAME` plus an `ai_register_tool("NAME", "doc", +{ ... }, &NAME)` call. The rewrite binds the params as named locals at the top of the body — it injects the equivalent of `my $city = $__args_ _->{city}` so that inside the body `$city` refers to the argument exactly as written, even though the model actually delivers tool arguments as a single JSON object. The author writes a normal typed function; the pre-pass handles the unpacking. The optional `-> Return Type` is parsed and used purely as documentation — the model never sees a return type, so it carries no schema weight.

The collapse here is three restatements down to one. The JSON schema, the description, and the implementation are not three things you keep in sync; they are three projections of one declaration, generated at build time from the same source. A drift bug — where the hand-written schema says `int` but the function expects a string — is unrepresentable, because there is no second copy to drift from.

For anyone who'd rather not use the keyword, the runtime registry it desugars to is itself a public surface — `ai_register_tool($name, $desc, +{params}, sub {...})`, with `ai_unregister_tool`, `ai_clear_tools`, and `ai_tools_list` for inspection and management at runtime rather than only at definition. Either way, bare `ai($prompt)` auto-routes to the agent loop the instant there's a tool to call.

Collection ops: AI as control flow

The most distinctive part of the surface treats AI not as an API call but as a control-flow primitive over collections:

```

@docs.ai_filter "is about cooking"
@articles.ai_map "summarize in one sentence"
@candidates.ai_sort by: "relevance to backend engineering"
@items.ai_classify into: ["urgent", "normal", "ignore"]
@names.ai_dedupe "treat misspellings as the same person"

if $email ai_match "spam" { discard }

```

The critical design property is cost: each of these compiles to a *single batched LLM call* across the whole collection, not one call per item. `ai_filter @[1000]` is one prompt asking the model for a JSON array of 1,000 booleans, not 1,000 round-trips. That distinction is the difference between an operation you can run and one you can't afford — at a fraction of a cent per call, a thousand round-trips is real money and real latency, and the batched form collapses both by an order of three. The design makes the cheap path the default path, which is the only honest way to put an LLM behind a `map`-shaped API: if the natural-looking call quietly fanned out to N requests, the ergonomics would be a trap. The doc is blunt that this is cost-conscious by construction and that an `ai_*` op inside a `for` loop in production is a lint — the compiler statically tracks predicted cost when the prompt is a constant and warns on hot loops, so the expensive mistake is caught at build time, not on the invoice. The shipped table (Phase 3, marked **SHIPPED**) lists `ai_filter` (boolean per item), `ai_map` (string per item), `ai_classify` (label per item into a fixed set), `ai_match` (a single boolean for one item, the form that reads naturally inside an `if`), `ai_sort` (a best-first reordering), and `ai_dedupe` (collapse near-duplicates by a semantic hint). Each builds one batched prompt and forgivingly parses the

first balanced [...] out of the response, so the model can wrap its answer in explanatory prose — as instruction-tuned models tend to — without breaking the parse. The forgiveness is deliberate: a brittle parser that demanded a bare JSON array would fail on exactly the well-behaved, chatty responses these models produce by default.

Here is the actual collection pipeline from `ai_collection_ops.stk`, run in mock mode so it's deterministic:

```
val $ref = \@reviews

$ref |> ai_classify("sentiment", into => ["Positive", "Negative"])
|> e { p "Sentiment: $_" }

$ref |> ai_filter("constructive")
|> e { p "Constructive: $_" }

$ref |> ai_map("summarize in 3 words")
|> e { p "Summary: $_" }
```

Three semantic operations over a list of reviews — classify sentiment, filter to constructive ones, summarize each — each a single batched call, composed in a pipe chain like any other collection transform.

MCP: server and client, both shipped

The Model Context Protocol surface is native on both sides. As a client (`mcp.rs`, Phase 2, **shipped**), `mcp_connect("stdio:CMD ARGS...")` spawns a subprocess, runs the `initialize` + `notifications/initialized` handshake, and returns a handle; `mcp_tools` / `mcp_resources` / `mcp_prompts` enumerate it (cached), `mcp_call($h, $name, +{args})` invokes a tool, and `mcp_attach_to_ai($h)` marks the handle so the agent loop pulls its tools automatically — no re-registration. Stdio is the proven transport, and the HTTP streamable transport is also shipped (`mcp_connect("https://api.github.com/mcp")`), with SSE responses and `mcp-session-id` carried across calls; WebSocket is the one transport still pending.

The server side ships too. The declarative `mcp_server "name" { tool ... }` block desugars through the same source-level pre-pass as `tool fn` — into a set of private `fn _mcp_name_...` declarations plus an `mcp_server_start` call carrying their schemas — and the runtime stands up a stdio JSON-RPC loop exposing `initialize` / `tools/list` / `tools/call`. The symmetry is the point: a tool you'd expose to *your own* agent via `tool fn` and a tool you'd expose to *another* process via `mcp_server` are written the same way, because they desugar through the same machinery. The doc records this round-tripped end to end — a stryke client connecting to a stryke server, tools enumerating, calls returning — so the loop isn't a one-directional claim. The same binary that runs your script can *be* an MCP server, which means a stryke tool is reachable by any MCP client in any language, not just by stryke's own agent loop.

A real walkthrough: the RAG agent

The `ai_agent_rag.stk` example ties the pieces together into an agentic retrieval-augmented-generation loop, and it runs network-free because of `ai_mock`. The mocks are installed first so the example is deterministic in CI:

```
ai_mock_install('(?!i)^doc:.*', "Stryke info: Created by Menke Technologies.")
ai_mock_install('(?!i)^web:.*', "Tokyo weather: Sunny, 22°C.")
ai_mock_install('(?!i)Who created Stryke.*weather in Tokyo',
  "Stryke was created by Menke Technologies. The weather in Tokyo is sunny, 22°C.")
```

Then two tools are declared with `tool fn` — one searching local docs, one searching the web:

```

tool fn search_docs($query: string) "Search our local documentation for relevant info" {
  p " [TOOL] searching docs for: $query"
  ai "doc:$query"
}

tool fn web_search($query: string) "Search the web for real-time information" {
  p " [TOOL] web search for: $query"
  ai "web:$query"
}

```

And the agent runs as one `ai` call. Because the two `tool fn`s are in scope, the bare-looking call is actually an agent loop:

```

val $question = "Who created Stryke and what is the weather in Tokyo?"
val $answer = ai($question,
  system => "You are a helpful research assistant. Use the provided tools to find
    ~ accurate information.",
  max_turns => 5
)
p "Answer: $answer"

```

The loop, as the example's own comments narrate it: the model recognizes it needs the origin of Stryke and calls `search_docs`; it recognizes it needs Tokyo's weather and calls `web_search`; it synthesizes both tool results into a final answer. Two tool declarations, one `ai` call, a complete agent — and `STRYKE_AI_MODE=mock-only` makes it a deterministic test rather than a flaky network call. The doc notes that CI runs `s test` with that mode set, so any unmocked live AI call fails the build.

What's actually shipped

The design doc is unusually candid about status, and the honest summary is that the surface is far past a skeleton. The Phase 0 walking skeleton (`ai`, `prompt`, `stream_prompt`, `chat`, `embed`, `tokens_of`, `ai_cost`, the cache builtins, `ai_mock_install` / `ai_mock_clear`) is shipped. The agent loop is shipped, both with explicit `tools => [...]` and via the auto-registering `tool fn` keyword. The MCP client is shipped (stdio + HTTP), the `mcp_server` declarative block is shipped, and the collection builtins are shipped with batching.

Beyond the core, the doc marks a long list of capabilities **SHIPPED** that fill out a production AI surface: structured output (`ai($p, schema => +{...})` validating and coercing to a hashref), Anthropic prompt caching (`cache_control => 1`, ~10% input cost on repeat), extended thinking (`thinking => 1` with `ai_last_thinking()`), PDF and image and vision input (`pdf =>`, `image =>`), real `Stream<Str>` streaming with `on_chunk`, SQLite-backed RAG memory (`ai_memory_save` / `ai_memory_recall`), vector ops (`vec_cosine`, `vec_search`, `vec_topk`), conversational sessions, the Anthropic batch API at ~50% cost, retry/backoff, scoped budgets (`ai_budget`), and a five-provider matrix — Anthropic, OpenAI, Ollama, OpenAI-compatible local servers, and Google Gemini — with Voyage as the default embedding provider.

What's explicitly *not* done is named with the same candor: in-process linked llama.cpp is deferred (shell-out via Ollama or LM Studio is the supported local path), WebSocket MCP transport is pending, and the effects/capabilities integration (`Effect::AI`, `AIcap`) is future work. That honesty is the point — the doc doesn't claim a primitive is shipped when it isn't, and the ones it does claim are pinned to a runtime file and, where it matters, a verified round-trip.

The throughline back to the rest of this part of the book: AI is a primitive here for the same structural reason the sketches and the KV store are. A library bolts AI onto a language from the outside, and pays for it — manual schemas, manual tool wiring, manual batching. Stryke builds AI into the language from the inside, and the payoff is the collapse of ceremony: a tool is a function with one keyword, a batched judgment over a thousand items is one method call, an MCP server is one block, and an agent is one `ai`. Two letters, the doc says. Like `print`.

Chapter 19 – From Source to Bytecode

Every stryke program makes the same journey. It begins as a stream of UTF-8 bytes — a file on disk, a `-e` argument, a heredoc piped into the process — and ends as native machine code running on the silicon in front of you. Between those two points sit four transformations, each owned by a single Rust module, each handing a well-defined data structure to the next. There is no improvisation in this pipeline and no shortcut around it. Source becomes tokens, tokens become an abstract syntax tree, the tree becomes a `Chunk` of bytecode, and the bytecode is interpreted — and then, for the code that runs often enough to deserve it, compiled. This chapter is about the first three steps: the front end that turns text into something the machine can execute.

The front end matters more than it usually gets credit for. A language's reputation is made at runtime — benchmarks, parallelism, the JIT — but every nanosecond the runtime saves can be squandered if the front end is sloppy about what it hands over. stryke's front end is built on one conviction: decisions that can be made once, at compile time, must never be made repeatedly, at run time. The whole architecture downstream depends on that conviction being honored here, at the start.

It is worth being precise about the boundary the pipeline draws, because the boundary is the architecture. The lexer, the parser, and the compiler form a unidirectional cascade: text flows into the lexer and never flows back; a `Vec<Token, usize>` flows into the parser and never flows back; a `Program` flows into the compiler and never flows back; a `Chunk` flows into the runtime and never flows back. No stage holds a reference to the stage before it once its input has been consumed, and no stage can ask its predecessor to do more work. This is not a stylistic choice. It is what makes the rkyv cache of Chapter 21 possible at all — a `Chunk` is a self-contained artifact, severed from the source that produced it, so it can be archived to disk and reloaded on the next run with the lexer and parser never running. If any runtime decision still depended on the original text, that severance would be impossible and the cache could not exist. The pipeline's one-way shape is the precondition for skipping the pipeline.

The lexer

The lexer lives in `strykelang/lexer.rs` — about 4,300 lines of it. Its public entry point is `tokenize`:

```
pub fn tokenize(&mut self) -> StrykeResult<Vec<Token, usize>> {
    let mut tokens = Vec::new();
    loop {
        let tok = self.next_token()?;
        let line = self.token_start_line;
        if self.last_was_bare_positional {
            self.bare_positional_indices.insert(tokens.len());
        }
        if tok == Token::Eof {
            tokens.push((Token::Eof, line));
            break;
        }
        tokens.push((tok, line));
    }
    Ok(tokens)
}
```

The signature tells you the shape of the output: a vector of `(Token, usize)` pairs, where the `usize` is the source line the token started on. That second element is not decoration. stryke reports errors against real

line numbers, and the line a token belongs to is not always the line the lexer's cursor happens to be on when it emits the token — a POD block or a heredoc can advance the line counter by many lines before the real token appears. So the lexer stamps `token_start_line` at the point the token actually begins and carries that value out, rather than snapshotting `self.line` after the fact. This is the kind of detail that distinguishes a lexer that was written once from a lexer that has survived contact with real programs.

There is a second piece of state the lexer carries that a C-family tokenizer never needs: `bare_positional_indices`, the set of token positions where a bareword was used as an implicit closure parameter. The lexer tracks `last_was_bare_positional` as it goes and records the index into `bare_positional_indices` whenever the flag is set. This exists because stryke's implicit closure parameters — `_`, `_1`, `_2`, the calling card of the language — are lexically ambiguous with ordinary identifiers, and the disambiguation has to be made at the point the token is emitted, not reconstructed later. The lexer is doing parser-grade work here, settling a question of meaning rather than merely of shape, because the alternative is to push that ambiguity downstream where it would have to be resolved with less context.

The heavy lifting happens in `next_token`, which is where stryke's Perl heritage shows. A naive lexer for a C-family language can get away with a fixed character-class table. stryke cannot, because a sigil-prefixed identifier, a quote-like operator (`q`, `qw`, `m`, `qr`, `s`, `tr`), a regex literal, a triple-quoted interpolating string, and a bareword that merely *looks* like one of those constructs all have to be disambiguated by context. The canonical hard case is the letter `s`: standing alone it is the substitution operator `s/old/new/`; followed by `(` in a call position it is a function name; inside an interpolating string it is a literal character. Nothing about the byte `s` decides which — only the surrounding context does, and the lexer is the layer that holds that context. The lexer's test suite — over a hundred unit tests starting at `tokenize_empty_yields_eof` — exists precisely because each of these forms is a small parser in its own right: `tokenize_q_paren_balances_nested_parens_in_prototype`, `tokenize_triple_quoted_with_embedded_quotes`, `tokenize_double_string_braced_hex_unicode_escape`, `tokenize_spaceship_operator`. Each name is a hazard someone hit and pinned. The lexer resolves all of this before the parser ever sees a token. By the time `parse_program` runs, the ambiguity of “is `s` a function name or a substitution operator” is already settled, and the parser can treat its token stream as a sequence of unambiguous facts rather than a second tier of guesswork.

The parser

The parser is `strykelang/parser.rs`, and at roughly 23,000 lines it is the largest single file in the front end — larger than the compiler, larger than the VM. That size is a direct consequence of the surface area stryke chose to cover. It is not a clean little grammar; it is a recursive-descent parser that has to accept the full Perl 5 expression grammar plus stryke's own extensions — pipeline operators, thread macros, implicit closure parameters, `val/var` bindings, expression-bodied functions, algebraic `match`, classes and traits.

The entry point is `parse_program`:

```
pub fn parse_program(&mut self) -> StrykeResult<Program> {
    let mut statements = self.parse_statements()?;
    if !self.pending_synthetic_subs.is_empty() {
        let synthetics = std::mem::take(&mut self.pending_synthetic_subs);
        let mut combined = Vec::with_capacity(synthetics.len() + statements.len());
        combined.extend(synthetics);
        combined.append(&mut statements);
        statements = combined;
    }
    Ok(Program { statements })
}
```

The output is a `Program` — a flat vector of `Statement` nodes. `parse_statements` loops until end-of-file, dispatching each statement through `parse_statement`, which is where labels (`FOO:`), declarations, control flow, and bare expressions get sorted out. The `pending_synthetic_subs` dance at the top handles a Perl-compatibility wrinkle: anonymous overload handlers register synthetic, package-qualified subroutine stubs that have to appear *before* the body so the overload table resolves them at runtime. The parser prepends

them. This is the parser doing bookkeeping that the rest of the pipeline can then take for granted — a recurring pattern in how stryke front-loads work.

The parser is also where `--no-interop` enforcement lives, and it is worth seeing how literal that enforcement is. When no-interop mode is active and the parser encounters the `sub` keyword, it does not desugar or warn — it refuses:

```
"stryke uses `fn` instead of `sub` (--no-interop is active)",
```

The same goes for `say` (“stryke uses `p` instead of `say`”), for `wantarray`, for `$a/$b` outside sort blocks, and for a bare-integer tail expression in module position (the Perl 1; ritual). These rejections, scattered through `parser.rs` at the points where each construct is recognized, are the bot firewall described in Chapter 22. The important architectural fact here is that they happen *at parse time*. A no-interop violation never reaches the compiler. The syntax tree that comes out of `parse_program` under `--no-interop` is, by construction, free of Perl-isms.

The compiler

The compiler is `strykelang/compiler.rs`. Its job is to walk the `Program` and emit a `Chunk` — the bytecode container — through `compile_program`:

```
pub fn compile_program(mut self, program: &Program) -> Result<Chunk, CompileError> {
```

It does this in passes, and the passes reveal how much the compiler resolves ahead of time. The first thing it does is extract phase blocks — `BEGIN`, `UNITCHECK`, `CHECK`, `INIT`, `END` — into separate vectors, because those run at distinct points in the program lifecycle and cannot simply be threaded into the main instruction stream. Then comes a forward-declaration pass that walks every statement, tracks the current `package`, and registers each subroutine’s qualified name in the chunk’s name pool *before* compiling any bodies:

```
StmtKind::SubDecl { name, .. } => {
    let q = Self::qualify_sub_decl_pass1(name, &pending_pkg);
    let name_idx = self.chunk.intern_name(&q);
    self.chunk.sub_entries.push((name_idx, 0, false));
}
```

This is why a stryke program can call a subroutine defined later in the file — the same forward-reference freedom Perl has. The compiler resolves the name once, into an interned index, so the runtime never does a string lookup to find the sub. Only after this pass does the compiler filter out the declarations and phase blocks and compile the main body, arranging for the last expression statement to leave its value on the stack so the program has an implicit return value, exactly as Perl’s implicit return works.

Name interning, via `intern_name`, is the clearest example of the compile-time-once principle in action. Variable names, subroutine names, package-qualified stash keys — they all go through a deduplicating intern table and come out as `u16` indices. The runtime carries indices, not strings. Every place the VM needs a name, it has a number.

The payoff of interning compounds with every other compile-time decision. A string lookup in a hash on a hot path costs a hash of the key, a probe, and a comparison against the stored key — small in isolation, ruinous in a loop that resolves the same name a million times. An interned index costs an array dereference. The compiler pays the hashing cost once, when it interns; the runtime pays the dereference cost on every access, and never re-hashes. This is the same trade the no-tree-walker invariant makes at the level of execution engines and the rkyv cache makes at the level of whole runs: identify the work that is being redone, move it to the earliest layer that can do it, and make the later layers consume a settled result. The `u16` width is itself a decision — sixty-five thousand distinct interned names is far past anything a real program reaches, so the index fits in two bytes, and two-byte indices keep the op stream and the `sub_entries` table compact, which keeps them cache-friendly when the VM walks them.

The Chunk and the Op set

The output of all of this is the `Chunk`, defined in `strykelang/bytecode.rs`:

```
pub struct Chunk {
    pub ops: Vec<Op>,
    #[serde(with = "crate::script_cache::constants_pool_codec")]
    pub constants: Vec<StrykeValue>,
    pub names: Vec<String>,
    pub lines: Vec<usize>,
    pub op_ast_expr: Vec<Option<u32>>,
    pub ast_expr_pool: Vec<Expr>,
    pub sub_entries: Vec<(u16, usize, bool)>,
    pub blocks: Vec<Block>,
    pub block_bytecode_ranges: Vec<Option<(usize, usize)>>,
    pub static_sub_calls: Vec<(usize, bool, u16)>,
    // ... and more
}
```

The `Chunk` is the boundary between the front end and the runtime, and its layout is built for the runtime's convenience. `ops` is the instruction stream. `constants` is the literal pool — string literals, regex patterns, the values referenced by index from a `LoadConst` op. `names` is the interned name pool the compiler filled. `lines` is a parallel array, one entry per op, so the VM can report the source line of any instruction without storing line numbers inline in the hot instruction stream. `sub_entries` records, for each compiled subroutine, its name index, its entry instruction pointer, and whether it uses the stack-args calling convention — `(u16, usize, bool)`. `blocks` and `block_bytecode_ranges` hold the bodies of `map { }`, `grep { }`, `sort { }`, and the parallel builtins, lowered to their own bytecode ranges so they execute on the same VM path as everything else rather than through a separate interpreter.

The `Op` enum itself is the instruction set, and it is large — the source defines 348 variants. Most are what you would expect: load a constant, get a scalar slot, call a sub, add two numbers, jump if false. But a meaningful fraction of them exist only because the compiler refuses to leave performance on the table. These are the fused superinstructions, and they are the subject of the next chapter's first section — but they originate here, in a peephole pass over the freshly emitted ops. Sequences like a slot-relative compound assignment followed by a discard collapse from two ops into one:

```
Op::AddAssignSlotSlot(d, s) => Some(Op::AddAssignSlotSlotVoid(*d, *s)),
```

and entire counted-loop bodies — `while $i < limit { $sum += $i; $i += 1 }` — fuse down to a single op like `AccumSumLoop` that the VM runs as a tight native Rust loop with no per-iteration dispatch. The compiler recognizes the shape; the VM gets a single instruction. Once again: the decision is made once, at compile time, and the runtime simply executes the conclusion.

The reason an instruction set grows to 348 variants rather than staying at the few dozen a textbook stack machine needs is exactly this fusion strategy. A minimal opcode set is easier to write and slower to run: every operation decomposes into generic primitives, and each primitive pays a fetch-and-dispatch on every step. `stryke` goes the other way. It spends opcode-space — the design budget that worries about how wide the `Op` enum gets — to buy dispatch cycles. Every fused superinstruction is one entry in the enum and one arm in the VM's match, and in exchange it erases several iterations of fetch-decode-dispatch from a loop that may run millions of times. The peephole pass is where that trade is struck: it runs over the freshly emitted ops, recognizes the small set of loop and assignment shapes that dominate real code and benchmarks, and rewrites each into its fused form. A wider instruction set is the deliberate cost of a faster interpreter, and it is paid in the front end where it costs nothing at run time. This is the through-line of the whole chapter stated in opcode terms: the compiler does the recognizing so the VM only has to do the executing.

That is the whole front end. Bytes in, a `Chunk` out — interned names, a literal pool, a parallel line table, pre-lowered blocks, and an instruction stream that has already been peeled of its obvious redundancies. The runtime that picks up this `Chunk` never re-tokenizes, never re-parses, never re-resolves a name to a string.

Everything that could be settled before the program ran has been settled. What is left is execution.

Chapter 20 – The Virtual Machine and the JIT

The `Chunk` arrives at the virtual machine and the program finally runs. This chapter is about how — about the dispatch loop that walks the bytecode, the superinstructions that let it skip work, the invariant that there is no other way for code to execute, and the layers of just-in-time compilation that turn the hottest stryke code into native machine instructions. Two distinct JIT systems are in play, and they are not the same thing: a shared three-tier Cranelift JIT inside the fusevm runtime that stryke rides on for the numeric core, and stryke's own two-tier method JIT for everything else. Keeping them straight is the spine of the chapter — they have different source files, different tier counts, different responsibilities, and different relationships to the rest of the stack. The distinction matters because the two systems answer two different questions: how does pure-integer arithmetic reach native code, and how does string-and-collection code reach native code. The answer is different machinery in each case. This chapter is the longest in this part of the book because it is where stryke earns its benchmark numbers.

The dispatch loop

The VM is `strykelang/vm.rs` — about 11,700 lines — and its beating heart is `run_main_dispatch_loop`:

```
fn run_main_dispatch_loop(
    &mut self,
    mut last: StrykeValue,
    op_count: &mut u64,
    init_dispatch: bool,
) -> StrykeResult<StrykeValue> {
```

The loop's structure is conventional in the best sense — this is what a fast bytecode interpreter looks like. It caches raw pointers to the three pools it touches on every iteration so the borrow checker doesn't force a re-borrow each time around:

```
let ops_ref: &Vec<Op> = &self.ops;
let ops = ops_ref as *const Vec<Op>;
let ops = unsafe { &*ops };
```

Then it spins: fetch the op at the current instruction pointer, dispatch on its variant through a large `match`, mutate the VM state, advance the pointer. The `match` arm for each `Op` variant is the implementation of that instruction. 348 arms, one per opcode — a `match` this wide compiles, on the architectures Cranelift and rustc target, to a jump table: the op's discriminant indexes directly into an array of arm addresses, so dispatch is one indirect branch regardless of how many variants exist. The width of the instruction set, which cost opcode-space in the front end, costs nothing here; the table is the same depth whether it has forty entries or three hundred and forty-eight. There is a hard ceiling — `const MAX_OPS: u64 = 1_000_000_000 (vm.rs:3639)` — as a runaway guard against an infinite loop in generated or eval'd code, and there are a handful of break conditions at the top of the loop for the cases where control has to leave the main loop cleanly: a JIT trampoline returning, a block region completing, the instruction pointer reaching the end.

Worth dwelling on is the choice to cache those pool pointers behind `unsafe`. The borrow checker cannot prove that `self.ops` won't be mutated while the loop holds a reference into it, because in the general case a stryke program *can* mutate its own instruction stream — `eval` of a string compiles new bytecode. The pointer

cache says, in effect, “for the duration of this dispatch the ops are stable,” and the loop’s break conditions are what make that promise true: any operation that would invalidate the cached view leaves the loop first. This is the kind of locally-unsafe, globally-sound construction that separates a fast interpreter from a merely correct one. The alternative — re-borrowing the pools on every iteration to satisfy the checker — would put a bounds-checked indirection on the hottest line of code in the entire runtime.

This is the only execution path. There is no fallback interpreter that walks the AST. That fact is not incidental; it is enforced, and the enforcement is worth a section of its own.

The no-tree-walker invariant

A common shape for a scripting-language implementation is to have two execution engines: a fast bytecode VM for the common case and a slower tree-walking interpreter that handles the awkward constructs the compiler never learned to lower. The trouble with that arrangement is that the two engines drift. They resolve names differently, they scope variables differently, and a feature that works on one path subtly misbehaves on the other. *stryke* had exactly this bug once, and the way it was fixed became a permanent architectural constraint.

The bug lived in aspect-oriented advice. The first implementation ran advice bodies through `Interpreter::exec_block` — a tree-walker — and that path did not carry the bytecode compiler’s *our*-qualified name resolution. The symptom: `$count` inside an advice body wrote to a storage key named `count`, while the surrounding code wrote to `main::count`. Two different variables that should have been one. The fix was to lower each advice body to bytecode at compile time and dispatch it through `run_block_region`, the same VM helper that runs `map { }` and `grep { }` blocks.

To make sure that fix can never be quietly undone, *stryke* ships `tests/tree_walker_absent_aop.rs`. The test is not a behavioral test; it is a *source-level* test. It reads `vm.rs`, locates the body of `dispatch_with_advice` by balanced-brace scanning, strips comments, and asserts that the body contains no call to any tree-walker entry point:

```
let needles = [
  "interp.exec_block",
  "interp.exec_block_with_tail",
  "interp.exec_block_no_scope",
  "interp.exec_block_no_scope_with_tail",
  "interp.exec_block_smart",
];
for needle in &needles {
  assert!(
    !body.contains(needle),
    "dispatch_with_advice still calls `{}` - AOP advice bodies must run \
      through the bytecode VM (run_block_region), not the tree-walker.",
    needle
  );
}
```

A companion test asserts the *positive* invariant — that `dispatch_with_advice` *does* call `run_block_region` — and a third checks that the `RuntimeAdviceDecl` still carries a `body_block_idx` so the VM knows which compiled region to dispatch. The triple matters: the negative test alone could be satisfied by a dispatch path that simply does nothing, and the positive test alone could be satisfied by a path that calls `run_block_region` for show but still falls through to a tree-walker for the real work. Pinned together, they describe the only acceptable shape — advice resolved at compile time to a block index, dispatched at runtime through the same bytecode helper as `map`, with no escape hatch back to AST interpretation.

The body of `dispatch_with_advice` itself shows why the resolution has to be compile-time. Advice runs against a live call: it pops the operands, resolves the matching intercepts by glob pattern, and declares context variables — `INTERCEPT_NAME`, `INTERCEPT_ARGS` — that the advice body can read, mirroring *zshrs*’s intercept-name and intercept-args convention. Those context variables are scalars and arrays in the same scope the advice body executes in. If the body ran on a separate engine with separate name resolution,

`INTERCEPT_NAME` inside the advice would resolve to a different storage key than the one the dispatcher just declared — which is precisely the class of bug the `our`-qualified `$count` incident was. One engine, one name resolver, one truth. The file's own doc comment calls it “the immune system,” and names its lineage: it is a mirror of zshrs's `tests/tree_walker_absent.rs`, the test that pinned the deletion of zshrs's tree walkers when those constructs moved onto the fusevm bytecode path. The pattern is a class of test whose job is to fail loudly if anyone reintroduces an architecture the project has deliberately deleted. You do not get to add a second execution engine back. The test will catch you.

That single-engine discipline is what makes the rest of this chapter possible. Because every construct lowers to the same bytecode and runs through the same loop, the JIT only ever has to understand one thing — bytecode — to accelerate all of it.

Fused superinstructions

The first speedup the VM gets is free, in the sense that it costs no JIT compilation: the compiler has already replaced common multi-op sequences with single superinstructions, and the dispatch loop simply has an arm for each. A loop body like `$sum += $i; $i += 1; jump-if-less` is, in unfused form, five or six separate opcodes — get slot, load int, compare, jump, get slot, add, set slot — each paying the cost of a fetch and a `match` branch on every iteration. The peephole pass in `bytecode.rs` recognizes the pattern and emits one op, `AccumSumLoop`, whose VM arm runs the entire loop as a native Rust `while` with the iteration variables held as plain integers. No stack traffic, no per-iteration dispatch.

There is a family of these: `AddAssignSlotSlotVoid` for slot-to-slot compound assignment with no stack push, `SlotIncltIntJumpBack` for the increment-compare-branch backedge, fused variants for counted string-append loops (`$s .= CONST`), array-push loops (`push @a, $i`), hash-insert loops (`$h{$i} = ...`), and the hash-sum inner loop (`for my $k (keys %h) { $sum += $h{$k} }`). Each one targets a loop shape that shows up constantly in real code and in benchmarks, and each one collapses a dispatch-heavy inner loop into straight-line native arithmetic. This is interpreter-level optimization — it happens before any JIT — and it is a large part of why stryke is fast even on code that never gets hot enough to compile.

The shared fusevm JIT: three tiers

When code *does* run often enough, just-in-time compilation engages. The first JIT that gets a chance is not stryke's own — it is shared.

stryke is a fusevm frontend. The header of `strykelang/fusevm_bridge.rs` states it plainly: “strykelang is a `[fusevm]` frontend: a conservative, universal-integer subset of its bytecode is translated to `fusevm::Op` and executed on `fusevm::VM`, reusing fusevm's interpreter and three-tier Cranelift JIT for the numeric hot path.” fusevm is the runtime substrate that zshrs also builds on — the bridge “mirrors the awkrs/zshrs integration pattern: each frontend owns its own” extension-op ID space and hands fusevm a subset of work it knows how to JIT.

That JIT has three tiers, not one, and the fusevm crate's own header (`fusevm/src/lib.rs`) lists them:

```
//! - Three-tier Cranelift JIT:
//!   - Linear (straight-line, compile on first call)
//!   - Block (whole-chunk CFG, threshold 10)
//!   - Tracing (hot loop body, threshold 50, full side-exit machinery)
```

The **linear** tier compiles a straight-line, branch-free op sequence on its first call — no warmup, because a branch-free sequence has no loop to get hot and nothing to profile, so compiling it eagerly is always a win. The **block** tier compiles a whole chunk's control-flow graph once it has been invoked ten times (`fusevm/src/lib.rs:9`); ten is low because a chunk that runs ten times will almost certainly run far more, and the block JIT's reward — running the whole chunk “in native code with zero interpreter dispatch,” in the words of `VM::run`'s own doc comment (`fusevm/src/vm.rs:738-741`) — is large enough to justify compiling early. The **tracing** tier is the

elaborate one. At a backward branch in a hot loop — threshold fifty (`fusevm/src/lib.rs:10`) — fusevm arms a recorder, captures the actual ops executed for one trip through the loop body, and compiles *that linear trace* to native code. A trace is speculative: it assumes the values it saw stay the shape it saw them in, and the loop body it records may contain branches whose outcome it has only seen go one way. When an iteration diverges — a branch goes the other way, an integer slot turns out to hold a float — the trace takes a **side exit**: it deoptimizes by reconstructing the interpreter’s value stack and frame from the trace’s recorded state, and hands control back to the interpreter at the exact instruction pointer the trace would have reached, so execution continues correctly as though the trace had never run.

The capability matrix in fusevm’s own header (`fusevm/src/lib.rs:12-24`) enumerates how far that tracing machinery reaches, phase by phase: loop bodies over integer slots with no calls (Phase 1); cross-call inlining of branchless callees (Phase 2); caller-frame `if/else` with side-exits (Phase 3); callee-frame branches with frame materialization on deopt (Phase 4); value-stack reconstruction on deopt for both `Int` and `Float` cells (Phase 5 and 5b); a per-side-exit deopt counter that auto-blacklists a trace that bails too often (Phase 6); persistent trace metadata export and import via `TraceMetadata` (Phase 7); bounded recursion inlining to depth four (Phase 8); and side-trace stitching that compiles a second trace starting from a hot deopt site (Phase 9). Each of these is the kind of feature a production tracing JIT needs and a toy tracing JIT omits — the blacklist is what stops a pathological trace from re-deoptimizing forever, the side-trace stitching is what keeps a loop with two common paths from falling back to the interpreter on the second path, the bounded-depth recursion inlining is what lets a recursive numeric kernel trace without unbounded code growth. None of this is stryke code; stryke inherits the entire pipeline by compiling to `fusevm::Op`. That is the leverage of building on a shared substrate: the tracing JIT is the hardest piece of machinery in the runtime, and stryke did not have to write it, maintain it, or test it — it is the same engine that JITs the shell.

At run time fusevm dispatches these tiers in priority order. Its `VM::run` doc comment (`fusevm/src/vm.rs`) spells out the policy: **block JIT first**, because when a whole chunk is block-eligible it runs in native code “with zero interpreter dispatch”; then the **tracing JIT**, “when block JIT doesn’t apply, the dispatch loop runs with the recorder armed at backward branches”; then the **interpreter** as “fallback for cold code and chunks neither tier handles.” Block-eligible chunks short-circuit before the tracing recorder captures anything, so the two never compete on the same chunk.

The qualifier “conservative, universal-integer subset” is the whole story of how stryke reaches that JIT. fusevm does not try to understand all of stryke’s bytecode. It understands a strict, identifiable subset — integer and float loads, stack shuffles, `+` `-` `*`, unary negate, the comparison operators, slot get/set — code that maps one-to-one onto `fusevm::Op` without any stryke-specific semantics leaking in. The bridge has a predicate that decides, segment by segment, whether a run of stryke ops is admissible:

```
/// True when every op in `seg` is in the strict 1:1 universal-integer/slot subset
```

If a subroutine’s hot segment fits that subset, it is translated to `fusevm::Op`, handed to fusevm’s VM, and run through that three-tier Cranelift pipeline. stryke gets native code for the numeric core for free, riding on the same runtime that hosts the shell. Where stryke needs an operation fusevm doesn’t have natively — Perl’s floored modulo, for instance — it reserves an extension-op ID (`fusevm::Op::Extended(u16, u8)`) and registers a JIT extension that emits native code for that opcode into fusevm’s Cranelift pipeline.

stryke’s own method JIT: two tiers

Code that does *not* fit the universal-integer subset — anything touching strings, arrays, hashes, references, the richer half of the value space — is where stryke’s own JIT takes over. This is a *separate* JIT system from fusevm’s, with its own source and its own tier count: it lives in `strykelang/jit.rs`, about 5,500 lines, and its header describes it as a “**Method JIT** (Cranelift): compiles stack bytecode to native code.” Where fusevm’s JIT has three tiers, stryke’s method JIT has two, and they do not include tracing. The **linear** tier compiles straight-line, branch-free op sequences into a single Cranelift basic block. The **block** tier handles control flow — jumps included — emitting a Cranelift control-flow graph computed by fixpoint merge, where merged slots become block parameters typed `i64` or `f64`. There is no tracing tier and no side-exit machinery in stryke’s own JIT; speculative trace compilation lives only in fusevm, and stryke gets at it only through the universal-integer

bridge.

Both of stryke's tiers compile through Cranelift — `cranelift_codegen`, `cranelift_frontend`, `cranelift_jit`, `cranelift_module` — which means the native code stryke emits targets the same architectures Cranelift targets: x86-64 and aarch64. That covers the daily-driver macOS aarch64 machine and the x86-64 and aarch64 Linux servers in the stack without any per-architecture assembly in stryke itself.

The eligibility rules for what stryke's method JIT will actually compile reveal how much care goes into staying correct under speculation. Both tiers handle integer and floating-point arithmetic, with the int/float promotion the interpreter performs preserved exactly — `jit.rs` widens stack slots at control-flow merges via an abstract `join_cell` so an integer predecessor and a float predecessor can join into a single Cranelift block parameter typed `i64` or `f64` per merged slot. The simple data ops translate directly: `LoadInt`, `LoadFloat`, `Add`, `Sub`, `Mul`, `Negate`, the comparison family, the bit operators with the shift amount masked to six bits. The arithmetic that can fault is gated. `Div` is JIT'd only when the VM would take the exact-integer-quotient path — when `a % b == 0` — because anything else changes the result type. `Mod` is JIT'd only when the divisor is provably never dynamically zero, either a constant non-zero or a folded stack value, so the native code can never hit a divide-by-zero trap. `Pow` is JIT'd only when the VM's integer `wrapping_pow` path applies — exponent in `0..=63` and the operands shaped so the result stays integer. The rule in every case is the same: emit native code only when it provably matches what the interpreter would have computed, and leave the op interpreted otherwise. A JIT that is occasionally wrong is worse than no JIT, so stryke's refuses to compile anything it cannot prove equivalent to the slow path.

The two JIT systems are not independent choices the user makes; they are layered, and stryke's dispatch loop tries them in order. The trigger logic sits right inside `run_main_dispatch_loop`. On reaching a subroutine entry point, the VM bumps that entry's invocation counter and, once the counter passes a threshold, attempts compilation:

```
let should_try_jit = *c > self.jit_sub_invoke_threshold
    && (!self.sub_jit_skip_linear_test(sub_ip)
        || !self.sub_jit_skip_block_test(sub_ip));
if should_try_jit {
    if self.try_fusevm_subroutine()? {
        continue;
    }
    if !self.sub_jit_skip_linear_test(sub_ip) && self.try_jit_subroutine_linear()? {
        continue;
    }
    if !self.sub_jit_skip_block_test(sub_ip) && self.try_jit_subroutine_block()? {
        continue;
    }
}
```

The order is the policy. Try fusevm first — if the subroutine is in the universal-integer subset, the shared runtime handles it. Fall through to stryke's linear JIT, then to stryke's block JIT. If a tier reports it can't compile this code (the `skip` tests cache that verdict so cold code never pays the analysis cost twice), the next tier gets a turn, and if none apply, the interpreter keeps running it. Cold code stays interpreted; only code that crosses the threshold pays Cranelift's compilation cost.

That threshold is `jit_sub_invoke_threshold`, and its default and override are spelled out in the VM source — “Minimum invocations before attempting subroutine JIT. Override with `STRYKE_JIT_SUB_INVOKES` (default 50).” A subroutine is interpreted for its first fifty calls; on the fifty-first, the VM tries to compile it. Fifty is a deliberate balance: low enough that hot loops compile almost immediately, high enough that a sub called a handful of times never wastes Cranelift's effort. The environment variable exists so that benchmarking and debugging can move the line — set it to zero to force compilation on the first call, set it high to keep everything interpreted — without recompiling stryke.

The fusevm migration: toward one runtime

Everything described so far runs on stryke's own machinery: `vm.rs` for the dispatch loop, `jit.rs` for the method JIT, with fusevm reached only through the numeric bridge. That is the present, but it is not the destination. A second path is being built underneath the first, one phase at a time, whose endpoint is the deletion of `vm.rs` and `jit.rs` entirely — about 11,800 and 5,500 lines respectively — and the running of *whole programs* on the shared fusevm runtime. The module that holds it is `strykelang/fusevm_native.rs`, and its header states the goal without hedging: it is “Phase 1 of retiring strykelang's own VM and JIT in favor of running whole programs on the shared fusevm runtime using native fusevm Values.”

The distinction between this path and the bridge of the previous sections is exact and worth stating, because the two look similar and are not. The fusevm *bridge* (`fusevm_bridge.rs`) offloads eligible numeric segments to fusevm using i64 bit-handles — a conservative integer subset, with everything else staying on `vm.rs`. The fusevm *native* path lowers the entire program to a fusevm `Chunk` built from universal ops plus `Extended` ops backed by a stryke host, and runs it on `fusevm::VM` with native fusevm `Values` rather than bit-handles. The bridge accelerates a hot loop inside an interpreted program; the native path replaces the interpreter. They share a runtime and nothing else.

The migration's discipline is the same discipline that governs the rest of the runtime: it must never silently diverge. The path is opt-in behind the `STRYKE_FUSEVM_ONLY` environment variable, and its lowering entry point, `try_run_native`, **aborts to None on any op outside the covered subset** — at which point the caller falls back to `vm.rs` and runs the program the old way. Coverage therefore only ever grows behind passing parity checks; an uncovered construct degrades to the proven path instead of producing a wrong answer. Two more flags exist for measuring the frontier: `STRYKE_FUSEVM_STRICT` turns an uncovered program into a hard error instead of a fallback (so a parity run cannot accidentally pass by never exercising the native path), and `STRYKE_FUSEVM_TRACE` logs the first uncovered op a program hits, which is how the next phase's target is chosen.

The progress is tracked in `FUSEVM_MIGRATION_CHECKLIST.md`, and it is concrete. At the end of Phase 36, **115 of stryke's 351 Op variants have native lowering arms**; the example/benchmark corpus runs 12 of 12 fully native and parity-clean; and the full library suite — 4,200 tests — passes on the path. The active measurement corpus is `parity/cases/*.pl`, 19,505 generated files sharing a common op template, so each phase that clears one more op tends to unlock a large fraction of the corpus at once. The phases recorded in the checklist read like a tour of the bytecode from the bottom up: scalar arithmetic and string concat first (Phases 1–8), then control flow and scalar variables (Phases 5–6), arrays and hashes via the interp host (Phases 10–11), print and interpolation (Phase 12), function calls and recursion and closures (Phases 13–14), regex match and substitution (Phases 15–16), the whole block-builtin family — `map/grep/sort/PMapWithBlock` (Phases 19, 22, 30, 31) — the fused loop superops (Phase 29), bitwise and shift ops with their set-and-sketch overloads preserved (Phase 32), reference and deref construction (Phases 34–35), and the short-circuit keep-jumps decomposed into `Dup` + truthiness + conditional jump because fusevm's own keep-jumps use fusevm truthiness rather than Perl's (Phase 36). The next wall is named explicitly: `Op::Call`, dynamic name-based dispatch for builtins and user subs, the most VM-coupled op left and the one the checklist warns against rushing because of the silent-divergence risk.

Native AOT: `stryke build --native`

The migration has already paid its first dividend, and it is the one the whole single-binary philosophy was aiming at: ahead-of-time compilation to native machine code. As of v0.17.33, `stryke build --native SCRIPT` no longer embeds source into a copy of the interpreter — it compiles. stryke was the last of fusevm's five frontends to gain this, precisely because it is not a *pure* fusevm frontend; it has its own VM, so the native path had to be wired up by hand rather than inherited. The plumbing lives in `aot_native.rs` and `fusevm_native.rs`: `build_native` parses and compiles the script on stryke's own front end, lowers the resulting chunk to a self-contained fusevm `Chunk` via `lower_to_fusevm_aot`, hands that to `fusevm::aot::compile_object` — which emits a relocatable Cranelift object — and links the object plus a small C entry stub against strykelang's

runtime staticlib `libstryke.a` into a standalone executable. The result is a real native binary, the same design the `--native` builds of `zshrs`, `vimlrs`, and `awkrs` use.

Because the native path can only AOT what lowers self-contained, the build is honest about its scope. The eligibility gate `aot_safe_ext` allowlists the subset that runs without the interpreter host — arithmetic, strings, scalars, slots, arrays and hashes, and `print/say/printf` — and `rejects` programs that reach for subs, closures, regex, the builtin surface, or block `map/grep/sort` with a clear message pointing the user back to plain `stryke build` (the source-trailer mode) for the rest. This is the same refuse-rather-than-miscompile rule the method JIT follows: a program that the native compiler cannot prove it can lower faithfully is turned away at build time, not silently miscompiled at run time. The two `stryke build` modes therefore divide the territory cleanly — the default source-trailer build runs the full language by carrying an embedded interpreter, while `--native` trades coverage for genuine machine code over the self-contained subset, and the subset grows with every phase of the migration above. The endgame the checklist names is the point where the two converge: once `Op::Call` and the remaining ops lower, `vm.rs` and `jit.rs` are deleted, stryke becomes a true fusevm frontend, and `--native` compiles the whole language. The current `fusevm` dependency is 0.14.5, bumped specifically for wider native-AOT op coverage through heap values.

The LuaJIT comparison

stryke's bar is LuaJIT, and it states the position in measurable terms. On single-threaded work, benchmarked against Perl 5.42.2, Python 3.14.4, Ruby 4.0.2, Julia 1.12.6, Raku, and LuaJIT on an Apple M-series machine — mean of ten hyperfine runs with three warmups, and critically including process startup rather than steady-state, because that is the workload a scripting language actually faces — stryke is second only to Mike Pall's LuaJIT. It beats LuaJIT outright on three of the eight benchmarks: `loop` (0.74x), `array` (0.17x), and `regex` (0.07x), where values below 1.0 mean stryke is faster. It reaches near-parity on `string` (1.21x) and `fib` (1.43x). LuaJIT wins on `hash` (3.4x) and `map_grep` (4.2x), the two benchmarks where its tracing JIT eliminates dispatch overhead that stryke still pays. Against every interpreter that is not LuaJIT — Perl, Python, Ruby, Julia, Raku — stryke is faster on every benchmark. The NaN-boxing this book describes in the next chapter is the same trick LuaJIT uses; that is not coincidence, it is stryke measuring itself against the bar and adopting the technique that put the bar where it is.

The architecture of this chapter is what makes that comparison legitimate rather than aspirational. A single execution engine means there is one thing to optimize and no slow path dragging the average down. Fused superinstructions mean the interpreter is fast before any compilation happens, so the JIT is an accelerator, not a crutch — the `loop` and `array` wins come substantially from fusion, where the inner loop runs as native Rust arithmetic before any Cranelift compilation has happened at all. Two JIT systems — the shared three-tier fusevm JIT for the universal-integer subset and stryke's own two-tier method JIT for everything richer — mean both the pure-numeric hot loops and the string-and-collection-heavy code get native code. The two benchmarks LuaJIT wins, `hash` and `map_grep`, are exactly the collection-heavy workloads where LuaJIT's single unified tracing JIT erases dispatch overhead that stryke's method JIT, lacking a tracing tier of its own outside the universal-integer bridge, still pays in part — which is the honest shape of the gap, not a rhetorical one. And the multi-threaded lead is something LuaJIT structurally cannot match: LuaJIT has no parallel primitives at all, while stryke's parallel builtins distribute across cores through rayon's work-stealing scheduler (the `use rayon::prelude::*` and `into_par_iter` calls throughout `vm.rs`, twenty such call sites in the file). On a multi-threaded benchmark stryke is not competing against LuaJIT on one core; it is running on every core a machine has while LuaJIT runs on one. The single-threaded number says the engine is sound. The multi-threaded number says the engine is built for the machine people actually run it on.

Chapter 21 – Values and the Cache

Two design choices govern how fast stryke moves data: how it represents a value in memory, and how it avoids redoing work between runs. The first is NaN-boxing, which makes the common case — an integer or a float — cost nothing but a register. The second is the rkyv script cache, which makes the second run of a program skip the entire front end. Both are measurable, and both are in the source.

NaN-boxing: one u64 to hold anything

A `StrykeValue` is a single 64-bit integer. The definition in `strykelang/value.rs` is two lines:

```
#[repr(transparent)]
pub struct StrykeValue(pub(crate) u64);
```

`#[repr(transparent)]` is load-bearing. It guarantees that a `StrykeValue` has exactly the layout of a `u64` — no wrapper overhead, no discriminant byte, no padding. A value moves through a function call, sits in a register, lands on the VM's value stack, all at the cost of moving eight bytes. The cleverness is in how those eight bytes are interpreted, and that logic lives in its own small module, `strykelang/nanbox.rs` — 133 lines, the densest payload-per-line in the codebase.

The trick exploits the structure of an IEEE 754 double. A 64-bit float has an 11-bit exponent, and when every bit of that exponent is set the value is “not a number” — NaN — of which there are billions of distinct bit patterns that all mean the same thing. NaN-boxing steals those unused patterns to encode non-float values, while letting any *actual* finite float pass through as its own raw bits. stryke partitions the space with two tags:

```
pub(crate) const QNAN_IMM: u64 = 0x7FF8_0000_0000_0000;
pub(crate) const SUB_UNDEF: u64 = 1;
pub(crate) const SUB_INT32: u64 = 2;
pub(crate) const HEAP_TAG_HI: u64 = 0x7FF9;
```

A value whose top sixteen bits are `0x7FF8` is an immediate — a small integer or `undef` — with a sub-tag in the next bits distinguishing which. A value whose top sixteen bits are `0x7FF9` is a heap pointer, with the pointer itself living in the low 48 bits. Anything else is a genuine float, stored as its own raw IEEE bits. The asymmetry here is deliberate and is the entire reason the scheme works: floats are the *default* interpretation, not a tagged case. Any finite double — the overwhelming majority of floats a program ever holds — has an exponent that is not all-ones, so its raw bits never collide with the `0x7FF8/0x7FF9` tag region and it passes through as itself with zero encoding work. `is_raw_float_bits` is precisely this check: extract the eleven exponent bits and confirm they are not all set (`(u >> 52) & 0x7FF != 0x7FF`). The only floats that *would* collide are the non-finite ones — infinities and NaNs, whose exponent is all-ones — and those are the rare case that gets boxed onto the heap (`float_needs_box` flags exactly them). So the common float costs nothing and the pathological float pays an allocation, which is the right way around. The three checks are trivial bit operations:

```
pub(crate) fn is_imm(u: u64) -> bool {
    (u >> 48) == 0x7FF8
}
pub(crate) fn is_heap(u: u64) -> bool {
    (u >> 48) == HEAP_TAG_HI
}
pub(crate) fn is_raw_float_bits(u: u64) -> bool {
    let exp = (u >> 52) & 0x7FF;
    exp != 0x7FF
}
```

Encoding a 32-bit integer is a couple of ORs and a shift:

```
pub(crate) const fn encode_imm_int32(n: i32) -> u64 {
    QNAN_IMM | (SUB_INT32 << 32) | (n as u32 as u64)
}
```

and decoding is the reverse — check the immediate tag, check the sub-tag, mask out the payload. Both are `#[inline]` and the encode path is `const`. There is no allocation, no boxing, no branch into the heap. A small integer in *stryke* is a `u64` with some high bits set, and arithmetic on it is arithmetic on that `u64`.

Why this makes arithmetic fast

Consider the alternative most dynamic languages reach for: a tagged enum, where every value is a discriminant plus a payload, often heap-allocated for anything bigger than a machine word. Under that scheme, adding two integers means loading two enum values, matching on both discriminants, extracting both payloads, adding, and constructing a new enum. Under NaN-boxing, the VM checks whether both operands carry the immediate-int tag — one comparison each — and if so adds the low 32 bits directly. You can see the shape of this all over `value.rs`: arithmetic and comparison routines lead with a fast path that calls `nanbox::as_imm_int32` on both operands and, when both are present, operates on raw `i32` values, only falling back to the general heap path when an operand turns out to be a string, a reference, a float that needed boxing, or a bignum.

There is a subtlety in the integer story worth making explicit. The immediate space holds a 32-bit integer — `encode_imm_int32` takes an `i32` — but *stryke*'s integers are 64-bit. The two are reconciled by the `HeapObject::Integer(i64)` variant: values that fit in 32 bits ride in the immediate, allocation-free, while a value that exceeds the `i32` range spills to a heap `Integer`. The overwhelming majority of integers in real code — loop counters, indices, small accumulators, the lengths of things — fit in 32 bits and never leave the register. The full 64-bit range is available; you pay for it only when you actually use it. This is the same graduated-cost philosophy the JIT follows with its invocation threshold and the cache follows with its lazy mmap: the common case is free, and the expensive case is paid for precisely when it occurs and not before.

The `HeapObject` enum (`value.rs`) is where the rich values live — `Integer(i64)`, `Float(f64)`, `String`, `Array`, `Hash`, the reference types, `CodeRef`, `Regex`, `Blessed`, and `BigInt` for `--compat` overflow promotion. But a `StrykeValue` only becomes a heap pointer when it has to. A loop counter, an array index, the accumulator in a tight numeric loop — these never touch the heap. They live in the immediate space, packed into the same `u64` that would otherwise hold a pointer to them.

This is the same trick LuaJIT uses, and the convergence is not coincidence. LuaJIT's speed on numeric code comes substantially from never boxing a number it doesn't have to, and *stryke*'s NaN-boxing buys the same property: the hot arithmetic path is register-resident and allocation-free. It is the value-representation half of the answer to “how is a dynamic language this fast.” The JIT from the previous chapter is the other half — and the two compound, because the integers the JIT compiles to native `i64` arithmetic are the same immediates NaN-boxing already kept out of the heap. There is no impedance mismatch between the value representation and the compiler: an immediate integer is already a machine integer with tag bits, the JIT strips the tag and operates on the `i64`, and the result is re-tagged on the way back into the value stack. Where a tagged-enum representation would force the JIT to emit a match-and-extract before every arithmetic op and a

construct-and-store after, NaN-boxing lets the generated code look almost exactly like the native arithmetic a static compiler would emit. The value design and the JIT design were made for each other, which is why the two chapters describe one system rather than two.

The rkyv script cache

The second way stryke avoids work is across runs, not within them. Compiling a script — lex, parse, compile to a **Chunk** — is not free, and for a program you run repeatedly it is pure waste to redo it every time the source hasn't changed. stryke caches the compiled **Chunk** and reloads it on the next run if the source is unchanged.

The cache used to be a SQLite database. It is now a single rkyv-archived shard, and the migration is documented in [docs/CACHE_RKYV_MIGRATION.md](#) — marked **SHIPPED**, live in [strykelang/script_cache.rs](#). The reason for the move is a workload mismatch. The cache does exactly two operations: look up a compiled program by its source path, and replace it by path. No range queries, no joins, no secondary indexes. SQLite's machinery — a bundled C library, WAL mode, SQL parsing, connection setup — is built for a query workload this cache never has. As the migration doc puts it, the SQL B-tree probe was sub-microsecond and never the problem; the cost was everything around it. Each `s test t` invocation spawns a separate process per test file, and each process paid SQLite's full connection-open, WAL pragma, and statement-prepare tax for a single lookup. That is a fixed per-process cost that scales with the number of files, not the amount of work.

rkyv collapses the read path to four real syscalls: open the file, mmap it, validate the archive root once with `check_archived_root`, and look up the entry in a zero-copy `ArchivedHashMap`. No connection, no pragmas, no query parser. The `ArchivedHashMap` is the load-bearing piece — it is a hashbrown table whose on-disk byte layout is identical to its in-memory layout, so the lookup runs directly against the mmap'd bytes with no deserialization of the map itself. The OS faults in the pages the probe touches and nothing more. What remains is a single bincode pass on the matched entry's inner `Program/Chunk` blob, which is the one piece of work Phase 2 would eventually remove. Everything around that — the connection, the WAL pragmas, the prepared statement, the page-cache warm — that SQLite paid on every invocation simply does not exist in the rkyv path.

The measured result is in the doc, and it is the number to remember:

Format		p50		p99		total	
rkyv		22 μ s		36 μ s		22.95 ms	
sqlite		241 μ s		499 μ s		263.62 ms	

That is the per-process workload — open, one lookup, close, repeated a thousand times — which is exactly what running a test suite looks like. rkyv is 10.8x faster at the median and 11.5x faster in total. The gap is wider than the steady-state number (3.46x) precisely because the per-process case is dominated by the setup cost SQLite pays on every single invocation and rkyv pays once. At 1,033 cached scripts, the migration doc works out the arithmetic: about 226 ms saved per warm test run, just from the cache layer.

The steady-state number — open the cache once and do ten thousand lookups against it — tells the other half of the story: rkyv at 2.49 μ s per hit against SQLite's 8.60 μ s, a 3.46x win. That gap is the read path alone, with connection setup amortized away. The fact that the per-process gap (10.8x) is so much wider than the steady-state gap (3.46x) is the whole argument for the migration in one comparison: the cache's real workload is per-process, not steady-state. A developer iterating on a test suite spawns a fresh process per file, and each process does one lookup and exits, so the cost that dominates is exactly the setup cost SQLite pays every time and rkyv pays once. Optimizing the steady-state path would have been optimizing the workload stryke doesn't have.

The cost is on disk. The rkyv file is 3.2x bigger — 270 KB versus 84 KB for the 100-script corpus — entirely because the migration dropped the zstd compression SQLite was applying to the bincode blobs, and AST-plus-bytecode bincode is highly compressible. The trade was deliberate: mmap-friendly uncompressed bytes that the OS can fault in directly, against a few hundred kilobytes of disk on a developer's machine. The doc notes the compression could be added back inside the rkyv container if a space-constrained CI environment ever made it worth the decode CPU.

There is one more reason the choice fits, beyond raw speed. `stryke` embeds in `zshrs`, and `zshrs` had already moved to an `rkyv` shard pattern — file-per-shard, atomic-rename writes, advisory `flock`, `mmap` reads — for its own bytecode cache. Aligning `stryke`'s persistence layer to the same crate and the same code shape means one architectural story across the stack instead of “`stryke` uses `SQLite`, except where it doesn't.” `rkyv` also gives explicit, versioned format headers, so a format change cleanly invalidates old shards on read — the migration-safety property the endgame demands of anything that persists to disk.

What the cache does not yet do is true zero-copy on the inner data. The shard wraps bincode-encoded `Program` and `Chunk` bytes, and decoding still costs a bincode pass on each hit. Phase 2 — deriving `rkyv`'s archive traits across the whole `Chunk/Program/StrykeValue` graph — is deferred, because `StrykeValue`'s `Arc`-shared, interior-mutable graph isn't trivially archivable in zero-copy form, and the estimated win (1–3 μ s of bincode-decode per hit) doesn't yet justify the churn. The 22 μ s number is what the shipped Phase 1 delivers.

Chapter 22 – Compatibility and Conviction

stryke makes two promises that sound contradictory. The first: it will run your existing Perl. The second: it will not let you write Perl when you mean to write stryke. Both are enforced by flags, both are backed by real machinery in the source, and the tension between them is not a flaw — it is the design. This chapter is about `--compat`, about `--no-interop`, about the ten style rules that turn taste into a parser error, and about why a language built for one person's hands draws these lines exactly where it does.

`--compat`: the parity corpus as specification

`--compat` puts stryke into Perl 5 strict-compatibility mode. The implementation is a process-wide flag in `strykelang/lib.rs`:

```
static COMPAT_MODE: AtomicBool = AtomicBool::new(false);

pub fn set_compat_mode(on: bool) {
    COMPAT_MODE.store(on, Ordering::Relaxed);
}

#[inline]
pub fn compat_mode() -> bool {
    COMPAT_MODE.load(Ordering::Relaxed)
}
```

The doc comment states the contract: “When `true`, all stryke extensions are disabled and only stock Perl 5 syntax / builtins are accepted.” The flag is read by the parser, the compiler, and the interpreter — `compat_mode()` is `#[inline]` because it is checked on hot paths. The visible behavioral consequences include things like integer arithmetic: in `--compat`, an `i64` operation that overflows promotes to an arbitrary-precision `BigInt` (the `HeapObject::BigInt` variant), because that is what Perl does; native stryke without `--compat` never creates a `BigInt` from overflow.

But a flag that claims “stock Perl 5” is only as credible as the evidence behind the claim, and stryke’s evidence is a corpus of twenty thousand tests. `docs/patent.md` frames why the number is the point and not the decoration: the corpus “IS the operational spec; competitors must reproduce 20,000 tests passing to make a comparable claim.” This is the right way to think about a compatibility promise. “We support Perl” is marketing — every Perl-influenced language has said it, and none of it is falsifiable. “We pass these twenty thousand tests, pinned against upstream Perl 5 behavior” is a specification you can run, and one that fails loudly the moment a change breaks it. The corpus is not documentation of compatibility; it is the *definition* of it. If a behavior isn’t in the corpus, it isn’t promised; if it is, it’s verified on every change. This inverts the usual relationship between docs and code: the tests are not checking that the implementation matches some prose description of Perl, because there is no such description that is precise enough to check against — Perl’s behavior *is* what Perl does, and the corpus captures what Perl does by running both and comparing. A compatibility claim grounded in a runnable corpus is the same kind of object as the no-tree-walker invariant of Chapter 20: a property the project cares about, pinned by a test that fails if the property regresses, so the property cannot quietly rot.

--no-interop: the bot firewall

`--no-interop` points the other direction. Where `--compat` accepts all of Perl, `--no-interop` rejects the parts of Perl that stryke has replaced. Its state lives alongside `--compat` in `lib.rs`, but with a twist — it has both a process-wide default and a per-thread override:

```
static NO_INTEROP_DEFAULT: AtomicBool = AtomicBool::new(false);

thread_local! {
    static NO_INTEROP_TLS: Cell<Option<bool>> = const { Cell::new(None) };
}
```

The thread-local exists for a concrete reason: stryke’s parallel builtins parse and evaluate code from rayon worker threads, and a worker needs to be able to set its own no-interop mode without racing siblings on a shared atomic. `no_interop_mode()` reads the thread-local override first (`Some(v)` wins) and falls back to the process-wide `NO_INTEROP_DEFAULT` only when the thread has set no override. That ordering is the whole mechanism: a worker thread evaluating a code fragment under different interop rules than the main thread sets its own TLS cell, gets its own answer, and disturbs nobody. A shared atomic could not express “this thread, right now, has a different mode than the rest” without a lock or a race. The thread-local is lock-free, contention-free, and per-thread by construction. That is the kind of detail that exists only because someone actually ran the feature under parallelism and hit the case where two workers needed different modes at the same instant.

What `--no-interop` does is reject Perl-isms at parse time, and the rejections are explicit, scattered through `parser.rs` at exactly the points where each banned construct is recognized:

```
"stryke uses `fn` instead of `sub` (--no-interop is active)",
"stryke uses `p` instead of `say` (--no-interop)",
"stryke uses `fn {}` instead of `sub {}` (--no-interop)",
```

`wantarray` is rejected. `$a` and `$b` outside a sort or reduce block are rejected. A bare-integer tail expression in module position — Perl’s `1;` ritual — is rejected. The flag is the bot firewall, and the reasoning behind that name is worth stating plainly: documentation does not stop a code generator from emitting `sub` and `print` and trailing semicolons, because a model trained on decades of CPAN reaches for Perl idioms by default. A `parser` stops it. Under `--no-interop`, Perl-shaped code does not compile, so the firewall enforces stryke’s idioms at the only layer that a generator cannot ignore. Parser rejections become free quality assurance: every rejection is a place where the code drifted back toward Perl and got caught before it ran.

The ten hard rules

The taste that `--no-interop` enforces is written down. `docs/STYLE_GUIDE.md` opens with “the ten hard rules,” and they are reproduced here verbatim because they are the constitution of the language’s surface:

1. **fn not sub.** `sub` is Perl 5 baggage and is rejected by `--no-interop`.
2. **p not print, never say.** `p` is the stryke verb, has the record separator built in, costs one character. `say` is rejected by `--no-interop`.
3. **Use `|> / ~>` over nested calls.** `f(g(h(x)))` reads inside-out; `x |> h |> g |> f` reads left-to-right and shows up at the call site without diving the cursor through three closing parens. Always pipeline.
4. **No semicolons at end of line.** stryke statement-terminates on newline. The only place `;` belongs is between statements on the same line.
5. **Implicit closure parameters always.** Never write `sub { my $x = shift; ... }`. Use `_` for slot 0 and `_N` for slot N.
6. **Expression-bodied fn for one-liners.** `fn double = _ * 2` over `fn double($a) { $a * 10 }` over `fn double { _ * 2 }`.
7. **Implicit return — never return on the tail expression.** The last expression in any `fn` body is the return value. `return` is only correct as an early-return guard.

8. **No parens on fn calls unless precedence demands them.** `p "hello", len @a, uc rev "abc"` — paren-less.
9. **No trailing 1 at the end of a library / module file.** That's Perl 5's `package returns true` ritual; stryke's `use/import` resolution does not consult the file's tail expression.
10. **`val` for immutable, `var` for mutable — never bare `my` in new code.** stryke flips Perl 5's mutable-by-default `my` into immutable-by-default. `val` marks the binding frozen at compile time; `var` is the mutable counterpart. Both desugar to `my` in the AST — same scope, same bytecode, same slot allocation — so the cost is zero.

The style guide pairs the rules with a single example that violates all of them at once, then walks the violations one by one. The right version:

```
fn shout = uc(_) . "!"
@lines |> maps { trim(_) |> shout } |> greps { len(_) > 0 } |> ep
```

The wrong version — `sub`, `print`, nested `length(...)`, trailing semicolons, explicit `my $s = shift`, block-bodied one-liner, explicit `return` — is, in the guide's words, “7× the characters and reads bottom-up.” Every one of those wrongs is a rule, and the rules that can be mechanically checked are the ones `--no-interop` enforces. The style guide is the human-readable form; the parser is the executable form; the two say the same thing.

Why the lines are drawn here

A reasonable objection: why ship a flag whose only job is to reject code that would otherwise run? The answer is in who stryke is for. This is a language whose every design call defaults to the terser form — `p` over `print`, `_` over `$_`, `val` over `my`, paren-less calls, implicit returns. Those choices are not arbitrary minimalism; they are the calling card of a tool built for a power user with millions of keystrokes of muscle memory, where one character saved per verb compounds across a career at the terminal. The cost of that terseness is that the wrong idioms are *also* short and reachable — Perl's `print` and `sub` are right there, a few characters away, and a generator or a tired hand will reach for them. `--no-interop` is how stryke keeps the surface it chose. It is not a safety rail for beginners. It is a discipline enforcer for an expert who has decided exactly how this language should read and wants the parser to hold the line.

There is one more property worth naming, because it ties the two flags back to the runtime chapters. `--no-interop` is enforced in the parser — the cheapest possible place — which means it costs nothing at run time. A program compiled clean under `--no-interop` produces exactly the same `Chunk` as the same program would without the flag; the flag only ever subtracts programs from the set of accepted inputs, never adds runtime checks. `--compat`, by contrast, is read on hot paths — `compat_mode()` is `#[inline]` precisely because the arithmetic routines consult it to decide between i64-overflow-to-BigInt (Perl semantics) and i64-overflow-to-float (native stryke). That asymmetry is correct. Rejecting bad idioms is a compile-time decision and should cost compile-time only. Reproducing Perl's runtime semantics is a runtime decision and has to be checked where the semantics diverge. The flags sit at the layers their jobs belong to, which is the same discipline the no-tree-walker invariant, the JIT threshold, and the NaN-box fast paths all express in their own domains: do the work at the cheapest layer that can do it, and do it once.

`--compat` and `--no-interop` are the two faces of the same conviction. `--compat` says: stryke is a strict superset of Perl, and here are twenty thousand tests proving the subset is faithful. `--no-interop` says: but when you are writing stryke, write stryke — the parser will not let the old language leak in. One flag honors the past so nothing that worked yesterday breaks. The other guards the present so the language doesn't dissolve back into the thing it grew out of. Between them sits a runtime — single-engine, NaN-boxed, twice-JIT'd, rkyv-cached, parallel across every core — fast enough to make the whole proposition worth the discipline. The compatibility is the bridge in. The conviction is what's on the other side.

Chapter 23 – Worked Examples: Algorithms and Numerics

The fastest way to learn a language is to read good programs in it. This chapter draws five from the `examples/` corpus — pathfinding, multi-pattern matching, numerical integration, gradient descent, and shortest paths with negative weights — and walks the core of each. The full A* listing appears as the chapter’s centerpiece; for the rest, the excerpts focus on the lines that carry the idiom and the prose carries the algorithm. Each is a real file you can run today, and each earns its place by being idiomatic: namespaced functions, colon ranges, `len` for sizes, `#{...}` interpolation, implicit tail returns, and the self-checking invariant blocks that make these examples double as their own test suites.

A* Pathfinding — `a_star_no_interop.stk`

The first program is a textbook A* search on a 2-D character grid. It finds the lowest-cost path from a start cell to a goal using $f(n) = g(n) + h(n)$, where `g` is the cost accrued so far and `h` is the Manhattan-distance heuristic — admissible for a 4-connected grid, which is what guarantees A* returns an optimal path.

Look at the heuristic on line one: `fn Star::heur = abs(_ - _2) + abs(_1 - _3)`. That's the whole function — a single expression with no braces, taking its four arguments through stryke's positional block parameters `_, _1, _2, _3`. There is no `($a, $b, $c, $d)` signature to read past; the parameters are the gaps in the arithmetic. The admissibility argument is the load-bearing detail: Manhattan distance never *overestimates* the true cost on a 4-connected grid, because any path between two cells must change row and column by at least their coordinate differences, one unit-cost step at a time. An admissible heuristic is exactly the precondition under which A* is guaranteed to return an optimal path — drop admissibility (use, say, Euclidean distance scaled up) and A* still terminates but can return a suboptimal route. So this one-liner isn't just terse; it's the line that makes the whole search correct.

The open set is a flat array of hash nodes, each carrying `r`, `c`, the accrued cost `g`, the priority `f = g + h`, and a `parent` pointer back into the array. Each iteration scans linearly for the minimum-`f` node and pops it with `splice`. The comment is candid that a binary heap would make selection $O(\log n)$ instead of $O(n)$ — for a 5×5 demo grid the linear scan is irrelevant, but on a real map the priority queue is the difference between A* and a quadratic crawl. The `parent` chain is what makes path reconstruction trivial: once the goal pops, the loop walks `parent` links backward, `unshift`-ing each cell so the returned path emerges start-to-goal with no post-hoc reversal. Two design choices are worth naming. First, `var %closed` keyed by the interpolated string `"$nr,$nc"` uses a hash as a coordinate set — idiomatic stryke, and it sidesteps allocating a 2-D boolean array sized to the grid. Second, the neighbor-expansion loop carries an explicit “already in open with a better `g`” check; without it, the same cell could be enqueued repeatedly with stale costs, inflating the open set. That check is doing the job a decrease-key operation would do in a heap-backed implementation.

What turns this from a demo into a verified artifact is the second half of the file. `Star::valid_path` re-derives correctness *independently* of the search: it walks the returned path and asserts every consecutive pair differs by exactly one Manhattan step (`($dr + $dc) == 1`), no cell sits on a wall, and the endpoints match start and goal. This is the right way to test a search — the validator shares no code with `find_path`, so a bug in the search can't hide behind the same bug in the check. The driver then runs four scenarios that exercise the distinct branches: a wall detour that forces the 9-cell optimal path (proving the search doesn't just find a path but the *shortest* one), an open 3×3 grid, a goal walled off so `find_path` returns `undef` (the unreachable branch), and the trivial start-equals-goal early return. The closing loop reasserts the universal invariant that every returned path validates. The program proves the path is a valid shortest path before printing `ALL PASS`.

Aho-Corasick Multi-Pattern Matching — `aho_corasick.stk`

Aho-Corasick finds all occurrences of a set of patterns in a single pass over the text. The naive alternative — running a separate substring search per pattern — costs $O(\text{text} \times \text{patterns})$; Aho-Corasick collapses that to $O(\text{text} + \text{matches})$ by precompiling all patterns into one finite automaton. It builds a trie of the patterns, adds “failure links” that let the automaton fall back gracefully when a partial match breaks, then scans the text without ever rewinding the input cursor. The whole automaton is one flat array of node hashes — the data structure is the entire trick.

The node shape and the construction's core idea fit in a few lines:

```

fn _node() { { ch => {}, fail => 0, out => [] } }

fn build($patterns) {
  var @nodes = ()
  push @nodes, AC::_node() # node 0 is root
  var $pi = 0
  for val $pat (@$patterns) {
    var $cur = 0
    val $n = len $pat
    for val $i (0:$n-1) {
      val $c = substr($pat, $i, 1)
      if (!exists $nodes[$cur]->{ch}->{$c}) {
        push @nodes, AC::_node()
        $nodes[$cur]->{ch}->{$c} = len(@nodes) - 1
      }
      $cur = $nodes[$cur]->{ch}->{$c}
    }
    push @{$nodes[$cur]->{out}}, $pi
    $pi = $pi + 1
  }
  # ... BFS fail-link pass elided ...
}

```

Each node is `{ ch => {}, fail => 0, out => [] }` — a child map keyed by character, a single fail-back index, and a list of pattern indices that *end* at this node. The decision that makes the rest of the algorithm tractable is that nodes refer to each other by **integer index into @nodes**, never by Perl-style reference. A trie node is `@nodes[$i]`; its child on character `$c` is the integer `$nodes[$i]->{ch}->{$c}`. This is deliberate: the failure-link BFS needs to read a node's fail-target's *output set* as `$nodes[$nodes[$v]->{fail}]->{out}`, and that double-index only works cleanly when fail-targets are array offsets. A reference-based trie would force a parallel index map to do the same thing, so the flat-array representation isn't an incidental choice — it's what keeps the construction one expression deep.

`build` runs two passes. The trie pass above walks each pattern character by character, extending the trie and recording the terminal node's pattern index in `out`. The elided second pass is a breadth-first traversal that sets failure links: the root's children fail to the root, and every deeper node `v` fails to wherever its parent's fail-state can continue on the same character — `while ($f != 0 && !exists $nodes[$f]->{ch}->{$c}) { $f = $nodes[$f]->{fail} }` chases the chain until it finds a node that *can* continue on `$c` or hits the root. The subtle step is that the BFS also copies outputs up the fail chain (`for val $o (@{$nodes[$nodes[$v]->{fail}]->{out}}) { push @{$nodes[$v]->{out}}, $o }`); this is what lets one node report *several* overlapping patterns at once, the property that distinguishes Aho-Corasick from a plain trie.

The scan loop is the payoff and the part worth seeing in full:

```

for val $i (0:$n-1) {
  val $c = substr($text, $i, 1)
  while ($cur != 0 && !exists $nodes->[$cur]->{ch}->{$c}) {
    $cur = $nodes->[$cur]->{fail}
  }
  if (exists $nodes->[$cur]->{ch}->{$c}) {
    $cur = $nodes->[$cur]->{ch}->{$c}
  }
  for val $pi (@{$nodes->[$cur]->{out}}) {
    val $plen = len $pats->[$pi]
    push @out, { pat => $pi, pos => $i - $plen + 1 }
  }
}

```

One left-to-right pass, and the input index `$i` never moves backward — every character is consumed exactly once. When the current state has no edge for the next character, the inner `while` follows fail links (a state the

automaton already knows is a valid suffix of what's been read) rather than re-scanning text. The amortized cost of those fail hops over the whole input is linear, which is the entire reason the algorithm exists. Each match's start position falls out arithmetically as `$i - $plen + 1`. Running it on "ushers" with ("he", "she", "his", "hers") surfaces she, he, and hers — the overlap of she/he and the embedded he inside hers all reported from a single cursor sweep. In a language without first-class hashes you'd hand-roll a character-to-index transition table; stryke's nested {ch}->{\$c} hash is the transition function written as data, which is why the construction reads like the textbook pseudocode rather than an array-bookkeeping exercise.

Adaptive Simpson Integration – adaptive_simpson_no_interop.stk

Numerical integration is where stryke's first-class function values shine. This program approximates a definite integral by recursively bisecting the interval and refining only where the curve demands it. Because the file carries seven invariant blocks, the listing below is a complete coherent excerpt — the core algorithm plus the driver — with the trailing invariant checks summarized after.

The functions to integrate are passed as *values*. The `@cases` table stores code references — `f => \&Sim::f_sin` and friends — and the integrator invokes them through the call-arrow `$f->(x)`. This is the idiom the whole program rests on: stryke functions are ordinary values you can stuff into a hash and call later, which is exactly what a numerical-methods API needs, because the integrand is the *parameter*, not the algorithm. The alternative in a language without first-class functions is a callback interface, a function-pointer table, or a `std::function` wrapper; here the function is just data in the `@cases` table next to its known exact value. `Sim::simpson` is the parabolic-area rule for one interval — it fits a parabola through three points and integrates that exactly — written as a single tail expression with no `return`.

The adaptive machinery is in `Sim::recurse`, and the algorithm is a textbook application of *Richardson extrapolation as an error estimator*. It evaluates the function at the two quarter-points, computes Simpson estimates `S_left` and `S_right` for the halves, and compares their sum against the parent estimate `$S`. The difference `$delta = S_left + S_right - S` is not just a stopping signal; because Simpson's rule has a known $O(h^4)$ error term, halving the interval cuts that error by $2^4 = 16$, so `$delta` approximates $15\times$ the error of the refined estimate. That's where the `15.0 * $eps` comparison and the `$delta / 15.0` correction both come from — the same constant, used first to decide whether the refinement is good enough and then to *extrapolate* the refined pair to one order higher accuracy. When the panel is smooth enough that `$delta` is within 15ϵ , it returns `S_left + S_right + $delta / 15.0`; otherwise it recurses into each half with the tolerance halved (so the per-panel errors still sum to the global ϵ). The result is *global adaptivity*: smooth regions terminate after one subdivision while a peaked region like `f_sharp` — $1/(0.01 + (x-0.5)^2)$, which spikes hard at 0.5 — keeps splitting exactly where the curvature lives, spending evaluations only where they buy accuracy. A fixed-grid Simpson rule with the same evaluation budget would either under-resolve the spike or waste points on the flat regions.

The omitted tail of the file is seven invariant blocks, each asserting a property any correct integrator must satisfy: the estimate matches the closed form to $1e-8$; Simpson beats the trapezoid rule on the same panels (a relative-accuracy check, not just an absolute one); additivity (the integral over `[a, b]` equals the integral over `[a, c]` plus the integral over `[c, b]`); the zero-width integral is exactly zero; reversing the limits negates the result; scalar linearity (integrating `c * f` returns `c` times the integral of `f`); and convergence on the peaked function. Notice these are *algebraic* properties of integration, not value comparisons against a table — additivity and limit-reversal must hold for *any* integrand, so they catch a class of bugs (sign errors, interval-splitting mistakes) that a single golden-value check would miss. The program is its own numerical-analysis test suite.

Adam Optimizer — `adam_optimizer_no_interop.stk`

Adam is the workhorse optimizer of modern machine learning — adaptive-moment stochastic gradient descent that keeps per-parameter running averages of the gradient and its square. This file is on the longer side, so the listing below is a complete coherent excerpt covering the optimizer state, the update step, and one representative driver; the remaining drivers and seven invariant checks are described afterward.

```
#!/usr/bin/env stryke

# Adam optimizer — adaptive-moment stochastic gradient descent. Maintains
# per-parameter first-moment (m) and second-moment (v) running averages,
# applies the bias correction ( $\hat{m} = m / (1 - \beta_1^t)$ ,  $\hat{v} = v / (1 - \beta_2^t)$ ),
# and steps by  $\alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ . On well-behaved convex problems
# Adam converges to the optimum; we cross-validate by minimizing
# several closed-form-known functions and checking the final position
# is within tolerance.

# — factory: fresh optimizer for a parameter vector of length n —

fn Adam::new($n, $lr, $beta1, $beta2, $eps) {
  val @m = (0.0) x $n
  val @v = (0.0) x $n
  +{
    n      => $n,
    lr     => $lr,
    beta1  => $beta1,
    beta2  => $beta2,
    eps    => $eps,
    m      => \@m,
    v      => \@v,
    t      => 0,
  }
}

# — single Adam update step (in place on $params) —————

fn Adam::step($opt, $params, $grads) {
  $opt->{t}++
  val $n = $opt->{n}
  for val $i (0:$n - 1) {
    $opt->{m}->[$i] = $opt->{beta1} * $opt->{m}->[$i] + (1.0 - $opt->{beta1}) *
      $grads->[$i]
    $opt->{v}->[$i] = $opt->{beta2} * $opt->{v}->[$i] + (1.0 - $opt->{beta2}) *
      $grads->[$i] * $grads->[$i]
    val $m_hat = $opt->{m}->[$i] / (1.0 - $opt->{beta1} ** $opt->{t})
    val $v_hat = $opt->{v}->[$i] / (1.0 - $opt->{beta2} ** $opt->{t})
    $params->[$i] = $params->[$i] - $opt->{lr} * $m_hat / (sqrt($v_hat) + $opt->{eps})
  }
}

# — 1-D quadratic:  $f(x) = (x - a)^2$ , gradient  $2(x - a)$ . Minimum:  $x = a$  —

fn Adam::run_quadratic($target_a, $x0, $steps, $lr) {
  val $opt = Adam::new(1, $lr, 0.9, 0.999, 1e-8)
  val @params = ($x0)
  for val $i (1:$steps) {
    val @grads = (2.0 * ($params[0] - $target_a))
    Adam::step($opt, \@params, \@grads)
  }
  $params[0]
}

```

The optimizer is just a hash. `Adam::new` builds the zero-initialized first-moment vector `@m` and second-moment vector `@v` with the replication operator — `val @m = (0.0) x $n` creates an `n`-long array of zeros — and bundles them with the hyperparameters and a step counter `t`. The leading `+{` on the return is stryke’s disambiguator that says “this brace is a hash literal, not a block,” which is needed precisely because a hash is the tail expression of the function: a bare `{ ... }` in tail position would parse as a block.

`Adam::step` is the algorithm transcribed line for line, and each line maps to one term of the published update rule. The two moment updates are exponential moving averages: `m` tracks the mean gradient (momentum), `v` tracks the mean *squared* gradient (per-coordinate variance). The bias correction — dividing by $(1 - \beta^t)$ via the `**` power operator against the step counter `t` — exists because both moments start at zero, so early steps are biased toward zero; dividing out $(1 - \beta^t)$ inflates the early estimates back to their unbiased values, and as `t` grows the correction factor approaches 1 and vanishes. The final step $\alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ is what makes Adam *adaptive*: dividing by $\sqrt{\hat{v}}$ gives each coordinate its own effective learning rate, large for low-variance directions and small for noisy ones, with ϵ guarding the divide-by-zero. The whole vector is mutated in place through the `$params->[$i]` arrow-deref, so the caller's array updates directly — no copy, which matters once the parameter vector is large. `run_quadratic` drives it on $f(x) = (x - a)^2$, whose gradient $2(x - a)$ is computed inline; the analytic gradient is what keeps the demo self-contained, but in real use this is where backprop or autodiff would supply `@grads`.

The file's full body adds a 2-D Rosenbrock-like valley (the classic ill-conditioned test that breaks naive gradient descent), a 5-D separable quadratic, and seven invariants the listing leaves out. Several of them are doing real specification work rather than smoke-testing: invariant 4 (starting *at* the minimum stays there) pins the fixed-point property — a correct optimizer must not drift off a zero-gradient point; invariant 5 checks that bias correction makes the very first step $\approx -lr \cdot \text{sign}(\text{grad})$, which is the observable signature that the $(1 - \beta^t)$ division is wired correctly rather than accidentally omitted; invariant 6 (zero gradient $\Rightarrow$ zero update across many steps) catches any spurious accumulation in the moment vectors; and invariant 7 demands the loss be *essentially* monotone — it tolerates a small count of bias-correction-transient bumps in the first few steps rather than demanding strict monotonicity, which is the honest property Adam actually has. Together they pin the behavior the way a numerical-library test suite would, distinguishing “it converged on this example” from “the update rule is correct.”

Bellman-Ford Shortest Paths — `bellman_ford_no_interop.stk`

Dijkstra can't handle negative edge weights; Bellman-Ford can, and it detects negative cycles into the bargain. The algorithm is almost suspiciously simple — relax every edge $V-1$ times, then one more time to check whether anything still improves — and this file pairs that simplicity with thorough validation.

```
#!/usr/bin/env stryke

# Bellman-Ford shortest paths – single-source SSSP that tolerates
# negative edge weights and detects negative cycles.  $O(V \cdot E)$ . Relax
# every edge  $V-1$  times; a  $V$ th round of relaxation must NOT improve
# anything, else there's a negative cycle on the path. Parses clean
# under `--no-interop`.

# Returns { dist => [d0, d1, ...], pred => [...], has_neg_cycle => 0/1 }.
# Edges are arrayrefs [u, v, w]. Unreachable distance =  $\infty$ .
fn BF::solve($v, $src, $edges) {
  val $INF = 1e18
  var @dist = ($INF) x $v
  var @pred = (-1) x $v
  $dist[$src] = 0
  for (1:$v - 1) {
    var $any_relax = 0
    for val $e (@$edges) {
      var ($u, $w, $cost) = @$e
      next if $dist[$u] >= $INF
      if ($dist[$u] + $cost < $dist[$w]) {
        $dist[$w] = $dist[$u] + $cost
        $pred[$w] = $u
        $any_relax = 1
      }
    }
    last unless $any_relax
  }
  # One more pass: any further relaxation = negative cycle reachable.
  var $neg = 0
  for val $e (@$edges) {
    var ($u, $w, $cost) = @$e
    next if $dist[$u] >= $INF
    if ($dist[$u] + $cost < $dist[$w]) {
      $neg = 1
      last
    }
  }
  +{ dist => \@dist, pred => \@pred, has_neg_cycle => $neg }
}

fn BF::is_inf = _ >= 1e17
```

`BF::solve` initializes a distance array to `1e18` (the sentinel for ∞) and a predecessor array to `-1`, with the source distance zeroed; the replication operator builds both — `($INF) x $v` and `(-1) x $v`. The whole algorithm is two phases. The first relaxes every edge up to $V-1$ times. The $V-1$ bound isn't arbitrary: a shortest path in a graph with no negative cycle visits at most V vertices, so $V-1$ edges, and each full relaxation pass is guaranteed to finalize at least one more edge of every shortest path. After $V-1$ passes, every shortest path is fully relaxed. The early-exit `last unless $any_relax` is the standard refinement — if a pass changes nothing, the distances have already converged and the remaining rounds are dead work, so a near-tree graph pays far less than the $O(V \cdot E)$ worst case. Edges are plain `[u, w, cost]` triples, destructured inline with `var ($u, $w, $cost) = @$e`.

The negative-cycle detection is the part that makes Bellman-Ford strictly more capable than Dijkstra, and it costs exactly one more pass. The invariant is sharp: after $V-1$ rounds the distances are settled *if and only if* no negative cycle is reachable from the source. So a V th relaxation pass that *still* finds an improvement is a proof — not a heuristic — that a reachable negative cycle exists, because the only way an already-converged distance can keep dropping is an arbitrarily-traversable loop with negative total weight. The code runs that one extra pass, flags `$neg`, and bails. Dijkstra can't do this at all; its greedy “finalize the nearest frontier vertex”

step is wrong the moment a negative edge can retroactively cheapen a finalized vertex, which is the entire reason this algorithm trades Dijkstra's $O((V+E) \log V)$ for a flat $O(V \cdot E)$. The one-liner `fn BF::is_inf = _ >= 1e17` — implicit single parameter `_`, no braces — is the unreachable predicate, threshold-checking against the sentinel rather than testing equality so that a single relaxation off `1e18` still reads as infinite.

The validation sweep exercises every branch: the CLRS Figure 24.4 graph with its negative edges (expected distances `0, 2, 7, 4, -2`) confirms correctness with negatives present; a disconnected graph checks that genuinely unreachable vertices stay at ∞ ; a deliberate cycle `0→1→2→0` summing to `-1` must trip `has_neg_cycle`; a single-vertex graph covers the degenerate `V=1` case; and a closing loop asserts `dist[u] + cost ≥ dist[w]` for every edge with reachable `u`. That last check is the mathematical definition of “these are shortest paths” — it's the relaxation condition stated as a postcondition, so passing it means no edge can improve any distance, which is precisely optimality.

Chapter 24 – Worked Examples: Parallel and Streaming

Parallelism in *stryke* is built into the collection verbs rather than bolted on as a threading library. This chapter walks through five programs that put data across threads with no ceremony: a one-line parallel prime finder, a producer-consumer over a bounded channel, a concurrent HTTP link checker, the streaming *pmaps/pgreps/pflat_maps* family, and an adaptive worker-pool pattern. The recurring lesson is that going parallel often costs a single keystroke — *map* becomes *pmap* — when the work is order-insensitive; the explicit *spawn/pchannel/congregation* forms are there for the cases that aren't.

Parallel Prime Finder – *parallel_prime_finder.stk*

The shortest program in this book, and a complete illustration of *stryke*'s parallel-pipeline philosophy. It finds the primes in *1:100* across worker threads and verifies the count.

```
#!/usr/bin/env stryke
# Run: stryke --no-interop examples/parallel_prime_finder.stk

p "--- Primes pgreps ---"
val @res = ~> (1:100) pgreps { is_prime _ } collect sort { _ <=> _1 }

p "Found #{len @res} primes: #{ join ', ', @res }"

len(@res) |> aeq 25, "1:100"
@res[0]   |> aeq 2,  "first"
test_run
```

The single substantive line does everything: *~> (1:100) pgreps { is_prime _ } collect sort { _ <=> _1 }*. The thread-first macro *~>* feeds the range *1:100* into *pgreps*, the *streaming* parallel filter, which fans the primality test across worker threads. The block *{ is_prime _ }* uses the implicit topic *_* for the current element and the built-in *is_prime* predicate. Because *pgreps* returns results in completion order rather than input order, the chain then *collects* them and *sorts* with the spaceship comparator *{ _ <=> _1 }* — where *_* and *_1* are the two elements being compared — to restore ascending order.

The two assertions use the pipe-forward operator *|>* into *aeq*, *stryke*'s assertion-equals helper: there must be exactly 25 primes below 100, and the first must be 2. *test_run* at the end tallies and reports the assertions. What's notable is the *absence* of machinery — no thread spawning, no join, no result collection by hand. The parallelism is a property of the verb *pgreps*, and swapping it for the serial *grep* would change nothing but the scheduling, because the program never relies on ordering from the parallel stage — the explicit *collect sort { _ <=> _1 }* reimposes order afterward. That decoupling is what makes the swap safe: a correctly written *pgreps* chain that sorts downstream is identical in output to its *grep* equivalent, so going parallel is a one-keyword edit with no restructuring. The cost is the non-deterministic completion order, which is exactly why the *sort* is present and not optional.

Multi-Threaded Channels — `multi_threaded_channels.stk`

Where the prime finder hides threads behind a pipeline verb, this program exposes them directly: a producer thread and a consumer thread communicating over a bounded channel, the canonical concurrency primitive. The whole structure is four constructs — `pchannel`, `spawn`, `send`, `recv`:

```
# 1. Create a bounded channel (capacity 2)
val ($tx, $rx) = pchannel(2)

# 2. Spawn Producer
p "Spawning Producer..."
spawn {
  for (1:5) {
    p " [Producer] Sending: $_"
    $tx->send(_)
    sleep 0.01
  }
}

# 3. Consume in Main Thread
p "Consumer starting..."
var @results = ()
for (1:5) {
  var $val = $rx->recv()
  p " [Consumer] Received: $val"
  push @results, $val
}
```

`pchannel(2)` returns a `($tx, $rx)` sender/receiver pair with capacity two — a *bounded* channel, and the bound is the point. When the producer runs two items ahead of the consumer, its next `send` blocks until the consumer drains one. That blocking is backpressure: it caps the in-flight buffer at two regardless of how fast the producer runs, so a producer that outpaces its consumer can't grow the queue without limit and exhaust memory. An *unbounded* channel would let the producer race ahead and accumulate all five (or five million) items in the buffer; the capacity argument is what converts an unbounded-memory hazard into a bounded one. `stryke` hands the whole thing back as a two-element list destructure, the same shape Rust's `mpsc::sync_channel` or Go's buffered `make(chan T, 2)` expose, but without the import or the type annotation.

`spawn { ... }` launches a detached background thread running the block; inside, the producer loops `1:5` and pushes each value with `$tx->send(_)` — the method-call arrow on the sender handle, consuming the bare `_` topic. The `sleep 0.01` only interleaves the output so the handoff is observable; it's not load-bearing. The main thread is the consumer, pulling values with the blocking `$rx->recv()`. Because a channel preserves send order, the assertions can demand `@results` equal exactly `1..5` with the right first and last elements — order-preservation is what makes that a meaningful check rather than just a count.

This is the explicit, hand-wired side of `stryke` concurrency, and the contrast with the prime finder is the lesson of the chapter. `pgreps/pmap` are for *data parallelism* — independent work over a collection, where the runtime owns the scheduling. `spawn + pchannel` is for *task parallelism with a dependency* — a producer and consumer that must hand off in a controlled stream. You reach for channels exactly when the work doesn't decompose into an order-insensitive map, and the blocking `recv` is doing the synchronization a `join` or a condition variable would do in a lower-level threading API.

Parallel Link Checker — `parallel_link_checker.stk`

Network I/O is the classic case where parallelism pays for itself: checking four URLs serially costs the *sum* of four round-trip latencies, while checking them concurrently costs the *max* of one. Most of the wall-clock

time is the worker sitting idle on a socket, so overlapping that idle time is nearly free throughput. The whole concurrent stage is one `pmap` block:

```
var @results = pmap {
  val $res = eval { http_request(_, +{ timeout => 5 }) }

  if ($@) {
    val $err = $@ =~ s/ at .*//r
    +{ url => _, ok => 0, status => 0, err => $err }
  } else {
    +{ url => _, ok => ($res->{status} < 400), status => $res->{status} }
  }
} @urls
```

`pmap` is the *eager* parallel map: it dispatches each URL to a worker, runs the block, and — unlike the streaming `pmaps` — returns results in input order, so `@results[0]` corresponds to `@urls[0]`. That ordering guarantee is the right tradeoff for a link checker, where the report should list URLs in the order you supplied them; the streaming variant would trade that for lower latency you don't need here. Inside the block, `eval { http_request(_, +{ timeout => 5 }) }` is the load-bearing pattern for *any* parallel I/O: the `eval` confines a failure to the single worker that hit it. Without it, one unreachable host would propagate an exception out of the worker and could take down the batch; with it, a DNS failure or a connection refused populates `$@` locally and the block converts it into a failure hash. The `+{ timeout => 5 }` options hash bounds each request so a single hung host can't stall the whole batch indefinitely — timeout-bounding is not optional in concurrent network code, because the slowest worker sets the wall-clock floor.

The error path cleans the message with the non-destructive substitution `$@ =~ s/ at .*//r` — the `/r` flag returns the edited copy instead of mutating `$@` in place — and crucially, *both* branches return a hashref of the same shape. That uniform return type is what lets the downstream loop treat every result identically; there's no “did this one throw” branching at the consumer, because the throwing was already normalized into `{ ok => 0, ... }` at the worker. The HTML-scraping half (elided here) uses `match_all('( ?i)href="([ ^"]+)"', $html)` to pull every link with the pattern as its *first* argument, then reduces each match to its URL with another `/r` substitution. The closing assertion `len(@results) == len(@urls)` is meaningful precisely because `pmap` preserves order and count: a mismatch would mean a worker silently vanished. This is the shape of production concurrent I/O in *stryke* — parallel, timeout-bounded, and per-element failure-tolerant, with no explicit thread pool, future, or `await` in sight.

Streaming Parallel Builtins — `streaming_parallel_no_interop.stk`

The trailing-*s* family — `pmaps`, `pgreps`, `pflat_maps` — are lazy iterators backed by background worker threads. They return results in *completion* order rather than input order, which is the right tradeoff whenever the downstream stage doesn't care about ordering. The file is a guided tour of all three; the listing below is a coherent excerpt covering each builtin, with two intermediate sections (a prime-count under `pgreps` and a parallel char-class tally) left out for length.

```
#!/usr/bin/env stryke

# Demo: the streaming parallel builtins `pmaps` / `pgreps` /
# `pflat_maps` - lazy iterators backed by background worker threads.
#
# Contrast with `pmap` / `pgrep` / `pflat_map` (no trailing-s):
# * eager variants return in INPUT order (collect + re-sort)
# * streaming variants return in COMPLETION order (lower latency,
#   non-deterministic order)

p "— 1. pmaps: streaming square —"
val @squared = pmaps { _ * _ } 1:10
val @sorted = sort { _ <=> _1 } @squared
p "  set-equal to serial map: @sorted"
die "set differs" unless "@sorted" eq "1 4 9 16 25 36 49 64 81 100"

# Stream-then-sum is the classic order-insensitive pattern.
val $sum_squares = pmaps { _ * _ } 1:100 |> sum
p "  sum of squares 1..100:    $sum_squares"
die "sum-squares" unless $sum_squares == 338350    # n(n+1)(2n+1)/6

# — 2. pgreps: streaming filter —

p ""
p "— 2. pgreps: streaming filter —"
val @evens = pgreps { _ % 2 == 0 } 1:20
val @evens_sorted = sort { _ <=> _1 } @evens
p "  evens 1..20:              @evens_sorted"
die "evens" unless len(@evens_sorted) == 10

# — 3. pflat_maps: 1-to-N streaming flat-map —

p ""
p "— 3. pflat_maps: streaming 1-to-N expansion —"
val $total_items = len pflat_maps { (_, _ * 10, _ * 100) } 1:10
p "  10 inputs × 3 outputs:    $total_items"
die "expand" unless $total_items == 30

# Combine with downstream uniq - pflat_maps emits, uniq dedupes.
val @u = uniq(pflat_maps { (_, _ % 3, _ % 5, _ % 7) } 1:50)
val @uniq_sorted = sort { _ <=> _1 } @u
p "  uniq mod-3/5/7 residues: @uniq_sorted"
die "residues" unless len(@uniq_sorted) == 7    # {0..6}

# — 5. fan-out then aggregate via pflat_maps + freq —

p ""
p "— 5. pflat_maps fan-out + freq aggregate —"
val @words = ("apple", "ant", "banana", "berry", "cherry")
val @letters = pflat_maps { split //, _ } @words
val $by_letter = freq(@letters)
printf("  most common letter: %s (%d times)\n",
  (sort { $by_letter->{_1} <=> $by_letter->{_} } keys %$by_letter)[0],
  (sort { $by_letter->{_1} <=> $by_letter->{_} } values %$by_letter)[0])
die "letter count (got " . len(@letters) . ")" unless len(@letters) == 25
```

The defining contrast is spelled out in the header: the no-*s* variants (*pmap*, *pgrep*, *pflat_map*) collect and re-sort to return in input order, paying latency for determinism; the *-s* variants stream results as they finish, lower-latency but unordered. The first demo makes the consequence explicit — *pmaps* { _ * _ } 1:10 produces the right set of squares, but the program must *sort* before comparing to the ordered string. The

`block { _ * _ }` squares the topic with two references to the same implicit parameter.

The order-insensitive sweet spot is the second example: `pmaps { _ * _ } 1:100 |> sum`. Summation doesn't care what order the squares arrive in, so streaming is purely a win — results feed straight into `sum` as workers complete, with no collect-and-sort overhead. This is the pattern the header calls “stream-then-sum,” and it generalizes to any commutative aggregate: `count`, `uniq`, `freq`, or any sketch.

`pflat_maps` is the one-to-many variant. `pflat_maps { (_, _ * 10, _ * 100) } 1:10` emits three values per input for 30 total, and because flat-map's whole job is to flatten a stream of variable-length results, completion-order streaming fits it naturally. The final example chains it into `freq` for a parallel letter-frequency count over a word list — `split //`, `_` explodes each word into characters across threads, and `freq` tallies the merged stream. The nested sort comparators `{ $by_letter->[_1] <=> $by_letter->[_] }` rank by descending count using the two-element `_/1` convention. The rule the three demos converge on is concrete: prefer the streaming `-s` variant when the next stage is a commutative reduction (`sum`, `count`, `uniq`, `freq`, any sketch) and the eager variant when downstream consumers need input order. The streaming form wins only because the ordering it sacrifices was never needed.

Adaptive Parallelism — `adaptive_parallelism.stk`

The previous examples used a fixed amount of parallelism. This one scales the worker fleet at runtime — start small, observe the bottleneck, add capacity — using stryke's multi-pool congregation API. The arc is provision → measure → grow → tear down; stage 1, the provision-and-measure step, shows the shape of every stage:

```
# Stage 1: start small. Two workers to feel out the workload.
val @stage1 = congregation(2)
printf "stage 1: spawned %d primary workers\n", len(@stage1)

# Initial job: sum 1..50000 on each worker (CPU-bound).
val $job_code = "var \s = 0; for var \i (1:50_000) { \s += \i } \s"
val %r1 = harvest $job_code, @stage1, 30_000
printf "stage 1 result: %d workers returned %d each\n",
    len(keys %r1),
    (values %r1)[0]
```

Stages 2 and 3 follow the same `anoint/harvest/excommunicate` vocabulary, described below.

The vocabulary is deliberately thematic — `congregation`, `anoint`, `harvest`, `excommunicate` — but the mechanics are a worker-pool API. `congregation(2)` spins up a primary pool of two workers and returns their handles. `harvest $job_code, @stage1, 30_000` distributes a job to those workers with a 30-second timeout and returns a hash of per-worker results keyed by worker. The detail that makes this a *distributed* job runner rather than a thread pool is that the job is passed as a **string of stryke source** — `"var \s = 0; for var \i (1:50_000) { \s += \i } \s"` — which each worker compiles and runs independently, returning the tail value `\s`. Shipping source rather than a closure is what lets workers live in separate processes (or, in the full API, separate machines): there's no shared address space to capture from, so the unit of work has to be self-contained code, not a heap-referencing lambda. The backslash-escaped sigils (`\s`, `\i`) are there because the job is a double-quoted string in the *host* program; the worker sees plain `\s/\i`. The digit-group separators (`50_000`, `30_000`) are stryke's readable numeric literals.

The adaptive move is `anoint(3)`, which adds a *secondary* pool of three workers under an independent controller. The separate-controller design is the isolation guarantee the comments stress: anointing more capacity cannot disturb the primary pool's in-flight state, so you can scale up a running system without risking the work already dispatched. Stage 3 re-runs the identical job on `@stage1` to demonstrate the primary pool stays live and responsive after the fleet grew. The comment in the real file is candid that *using @stage2* for the same job would require switching the current controller — an operation a higher API tier exposes — so the demo proves the capacity exists rather than routing work to it.

The pattern is for workloads whose parallelism demand isn't known up front: provision conservatively, measure with a real job, grow the fleet only when a stage proves to be the bottleneck, then `excommunicate(@stage1)`

tears it down cleanly. The example is CI-safe by construction — pure loopback forks, no network — which is what lets a demo that genuinely spawns and scales worker pools run in a headless test environment without external infrastructure.

Chapter 25 — Worked Examples: Text, Data, and Simulations

This chapter shows stryke doing the everyday work of a scripting language and then some: translating biological sequences, rendering 3D graphics to ASCII, simulating cellular automata, rate-limiting with a token bucket, building an observability dashboard from probabilistic sketches, and doing file I/O the stryke way. The throughline is that text-munging, simulation, and data aggregation all read cleanly with the same small set of idioms — hashes, ranges, pipelines, and the thread-first macro.

RNA Codon Translator — `rna_codon_translator.stk`

Translating RNA to a protein is a hash lookup over three-letter chunks — a clean fit for stryke's string slicing and hash-based dispatch. The program walks a sequence in codon-sized steps and stops at the first STOP codon, mirroring how a ribosome reads mRNA. The stepping loop is the idiom worth isolating:

```
fn Bio::translate_codon($codon) = $CODONS{uc $codon} // "Unknown"

fn Bio::translate_sequence($seq) {
  var @amino_acids
  val $s = uc $seq

  # 1. Split into 3-char chunks via range
  # 2. Translate and collect
  # 3. Stop on STOP codon
  for val $i (0:len($s)-3:3) {
    val $codon = $s[$i:$i+2]
    val $aa = Bio::translate_codon($codon)
    last if $aa eq "STOP"
    push @amino_acids, $aa
  }

  \@amino_acids
}
```

The genetic code itself is a 64-entry hash literal `%CODONS` (elided above; the real file spells out every codon), and the lookup `$CODONS{uc $codon} // "Unknown"` is the whole translation table reduced to one defined-or-expression. The `uc` normalization on both the key and the input means case never matters to the caller. The biology maps cleanly onto two stryke features. First, `for val $i (0:len($s)-3:3)` is a **three-part colon range** — start, end, *step* — and the step of 3 is the codon frame: RNA is read in non-overlapping triplets, so a stride-3 range is the literal expression of “the reading frame.” A two-part range with manual `$i += 3` bookkeeping would work but would put the stride in the loop body where it's easy to get wrong; here the stride is in the range itself. Second, `$s[$i:$i+2]` is a start-end character slice that lifts the codon out as a substring without an explicit `substr` call. The `last if $aa eq "STOP"` halts at the first stop codon exactly as translation terminates at a stop, so the early exit isn't an optimization — it's the biology.

The driver invokes the function two ways on purpose: pipe-forward `$sample_rna |> Bio::translate_sequence` and thread-first `~> $long_rna Bio::translate_sequence`. For a single-stage call these are interchangeable, and showing both side by side is the file's way of demonstrating that `|>` and `~>` thread their left operand into the first argument position identically — the difference only appears in multi-stage chains,

where `~>` is the preferred form. The assertions pin the output rather than just printing it: three amino acids for `AUGUUUCUCAA` starting with Methionine, and seven for the longer sequence ending in Threonine after its mid-sequence stop codon. Checking the *last* amino acid of the long sequence specifically proves the mid-sequence STOP truncation worked — a correctness property the short example can't exercise.

ASCII Ray Tracer — `ascii_raytracer.stk`

A ray tracer in a scripting language is a good stress test of numeric performance and abstraction. This one renders a shaded sphere to the terminal using a `struct` for 3D vectors with methods, ray-sphere intersection math, and ASCII shading by surface normal.

The `struct Vec3` declaration packs typed fields and methods together. `x => Float, y => Float, z => Float` declares the typed shape, and the methods — `add, sub, mul, dot, mag, unit` — are all single-expression `fn ... = ...` definitions that operate on `$self`. The vector constructor is called by name with keyword arguments, `Vec3(x => ..., y => ..., z => ...)`, and methods chain naturally: `Vec3(x => 1.0, y => -1.0, z => -1.0)->unit()` builds and immediately normalizes the light direction. This is object-oriented stryke without ceremony — the struct is the data, the methods are the algebra.

`RT::hit_sphere` is textbook ray-sphere intersection by the quadratic formula. It forms the coefficients `qa, qb, qc` from dot products, computes the discriminant, and returns the near intersection distance $(-qb - \sqrt{\text{disc}}) / 2qa$ — or `-1.0` to signal a miss when the discriminant is negative. Every operation flows through the `Vec3` methods, so the math reads like the vector equations it implements rather than a soup of component arithmetic.

The render loop casts one ray per character cell. It maps pixel `(x, y)` to screen coordinates — with an aspect correction for the fact that terminal characters are taller than they are wide — builds a normalized ray direction, and tests it against the sphere. On a hit it computes the surface normal, applies Lambertian diffuse shading via `$normal->dot($LIGHT_DIR)` clamped at zero, and indexes into the ASCII brightness ramp `" .:-=+*#%@"` to pick a character. The self-test at the end fires a ray straight down the z-axis and asserts it hits the sphere at distance `3.5` — the camera-to-near-surface distance for a radius-1.5 sphere centered at `z=5` — pinning the intersection math to within `1e-4`.

ASCII Game of Life — `game_of_life_cli.stk`

Conway's Game of Life is the canonical cellular automaton, and stryke renders it with nested ranges and a step function whose entire ruleset is one ternary. This version evolves a glider one generation on a 5×5 grid. The two functions that carry the algorithm:

```
fn life_count($grid, $r, $c) {
  var $count = 0
  for val $dr (-1:1) {
    for val $dc (-1:1) {
      next if $dr == 0 && $dc == 0
      val $nr = $r + $dr
      val $nc = $c + $dc
      $count++ if $nr >= 0 && $nr < 5 && $nc >= 0 && $nc < 5 && $grid->[$nr][$nc]
    }
  }
  $count
}

fn life_step($grid) {
  var @next
  for val $r (0:4) {
    var @row
    for val $c (0:4) {
      val $n = life_count $grid, $r, $c
      val $alive = $grid->[$r][$c]
      push @row, $alive ? ($n == 2 || $n == 3 ? 1 : 0)
                        : ($n == 3 ? 1 : 0)
    }
    push @next, \@row
  }
  \@next
}
```

`life_count` sweeps the 3×3 offset window with two nested colon ranges `(-1:1)` over `$dr` and `$dc`, skipping the center with `next if $dr == 0 && $dc == 0`. The signed range `-1:1` is doing real work here — it directly expresses “the eight neighbors are the offsets from -1 to +1 in each axis, minus the origin,” which is the Moore neighborhood by definition. The bounds checks fold into the increment condition (`$count++ if`

`$nr >= 0 && ... && $grid->[$nr][$nc]`), so off-grid neighbors are simply false and contribute nothing; this is the standard dead-border convention for a finite Life grid, and folding it into the guard avoids a separate clamping pass.

`life_step` collapses Conway's four rules (underpopulation, survival, overpopulation, birth) into a single nested ternary: a live cell survives only on 2 or 3 neighbors, a dead cell is born only on exactly 3, everything else is dead. The structurally important choice is that it builds `@next` fresh and returns `\@next` rather than mutating `$grid` in place. Cellular automata *require* this: every cell's next state depends on the *current* state of its neighbors, so mutating cells as you scan would let an already-updated cell corrupt the count of a cell processed later in the same generation. The fresh-grid double-buffer is correctness, not style — an in-place update would compute a different, wrong automaton.

Rendering is a tidy pipeline — `join ' ', map { _ ? '#' : '.' } @$row` maps each cell to a glyph and joins the row. The assertions check the glider's next phase by *exact coordinate*: `$g2->[1][0]`, `$g2->[1][2]`, `$g2->[2][1]`, `$g2->[3][1]` are the four live cells of the advanced glider. Verifying specific coordinates rather than a live-cell count is what makes this a real test — a count check would pass for any 4-cell pattern, but a glider that moved the wrong direction, or a still-life that didn't move, would fail these. Pinning the exact next-generation positions is the difference between asserting “the automaton ran” and “the automaton computed Conway's rules correctly.”

Token Bucket Rate Limiter — `token_bucket.stk`

A token bucket is the standard rate-limiting algorithm, and the design decision that defines this implementation is making the clock an *explicit argument*. The caller supplies “now,” so the algorithm is fully deterministic and testable without any real sleeping. The refill helper plus one public operation show the whole technique:

```
fn _refill($bk, $now) {
  val $elapsed = $now - $bk->{last}
  if ($elapsed > 0) {
    val $add = $elapsed * $bk->{rate}
    val $new = $bk->{tokens} + $add
    $bk->{tokens} = $new > $bk->{cap} ? $bk->{cap} : $new
    $bk->{last} = $now
  }
}

fn try_consume($bk, $now, $n) {
  TB::_refill($bk, $now)
  if ($bk->{tokens} >= $n) {
    $bk->{tokens} = $bk->{tokens} - $n
    return 1
  }
  0
}
```

The bucket is a hash holding capacity, refill rate, current token count, and the timestamp of the last refill. Every public operation — `try_consume`, `available`, `time_until` — calls `TB::_refill($bk, $now)` first, and `_refill` is the heart of the algorithm: it adds `elapsed * rate` tokens accrued since the last call, clamps at capacity with a ternary, and advances `last`. This is **lazy refill on read**, and it's the standard token-bucket trick. The naive implementation runs a background timer that drips tokens into every bucket on a fixed tick; that scales terribly — a million idle buckets cost a million timer wakeups. Lazy refill inverts it: nothing happens until someone asks, and the answer is computed in `O(1)` from the elapsed time. A bucket nobody touches for an hour costs zero work, and when it's finally read, a single multiply reconstructs exactly how many tokens it should hold (capped). The clamp matters — without it, a long idle period would let `tokens` exceed capacity, which would defeat the burst limit the bucket exists to enforce.

The explicit-clock design is what makes the algorithm testable, and that is the point the file is making. Every

function takes `$now` as a parameter instead of calling a real time source, so the driver can advance a plain integer `$t` — 0, then 2, then 100 — and the bucket responds deterministically: burst-consume 8 of 10 succeeds, a follow-up 5 fails (only 2 left), “waiting 2 seconds” refills 4 tokens at rate 2/sec so the next 5 succeeds, and a long wait caps back at 10. There is no `sleep`, no wall-clock dependency, and therefore no flakiness — a test that depended on real elapsed time would be slow and intermittently wrong on a loaded CI box. Injecting the clock as an argument is the difference between a rate limiter you can unit-test in microseconds and one you can only test by waiting. The `package TB; / package main;` split keeps `_refill` an internal detail while exposing `new`, `try_consume`, `available`, and `time_until`; the last computes how long a caller must wait for `n` tokens as `deficit / rate`, the back-off hint a real limiter hands clients so they can retry intelligently rather than spin.

API Health-Check Dashboard — `api_health_check.stk`

Observability tooling lives or dies on probabilistic data structures, and stryke ships them as builtins. This dashboard simulates endpoint pings and computes per-endpoint latency percentiles with t-digests, distinct-cardinality with HyperLogLog, and noisiest-endpoint rankings with a top-k sketch. The file is large, so the listing is a complete coherent excerpt of the sketch setup, simulation loop, and dashboard; the alerts and rollup sections are summarized after.

The architecture is a *hash of sketch handles*. `$latency{$ep} = t_digest(100)` gives each endpoint its own t-digest — a structure that estimates quantiles in bounded memory — and `td_add` streams latency samples into it. When the dashboard needs percentiles, `td_quantile($latency{$ep}, 0.99)` reads them back. The thread-first macro tidies the conversion: `~> td_quantile($latency{$ep}, 0.50) int` pipes the quantile straight into `int`. Response classification needs no regex at all — `int($resp->{code} / 100)` maps any 2xx/4xx/5xx code to its class 2/4/5 with integer division.

Three different sketches cover three different questions. The per-endpoint t-digests answer “what’s the tail latency.” `hll(14)` — a HyperLogLog with 2^{14} registers — answers “how many *distinct* request signatures did we see,” fed by `hll_add($hll_req, "sep#i#cls")` and read with `hll_count`. And `topk(3)` answers “which endpoints are noisiest,” fed `topk_add($tk_noisy, $ep)` on every 4xx/5xx and read with `topk_top`. Each is a bounded-memory probabilistic answer to a question that would otherwise require storing every event.

The elided tail of the file uses these accumulated sketches three ways. An alerts pass flags any endpoint whose error rate exceeds 15% (the `/api/payments` endpoint with its 0.25 simulated failure rate is the designed offender). A noisiest-endpoints section reads the top-k sketch. And a global latency rollup demonstrates *sketch algebra* — `$latency{...} + $latency{...} + ...` merges all six per-endpoint t-digests into one, then reads global p50/p90/p99 from the merged digest. That mergeability is the whole reason t-digests beat naive percentile tracking in distributed systems: partial digests computed on different hosts combine into a correct global answer.

Advanced File Operations — `advanced_file_ops.stk`

File I/O is where stryke’s terseness pays off daily. This program shows the slurp prefix `c"..."`, the `spurt` writer, lazy line iteration, and zsh-style glob qualifiers — the everyday vocabulary of shell-adjacent scripting. The four idioms in sequence:

```
# 1. Spurt: write data to file in one call.
spurt $tmp, "line 1: hello\nline 2: world\nline 3: stryke\n"

# 2. Slurp prefix: `c"..."` is the shortest path to file contents.
val $content = c"$tmp"

# 3. Lazy line iteration with filtering.
c"$tmp" |> lines |> greps { /stryke/ } |> e p

# 4. Zsh-style globbing with qualifiers - `(.)` = plain files only.
var @files = glob("$examples_dir/*.stk(.)")
```

`spurt $tmp, "..."` writes a whole string to a file in one call, the inverse of slurping. Reading it back is `c"$tmp"` — the `c"..."` slurp prefix interpolates the path and returns the entire file as a string. The pairing reduces the two most common file operations to the fewest characters that can still name them; in Perl the same round-trip is a three-line `open/print/close` plus an `open/local-$/<$fh>` slurp, and in most languages it’s a `with open(...)` block in each direction.

The lazy line pipeline is the part with a real performance consequence: `c"$tmp" |> lines |> greps { /stryke/ } |> e p`. It splits the file into lines with `lines`, filters with the streaming `greps` and a bare regex `{ /stryke/ }` matching the topic, and prints survivors through `e p` (the each-iterator applied to `p`). The laziness is load-bearing — `lines` yields one line at a time rather than materializing the file as an array, and `greps` is streaming, so the pipeline’s peak memory is one line regardless of file size. The same logic written as `grep { /stryke/ } split /\n/, slurp($f)` would hold the entire file *and* the entire line array in memory at once; the lazy spelling scales to files larger than RAM, which is exactly when it matters.

The globbing borrows zsh’s qualifier syntax directly — `glob("$examples_dir/*.stk(.)")` matches `.stk` files where the trailing `(.)` restricts to plain files, excluding directories. That this is identical to zsh’s glob qualifier isn’t coincidence; it’s stryke’s shell lineage surfacing, so a zsh user’s muscle memory transfers verbatim. `dirname(__FILE__)` resolves the script’s own directory and `@files[0:2]` slices the first three

with a start-end slice. The assertions use `aok` with a bound regex and a count check to confirm the write-then-read round-trip. For anyone coming from shell scripting, every operation here has a one-character or one-word spelling, which is the design intent: file plumbing should disappear into the script, not dominate it.

Chapter 26 – Worked Examples: AI and Agents

stryke treats AI as a language primitive, not a library. `ai` is a builtin alongside `print`; `ai_classify`, `ai_filter`, `ai_map`, and `ai_sort` are collection operations; tools are exposed with `tool fn`; and `ai_mock` makes every one of them deterministic and network-free for testing. This chapter shows four programs that put those primitives to work — an agentic RAG loop, batch collection processing, structured schema extraction, and dropping a process into distributed agent mode.

Agentic RAG – `ai_agent_rag.stk`

Retrieval-augmented generation with an autonomous agent loop: the `ai` builtin decides which tools to call, calls them, and synthesizes an answer. The whole thing runs deterministically under mock mode, which is how AI code becomes testable.

```
#!/usr/bin/env stryke

# Agentic RAG (Retrieval-Augmented Generation) – an AI agent that searches
# local memory and the web to answer complex questions.
#
# This example demonstrates:
# * `ai_memory_save` / `recall` for semantic document retrieval.
# * `tool fn` to expose custom logic to the AI agent.
# * The `ai` builtin's autonomous agent loop.
# * `ai_mock` for deterministic, network-free testing.
#
# Run: STRYKE_AI_MODE=mock-only stryke examples/ai_agent_rag.stk

# — 1. Setup Mock responses —————

# We mock the embedding probe to enable mock-only AI memory
ai_mock_install("embed:probe", "")

# Mock for the search and memory recall
# We prefix mocks with [MOCK] to distinguish them from tool outputs
ai_mock_install("(?i)^doc:.*", "Stryke info: Created by Menke Technologies.")
ai_mock_install("(?i)^web:.*", "Tokyo weather: Sunny, 22°C.")

# Mock for the full agent query to allow passing in mock-only mode
ai_mock_install("(?i)Who created Stryke.*weather in Tokyo",
  "Stryke was created by Menke Technologies. The weather in Tokyo is sunny, 22°C.")

# ... later ...

# Use explicit mocks in tools to ensure they work in mock-only mode
tool fn search_docs($query: string) "Search our local documentation for relevant info" {
  p " [TOOL] searching docs for: $query"
  # Map the tool query to a mock probe
  ai "doc:$query"
}

tool fn web_search($query: string) "Search the web for real-time information" {
  p " [TOOL] web search for: $query"
  # Map the tool query to a mock probe
  ai "web:$query"
}

# — 4. Run the Agent —————

p ""
p "— running agentic RAG query —————"

val $question = "Who created Stryke and what is the weather in Tokyo?"
p "Question: $question"

# The `ai` builtin will:
# 1. Identify that it needs information about "Who created Stryke".
# 2. Call `search_docs`.
# 3. Identify that it needs information about "weather in Tokyo".
# 4. Call `web_search`.
# 5. Synthesize the final answer.

val $answer = ai($question,
  system => "You are a helpful research assistant. Use the provided tools to find
    ~ accurate information.",
  max_turns => 5
)

p ""
p "Answer: $answer"

p ""
```

The headline construct is `tool fn`. Declaring `tool fn search_docs($query: string) "Search our local documentation..." { ... }` defines an ordinary function *and* registers it as a tool the AI agent can invoke. The typed parameter `$query: string` and the trailing string-literal description aren't decoration — they become the tool's schema and the natural-language hint the model uses to decide when to call it. This is how you give an LLM agent custom capabilities in stryke: write a normal function with a type signature and a docstring, prefix it with `tool`, and it's callable from the agent loop.

The `ai($question, system => ..., max_turns => 5)` call runs the autonomous loop. Given the question "Who created Stryke and what is the weather in Tokyo?", the agent recognizes it needs two pieces of information, calls `search_docs` for the first and `web_search` for the second, then synthesizes a combined answer — all bounded by `max_turns => 5` so it can't loop forever. The `system` argument sets the agent's role. This is a full ReAct-style tool-using loop driven by a single builtin call.

What makes it *testable* is `ai_mock`. Every `ai_mock_install(pattern, response)` registers a deterministic answer keyed by a regex against the prompt, and running under `STRYKE_AI_MODE=mock-only` means no network call ever fires. The mocks here cover the embedding probe, the `doc:` and `web:` tool probes, and the full agent query. The result is an agentic RAG pipeline that runs identically in CI with zero API keys and zero latency — the mock layer is what turns nondeterministic LLM behavior into a reproducible test.

AI Collection Operations — `ai_collection_ops.stk`

Beyond the agent loop, stryke exposes LLM-backed *collection* operations that read exactly like their deterministic cousins: `classify`, `filter`, `map`, `sort`. This program runs a batch of product reviews through three of them with the pipe-forward operator:

```
val $ref = \@reviews

# Using pipe-forward for clarity with multi-arg calls
$ref |> ai_classify("sentiment", into => ["Positive", "Negative"])
      |> e { p "Sentiment: $_" }

$ref |> ai_filter("constructive")
      |> e { p "Constructive: $_" }

$ref |> ai_map("summarize in 3 words")
      |> e { p "Summary: $_" }
```

The three stages are the LLM-backed analogs of `classify`, `grep`, and `map`. `ai_classify("sentiment", into => ["Positive", "Negative"])` labels each review against a fixed set; `ai_filter("constructive")` keeps the ones the model judges constructive; `ai_map("summarize in 3 words")` transforms each into a terse summary. The shape is identical to the deterministic verbs — array in, array out, threaded with `|>` — and the only thing that changes is that the *predicate is a natural-language string* instead of a block. Where `grep { /good/ }` matches a regex, `ai_filter("constructive")` matches a judgment the model makes; where `map { uc _ }` applies a mechanical transform, `ai_map("summarize in 3 words")` applies a semantic one. That symmetry is the design intent: an engineer who already knows stryke's collection vocabulary has nothing new to learn except that the work is now done by a model. The `into => [...]` named argument is also why pipe-forward is the right operator here rather than the thread macro — `|>` threads the data into the first argument while leaving room for the trailing named args that multi-argument AI calls need.

The mocking strategy is the part worth studying, because it's what makes nondeterministic LLM calls into a deterministic test. Each `ai_mock_install` matches the *internal prompt template* a given verb generates — `'Classify each item.*labels: Positive, Negative'` for `ai_classify`, `'Decide whether.*criterion: constructive'` for `ai_filter`, `'Apply this instruction.*summarize'` for `ai_map` — and returns canned JSON. This only works because the verbs emit *structured* prompts with stable phrasing: the mock can pattern-match the template the verb produces, so installing a mock is effectively stubbing the verb's behavior without knowing its implementation. The closing assertion installs one more mock

for a “positive sentiment” filter, runs `ai_filter($ref, "positive sentiment")`, and checks that exactly two reviews pass and the first mentions loving Stryke. Under `STRYKE_AI_MODE=mock-only` no network call fires, so that is a genuine, repeatable unit test of an LLM-backed filter — the kind of thing usually considered untestable because the model’s output isn’t deterministic. The mock layer relocates the determinism boundary so the *stryke* logic around the model can be pinned even when the model itself can’t be.

AI SQL Generator — `ai_sql_generator.stk`

When you need *structured* output from an LLM, not free text, *stryke*’s `ai` builtin takes a `schema` argument and returns a typed hashref. This program turns natural-language requests into SQL with type-safe extraction. The helper is the whole pattern:

```
fn SqlGen::generate($request) {
  p "Request: $request"

  val $schema = +{
    sql      => "string",
    tables   => "array",
    complexity => "int"
  }

  val $res = ai("Convert to SQL: $request",
    schema => $schema,
    system => "You are a SQL expert. Return ONLY valid JSON matching the schema.")

  p "  SQL: #{ $res->{sql} }"
  p "  Tables: #{ join(' ', '@{$res->{tables}}') }"
  p "  Complexity: #{ $res->{complexity} }"

  return $res
}
```

The `schema` argument is the feature that changes the interaction. `$schema = +{ sql => "string", tables => "array", complexity => "int" }` declares the *shape* the model must return, and passing it as `ai("...", schema => $schema, ...)` constrains the output to a typed hashref instead of free-form text. The result is destructured immediately — `$res->{sql}`, `$res->{tables}`, `$res->{complexity}` — with each field present and correctly typed. This is the difference between two failure-prone steps (prompt the model for SQL, then hand-parse its prose answer with a regex and hope it didn’t wrap the SQL in a code fence or add an apology) and one reliable step (declare the shape, get back an object you can index). The `system` prompt reinforces it by pinning the model to “return ONLY valid JSON matching the schema”; schema plus system prompt together are what make the extraction dependable rather than a parsing gamble.

The use case is natural-language-to-DSL: “Show me all users who are active” becomes `SELECT * FROM users WHERE active = true` plus the metadata that it touches the `users` table and has complexity 1. Wrapping the call in `SqlGen::generate` gives a reusable namespaced helper. The assertions are where the type guarantee earns its keep: `$r2->{tables}->[0] |> aeq "products"` indexes into `tables` *without* first checking that the field exists or that it’s an array. In code that parsed raw model text, that indexing would need a defensive `ref($res->{tables}) eq "ARRAY"` guard because the model might have returned a string, a missing key, or malformed JSON. The schema removes that whole class of guard — `tables` is an array because the extraction enforced it — and the test exercises exactly that contract, asserting the structured field directly rather than re-validating the model’s output shape.

Agent Mode — `agent_become.stk`

The final example is operational rather than algorithmic: it drops the running stryke process into distributed agent mode and connects to a controller. The entire program is five lines of logic around one builtin:

```
val $addr = $ENV{CONTROLLER_ADDR} // "localhost:9999"
val $name = $ENV{AGENT_NAME}      // $ENV{HOSTNAME} // "anonymous"

p "joining controller at $addr as '$name'"

val $rc = agent($addr, $name)
if ($rc != 0) {
    p "no controller reachable - demo is a no-op without one (exit 0)"
    exit 0
}
exit $rc
```

The `agent($addr, $name)` builtin is the whole point: calling it transforms the current process into a distributed worker that connects to a controller, accepts jobs, and runs them — exactly equivalent to launching `stryke agent --controller HOST --port PORT` from the command line. The reason to expose it as a *builtin* and not only a CLI subcommand is wrapping. As a function call, agent-mode entry sits inside whatever startup logic the script needs first: structured logging, environment validation, secrets fetching from a vault, or the exponential-backoff reconnect loop the file's header sketches as `while (1) { agent($addr, $name) == 0 ? last : sleep 5 }`. The CLI form `stryke agent ...` is a fixed entry point you can't decorate; the builtin lets the agent become a worker *after* the script has done its own initialization. The CLI subcommand is the convenience; the builtin is the primitive the subcommand is built on.

Configuration comes from the environment with defined-or chaining, and the two-level fallback on the name is the idiom to note: `$ENV{AGENT_NAME} // $ENV{HOSTNAME} // "anonymous"` tries an explicit name, then the hostname, then a constant, each `//` supplying the next rung when the previous is unset. That reads top-to-bottom as a precedence order, which is exactly what layered configuration is — explicit override beats inferred default beats hard fallback. The exit-code handling is what lets this ship as a runnable demo: `agent` returns 0 on a clean controller-initiated shutdown and nonzero on a connection failure, so the script catches the failure and exits 0 anyway. The demo's job is to show the *invocation*, not to require a live controller at demo time, so converting “no controller reachable” into a clean exit means the example runs in CI without standing up cluster infrastructure first — the same CI-safety discipline the adaptive-parallelism example used, applied to a network primitive instead of a fork.

Chapter 27 — Worked Examples: Graph Algorithms

Graphs are where a language's data-structure ergonomics earn their keep. The classic graph algorithms — shortest paths, spanning trees, connectivity — are short on arithmetic and long on bookkeeping: adjacency lists, visited sets, parent-pointer chains, priority queues, disjoint sets. A language that makes this bookkeeping verbose makes graph code unreadable, and a language that makes it implicit makes graph code untrustworthy. Stryke threads this needle by giving you Perl-lineage references and sigils for the structure, hash-and-array literals for the graph itself, and a small set of idioms — expression-bodied functions, the topic variable, `:` ranges, list-assignment destructuring — for the inner loops. This chapter walks six real programs from the `examples/` tree, all of them written in the strict `--no-interop` dialect (the `_no_interop.stk` suffix), which is simply the form that parses without the interop-compatibility relaxations. Every excerpt below is copied verbatim from the file it names; the algorithms are the textbook ones, and the point of the chapter is to watch how stryke expresses them.

Dijkstra's shortest paths with a hand-rolled binary heap

The `dijkstra_no_interop.stk` program is the most complete shortest-path example in the tree because it doesn't lean on a library priority queue — it builds one. The file opens with `package Dijkstra` and defines the min-heap as three free functions over a plain array. The constructor is a single expression-bodied function, `fn pq_new = []`, which is stryke's terse `fn name = expr` form: no braces, no explicit `return`, the body *is* the value. The push operation is the classic sift-up, and it shows several stryke idioms at once:

```
fn pq_push($q, $pair) {
  push @$q, $pair
  var $idx = (len @$q) - 1
  while ($idx > 0) {
    var $p = int(($idx - 1) / 2)
    last if $q->[$p]->[0] <= $q->[$idx]->[0]
    ($q->[$p], $q->[$idx]) = ($q->[$idx], $q->[$p])
    $idx = $p
  }
}
```

The queue is passed by reference — `$q` holds an arrayref, and `push @$q, $pair` dereferences it to push in place, so the mutation is visible to the caller. Heap entries are `[distance, node]` pairs, indexed with the chained arrow form `$q->[$p]->[0]` to read the distance component. The swap on the penultimate line is the part that would be three statements and a temporary in C: `($q->[$p], $q->[$idx]) = ($q->[$idx], $q->[$p])` is list assignment, evaluating the right-hand side as a list and binding it to the left-hand targets atomically, which makes the swap a single readable line. The statement modifier `last if ...` reads as the loop-exit condition it is — the parent is already no larger than the child, so the heap property holds and we stop sifting.

The pop operation mirrors it with a sift-down, and it leans on the same destructuring to find the smaller child:

```

fn pq_pop($q) {
  return undef if !$q
  val $res = $q->[0]
  val $last = pop @$q
  if (@$q) {
    $q->[0] = $last
    var $idx = 0
    val $n = len @$q
    while (1) {
      val $l = 2 * $idx + 1
      val $r = 2 * $idx + 2
      var $smallest = $idx
      $smallest = $l if $l < $n && $q->[$l]->[0] < $q->[$smallest]->[0]
      $smallest = $r if $r < $n && $q->[$r]->[0] < $q->[$smallest]->[0]
      last if $smallest == $idx
      ($q->[$idx], $q->[$smallest]) = ($q->[$smallest], $q->[$idx])
      $idx = $smallest
    }
  }
  $res
}

```

Two stryke distinctions matter here. The first is **val** versus **var**: **val \$res** and **val \$last** are immutable bindings — the result and the popped tail never need to change — while **var \$idx** and **var \$smallest** are the loop counters that do. The discipline of marking everything that *can* be immutable signals intent to the reader before they read the body. The second is the boolean coercion **!@\$q**: an array in boolean context is true when non-empty, so **return undef if !\$q** is the empty-queue guard, and **if (@\$q)** after the **pop** asks “is anything left to re-heapify?”. The function’s last expression, the bare **\$res**, is the implicit return — stryke returns the value of the final expression with no **return** keyword.

With the heap in place, the algorithm itself is compact. The graph is an adjacency list keyed by node, each value a list of [**neighbor**, **weight**] edges, and the search keeps two hashes — distances and predecessors:

```

fn shortest_paths($graph, $start) {
  var %dist
  var %prev
  val $pq = pq_new()

  # Initialize start node
  $dist{$start} = 0
  pq_push $pq, [0, $start]

  while (val $pair = pq_pop $pq) {
    val ($d, $u) = @$pair

    # Optimization: skip if we've already found a shorter path
    next if defined $dist{$u} && $d > $dist{$u}

    val $neighbors = $graph->{$u} // []
    for val $edge (@$neighbors) {
      val ($v, $w) = @$edge
      val $new_d = $d + $w

      if (!defined $dist{$v} || $new_d < $dist{$v}) {
        $dist{$v} = $new_d
        $prev{$v} = $u
        pq_push $pq, [$new_d, $v]
      }
    }
  }
  { dist => \%dist, prev => \%prev }
}

```

The `while (val $pair = pq_pop $pq)` header binds the popped pair and tests its truth in one move, so the loop runs until the queue drains. `val ($d, $u) = @$pair` deconstructs the dereferenced pair into distance and node. This is the lazy-deletion variant of Dijkstra: rather than decrease-key on the heap, stale entries are left in and skipped with `next if defined $dist{$u} && $d > $dist{$u}`, which discards any popped distance worse than the best already recorded. The `//` defined-or operator gives the empty-list fallback for nodes with no outgoing edges, `$graph->{$u} // []`, so the inner `for` never iterates over `undef`. The function returns a hash literal wrapping references to both result hashes, `{ dist => \%dist, prev => \%prev }`, and the `\%dist` backslash takes a reference to the hash so it survives past the function's scope.

The `main` package consumes the result by rebuilding the path. It walks the predecessor chain backward from the target and reverses it, a pattern that recurs in every path algorithm in this chapter:

```

var @path
var $curr = $target
while (defined $curr) {
  push @path, $curr
  $curr = $res->{prev}->{$curr}
}
@path = rev @path
p " Path to #{ $target }: #{join ' -> ', @path}"

```

Note the `#{...}` interpolation in the final `p` call, which is stryke's Ruby-style string interpolation embedding `join ' -> ', @path` directly inside the string, and the `p` builtin itself — the terse print that the language inherits as a first-class verb rather than a function you import.

Breadth-first search on an unweighted graph

Where Dijkstra needs a heap, BFS on an unweighted graph needs only a FIFO queue, and `graph_bfs_no_interop.stk` makes that the whole story. The core function namespaces itself directly in its name — `fn BFS::shortest(...)` — which is stryke’s way of declaring a function inside a package without a surrounding `package` block. The queue is a plain array used as a FIFO via `push` to enqueue at the tail and `shift @q` to dequeue at the head:

```
fn BFS::shortest($graph_ref, $src, $dst) {
  return [$src] if $src eq $dst
  var %parent
  var %seen = ($src => 1)
  var @q = ($src)
  while (len(@q) > 0) {
    val $u = shift @q
    val $neighbours = $graph_ref->{$u}
    next unless defined $neighbours
    for val $v (@$neighbours) {
      next if exists $seen{$v}
      $seen{$v} = 1
      $parent{$v} = $u
      if ($v eq $dst) {
        # Walk back to reconstruct.
        var @path = ($dst)
        var $cur = $dst
        while (exists $parent{$cur}) {
          $cur = $parent{$cur}
          unshift @path, $cur
        }
        return \@path
      }
      push @q, $v
    }
  }
  # No path.
  [ ]
}
```

The visited set and the parent map are two hashes, with the source pre-seeded into `%seen` at construction via the literal `($src => 1)`. String equality uses `eq`, not `==` — stryke keeps the Perl distinction between numeric and string comparison operators, and node ids here are strings like `"A"`. The reconstruction uses `unshift @path, $cur` to prepend during the backward walk, which means the path comes out already in source-to-destination order without a reversal step — a small contrast with the Dijkstra example, which collected forward and reversed at the end. When the destination is unreachable the function falls through both loops and returns the empty arrayref `[ ]`. A second function builds on the first to report distances rather than paths, and it shows the ternary doing real work: `len(@$path) == 0 ? -1 : len(@$path) - 1`, where an empty path means unreachable and otherwise the hop count is one less than the cell count. The driver doubles as a test harness — a table of `+{ src => ..., dst => ..., dist => ... }` hashref cases (the `+{ ... }` forces a hashref literal, disambiguating from a code block) — and finishes by checking the triangle inequality $d(A,C) \leq d(A,B) + d(B,C)$, a graph-theoretic invariant that catches a broken BFS a single hardcoded expected value would not.

A* pathfinding on a grid

`a_star_no_interop.stk` generalizes BFS to weighted, heuristic-guided search over a 2-D grid. The heuristic is the Manhattan distance, and it is the chapter’s purest demonstration of stryke’s topic variables — the implicit

positional parameters `_, _1, _2, _3`:

```
fn Star::heur = abs(_ - _2) + abs(_1 - _3)
```

There is no parameter list. In an expression-bodied function the arguments are bound to `_, _1, _2, _3` in order, so this reads the four coordinates as `(r, c, gr, gc)` and computes `abs(r - gr) + abs(c - gc)`. For a one-line pure function this is dramatically tighter than naming four parameters, and the file's comment marks it admissible — it never overestimates true cost on a 4-connected grid, the precondition for A* optimality. Neighbor expansion walks the four orthogonal moves as an inline list of offset pairs and destructures each:

```
for var $d ([-1, 0], [1, 0], [0, -1], [0, 1]) {
  val ($dr, $dc) = @$d
  val $nr = $cur->{r} + $dr
  val $nc = $cur->{c} + $dc
  next if $nr < 0 || $nr >= $rows || $nc < 0 || $nc >= $cols
  val $row = $grid->[$nr]
  next if $row[$nc] eq "#"
  next if exists $closed{"$nr,$nc"}
  val $tentative_g = $cur->{g} + 1
```

The closed-set key is built by interpolating the coordinates into a comma-joined string, walls are the literal `"#"` compared with `eq`, and the grid is an arrayref of strings indexed first to a row then to a cell. The open set is a flat array of node hashrefs; the lowest-`f` node is found with a linear scan over a `:` range of indices (`for val $i (1:len(@open) - 1)`), and the reconstruction at the goal walks a `parent` chain with `unshift @path, [$n->{r}, $n->{c}]` to emit `[row, col]` cells in start-to-goal order. The driver asserts concrete path lengths — the 5×5 grid with a wall column forces a 9-cell detour, the open 3×3 yields a 5-cell L-shaped path, a walled-off goal returns `undef`, and `start == goal` returns a 1-cell path — and those exact counts are the test's teeth: an A* that found a path but not the *optimal* one would fail the length assertion.

Union-Find with path compression and union-by-rank

`union_find_no_interop.stk` implements Tarjan's disjoint-set structure, the substrate beneath Kruskal's MST and a dozen other algorithms. Its constructor is the chapter's densest single line:

```
fn UF::new = +{ parent => [0:_ - 1], rank => [(0) x _], count => _ }
```

Here `_` is the single argument, the element count `n`. The `parent` array is initialized so every element is its own root via the range literal `[0:_ - 1]`, which materializes `0, 1, ..., n-1` directly into an array. The `rank` array is `[(0) x _]` — the list-repetition operator `x` produces `n` copies of `0` — and `count` records the live component total. Three structural idioms (range fill, list repetition, hashref literal) compress what would be a loop and several statements into one expression. The `find` operation is recursive with path compression written as an assignment in the recursive step:

```
fn UF::find($uf, $x) {
  return $x if $uf->{parent}[$x] == $x
  $uf->{parent}[$x] = UF::find($uf, $uf->{parent}[$x])
  $uf->{parent}[$x]
}
```

The base case — a self-parent is a root — returns immediately. Otherwise the recursive call resolves the root, and the result is written *back* into `parent[$x]` before being returned, flattening the tree so subsequent finds short-circuit. The union routine implements union-by-rank, swapping so the shorter tree always hangs under the taller and bumping rank only on a tie, with `return 0 if $ra == $rb` so the function reports whether a merge actually happened. The stress test chains a thousand elements with a postfix `for, UF::union($big, _, _ + 1) for 0:$N - 2`, where `_` is the loop value, collapsing the whole chain to a single component whose `count` must end at `1`.

Kruskal's minimum spanning tree

`kruskal_no_interop.stk` is the payoff for the union-find work — it inlines a stripped-down disjoint set under the `MST::UF::*` namespace and uses it for cycle detection while greedily accepting edges:

```
fn MST::kruskal($n, $edges) {
  var @sorted = sort { _->[2] <=> _1->[2] } @$edges
  val $uf      = MST::UF::new($n)
  var @tree    =
  var $cost    = 0
  for var $e (@sorted) {
    if (MST::UF::union($uf, $e->[0], $e->[1])) {
      push @tree, $e
      $cost += $e->[2]
      last if len(@tree) == $n - 1
    }
  }
  +{ tree => \@tree, cost => $cost }
}
```

The sort comparator is the feature to study: `sort { _->[2] <=> _1->[2] }` uses stryke's bareword positional parameters `_` and `_1` for the two elements being compared instead of Perl's `$a/$b` globals — the style guide calls this out as rule 11. Each edge is `[u, v, weight]`, so `_->[2]` and `_1->[2]` extract the weights and `<=>` gives ascending weight order. The acceptance test is the union call: `MST::UF::union` returns truthy only when it actually merged two distinct components, so a `1` means the edge is safe to add and a `0` means it would close a cycle and is dropped. The `last if len(@tree) == $n - 1` is the early exit. The driver validates against the Cormen nine-vertex graph whose known MST weight is 37, then proves a stronger property by re-running a *fresh* union-find over only the chosen tree edges and asserting every union succeeds — if any returned `0`, the “tree” contained a cycle. That is the most instructive testing pattern in the chapter: it asserts the *defining property* of a spanning tree — acyclicity — through the same primitive the algorithm uses.

Tarjan's strongly-connected components

`scc_tarjan_no_interop.stk` closes the chapter with the one genuinely stateful algorithm — a single DFS pass that discovers strongly connected components by tracking each vertex's discovery `index` and its `lowlink`, the lowest index reachable from its subtree. Stryke carries the entire algorithm state in one hashref so the recursion can mutate it in place:

```
val $state = +{
  adj      => \%adj,
  index    => 0,
  idx_of   => +{},
  low_of   => +{},
  on_stack => +{},
  stack    => [],
  sccs     => [],
};
```

Bundling `index`, the `idx_of/low_of/on_stack` hashes, the working `stack`, and the result `sccs` into a single passed-by-reference record is the stryke way to give a recursive helper shared mutable state without globals. The recursive heart folds a recursed neighbor's lowlink back, while a neighbor already on the stack contributes its `index` (not its lowlink) — the subtle distinction that makes Tarjan correct:

```

for val $w (@{$s->{adj}}{$v}) {
  if (!exists $s->{idx_of}{$w}) {
    SCC::strong_connect($s, $w)
    $s->{low_of}{$v} = $s->{low_of}{$w} if $s->{low_of}{$w} < $s->{low_of}{$v}
  } elsif ($s->{on_stack}{$w}) {
    $s->{low_of}{$v} = $s->{idx_of}{$w} if $s->{idx_of}{$w} < $s->{low_of}{$v}
  }
}
}

```

The component pop fires when a vertex's lowlink equals its own index, identifying it as the root of an SCC; it drains the stack down to and including the root, clearing each popped vertex from the `on_stack` set via `delete`, and pushes the collected component as a new arrayref onto `sccs`. The driver canonicalizes output for comparison — sorting within each component and then sorting the components lexicographically by their comma-joined string form with the string comparator `cmp` — and the closing invariant is the partition property: every vertex of every SCC is walked, none may appear twice (checked via a `%seen` hash), and the total must equal `N`.

Chapter 28 – Worked Examples: Simulation and Physics

Simulation is where a programming language's expressiveness meets its performance budget head-on. Every simulator is a loop over time: a state vector, a transition rule applied once per tick, and some accounting of what changed. The interesting variation lies in how the state is shaped — a flat array of particles, a two-dimensional grid of cells, a recursive spatial tree — and in how the transition rule is parallelised across the available cores or worker processes. This chapter walks six real Stryke programs drawn directly from the [examples/](#) corpus; every excerpt is copied verbatim from a file read in full. The programs span the major simulation idioms: continuous rigid-body dynamics with collision response, two distinct N-body force solvers (one brute-force and distributed across worker processes, one a recursive Barnes-Hut quadtree), two cellular/agent automata on a grid, and the canonical embarrassingly-parallel Monte Carlo estimator expressed both with in-process thread chunking and with cross-process worker congregations.

physics_engine_simulation.stk – rigid bodies, impulses, symplectic Euler

The most structurally complete simulator in the set is the 2-D rigid-body engine in [physics_engine_simulation.stk](#). It builds the entire stack from primitives rather than leaning on library types, so every layer of a physics engine is visible in source. The vector layer is a set of namespaced free functions over a two-element array — there is no `Vec2` struct, the vector *is* the pair `[$x, $y]`, and the operations are expression-bodied functions written with the `fn Name = expr` form:

```
fn Vec2::new($x=0, $y=0) = [$x, $y]
fn Vec2::add($v, $w)     = [$v->[0] + $w->[0], $v->[1] + $w->[1]]
fn Vec2::sub($v, $w)     = [$v->[0] - $w->[0], $v->[1] - $w->[1]]
fn Vec2::mul($v, $s)     = [$v->[0] * $s, $v->[1] * $s]
fn Vec2::dot($v, $w)     = $v->[0] * $w->[0] + $v->[1] * $w->[1]
fn Vec2::mag_sq($v)      = $v->[0]**2 + $v->[1]**2
fn Vec2::mag($v)         = sqrt(Vec2::mag_sq $v)
```

This is a representative slice of Stryke's economy: default parameter values (`$x=0, $y=0`), the `->[0]` indexing arrow into an arrayref, the `**` exponent operator in `mag_sq`, and the call-without-parens style in `sqrt(Vec2::mag_sq $v)` where `Vec2::mag_sq $v` is a function application whose result feeds into `sqrt`. A body is a hash literal built with the `+{ ... }` anonymous-hashref syntax, holding position and velocity vectors, mass, an `inv_mass` precomputed as `$mass > 0 ? 1.0 / $mass : 0.0` (so static bodies with mass 0 get inverse mass 0 and become immovable), a restitution coefficient, a `shape` tag drawn from `enum ShapeKind { Circle, Box }`, and a `data` field overloaded by shape — a radius for circles, a `[width, height]` pair for boxes. The collision layer dispatches on the shape pair, with the circle-circle detector the cleanest expression of the pattern — compute the separation vector, compare its magnitude to the sum of radii, early-return `undef` on no overlap, otherwise return a manifold hash with a contact normal and penetration depth:

```

fn Physics::collide_cc($body1, $body2) {
  val $dist_v = Vec2::sub $body2->{pos}, $body1->{pos}
  val $dist    = Vec2::mag $dist_v
  val $radius_sum = $body1->{data} + $body2->{data}

  return undef if $dist > $radius_sum

  val $normal = $dist > 0 ? Vec2::mul($dist_v, 1.0 / $dist) : Vec2::new(0, 1)
  val %m = (normal => $normal, depth => $radius_sum - $dist)
  \%m
}

```

The time-step `World::step` is the canonical symplectic (semi-implicit) Euler loop and shows two different `for` forms in one function — a `foreach` over the dereferenced array for integration, and a classic C-style triple-clause `for` for the pairwise broad-phase:

```

fn World::step($bodies, $dt) {
  val $gravity = Vec2::new(0, 9.8)

  # 1. Integrate forces (gravity) and update positions
  for var $body (@$bodies) {
    next if $body->{inv_mass} == 0
    $body->{vel} = Vec2::add $body->{vel}, Vec2::mul($gravity, $dt)
    $body->{pos} = Vec2::add $body->{pos}, Vec2::mul($body->{vel}, $dt)
  }

  # 2. Collision detection and resolution
  for (var $i = 0; $i < len(@$bodies); $i++) {
    for (var $j = $i + 1; $j < len(@$bodies); $j++) {
      val $body1 = $bodies->[$i]
      val $body2 = $bodies->[$j]
      next if $body1->{inv_mass} == 0 && $body2->{inv_mass} == 0

      val $manifold = Physics::collide $body1, $body2
      Physics::resolve($body1, $body2, $manifold) if $manifold
    }
  }
}

```

The integration order — velocity first from gravity, then position from the updated velocity — is what makes this symplectic rather than naive explicit Euler, and the `next if $body->{inv_mass} == 0` guard skips static geometry. The `Physics::resolve` step computes relative velocity, projects it onto the contact normal with `Vec2::dot`, bails out early if the bodies are already separating, computes the scalar impulse $-(1 + \epsilon) * v_{\text{along_normal}}$ over the sum of inverse masses, applies equal-and-opposite velocity changes weighted by inverse mass, and adds a positional correction with a 20% bias and 0.01 slop to stop stacked bodies from slowly interpenetrating. The driver seeds a static floor and two walls (mass 0), drops four dynamic circles and a box, and runs thirty steps with `$dt = 0.1`, the whole thing a standalone engine with an embedded `test_physics` assertion suite that dies on a wrong manifold before the simulation even starts.

distributed_nbody_step.stk — one gravitational step across worker processes

The N-body programs compute long-range gravitational attraction, and `distributed_nbody_step.stk` is the most direct illustration of Stryke's cross-process parallelism applied to physics. Workers each compute the force and acceleration for a subset of bodies given the current positions of all bodies. The parallel substrate is a worker pool sorted deterministically: `val @workers = congregation(3)` then `var @sorted_workers`

`= sort { _ <=> _1 } @workers`. The comparator `{ _ <=> _1 }` is the topic-variable idiom — `_` the first comparand, `_1` the second — ordering the worker session ids ascending; sorting matters because shard assignment is positional. The crucial design point — and the reason this is an N-body solver rather than a trivially partitioned problem — is that every worker needs the *entire* position list to compute forces on its subset even though it only writes accelerations for its own range. The work is shipped as a string of Stryke source built by interpolation, with the inner sigils escaped so they evaluate remotely:

```
val $code = "
  var \bodies = from_json(q{$bodies_json});
  var %acc;
  var $n = len(\@\bodies);
  var $g = $g;
  for var $i ($start:$end) {
    var $ax = 0.0;
    var $ay = 0.0;
    var $bi = \bodies->[$i];
    for var $j (0:$n - 1) {
      next if $j == $i;
      var $bj = \bodies->[$j];
      var $dx = $bj->[0] - $bi->[0];
      var $dy = $bj->[1] - $bi->[1];
      var $r2 = $dx * $dx + $dy * $dy + 0.01; # softening
      var $r = sqrt($r2);
      var $f = $g * $bj->[2] / $r2;
      $ax += $f * $dx / $r;
      $ay += $f * $dy / $r;
    }
    $acc{$i} = sprintf('%.4f,%.4f', $ax, $ay);
  }
  to_json(\%acc)
"
```

This is the gravitational inner loop in textbook form: for each body in the worker's range, sum the contribution of every other body, where the force magnitude is $G * m_j / r^2$ decomposed into x and y components by multiplying by the unit separation. The `+ 0.01` softening term on `$r2` prevents a singularity when two bodies are nearly coincident. The parameters that vary per worker (`$start`, `$end`, `$g`, the serialised `$bodies_json`) are interpolated unescaped so they bake into the source string, while the runtime variables (`\$i`, `\$j`, the `\@\bodies` deref) are escaped so they remain code. The dispatch and collection use the prayer primitives: `pray $code`, `$w` sends the code to one worker, `annex $div` blocks for that worker's reply, the JSON acceleration map is merged into the master `%accelerations`, and the pool is released with `excommunicate(@workers)`.

barnes_hut_no_interop.stk – the $O(N \log N)$ quadtree N-body

`barnes_hut_no_interop.stk` solves the same gravitational problem but trades brute-force $O(N^2)$ summation for the Barnes-Hut approximation, $O(N \log N)$, built on a recursive spatial data structure rather than a parallel decomposition. The node is a plain hash with geometry, an aggregate `mass` and center-of-mass, a single `body` slot for leaves, and four child slots `nw/ne/sw/se`. Insertion is the standard quadtree build with center-of-mass accumulation — the mass-weighted average that lets a whole subtree later be treated as a single point mass. The force calculation is where the approximation pays off:

```

fn calculate_force($node, $body, $theta, $g) {
  return [0.0, 0.0] if $node->{mass} == 0.0

  val $dx = $node->{center_x} - $body->{x}
  val $dy = $node->{center_y} - $body->{y}
  val $dist_sq = $dx * $dx + $dy * $dy + 1e-9 # softening
  val $dist = sqrt($dist_sq)

  if (defined $node->{body}) {
    return [0.0, 0.0] if $node->{body}->{id} == $body->{id}
    val $f = ($g * $node->{mass} * $body->{mass}) / $dist_sq
    return [ $f * $dx / $dist, $f * $dy / $dist ]
  }

  if ($node->{size} / $dist < $theta) {
    val $f = ($g * $node->{mass} * $body->{mass}) / $dist_sq
    return [ $f * $dx / $dist, $f * $dy / $dist ]
  }

  var @total_f = (0.0, 0.0)
  for val $child ($node->{nw}, $node->{ne}, $node->{sw}, $node->{se}) {
    if (defined $child) {
      val $f = BarnesHut::calculate_force($child, $body, $theta, $g)
      $total_f[0] += $f->[0]
      $total_f[1] += $f->[1]
    }
  }
  \@total_f
}

```

A leaf node contributes its exact pairwise force (skipping a body's interaction with itself via the `id` check); an internal node whose angular size is small enough — `if ($node->{size} / $dist < $theta)` — is approximated as a single point mass at its center of mass; otherwise the function recurses into all four children and sums their contributions. The `$theta` opening-angle parameter is the accuracy-versus-speed knob; the driver passes `0.5`. The time-step rebuilds the tree from scratch each tick, computes each body's force against the tree, and applies a two-phase integration: update all velocities from accelerations, then all positions from the new velocities. The contrast with the distributed stepper is the whole lesson: same physics, but one program attacks the cost with a smarter algorithm (the tree) and the other with more processors (the congregation), and Stryke expresses both naturally.

conway_life.stk and langtons_ant.stk — automata on a grid

Cellular automata replace continuous vectors with a discrete grid and a local neighbourhood rule. `conway_life.stk` represents the state as an array-of-arrayrefs of 0/1 cells and computes the eight-direction neighbour count with a doubly-nested loop over `(-1, 0, 1)` in each axis that skips the center and any off-grid neighbour:

```

fn Demo::Life::count_neighbors($g, $r, $c) {
  var $n = 0
  for val $dr (-1, 0, 1) {
    for val $dc (-1, 0, 1) {
      next if $dr == 0 && $dc == 0
      val $nr = $r + $dr
      val $nc = $c + $dc
      next if $nr < 0 || $nr >= $H || $nc < 0 || $nc >= $W
      $n += $g->[$nr]->[$nc]
    }
  }
  $n
}

```

The generation step applies the Conway rule to a *fresh* output grid so the update is synchronous — a live cell survives with two or three neighbours, a dead cell is born with exactly three — and the driver instruments each generation with a t-digest, a HyperLogLog, and a top-k sketch, reporting p50/p99 step latency via `td_quantile($td_step, 0.50)/td_quantile($td_step, 0.99)`; the streaming-statistics builtins drop straight into the loop without ceremony. `langtons_ant.stk` keeps the grid but replaces the global rule with a single agent — turn right on white, left on black, flip the cell, move one square forward with toroidal wraparound (`(($x + $DX[$new_dir] + $size) % $size, the + $size keeping the index non-negative before the modulo, and 1 - $cell the branchless flip). What distinguishes it is that it is written as an executable specification: die unless invariant pins assert the flip count equals the step count, a parity invariant, and — the strongest pin — that re-running the identical 200-step simulation into a fresh parallel grid produces the same final position, direction, and black count. A deterministic simulation can be made self-verifying by re-running and comparing.`

Two ways to parallelise the same Monte Carlo estimator

The final pair shows one simulation expressed under both of Stryke's parallel models. Monte Carlo π throws random darts at the unit square and estimates $\pi \approx 4 \times \text{inside} / \text{total}$. The in-process version, `parallel_pi_monte_carlo.stk`, uses thread chunking through the `pmaps` parallel-map operator fed by a pipeline — it splits the samples into chunks, runs each chunk's full sample loop inside one thread, and sums the per-chunk counts:

```

val $inside_count = 1:$chunks |> pmaps {
  var $c = 0
  for (1:$per) {
    val $x = rand()
    val $y = rand()
    $c++ if $x*$x + $y*$y <= 1.0
  }
  $c
} |> sum

```

The pipeline reads as data flowing left to right: the range `1:$chunks` feeds into `pmaps`, whose block runs once per chunk in parallel, and the list of counts feeds into `sum`. The correctness subtlety, flagged in the file's header, is that each chunk must run *many* samples inside one thread so its RNG state diverges from its siblings; one `rand()` per parallel task would produce correlated outputs and a biased estimate. The distributed version, `distributed_monte_carlo_pi.stk`, instead spawns worker processes with `congregation($n_workers)` and ships each a source string that generates points and returns a hit count, solving the decorrelation problem differently — each worker reseeds with `srand($$ + time)` so the pid and clock make every sequence independent. Collection uses the one-shot `harvest` primitive — `harvest $code, @workers, 60_000` runs the code on every worker with a 60-second timeout and returns a session-id-to-hits hash. The two programs are the same simulation with the same convergence; the only difference is the unit of parallelism — `pmaps` chunks across threads in one process, `congregation/harvest` shards across worker processes —

and the matching RNG-decorrelation idiom for each.

Chapter 29 – Cryptography, Hashing, and Encoding

Cryptography is the discipline where a language’s character is most ruthlessly exposed, because the work it demands is unforgiving on two opposing axes at once. On one side sits the bit-level arithmetic of checksums, ciphers, and field operations — code that lives or dies by whether shifts, masks, and modular reductions behave exactly as the specification says, with no rounding, no silent widening, and no off-by-one in a sextet boundary. On the other side sits the high-level orchestration of a real security protocol — parsing a header, recomputing a keyed tag, enforcing a time window, refusing a replay — where the language must express intent clearly enough that a reviewer can see the security property holds. A language that is comfortable at only one end produces either elegant protocol descriptions resting on opaque library calls, or correct-but-illegible bit-twiddling that nobody dares touch. The stryke example corpus contains programs that sit firmly at both ends, and reading them together shows how the language’s design choices — Perl-style sigils, the topic variable `_`, colon ranges, native bit operators, and a battery of in-binary hash and HMAC builtins — let the same syntax describe a from-scratch Base64 codec and a production-shaped webhook verifier without changing register. This chapter walks through six real programs from the `examples/` directory, taking each in turn, and uses them to show what stryke offers a cryptographic programmer: where it leans on builtins, where it drops to the bare integer, and where its idioms make a security invariant legible.

crypto.stk – the builtin cryptographic surface

The cleanest place to begin is `crypto.stk`, because it is not an implementation of an algorithm at all; it is a tour of what stryke ships in the binary, and it establishes the vocabulary the rest of the chapter relies on. The file’s own header states the design stance plainly: “Stryke crypto primitives — all in-binary, no external deps. Hashes return hex strings; signing/verification round-trips cleanly.” That single sentence is the load-bearing claim. Every hash, HMAC, password-hashing, and token primitive used across the corpus is a builtin function callable with no import, and each one returns a hex string rather than a raw byte buffer, which is why downstream code can compare tags with ordinary string equality.

The hash section calls five distinct digest builtins on one message, each as a bare function applied to a value with no parentheses, in the Perl list-operator style:

```
p "— Cryptographic hashes —"
val $msg = "the quick brown fox"
printf "  md5      : %s\n", md5 $msg
printf " sha256    : %s\n", sha256 $msg
val $sha512_full = sha512 $msg
printf " sha512    : %s... (truncated)\n", $sha512_full[0:31]
printf " sha3_256   : %s\n", sha3_256 $msg
printf " blake3     : %s\n", blake3 $msg
```

The roster — `md5`, `sha256`, `sha512`, `sha3_256`, `blake3` — spans the legacy MD-family, the SHA-2 pair, a SHA-3 (Keccak-family) variant, and BLAKE3, the modern tree-hash. The truncation of the SHA-512 output via `$sha512_full[0:31]` is itself instructive: because the digest is a hex string, a colon-range slice produces the first thirty-two characters directly, the same slicing syntax used for arrays. There is no `bytes-versus-string` ceremony.

Keyed authentication arrives through `hmac_sha256`, which takes the key and message as two arguments

and is described in the file's comment as a "keyed authentication tag." The example proves the determinism property that every MAC scheme guarantees and every protocol depends on — the same key and message yield the same tag — by computing once and printing twice:

```
val $secret = "shared-secret-32-bytes-or-bigger"
val $tag    = hmac_sha256 $secret, $msg
printf "  hmac_sha256: %s\n", $tag
p "  same key+message always produces the same tag:"
printf "    pass 1   : %s\n", $tag[0:31]
printf "    pass 2   : %s\n", $tag[0:31]
```

Beyond raw hashing, the file demonstrates that stryke's crypto surface reaches into the application-server use cases the header names. Password storage is handled by Argon2id through `argon2_hash` and `argon2_verify`, where the verify function returns truthy for the correct password and falsy for a wrong one, and the stored hash is opaque and self-describing (the example slices the first fifty characters of it with `substr($hash, 0, 50)`). Token issuance is handled by `jwt_encode` and `jwt_decode`, which take a claims hash and a signing secret; the claims are an ordinary stryke hash literal whose values include calls to `time()` for the issued-at and expiry fields, showing that the JWT builtin consumes native data structures rather than requiring a pre-serialized string. The tamper case closes the loop on verification semantics by corrupting the last three characters of a valid token and confirming the decode rejects it, using stryke's `eval { ... }` block and the `$@` error variable in the Perl manner:

```
val $bad = substr($token, 0, -3) . "AAA";    # corrupt last 3 chars
var $err = 0
eval { jwt_decode($bad, "jwt-secret") }
$err = 1 if $@
p "  tampered token rejects (eval `\$@` set): $err"
```

The final section fuses HMAC with the key-value store: a manifest is serialized with `to_json`, tagged with `hmac_sha256`, both the JSON and the tag are written into a `kv_open`-backed store, committed, reopened, and the tag is recomputed over the retrieved JSON to confirm it survives a persistence round-trip. The verification is plain string equality, `($t2 eq $recomputed) ? 1 : 0`, which is only possible because the tag is a hex string. This file is the foundation: it shows that for the common cases — digesting, keyed authentication, password hashing, signed tokens — stryke wants you calling a builtin, not reimplementing a compression function.

base64_no_interop.stk – encoding from the bit up

Where `crypto.stk` leans entirely on builtins, `base64_no_interop.stk` does the opposite: it builds RFC 4648 Base64 from scratch, and in doing so it exercises stryke's byte-and-bit layer. The header lays out both the algorithm and the language features deliberately. The algorithm is the canonical one — "Pack 3 input bytes → 4 sextets (6-bit chunks), each sextet indexes the alphabet, pad with `=` so output length is multiple of 4" — and the listed "Stryke choices" name `ord/chr` for the byte-integer conversions, "bitops for sextet pack/unpack," a "Hash for reverse lookup table," and built-in `die` invariants so that every test vector round-trips.

The setup builds the sixty-four-character alphabet as a string, splits it into a character array, and constructs the decode table as a hash mapping each character back to its index, iterating with a colon range:

```
val $ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
val @ALPHA   = split(//, $ALPHABET)
var %REVERSE
for val $i (0:63) { $REVERSE{$ALPHA[$i]} = $i }
```

The `0:63` is stryke's colon-range form for the inclusive integer sequence zero through sixty-three; `split(//, $ALPHABET)` splits on the empty pattern to yield individual characters, the same idiom Perl uses. The encode function is the heart of the exercise, and every line is a direct expression of the packing arithmetic:

```

fn B64::encode($s) {
  val @bytes = map {ord} split(//, $s)
  val $n = len(@bytes)
  var $out = ""
  var $i = 0
  while ($i < $n) {
    val $b1 = $bytes[$i]
    val $b2 = $i + 1 < $n ? $bytes[$i + 1] : 0
    val $b3 = $i + 2 < $n ? $bytes[$i + 2] : 0
    val $combined = ($b1 << 16) | ($b2 << 8) | $b3
    $out .= $ALPHA[(($combined >> 18) & 0x3f)]
    $out .= $ALPHA[(($combined >> 12) & 0x3f)]
    $out .= ($i + 1 < $n) ? $ALPHA[(($combined >> 6) & 0x3f)] : "="
    $out .= ($i + 2 < $n) ? $ALPHA[$combined & 0x3f] : "="
    $i += 3
  }
  $out
}

```

The three input bytes are read with bounds-aware ternaries that substitute zero past the end of the string, then fused into a single twenty-four-bit integer with two left shifts and bitwise-or, `(($b1 << 16) | ($b2 << 8) | $b3)`. The four output sextets are extracted by right-shifting by eighteen, twelve, six, and zero and masking each with `0x3f` (the low six bits), and each sextet indexes the alphabet array. The hexadecimal mask literal, the C-family shift operators `<<` and `>>`, the bitwise-or `|`, and the bitwise-and `&` are all first-class in stryke, so the code reads exactly like the reference description of the algorithm. Padding is emitted by the same bounds ternaries that fed in the zero bytes: if there was no second input byte, the third sextet position becomes `=`, and likewise for the fourth. The string-append operator `.=` accumulates output. Note that `map {ord} split(//, $s)` converts the whole string to a byte array in one pass, with `{ord}` as a block applying the `ord` builtin to each element.

Decoding is the inverse, and it leans on the reverse-lookup hash and the defined-or operator `//` to default missing entries to zero, while first stripping padding with a `grep` block whose body uses the topic variable `_`:

```

fn B64::decode($s) {
  val @chars = grep { _ ne "=" } split(//, $s)
  var $out = ""
  var $i = 0
  while ($i < len(@chars)) {
    val $s1 = $REVERSE{@chars[$i]} // 0
    ...
    val $combined = ($s1 << 18) | ($s2 << 12) | ($s3 << 6) | $s4
    $out .= chr((($combined >> 16) & 0xff))
    $out .= chr((($combined >> 8) & 0xff) if $i + 2 < len(@chars))
    $out .= chr($combined & 0xff) if $i + 3 < len(@chars)
    $i += 4
  }
  $out
}

```

Here `grep { _ ne "=" }` filters the character list, with `_` standing for each element under test — stryke's adoption of the topic variable for the predicate. The four sextets are recombined into the twenty-four-bit word and split back into bytes with `>> 16`, `>> 8`, and `& 0xff`, each wrapped in `chr` to produce a character, and the trailing-byte emissions are guarded by postfix `if` conditions so that padded groups don't emit spurious bytes. The program's correctness is pinned against the RFC 4648 §10 test vectors stored as a list of hashes (`+{plain => "f", b64 => "Zg=="}` and so on), every case is run through encode and then decode, and any failure increments a counter that triggers `die` at the end. The closing line asserts "ALL PASS (8 RFC 4648 vectors, encode + round-trip both verified)." This file is the canonical demonstration that stryke's bit operators and `ord/chr` pair are sufficient to implement an encoding standard byte-exactly. It also doubles as a command-line tool: the program reads `@ARGV`, and when at least two arguments are present it destructures

them with `val ($mode, $arg) = @ARGV`, dispatching `encode` or `decode` against the single argument and exiting, or dying with `"mode must be 'encode' or 'decode'"` on anything else. That dual nature — a verified test harness when run bare, a usable codec when given arguments — is a recurring shape across the corpus, and it costs only a handful of lines because the same `B64::encode` and `B64::decode` functions serve both paths.

crc32_checksum.stk — table-driven bitwise checksumming at scale

`crc32_checksum.stk` extends the bit-level work of the Base64 codec into a checksum, then layers stryke's parallel and sketching facilities on top. The header is precise about the variant: "CRC-32 implementation (IEEE 802.3 polynomial 0xEDB88320, reflected)," and it cross-checks against "Perl/Python-known fixed checksums for canonical inputs." The reflected polynomial `0xEDB88320` is the standard zlib/Ethernet constant, and the implementation is the textbook table-driven form.

Table construction generates the 256-entry lookup table by running each byte value through eight rounds of the shift-and-conditional-xor that defines the reflected CRC:

```
fn Demo::CRC::build_table() {
    var @table
    for val $n (0:255) {
        var $c = $n
        for val $k (1:8) {
            if ($c & 1) {
                $c = ($c >> 1) ^ 0xEDB88320
            } else {
                $c = $c >> 1
            }
        }
        push @table, $c & 0xFFFFFFFF
    }
    \@table
}
```

The inner loop tests the low bit with `$c & 1`, and on a set bit shifts right and xors the polynomial, otherwise shifts right alone — the exact reflected-CRC recurrence. Each finished entry is masked to thirty-two bits with `& 0xFFFFFFFF` before being pushed, which keeps the values inside the CRC-32 domain. The function returns a reference to the array with `\@table`, the Perl reference-taking syntax, and the caller binds it once at top level. The per-string checksum is then the standard byte-stream update loop:

```
fn Demo::CRC::crc32($s) {
    var $crc = 0xFFFFFFFF
    for val $byte (split //, $s) {
        var $bv = ord $byte
        $crc = ($TABLE->[(($crc ^ $bv) & 0xFF)] ^ ((($crc >> 8) & 0x00FFFFFF))
    }
    ($crc ^ 0xFFFFFFFF) & 0xFFFFFFFF
}
```

The register starts at all-ones, each byte is folded by indexing the table with the low byte of `crc xor byte` and xoring against the right-shifted register, and the final result is the ones-complement of the register masked to thirty-two bits. The masks `& 0xFF`, `& 0x00FFFFFF`, and `& 0xFFFFFFFF` are doing the work of fixed-width integer arithmetic that a C implementation would get from the type system; in stryke the programmer asserts the width explicitly through masking. The table is dereferenced and indexed with `$TABLE->[...]`, the arrow-then-bracket form for a reference to an array.

Correctness is established against six known checksums — the empty string mapping to `0x00000000`, "a"

to `0xE8B7BE43`, `"123456789"` to the well-known `0xCBf43926`, and so on — printed with `%08x` format so the hex matches the published constants. Beyond verification, this file shows stryke's analytical primitives operating over cryptographic-style output. A workload of strings across length tiers from ten to ten-thousand bytes is checksummed in parallel through `pmap`, whose block uses `_` for the current element and times each computation with `now_ns()`:

```
var @cirs = pmap {
  val $t0 = now_ns()
  val $c = Demo::CRC::crc32(_)
  val $elapsed_us = (now_ns() - $t0) / 1000.0
  val $bytes_per_us = $elapsed_us > 0 ? len(_) / $elapsed_us : 0
  [len(_), $c, $bytes_per_us]
} @workload
```

The throughput numbers are fed into a t-digest (`t_digest`, `td_add`, `td_quantile`) for percentile estimation, the size tiers into a top-k sketch (`topk`, `topk_add`, `topk_top`), and every distinct CRC value into a Hyper-LogLog (`hll`, `hll_add`, `hll_count`) so the program can estimate the number of distinct checksums and reason about collision rate. The file closes with an avalanche demonstration that changes a single letter — `"the quick brown fox"` to `"the quick brown box"` — computes both CRCs, xors them, and counts the differing bits across `0:31`, reporting that roughly half the bits flip, the property a good checksum should exhibit. The lesson here is that stryke treats a hand-rolled bit algorithm and its statistical analysis as the same kind of code: bit operators for the kernel, `pmap` and the sketch builtins for the study of its behavior.

`ecc_no_interop.stk` — elliptic-curve arithmetic over a prime field

`ecc_no_interop.stk` moves from byte arithmetic to number theory, implementing elliptic-curve point operations over a prime field, with the curve form stated in the header as `"y^2 = x^3 + a*x + b mod p,"` supporting "point addition, doubling, and scalar multiplication via double-and-add," and modelling "the point at infinity ... as `+{ inf => 1 }`." The choice to represent the identity element as a hash with an `inf` flag, and finite points as `+{ x => ..., y => ..., inf => 0 }`, lets the geometry of the group law be expressed directly against hash fields.

Two number-theoretic helpers underpin everything. An always-positive modulus corrects stryke's `%` for negative dividends, and an extended-Euclidean modular inverse computes division in the field:

```
fn ECC::mod($n, $p) {
  val $r = $n % $p
  $r < 0 ? $r + $p : $r
}

fn ECC::mod_inv($n, $p) {
  var $t = 0
  var $newt = 1
  var $r = $p
  var $newr = ECC::mod($n, $p)
  while ($newr != 0) {
    val $q = int($r / $newr)
    val $tmp_t = $t - $q * $newt; $t = $newt; $newt = $tmp_t
    val $tmp_r = $r - $q * $newr; $r = $newr; $newr = $tmp_r
  }
  return undef if $r > 1
  ECC::mod($t, $p)
}
```

The inverse follows the iterative extended-Euclidean form, carrying the coefficient pair (`$t`, `$newt`) alongside the remainder pair (`$r`, `$newr`), and returns `undef` when the gcd is not one — meaning no inverse exists —

which is the correct failure mode for a field operation. The semicolon-joined multiple assignments on one line, `$t = $newt; $newt = $tmp_t`, are stryke's way of expressing the simultaneous coefficient update.

Point addition encodes the affine group law with all its special cases, and the header notes that “doubling collapses into the tangent-slope branch (`p1 == p2`)”:

```
fn ECC::add_points($p1, $p2, $ca, $p) {
  return $p2 if $p1->{inf}
  return $p1 if $p2->{inf}
  return +{ inf => 1 } if $p1->{x} == $p2->{x} && $p1->{y} != $p2->{y}

  var $m
  if ($p1->{x} == $p2->{x}) {
    return +{ inf => 1 } if $p1->{y} == 0
    val $num = ECC::mod(3 * $p1->{x} * $p1->{x} + $ca, $p)
    val $den = ECC::mod_inv(2 * $p1->{y}, $p)
    $m = ECC::mod($num * $den, $p)
  } else {
    val $num = ECC::mod($p2->{y} - $p1->{y}, $p)
    val $den = ECC::mod_inv(ECC::mod($p2->{x} - $p1->{x}, $p), $p)
    $m = ECC::mod($num * $den, $p)
  }

  val $x3 = ECC::mod($m * $m - $p1->{x} - $p2->{x}, $p)
  val $y3 = ECC::mod($m * ($p1->{x} - $x3) - $p1->{y}, $p)
  +{ x => $x3, y => $y3, inf => 0 }
}
```

The early `return ... if ...` postfix guards handle adding the identity on either side and the inverse-pair case that yields infinity. When the x-coordinates match, the slope is the tangent line, computed as $(3x^2 + a) / (2y)$ with the field inverse standing in for division; otherwise it is the chord slope $(y_2 - y_1) / (x_2 - x_1)$. The resulting point's coordinates $x_3 = m^2 - x_1 - x_2$ and $y_3 = m(x_1 - x_3) - y_1$ are each reduced through `ECC::mod`. Every reference into the point hashes is `$p1->{x}` and similar, the arrow-dereference form for hash references. Scalar multiplication is the double-and-add ladder, copying the starting point with the hash-spread `+{ %$pt }` and processing the binary expansion of the scalar:

```
fn ECC::scalar_mult($k, $pt, $ca, $p) {
  var $res = +{ inf => 1 }
  var $addend = +{ %$pt }
  var $n = $k
  while ($n > 0) {
    $res = ECC::add_points($res, $addend, $ca, $p) if $n % 2 == 1
    $addend = ECC::add_points($addend, $addend, $ca, $p)
    $n = int($n / 2)
  }
  $res
}
```

The accumulator starts at infinity, the addend doubles each iteration, and the addend is folded into the result whenever the current low bit of the scalar is set — $O(\log k)$ point operations as the comment states. The demo runs the curve $y^2 = x^3 + 2x + 2 \bmod 17$ with base point (5, 1), computes 2G, 3G, and 19G, and confirms that 19G lands on the point at infinity, consistent with the stated base-point order of nineteen. The interpolation `#{$G->{x}}` inside the printed strings shows stryke's Ruby-style string interpolation reaching into hash references. This file demonstrates that stryke is comfortable expressing field arithmetic and group laws with nothing but native integer operators, the `int` truncating-division builtin, hash-reference data structures, and `undef` for the no-inverse case.

rsa_no_interop.stk — a public-key cryptosystem end to end

`rsa_no_interop.stk` builds the RSA cryptosystem — key generation, encryption, decryption — and organizes it with stryke's package declarations rather than namespaced function names, opening with `package RSA` and later switching to `package main` for the driver. The same number-theoretic toolkit reappears in a different guise: a Euclidean `gcd`, a modular inverse, modular exponentiation, and a trial-division primality test. Modular exponentiation is the square-and-multiply routine that both encryption and decryption need:

```
fn mod_pow($base, $exp, $mod) {
  var $res = 1; $base = $base % $mod
  while ($exp > 0) {
    $res = ($res * $base) % $mod if $exp % 2 == 1
    $exp = int($exp / 2)
    $base = ($base * $base) % $mod
  }
  $res
}
```

The accumulator multiplies in the current base whenever the exponent's low bit is set, then squares the base and halves the exponent, reducing modulo `mod` at every step so intermediate values never grow unbounded — the same binary-ladder structure as the ECC scalar multiply, transplanted from the additive curve group to the multiplicative integer group. Primality is checked by `is_prime`, which handles the small cases and then trial-divides by $6k \pm 1$ candidates up to the square root, advancing `$i` by six each loop. Key generation enforces that both inputs are prime, computes the modulus and Euler totient, searches for the smallest encryption exponent coprime to the totient, and derives the private exponent as its modular inverse:

```
fn generate_keypair($p, $q) {
  die "Both numbers must be prime." if !RSA::is_prime($p) || !RSA::is_prime($q)
  val $n = $p * $q
  val $phi = ($p - 1) * ($q - 1)
  var $e = 3
  while ($e < $phi) {
    last if RSA::gcd($e, $phi) == 1
    $e++
  }
  val $d = RSA::mod_inverse($e, $phi)
  { public => [$e, $n], private => [$d, $n] }
}
```

The cross-package calls are fully qualified as `RSA::is_prime` and `RSA::gcd`, the `last` keyword breaks the search loop on the first coprime exponent, and the returned hash carries the public pair `[e, n]` and private pair `[d, n]` as array references. Encryption and decryption operate character by character, mapping each plaintext character to its `ord` value, raising it to the public exponent for encryption and to the private exponent for decryption, and reconstructing characters with `chr`:

```
fn encrypt($msg, $public_key) {
  val $e = $public_key->[0]; val $n = $public_key->[1]
  var @enc; val $len = len $msg
  for val $i (0:$len-1) {
    val $m = ord($msg[$i])
    push @enc, RSA::mod_pow($m, $e, $n)
  }
  \@enc
}
```

Decryption is the mirror, reading the private pair and reconstructing the message:

```

fn decrypt($enc, $private_key) {
    val $d = $private_key->[0]; val $n = $private_key->[1]
    var $msg = ""
    for val $c (@$enc) {
        val $m = RSA::mod_pow($c, $d, $n)
        $msg = $msg . chr($m)
    }
    $msg
}

```

The `for val $c (@$enc)` iterates over the dereferenced ciphertext array directly, raising each number to the private exponent and appending the recovered character with `.` concatenation. The driver in `package main` generates a key pair from the primes sixty-one and fifty-three, prints the public and private keys via interpolation, encrypts "HELLO", prints the ciphertext numbers joined with commas through `join ', '`, `@$enc`, and decrypts back to the plaintext. The `@$enc` is array-reference dereference in list context, feeding the joined output. Read alongside `ecc_no_interop.stk`, this file makes the structural point that stryke expresses both major public-key families — discrete-log over an elliptic curve and integer factorization for RSA — out of the same handful of integer primitives, with packages organizing the RSA version and the `Name::fn` namespacing organizing the ECC version, two different but equivalent modularity idioms the language supports. The two files together also reveal a deeper economy: `mod_pow` in RSA and `scalar_mult` in ECC are the identical square-and-multiply skeleton, the only difference being whether the repeated operation is integer multiplication modulo `n` or point addition on the curve. A reader who has internalized one understands the other, which is exactly the pedagogical payoff of writing both in the same idiom rather than reaching for a library that hides the ladder.

merkle_tree.stk and webhook_signature_verifier.stk — committing and authenticating

The last two programs return to the protocol end of the spectrum, where hashing is in service of an integrity guarantee rather than an end in itself. `merkle_tree.stk` builds a Merkle root and inclusion proofs, deliberately using “a deterministic ASCII hash (sum + xor + len) so tests don’t depend on an external crypto library” — a polynomial rolling hash standing in for SHA-256 so the structure, not the digest, is what the demo exercises. The leaf hash multiplies a running accumulator by thirty-one and folds in each character code under a large prime modulus, formatting the result with `%08x`:

```

fn MT::hash($s) {
    var $h = 0
    val $n = len $s
    for val $i (0:$n-1) {
        val $c = ord(substr($s, $i, 1))
        $h = (($h * 31) + $c) % 1000000007
    }
    sprintf("%08x", $h)
}

```

Internal nodes combine their children by hashing a delimited concatenation, `MT::hash("(" . $aa . "|" . $bb . ")")`, which is the domain-separation idea in miniature — the parenthesization and pipe ensure that combining two hashes is unambiguous. The tree builder hashes all leaves into the first level, then repeatedly pairs adjacent nodes — duplicating the last node when a level has odd length, via `($i + 1 < $n) ? $level[$i + 1] : $left` — and pushes each completed level as an array reference into a `@levels` list, so the full tree is retained for proof generation. The proof walker climbs from the leaf, at each level recording the sibling hash and which side it sits on:

```

fn MT::proof($tree, $idx) {
  var @proof = ()
  val $levels = $tree->{levels}
  val $depth = len(@$levels) - 1
  for val $d (0:$depth-1) {
    val $level = $levels->[$d]
    var $sib = ($idx % 2 == 0) ? $idx + 1 : $idx - 1
    if ($sib >= len(@$level)) { $sib = $idx } # odd-leaf duplication
    val $side = ($idx % 2 == 0) ? "right" : "left"
    push @proof, +{ hash => $level->[$sib], side => $side }
    $idx = int($idx / 2)
  }
  \@proof
}

```

Verification re-hashes the candidate leaf and folds in each recorded sibling on the correct side, combining left-then-right or right-then-left per the `side` field, and checks the final hash against the published root, returning one or zero. The demo builds a tree over four leaves, verifies an inclusion proof for each, and confirms that a forged leaf "eve" fails verification against an honest proof — the tamper-detection property a Merkle tree exists to provide.

`webhook_signature_verifier.stk` is the most production-shaped program in the set, modelling “the receiver side of a GitHub-/Stripe-style webhook endpoint.” It enforces three distinct security properties — authenticity, freshness, and idempotency — and its choice of `hmac_sha256` ties it back to the builtin surface from `crypto.stk`. Signing computes the tag over a timestamp-dotted body and packages the header string:

```

fn Webhook::sign($secret, $body, $timestamp = time()) {
  val $payload = "$timestamp.$body"
  val $sig = hmac_sha256 $secret, $payload
  +{ ts => $timestamp, sig => $sig, header => "t=$timestamp,v1=$sig" }
}

```

The default-argument form `$timestamp = time()` shows stryke supporting default parameter values. The security-critical comparison is constant-time, refusing the early-exit that a naive equality check would allow and that a timing attacker could exploit:

```

fn Webhook::eq_constant_time($a, $b) {
  return 0 if len($a) != len($b)
  var $diff = 0
  for val $i (0:len($a) - 1) {
    $diff |= ord($a[$i]) ^ ord($b[$i])
  }
  $diff == 0
}

```

It accumulates the bitwise difference across all positions with `|= ord(...) ^ ord(...)` and only checks for zero at the very end, so every byte is always examined regardless of where a mismatch first occurs. The full `Webhook::verify` parses the header into a hash by splitting on commas and then on the first `=`, rejects missing fields, coerces the timestamp numerically with `0 + $parsed->{ts}`, enforces a max-age window and a future-dated bound against `time()`, consults a seen-set to reject replays, recomputes the HMAC over `"$ts.$body"`, and compares it with the constant-time function before finally recording the signature as seen. The driver walks six adversarial cases — legitimate request, replay, tampered body, wrong secret, stale timestamp, malformed header — and then pins each outcome with assertion builtins piped through the `|>` operator:

```

$r1->{ok}           |> aeq 1, "legitimate request accepted"
$r2->{ok}           |> aeq 0, "replay rejected"
$r2->{reason} =~ /replay/ |> aok "replay reason mentions replay"

```

The `|>` pipe feeds the left value into `aeq` (assert-equal) or `aok` (assert-truthy), the `=~ /replay/` is a regex

match against the rejection reason, and a closing `test_run` executes the registered assertions. Across these two files the throughline is that stryke makes a security invariant legible: the Merkle combine function's delimiter, the constant-time comparison's deferred zero-check, the replay seen-set, and the freshness window are each a few plainly readable lines, and the assertion pipes turn the protocol's required behaviors into checked claims rather than prose.

Chapter 30 – Parsing, Text, and Bioinformatics

Text is the substrate on which the shell-adjacent world runs, and parsing is the act of recovering structure from that flat substrate. The stryke example corpus treats parsing not as a single technique but as a spectrum: at one end the hand-written recursive-descent walker that consumes a character stream one byte at a time and builds a value tree, and at the other end the regular expression that delegates structural recognition to a compiled automaton and lets the language fish captures out of the match. Bioinformatics sits naturally alongside these because nucleotide and amino-acid work is, at bottom, string processing with a domain vocabulary — codons are fixed-width tokens, sequence alignment is a two-dimensional dynamic-programming traversal over two character strings, and edit distance is the same minimum-cost transformation problem that powers spell correction. This chapter reads seven real programs from the `examples/` tree, each one self-contained and each one demonstrating a distinct point on that spectrum. The programs are `json_parser.stk`, `url_parser.stk`, `email_parser.stk`, `markdown_parser.stk`, `rna_codon_translator.stk`, `dna_alignment.stk`, and `damerau_no_interop.stk`. Across them you will see the regular expression operators `=~` and `s///g`, named capture groups via `${...}`, compiled patterns through `qr//`, the global match loop driven by the `/g` flag inside a `while`, the threading and pipe-forward macros `~>` and `|>`, the implicit topic `_`, and `split` in both its delimiter and its character-fission forms. None of the code below is invented; every line is copied from the file under discussion.

`json_parser.stk` – recursive descent without regex

The JSON parser is the purest expression of the hand-written end of the spectrum. It uses no regular expressions at all for structure; instead it carries a mutable cursor through the source string and dispatches on the next character. The parser state is a single hashref holding the source, the current position, and the precomputed length, constructed at entry and threaded by reference through every helper so that position advances are visible to the whole parse:

```
fn JP::parse($s) {
  var $st = +{ src => $s, pos => 0, n => len($s) }
  JP::_ws($st)
  val $v = JP::_value($st)
  JP::_ws($st)
  $v
}
```

The `+{ ... }` form builds an anonymous hashref directly, and because `$st` is passed as a reference, the whitespace skipper, the peek, and the take primitives all mutate the same `pos`. Whitespace skipping is a tight `while` that inspects one character at a time with `substr` and stops on the first non-space, non-tab, non-newline, non-carriage-return byte, incrementing `$st->{pos}` until the predicate fails. The dispatch heart of the parser is `JP::_value`, which peeks at the leading character and routes to the appropriate sub-parser. The routing reads as a stack of guarded early returns, which is the idiomatic stryke way to express a dispatch table without a `match`:

```

fn JP::_value($st) {
  JP::_ws($st)
  val $c = JP::_peek($st)
  return JP::_string($st) if $c eq '"'
  return JP::_array($st)  if $c eq "["
  return JP::_object($st) if $c eq "{"
  if ($c eq "-" || ($c ge "0" && $c le "9")) { return JP::_number($st) }
  if (substr($st->{src}, $st->{pos}, 4) eq "true") { $st->{pos} = $st->{pos} + 4;
    ~ return 1 }
  if (substr($st->{src}, $st->{pos}, 5) eq "false") { $st->{pos} = $st->{pos} + 5;
    ~ return 0 }
  if (substr($st->{src}, $st->{pos}, 4) eq "null") { $st->{pos} = $st->{pos} + 4;
    ~ return undef }
  die "unexpected: " . $c . " at " . $st->{pos}
}

```

The literal keywords `true`, `false`, and `null` are recognized by a fixed-width `substr` comparison against the source at the cursor, and on a hit the position jumps forward by the literal's length. The booleans are decoded to `1` and `0` and `null` to `undef`, so the resulting value tree is built from native stryke scalars rather than wrapper objects. String parsing consumes the opening quote with `JP::_take`, accumulates into a `var $out` string, and handles the escape repertoire by switching on the character after a backslash, translating `n`, `t`, `r`, the literal backslash, the double quote, and the forward slash into their decoded forms while passing any other escaped character through verbatim. The header comment is explicit about scope — “no exponents, no unicode escapes” — so the `\u` escape and the exponent grammar are deliberately out of bounds, which keeps the number parser to a single pass that walks digits and at most one decimal point.

The number parser is the one place where a regular expression appears, and it is used not for parsing but for type discrimination after the digits have been gathered. Having sliced the raw numeric text out with `substr`, the function tests whether that text contains a decimal point and produces either a float or an integer accordingly:

```

val $raw = substr($st->{src}, $start, $st->{pos} - $start)
($raw =~ /\./) ? ($raw + 0.0) : ($raw + 0)

```

The `=~ /\./` test is a boolean: does the raw token contain a literal dot. If it does, adding `0.0` coerces the string to a float; otherwise adding `0` coerces to an integer. This is the smallest possible use of the regex engine — a single anchored-nowhere literal match used purely as a predicate — and it sits inside an otherwise regex-free parser, which makes the file a clean illustration that stryke does not force you to choose a single parsing paradigm. Arrays and objects are parsed with the classic “parse first element, then loop on comma” structure, where the array parser pushes each `JP::_value` result onto a `var @out` and returns a reference to that array (`\@out`), while the object parser reads a string key, asserts the colon separator with a `die` on mismatch, parses the value, and stores it into a `var %out` hash returned as `\%out`. The demonstration at the bottom parses a flat object and a deeply nested array and then navigates the resulting tree with the arrow-deref chain `$v2->[2]->[2]->[0]`, confirming that the parser produced genuine nested references rather than a flattened structure.

url_parser.stk — destructuring by index and split

Where JSON requires recursion because its grammar nests arbitrarily, a URL has a fixed, shallow shape, and `url_parser.stk` exploits that by parsing entirely through `index` and `substr` with no recursion and no regex. The parser allocates a result hash with all six components — scheme, host, port, path, query, fragment — initialized to the empty string, then peels the URL apart in the order that the delimiters appear, narrowing a working copy `$s` after each extraction:

```

var $s = $url
val $col = index($s, "://")
if ($col >= 0) {
    $u{scheme} = substr($s, 0, $col)
    $s = substr($s, $col + 3)
}
val $hash = index($s, "#")
if ($hash >= 0) {
    $u{fragment} = substr($s, $hash + 1)
    $s = substr($s, 0, $hash)
}

```

The technique is destructure-and-shrink: each delimiter is located with `index`, the component to the right (or left) is sliced out, and the remaining substring is reassigned to `$s` so the next stage works on a smaller problem. After the scheme is removed by skipping past `://`, the fragment is pulled off the end at the `#`, then the query string at the `?`, then the path is separated from the authority at the first `/`. Only after the host-and-port chunk is isolated does the parser look for the colon that separates host from port. This ordering matters: extracting the fragment and query before the path keeps a `?` or `#` that might legally appear later from interfering with the path split, and isolating the authority before splitting on the colon keeps a colon inside the path from being mistaken for a port separator. The second function, `Url::query_params`, shows the delimiter form of `split` doing the work that `index` did for the top-level structure:

```

var @pairs = split /\&/, $url_obj->{query}
for val $kv (@pairs) {
    val $eq = index($kv, "=")
    if ($eq >= 0) {
        val $k = substr($kv, 0, $eq)
        val $v = substr($kv, $eq + 1)
        $params{$k} = $v
    } else {
        $params{$kv} = ""
    }
}

```

The query string is split on `&` into key-value pairs, and each pair is then bisected at its first `=` with `index` and `substr` rather than another `split`, which guards against values that themselves contain `=`. A pair with no `=` is recorded as a bare key with an empty value. The demonstration parses `https://api.example.com:8443/users/42?fmt=json&pretty=1#top` and prints each component, then sorts the parameter keys with `sort keys %$p` and dumps them, confirming that scheme, host, port, path, query, and fragment were all recovered and that the two parameters were correctly bisected. This program and the JSON parser bracket the manual end of the spectrum: both avoid the regex engine for structure, one because recursion is unavoidable and one because the structure is flat enough that linear scanning is simpler than a pattern.

shell_tokenizer.stk — quote-aware character-stream tokenizing

The shell tokenizer is the JSON parser's tokenizing cousin: a character-stream scanner that recognizes word boundaries while respecting single-quote, double-quote, and backslash-escape semantics, all without a single regular expression. The state is three locals — the accumulated current word, an `$in_word` flag distinguishing “I have started a token” from “I am between tokens,” and the cursor `$i` — and the main loop dispatches on the current character. Unquoted whitespace closes the current word if one is open and is otherwise skipped; a backslash copies the following character verbatim into the current word and advances by two, which is how a space can be embedded in a token:

```

if ($c eq "\\") {
  if ($i + 1 < $n) {
    $cur = $cur . substr($s, $i + 1, 1)
    $in_word = 1
    $i = $i + 2
    next
  } else {
    $i = $i + 1
    next
  }
}
}

```

The two quote forms differ exactly as a POSIX shell makes them differ, and the tokenizer encodes that difference structurally. A single quote opens an inner `while` that copies every character literally until the closing quote, with no escape processing at all, so backslashes inside single quotes are ordinary characters. A double quote opens an inner loop that does honor a restricted escape set — `\`, `\\`, and `\n` — translating the last into a real newline while passing the others through as their literal character:

```

if ($c eq '"') {
  $i = $i + 1
  $in_word = 1
  while ($i < $n) {
    val $cc = substr($s, $i, 1)
    last if $cc eq '"'
    if ($cc eq "\\") && $i + 1 < $n) {
      val $nx = substr($s, $i + 1, 1)
      if ($nx eq '"' || $nx eq "\\") || $nx eq "n") {
        if ($nx eq "n") {
          $cur = $cur . "\n"
        } else {
          $cur = $cur . $nx
        }
        $i = $i + 2
        next
      }
    }
    $cur = $cur . $cc
    $i = $i + 1
  }
  $i = $i + 1
  next
}
}

```

Setting `$in_word = 1` on entering a quote is what makes an empty quoted string — `"` — still produce a token, because the flag, not the accumulated length, decides whether a word is pushed at the next boundary. Any other character falls through to the default arm, which appends to the current word and marks `$in_word`, and at end of input a final open word is flushed. The demonstrations exercise the three modes together: `echo 'hello world' "foo bar" baz` must produce `echo`, `hello world`, `foo bar`, and `baz` as four tokens despite the embedded spaces, and `grep -e "a\"b" /etc/file\ with\ space` must keep the escaped quote inside the double-quoted token and the backslash-escaped spaces inside the final path token. This file and `json_parser.stk` together establish the manual scanner as a first-class stryke idiom: carry a cursor, peek and take one character at a time, and nest inner loops for the bracketed sublanguages — quotes here, strings and arrays there.

email_parser.stk – regex per header line and keyword scoring

The email parser introduces the regular expression as a line-level recognizer and pairs it with a keyword-scoring spam heuristic. An RFC-822-style message splits into headers and body at the first blank line, and the parser models that as a state machine over the lines produced by splitting the raw message on newline:

```
fn Demo::Email::parse($raw) {
  var @lines = split /\n/, $raw
  var %headers
  var @body_lines
  var $in_body = 0
  for val $line (@lines) {
    if (!$in_body) {
      if (len($line) == 0) {
        $in_body = 1
        next
      }
      if ($line =~ /^[A-Za-z\-\+]:\s*(.*)$/) {
        val $name = lc $1
        val $value = $2
        $headers{$name} = $value
      }
    } else {
      push @body_lines, $line
    }
  }
  val $body = join("\n", @body_lines)
  return +{
    headers => \%headers,
    body    => $body,
  }
}
```

While in the header region, a zero-length line flips `$in_body` and is skipped with `next`; otherwise the line is matched against `^[A-Za-z\-\+]:\s*(.*)$`, which captures the header name before the colon and the value after optional whitespace. The captured name is lowercased with `lc $1` before being stored, so downstream lookups can use a canonical case regardless of how the sender wrote the header. Everything after the blank line accumulates into the body, which is rejoined with newlines. The classifier walks a flat list of spam keywords and scores a message by counting how many of them appear, using `index` for the substring test rather than a regex because the keywords are plain literals and `index($text, lc($kw)) >= 0` is the direct way to ask “does this contain that”:

```
fn Demo::Email::is_spammy($parsed) {
  val $text = lc($parsed->{body} . " " . ($parsed->{headers}->{subject} // ""))
  var $score = 0
  var @hits
  for val $kw (@spam_keywords) {
    if (index($text, lc($kw)) >= 0) {
      $score = $score + 1
      push @hits, $kw
    }
  }
  +{ score => $score, keywords => \@hits }
}
```

The classifier concatenates the body with the subject (the latter defended with the `//` defined-or operator against a missing header) before lowercasing the whole thing once, so every keyword test runs against case-

folded text. The score is the count of matched keywords and the hit list is returned alongside it for explanatory output. The driver processes six raw messages — two ordinary, four with escalating spam vocabulary — and rolls the results up through several sketch structures: a t-digest `td_parse` records per-message parse latency, a HyperLogLog `hll_senders` estimates the distinct-sender cardinality, and a TopK `tk_subjects` tracks the most frequent subjects. The per-sender breakdown uses a comparator that sorts by descending message count, written with the implicit comparison operands `_1` and `_`:

```
for val $sender (sort { $by_sender[_1] <=> $by_sender[_] } keys %by_sender) {
    printf " %-30s : %d messages\n", $sender, $by_sender{$sender}
}
```

This is the stryke comparator idiom: `_` and `_1` stand for the two elements being compared, and `<=>` is the numeric spaceship, so `$by_sender[_1] <=> $by_sender[_]` orders senders from most to fewest messages. The final ham-versus-spam rollup uses `grep` with a block predicate over the processed list, `grep { _->{spam}->{score} >= 4 } @processed`, where the topic `_` is the current element and the arrow chain reaches into its nested score, returning the count of messages whose score crosses the spam threshold of four.

markdown_parser.stk — regex substitution as a transform pipeline

The Markdown converter is the clearest demonstration of `s///g` substitution used as a transformation stage rather than a recognizer. Inline formatting — code spans, bold, italic, and links — is applied as an ordered series of in-place substitutions on the line, and the ordering is load-bearing, which the source comments call out directly: code spans are converted first so that an asterisk living inside backticks is never mistaken for emphasis, and bold (`**`) is converted before italic (`*`) so the double-asterisk delimiter is consumed before the single-asterisk rule can see it:

```
fn Demo::MD::inline($text) {
    # Apply inline rules in order: code first (so * inside `code` isn't
    # treated as emphasis), then bold, italic, links.
    $text =~ s/`([^`]+)`/<code>$1</code>/g
    $text =~ s/\*\*([^*]+)\*\*/<strong>$1</strong>/g
    $text =~ s/\*([^*]+)\*/<em>$1</em>/g
    # Links: [text](url)
    $text =~ s/\[([^\]]+)\]\([^\)]+\)/<a href="$2">$1</a>/g
    $text
}
```

Each `s///g` mutates `$text` in place via `=~`, and the capture group inside the pattern is referenced by `$1` (and `$2` for the link's URL) inside the replacement, where the forward slash in the closing HTML tags is escaped as `<\/code>`. The link rule captures the bracketed text and the parenthesized URL separately and reassembles them into an anchor with the URL in the `href` and the text as the anchor body. Block-level structure is a line-by-line state machine over `split /\n/, $md`, tracking whether the parser is currently inside an unordered list with `$in_list` and buffering consecutive plain lines into `@para_buf` until a structural line or a blank line forces a paragraph flush. Headings are recognized by a single pattern that captures the run of hashes and the heading text, and the heading level is simply the length of the captured hash run:

```

if ($line =~ /^(#{1,6})\s+(.+)/) {
    Demo::MD::flush_para(@para_buf, @html)
    if ($in_list) {
        push @html, "</ul>"
        $in_list = 0
    }
    val $hashes = $1
    val $text = $2
    val $level = len $hashes
    push @html, "<h${level}>" . Demo::MD::inline($text) . "</h${level}>"
    next
}

```

The pattern `^(#{1,6})\s+(.+)/` captures one to six hashes and the following text; `len $hashes` turns the captured hash run into the numeric heading level, which is interpolated into the open and close tags as `<h${level}>` and `</h${level}>`. List items match `^\s+(.+)/`, blank lines match `^\s*$` and trigger paragraph and list flushes, and any other line accumulates into the paragraph buffer. After rendering, the program performs a tag-type rollup that shows the `/g` flag driving a global match loop inside a `while`, which is the canonical way stryke iterates over every match in a string:

```

while ($line =~ /<(\w+)/g) {
    $tags{$1}++
}

```

Each iteration of the `while` advances the match position and binds `$1` to the next tag name, so the loop visits every opening tag on the line and increments the corresponding bucket in `%tags`. This same `/g-in-while` idiom appears in the email and regex examples; it is the language's general mechanism for streaming all matches rather than just the first. The rollup is then sorted with the same `_1/_` descending comparator seen in the email parser, and the distinct tag count is cross-checked between a HyperLogLog estimate and the exact key count, while a TopK sketch reports the most frequently emitted tags.

regex_three_tier.stk – the three-tier engine made visible

If the manual scanners show what stryke does without regular expressions, `regex_three_tier.stk` shows what its regex engine actually is. The file's header documents that stryke does not bind to one regex backend but to three, and that the compiler picks the backend per pattern, escalating only when a pattern needs richer features. Tier one is the Rust `regex` crate, a linear-time DFA with no backreferences or look-around; tier two is `fancy-regex`, which adds backreferences and look-around; tier three is `pcre2`, reached only for PCRE-only verbs. The header states the user-facing consequence directly: “Same `m//`, `s///`, `qr//` syntax; engine selection is invisible.” A tier-one match with two numbered captures looks like ordinary regex, and the captures are bound to `$1` and `$2`:

```

val $s = "user_id=42 status=ok latency=137ms"
if ($s =~ /user_id=(\d+).*latency=(\d+)ms/) {
    p " user=$1 latency=${2}ms"
}

```

Named captures use the `(?<name>...)` syntax in the pattern and are read back through the `${name}` hash, which is the dedicated named-capture accessor:

```

val $email = "alice.bob+spam@example.com"
if ($email =~ /^(?<user>[\w.+]*)@(?<host>[\w.+]*)$/) {
    p " user = ${user}"
    p " host = ${host}"
}

```

The file then deliberately reaches into tier two. The palindrome test relies on backreferences — `\2\1` requires the engine to refer to earlier captured text, which the linear DFA cannot do — so the compiler escalates to `fancy-regex` for `/^(.)(.)*\2\1$/`. The look-around demonstration uses a positive look-ahead (`?= bar`) to find words that are followed by " bar" without consuming the " bar", and it pairs that with the `/g-in-while` global-match idiom to collect every such word:

```
while ($text =~ /(\w+)(?= bar)/g) {
  push @hits, $1
}
```

Substitution appears in two forms: a plain global replace `$piped =~ s/,/ | /g` that converts a comma-separated string to pipe-separated, and a capture-referencing replace `$words =~ s/(\w+)/<$1>/g` that wraps each word in angle brackets by referring to the captured word as `$1` in the replacement. The `qr//` form compiles a pattern once for reuse, and the compiled object is then matched against many inputs in a loop, with the captures available as `$1`, `$2`, `$3` after each successful match:

```
val $isodate = qr/^(\\d{4})-(\\d{2})-(\\d{2})$/
for val $d ("2026-05-15", "not-a-date", "1999-12-31") {
  if ($d =~ $isodate) {
    printf " %s → year=%s, month=%s, day=%s\\n", $d, $1, $2, $3
  } else {
    printf " %s → reject\\n", $d
  }
}
```

The closing section composes regex with the pipe-forward operator, filtering a log array down to its error rows by piping the array into a `grep` whose block predicate is a bare match against the topic, `@log |> grep {/^ERROR/}`. A bare `/^ERROR/` with no explicit `==` matches against `_`, the current element, which is why the predicate needs no left-hand operand. The whole file is the concrete grounding for the claim that stryke's regex surface is a single syntax over a tiered backend: every pattern here is written in the same `m//`, `s///`, `qr//` notation, yet some run on the linear DFA and others force escalation to backreference-and-look-around territory, with no annotation from the programmer marking which is which.

rna_codon_translator.stk — fixed-width tokenizing and the threading macro

The RNA translator moves the chapter into bioinformatics proper, and it is the canonical demonstration of the threading macro `>>` and the pipe-forward `|>` applied to a tokenizing problem. The genetic code is a lookup table from three-letter codons to amino-acid names, declared as a `val %CODONS` hash with the stop codons mapped to the sentinel string `"STOP"`. A single codon is translated by a one-line expression-bodied function that uppercases its argument, indexes the table, and falls back to `"Unknown"` with the defined-or operator:

```
fn Bio::translate_codon($codon) = $CODONS{uc $codon} // "Unknown"
```

The `fn name($arg) = expr` form is stryke's expression-bodied function syntax, and `$CODONS{uc $codon} // "Unknown"` looks the uppercased codon up and supplies a default for anything not in the standard code. Full-sequence translation is fixed-width tokenizing: the sequence is walked three characters at a time using a strided range, and each three-character slice is taken with the slice syntax `$s[$i:$i+2]`, translated, and pushed onto the accumulator until a stop codon ends the chain:

```
for val $i (0:len($s)-3:3) {
  val $codon = $s[$i:$i+2]
  val $aa = Bio::translate_codon($codon)
  last if $aa eq "STOP"
  push @amino_acids, $aa
}
```

The range `0:len($s)-3:3` is a start-stop-step range that produces codon-aligned starting indices, `$s[$i:$i+2]` slices out the inclusive three-character window, and `last if $aa eq "STOP"` terminates translation at the first stop codon, which models the biological behavior where the ribosome releases the chain at a stop. The two demonstration paths show that stryke offers two equivalent ways to feed a value into this function. The first uses pipe-forward:

```
val $translated = $sample_rna |> Bio::translate_sequence
```

The second uses the prefix threading macro, where `~>` takes the value and the function and applies the latter to the former:

```
val $long_translated = ~> $long_rna Bio::translate_sequence
```

Both `$sample_rna |> Bio::translate_sequence` and `~> $long_rna Bio::translate_sequence` feed a single sequence string through the translator; the file's header comment names the `~>` form the "thread-first macro." The assertions at the bottom pin the behavior exactly — the short sequence `AUGUUUCUCAA` must yield three amino acids ending before its trailing stop codon, and the longer `AUGUUUCUCUCUAGUCAUACGUAA` must yield seven, with the last being threonine — and the file uses `#{...}` interpolation inside the `die` messages so that a failure reports the actual computed value, as in `die "Translation count mismatch: #{len @$translated}"`.

dna_alignment.stk – Needleman-Wunsch as a 2-D DP over two strings

The DNA alignment program is the most computationally substantial parser-adjacent example in the set: it implements Needleman-Wunsch global alignment, which is the canonical dynamic-programming algorithm for optimally aligning two sequences. The scoring scheme is declared as three constants — match `+2`, mismatch `-1`, gap `-2` — and the algorithm builds two parallel two-dimensional tables, one holding the cumulative best score at each cell and one holding a back-pointer string (`"diag"`, `"up"`, or `"left"`) recording which neighbor produced that score. Both tables are arrays of arrayrefs, explicitly allocated and zero-filled before the boundary rows and columns are seeded with cumulative gap penalties. The fill loop is the heart of the algorithm, computing each interior cell as the maximum of three candidate moves and recording the winning direction:

```
val $ac = substr($seq_a, $i - 1, 1)
val $bc = substr($seq_b, $j - 1, 1)
val $diag = $table[$i - 1]->[$j - 1] + Demo::DNA::score($ac, $bc)
val $up = $table[$i - 1]->[$j] + $gap_penalty
val $left = $table[$i]->[$j - 1] + $gap_penalty
val $best = Demo::DNA::max3($diag, $up, $left)
$table[$i]->[$j] = $best
if ($best == $diag) {
    $trace[$i]->[$j] = "diag"
} elsif ($best == $up) {
    $trace[$i]->[$j] = "up"
} else {
    $trace[$i]->[$j] = "left"
}
```

The diagonal candidate adds the match-or-mismatch score for the two current characters, drawn with single-character `substr` calls; the up and left candidates each add the gap penalty, modeling an insertion of a gap into one sequence or the other. `Demo::DNA::max3` picks the largest of the three, and the back-pointer table records which move won so the path can be reconstructed. The traceback walks the back-pointer table from the bottom-right corner toward the origin, and each step `unshifts` a character (or a `-` gap marker) onto the front of the aligned-string accumulators, so that walking backward through the table builds the aligned strings forward:

```

if ($dir eq "diag") {
    unshift @a_aligned, substr($seq_a, $i - 1, 1)
    unshift @b_aligned, substr($seq_b, $j - 1, 1)
    $i = $i - 1
    $j = $j - 1
} elsif ($dir eq "up") {
    unshift @a_aligned, substr($seq_a, $i - 1, 1)
    unshift @b_aligned, "-"
    $i = $i - 1
} else {
    unshift @a_aligned, "-"
    unshift @b_aligned, substr($seq_b, $j - 1, 1)
    $j = $j - 1
}

```

A diagonal move consumes one character from each sequence; an up move consumes one from `seq_a` and aligns it against a gap in `seq_b`; a left move does the reverse. After alignment, the driver constructs a visual match line character by character — a `|` where the two aligned characters are equal, a space where either is a gap, and a `.` for a mismatch — and then counts each category using `grep` with the topic `_` over the characters produced by `split //`, `$match_line`:

```

val $matches      = grep { _ eq "|" } split //, $match_line
val $mismatches   = grep { _ eq "." } split //, $match_line
val $gaps         = grep { _ eq " " } split //, $match_line

```

Here `split //` fissions the match line into its individual characters and `grep` in scalar context counts how many satisfy each equality test, with `_` bound to each character in turn. This is the character-fission form of `split` — an empty pattern splits between every character — used here purely to tally a summary line. The program also instruments itself: a t-digest `$td_cell` accumulates the per-cell fill latency captured with `now_ns()` deltas, and the closing report prints p50 and p99 quantiles, then verifies the algorithm against an identity case where aligning `ACGTACGT` against itself must produce a score equal to the sequence length times the match score, $8 * 2 = 16$.

damerau_no_interop.stk — edit distance and the spell-correction connection

The Damerau-Levenshtein program closes the loop between text correction and bioinformatics by implementing the edit-distance metric that underlies typo correction, and it is structurally a sibling of the alignment DP: both are minimum-cost transformations computed over a two-dimensional table indexed by positions in two strings. The difference is the transposition rule, which the header explains makes “teh” → “the” cost one edit instead of two, matching how humans actually mistype. Both strings are fissioned into character arrays with `split //`, and the table is filled with the standard three-way Levenshtein recurrence using the `min` builtin, then conditionally lowered by the transposition rule when the two preceding characters are swapped:

```

val $cost = $sa[$i - 1] eq $ta[$j - 1] ? 0 : 1
$d[$i][$j] = min(
    $d[$i - 1][$j] + 1,      # deletion
    $d[$i][$j - 1] + 1,      # insertion
    $d[$i - 1][$j - 1] + $cost, # substitution
)
if ($i >= 2 && $j >= 2 &&
    $sa[$i - 1] eq $ta[$j - 2] &&
    $sa[$i - 2] eq $ta[$j - 1]) {
    $d[$i][$j] = min($d[$i][$j], $d[$i - 2][$j - 2] + 1)
}

```

The substitution cost is zero when the characters match and one when they differ; the three `min` arguments

correspond to deletion, insertion, and substitution; and the transposition clause, guarded so it only fires from index two onward, checks whether the current and previous characters of each string are crossed and, if so, allows a single transposition edit from the cell two steps back on the diagonal. The header comment notes that under `--no-interop` stryke “doesn’t autoviv nested arrayrefs,” so the table is explicitly pre-allocated with `for val $i (0:$m) { $d[$i] = [(0) x ($n + 1)] }`, where the list-repetition `(0) x ($n + 1)` builds a zero-filled row of the right width. The test harness is a table of cases each carrying the two strings, the expected distance, and a label, and the verification not only checks each expected value but also asserts the metric’s mathematical invariants — symmetry ($DL(s, t) == DL(t, s)$), the upper bound that distance never exceeds the longer string’s length, and the triangle inequality on a concrete triple. The failure messages use `#{...}` interpolation to surface the actual versus expected values, the same diagnostic idiom seen in the RNA translator. Taken together with the alignment program, this file shows that stryke treats two strings and a cost recurrence as a single reusable shape: change the recurrence and the scoring, and the same two-dimensional walk serves both global sequence alignment and fuzzy-match spell correction.

Chapter 31 – Optimization, Numerics, and Data Structures

A scripting language earns its place in the quantitative corner of computing not by exposing a single `solve` builtin but by making the underlying loops, tables, and recurrences pleasant to write by hand. The six programs studied in this chapter all take that route deliberately. They build dynamic-programming tables cell by cell, descend gradients step by step, walk parse trees node by node, isolate the lowest set bit of an index, recurse through the butterflies of a Fourier transform, and pivot rows of an augmented matrix until an identity emerges on the left. None of them hides the algorithm behind a library call; each one is a worked specimen of how stryke's array and hash literals, its `1:n` ranges, its arrayref dereference syntax, its hash-of-string-keyed records, and its small math vocabulary combine to express textbook numerical and combinatorial methods. The examples were chosen because they are substantial — each runs to well over a hundred lines, carries its own correctness checks, and exercises a distinct cluster of the language's features. Read together they form a tour of what the language looks like when it is asked to compute rather than to orchestrate.

Zero/One Knapsack via a Two-Dimensional DP Table

The `knapsack_optimizer.stk` program solves the canonical 0/1 knapsack problem: given items each carrying a weight and a value, and a bag of fixed capacity, choose the subset that maximises total value without exceeding capacity. The solver lives in a namespaced function, `fn Demo::KS::solve($items_ref, $capacity)`, and its first act is to allocate a two-dimensional table. stryke spells the empty-table idiom out in two stages — a scalar `var @dp` declared empty, then a loop that pushes a fresh zero-filled row for each item index:

```
fn Demo::KS::solve($items_ref, $capacity) {
  val $n = len @$items_ref
  var @dp
  for val $i (0:$n) {
    push @dp, [(0) x ($capacity + 1)]
  }
}
```

The expression `[(0) x ($capacity + 1)]` is the row constructor: the list-repetition operator `x` replicates the single-element list `(0)` `($capacity + 1)` times, and the surrounding brackets capture that list as an arrayref so that `@dp` becomes an array of arrayrefs — a genuine matrix indexed `$dp[$i]->[$cap]`. The dimensions are `$n + 1` rows (because the loop runs `0:$n`, inclusive of both endpoints) by `$capacity + 1` columns, matching the standard DP convention where row zero and column zero represent the empty-item and zero-capacity base cases. The two-stage allocation — declare `var @dp` empty, then push one fresh arrayref per iteration — matters precisely because the language treats `x` on a list as producing a flat run of values; without wrapping each replicated run in its own pair of brackets the rows would alias one another, and a write to one row would silently corrupt all the others. By pushing a separately bracketed `[(0) x ($capacity + 1)]` on every pass the program guarantees `$n + 1` distinct, independently mutable rows, which is the property the cell-by-cell fill depends on.

The fill phase is the classic recurrence written without a single library call. For each item index `1:$n` and each capacity `0:$capacity`, the cell is either inherited from the row above (the item does not fit, or is skipped) or computed as the better of skipping versus taking the item:

```

for val $i (1:$n) {
  val $item = $items_ref->[$i - 1]
  val $w = $item->{w}
  val $v = $item->{v}
  for val $cap (0:$capacity) {
    if ($w > $cap) {
      $dp[$i]->[$cap] = $dp[$i - 1]->[$cap]
    } else {
      val $skip = $dp[$i - 1]->[$cap]
      val $take = $dp[$i - 1]->[$cap - $w] + $v
      $dp[$i]->[$cap] = $skip > $take ? $skip : $take
    }
  }
}

```

Two stryke conventions are worth singling out here. First, items are records — hashrefs whose fields are extracted with the `->{w}` and `->{v}` arrow-and-brace syntax — and the item array is one-indexed against a zero-indexed source via `$items_ref->[$i - 1]`, so the DP table's row `$i` corresponds to “the first `$i` items.” Second, the ternary `$skip > $take ? $skip : $take` does the `max` inline, avoiding any helper. The take branch reaches back to `$dp[$i - 1]->[$cap - $w]`, the optimal value for the remaining capacity once the current item's weight is paid, then adds its value — the heart of the recurrence.

Having filled the table the program does not stop at the optimal value; it reconstructs which items were chosen by backtracking. Walking the item indices in reverse with `rev 1:$n`, it compares each cell to the cell directly above it: if they differ, the item must have been taken, so it is unshifted onto the front of a `@chosen` list and the running capacity is decremented by that item's weight.

```

var @chosen
var $cap = $capacity
for val $i (rev 1:$n) {
  if ($dp[$i]->[$cap] != $dp[$i - 1]->[$cap]) {
    unshift @chosen, $items_ref->[$i - 1]
    $cap -= $items_ref->[$i - 1]->{w}
  }
}

```

The `rev` modifier on the range reverses iteration order, and `unshift` prepends so that the recovered list comes out in original item order rather than reversed. The logic is the textbook reconstruction: if the optimal value for the first `$i` items at capacity `$cap` differs from the optimal value for the first `$i - 1` items at the same capacity, then item `$i` must have contributed to the optimum, so it is recorded and the residual capacity drops by that item's weight before the walk continues upward. The function returns a record `{ max_value => ..., chosen => \@chosen, used_w => $capacity - $cap }`, where `\@chosen` takes a reference to the local array so it survives the function's return — the same reference discipline seen in every other example here, since aggregates are passed and returned by arrayref rather than copied. The driver then defines four problem instances — `camping`, `burglary`, `groceries`, `rock-tour` — each an array-of-hashrefs of named items, and solves all four in parallel with `pmap`, the topic-variable form being `pmap { Demo::KS::solve(_->{items}, _->{capacity}) } @instances`, where the underscore `_` is the per-element topic. Beyond the core algorithm the program threads its results through three probabilistic sketches built at the top — a `t_digest(100)` for value-per-weight efficiency quantiles, a `topk(16)` for the most-frequently-chosen items, and an `hll(12)` HyperLogLog for distinct chosen-item names — closing with a cross-check of the HLL estimate against an exact count assembled from a `%seen` hash. The result is a single file that uses two-dimensional arrays, hashref records, ternary maximisation, reverse-range backtracking, parallel map, and streaming sketches in concert.

Mini-Batch Gradient Descent for Linear Regression

Where the knapsack program is combinatorial, `gradient_descent_linreg.stk` is continuous optimization, and its stated design principle — visible in the header comment — is that “the demo’s value is showing the optimization loop, not hiding it.” It fits the line $y \approx 3 \cdot x + 7$ from a synthetic noisy dataset. The data generation is itself instructive: a reproducible PRNG is seeded with `rand 42`, and Gaussian noise is produced by a hand-written Box-Muller transform rather than any normal-distribution builtin:

```
fn Linreg::gauss($s) {
  var $u1 = rand()
  val $u2 = rand()
  $u1 = 1e-10 if $u1 < 1e-10
  sqrt(-2.0 * log($u1)) * cos(2.0 * 3.14159265358979 * $u2) * $s
}
```

This single function exercises a slice of stryke’s math vocabulary — `rand()`, `sqrt`, `log`, `cos` — and the statement-modifier `if` clause `$u1 = 1e-10 if $u1 < 1e-10` guards against taking the log of zero. The function’s value is its tail expression (no explicit `return`), scaled by the requested standard deviation `$s`. The dataset loop draws a thousand points with `$x` uniform on `[-10, 10]` and `$y` the true line plus noise, pushing into parallel `@xs` and `@ys` arrays. Crucially, the program then captures arrayrefs of those arrays — `val $XS = \@xs` and `val $YS = \@ys` — with the comment explaining that “`pmap_reduce` workers reach these through lexical closure more reliably than via the array variables themselves.” This is a real, documented idiom for the parallel primitives: closures over scalar arrayrefs cross worker boundaries more dependably than closures over aggregate variables.

The loss is mean squared error, computed by an explicit accumulation loop over `0:$n - 1`. The optimization driver maintains scalar parameters `$w` and `$b`, a learning rate `$lr = 0.01`, and runs `$epochs = 120` epochs with a batch size of two hundred. Each epoch begins by shuffling an index array in place with a Fisher-Yates loop:

```
fn Linreg::shuffle($arr) {
  for var $i (rev(1:len(@$arr) - 1)) {
    var $j = int(rand() * ($i + 1))
    ($arr->[$i], $arr->[$j]) = ($arr->[$j], $arr->[$i])
  }
}
```

The reverse range `rev(1:len(@$arr) - 1)` walks from the last index down to one, picks a random partner `$j` in `0:$i`, and swaps the two elements via stryke’s parallel-assignment tuple syntax `($a, $b) = ($b, $a)` — no temporary variable, the right-hand list is evaluated before the left-hand binding. The shuffled indices are then sliced into batches with `[ @indices[$start:$end] ]`, the slice `@indices[$start:$end]` selecting a contiguous run and the brackets re-capturing it as a batch arrayref.

The gradient computation is the program’s centrepiece and uses `pmap_reduce`, a two-block parallel fold. The first block maps each batch to a partial gradient triple, the second block reduces two partials into one:

```
val $grads = @batches |> pmap_reduce {
  var $sum_dw = 0
  var $sum_db = 0
  for val $i (@$_) {
    val $err = $w * $XS->[$i] + $b - $YS->[$i]
    $sum_dw += 2 * $err * $XS->[$i]
    $sum_db += 2 * $err
  }
  [ $sum_dw, $sum_db, len(@$_) ]
} {
  [ _->[0] + _1->[0], _->[1] + _1->[1], _->[2] + _1->[2] ]
};
```

Several language details converge in this block. The pipe operator `|>` feeds `@batches` as the data argument. Inside the map block the topic `$_1` is the current batch arrayref, dereferenced for iteration with `@$_1` and indexed via the captured `$X5/$Y5` closures; the partial result is the triple `[sum_dw, sum_db, count]`. In the reduce block the two combinands are the topic `_` and the second positional topic `_1`, summed element-wise. The header comment flags a parsing subtlety — the trailing semicolon after the reduce block is “required” because `pmap_reduce` “otherwise greedily consumes the following statement as a third block argument.” After the parallel fold the parameters are nudged by the averaged gradient: `$w -= $lr * ($grads->[0] / $count)` and the matching update for `$b`. The program ends not with a print but with four convergence assertions written through the pipe-to-assertion idiom — `(abs($w - $TRUE_W) < 0.1) |> aok "w converged within 0.1 of true 3.0"` and three siblings checking `b`, a ninety-five-percent loss drop, and final loss within `1.3×` of the irreducible noise variance — concluded by `test_run`. The assertions make the file self-verifying: it is both a demonstration and a regression test.

Symbolic Differentiation Over an Expression AST

`symbolic_diff_no_interop.stk` shifts from numbers to symbols. It tokenizes an arithmetic expression, parses it into an abstract syntax tree of hashref nodes, differentiates that tree by the chain and product rules, simplifies the result, and prints it back out — a small computer-algebra pipeline. The whole thing lives in a package `SymbolicDiff` and switches to `package main` for the driver, the two-package structure being the `--no-interop` convention for separating library functions from the script body.

Tokenization leans on stryke’s regex substitution and split. The binding operator `=~` applies an `s///g` substitution that pads every operator and parenthesis with spaces, a second substitution pads the function names, and `split /\s+/` plus a `grep { len(_) > 0 }` filter yields a clean token list:

```
fn tokenize($s) {
  var $str = $s
  $str =~ s/([\\+\\-\\*\\/\\^\\(\\)])/ $1 /g
  $str =~ s/(sin|cos|log)/ $1 /g
  val @tokens = split /\s+/, $str
  val @clean = grep { len(_) > 0 } @tokens
  val $res = [ @clean ]
  return $res
}
```

The parser is a hand-written recursive-descent set of four mutually recursive functions — `parse_expr`, `parse_mul_div`, `parse_pow`, `parse_atom` — encoding standard precedence: addition and subtraction at the top, then multiplication and division, then exponentiation, with atoms (numbers, variables, parenthesised subexpressions, and the unary functions) at the bottom. Each level consumes tokens off the front of the shared token arrayref with `shift @$tk` and peeks with `$tk->[0]`. Nodes are hashrefs tagged with a `type` field — “op”, “num”, “var”, or “fn” — and an operator-precedence climb builds left-associative trees. The atom parser shows the node shapes and the integer-literal test `$t =~ /^-?\d+$/`:

```
if ($t eq "sin" || $t eq "cos" || $t eq "log") {
  val $arg = SymbolicDiff::parse_atom($tk)
  return { type => "fn", op => $t, arg => $arg }
}
if ($t =~ /^-?\d+$/) { return { type => "num", val => int($t) } }
{ type => "var", val => $t }
```

The differentiation function `diff($node, $v)` dispatches on the node’s `type` and implements the calculus rules structurally. A number differentiates to a zero node; a variable to one or zero depending on whether it matches the differentiation variable `$v`. The product rule builds a new sum-of-products tree, and the quotient rule assembles the $(u'v - uv')/v^2$ shape entirely out of freshly constructed hashref nodes. The chain rule for the unary functions is explicit — the derivative of `sin(u)` is `cos(u)·u'`, the derivative of `cos(u)` carries the `-1` factor, and `log(u)` becomes $(1/u)·u'$:

```

if ($type eq "fn") {
  val $op = $node->{op}; val $arg = $node->{arg}; val $ad = SymbolicDiff::diff($arg,
  ~ $v)
  if ($op eq "sin") {
    return { type => "op", op => "*", left => { type => "fn", op => "cos", arg =>
      ~ $arg }, right => $ad }
  }
  if ($op eq "cos") {
    val $ns = { type => "op", op => "*", left => { type => "num", val => -1 },
  ~ right => { type => "fn", op => "sin", arg => $arg } }
    return { type => "op", op => "*", left => $ns, right => $ad }
  }
}

```

The raw derivative tree is verbose, so `simplify($node)` performs constant folding and identity elimination: when both children of a `+`, `-`, or `*` node are numbers it folds them to a single number node, and it applies the algebraic identities $x + 0 = x$, $0 + x = x$, $0 * x = 0$, and $1 * x = x$. The simplifier is itself recursive — it simplifies the two children first, binding the results to `$ls` and `$rs`, before attempting to fold or rewrite the current node — and it is defensive about its inputs, guarding every test with `defined` and `ref($ls) eq "HASH"` checks so that an undefined or non-hashref child cannot crash the rewrite. That defensiveness is necessary because the differentiation rules construct deeply nested trees in which some branches (the derivative of a constant subterm, for example) collapse to zero nodes that the simplifier must then absorb. Because one pass does not reach a fixed point, the driver simplifies once and then loops three more times — `for (1:3) { $s_ast = SymbolicDiff::simplify($s_ast) }` — before stringifying. The `to_string` function reconstructs infix notation using stryke's `#{...}` interpolation directly inside string literals: a number node becomes `"#{ $node->{val} }"`, a function call `"#{ $node->{op} } (#{ $inner })"`, and a binary operation the parenthesised `"(#{ $ls } #{ $node->{op} } #{ $rs })"`. The whole program is a study in using hashref records as a tagged-union AST, recursion over that AST, and regex-driven lexing — with no numeric computation at all beyond the constant folding.

The Fenwick Tree: Bit-Tricks for Prefix Sums

`fenwick_tree_no_interop.stk` implements the binary indexed tree, the data structure competitive programmers reach for when they need both point updates and prefix-sum queries in logarithmic time. Its correctness rests on a single bit-manipulation trick — isolating the lowest set bit of an index — and stryke expresses that as a one-line function using the equation-form `fn name = expr` syntax the file's header explicitly cites from the style guide:

```
fn Fenwick::lsb = _ & -_
```

Here `_` is the implicit single argument and `_ & -_` is the two's-complement idiom that returns the value of the lowest set bit. Construction is equally terse, the `new` function building a record with a `size` and a one-padded zero array via `+{ size => _, bit => [(0) x (_ + 1)] }`, where the leading `+{ }` disambiguates a hash literal from a code block. The tree is one-indexed internally but exposes a zero-indexed interface, so the update function offsets by one and then climbs the tree by repeatedly adding the lowest set bit:

```

fn Fenwick::add($f, $i, $delta) {
  var $k = $i + 1
  while ($k <= $f->{size}) {
    $f->{bit}[$k] += $delta
    $k += Fenwick::lsb($k)
  }
}

```

The prefix-sum query is the mirror image — starting at `$i + 1` and walking down by subtracting the lowest set bit until the index reaches zero, accumulating the partial sums stored at each visited node. Both loops touch only $O(\log n)$ nodes because each `lsb` step clears or sets exactly one bit position. The `bit` array

is accessed as `$f->{bit}[$k]`, a hashref field that is itself an arrayref, indexed without an intermediate arrow — the language permits the shorthand `$f->{bit}[$k]` for `$f->{bit}->[$k]` once the first arrow has dereferenced into the hash. Range sums are then a difference of two prefix sums, with a special case for a left bound of zero where the subtraction of `prefix_sum(lo - 1)` would index position `-1`; the guard `return Fenwick::prefix_sum($f, $hi) if $lo == 0` handles that boundary directly. A `from_array` constructor folds an existing array into a tree by repeated `add`, which costs $O(n \log n)$ overall but keeps the construction code trivial rather than implementing the linear-time build.

The program's distinguishing feature is its verification discipline: it does not merely assert hard-coded answers but cross-checks the structure against a brute-force reference. After loading `[1..8]` and confirming a handful of known sums with `die "..."` `unless ...` guards, it builds a two-hundred-element array of random values in `[-50, 49]` and runs fifty random range queries, comparing each fast `range_sum` against an explicit $O(n)$ accumulation loop:

```
for val $check (1:50) {
  val $lo = int(rand 200)
  val $hi = $lo + int(rand(200 - $lo))
  var $brute = 0
  for val $i ($lo:$hi) { $brute += $big[$i] }
  val $fast = Fenwick::range_sum($big_f, $lo, $hi)
  die "disagreement [$lo,$hi]: fast=$fast brute=$brute" unless $fast == $brute
}
```

It closes with universal-invariant checks over all `lo/hi` pairs of a small array, asserting the decomposition identity `range_sum(lo, hi) == prefix_sum(hi) - prefix_sum(lo-1)` directly. The file demonstrates the equation-form function syntax, the `+{ }` hash-literal disambiguator, bitwise operators on integers, `while` loops driven by bit arithmetic, and a property-based testing style built entirely from `die ... unless ....`

The Cooley-Tukey FFT with Hand-Rolled Complex Arithmetic

`fft_no_interop.stk` computes the one-dimensional discrete Fourier transform in $O(N \log N)$ time and, because stryke has no native complex type, builds complex numbers as two-element `[real, imag]` arrays with a small algebra of helper functions. Addition and subtraction are component-wise, and multiplication implements the cross-term formula directly:

```
fn c_mul($u, $v) {
  val $r = $u->[0] * $v->[0] - $u->[1] * $v->[1]
  val $i = $u->[0] * $v->[1] + $u->[1] * $v->[0]
  [ $r, $i ]
}

fn c_exp($theta) {
  [ cos($theta), sin($theta) ]
}
```

The `c_exp` helper returns a unit complex number on the unit circle via Euler's formula, using stryke's `cos` and `sin` builtins — these are the twiddle factors of the transform. The recursive `transform($x)` is the Cooley-Tukey decimation-in-time algorithm. Its base case returns the input unchanged when length is one or less, then it splits the signal into even- and odd-indexed halves using a C-style three-clause `for` loop with an explicit increment of two:

```

var @even; var @odd
for (var $i = 0; $i < $n; $i += 2) {
    push @even, $x->[$i]
    push @odd, $x->[$i + 1]
}

val $even_fft = FFT::transform(\@even)
val $odd_fft = FFT::transform(\@odd)

```

The presence of the C-style `for (init; cond; step)` form alongside the `for val $i (0:$n)` range form is a deliberate stryke feature — the index-stepping loop is the natural fit for a stride-two split. After recursing on each half, the butterfly combination loop computes the twiddle factor $\$theta = -2.0 * \$pi * \$k / \n , multiplies it into the odd half's $\$k$ -th coefficient, and writes the sum into the first half of the output and the difference into the second:

```

for (var $k = 0; $k < $half; $k++) {
    val $theta = -2.0 * $pi * $k / $n
    val $t = FFT::c_exp($theta)
    val $term = FFT::c_mul($t, $odd_fft->[$k])

    $res[$k] = FFT::c_add($even_fft->[$k], $term)
    $res[$k + $half] = FFT::c_sub($even_fft->[$k], $term)
}

```

The choice to write the output array first and then assign into it by index — `var @res` is pre-filled with `[0.0, 0.0]` placeholders for all n slots before the butterfly loop runs — is what lets a single loop over the first half populate both halves at once, `$res[$k]` and `$res[$k + $half]` being written in the same iteration. This is the in-place character of the radix-2 butterfly: each twiddle-multiplied odd coefficient is added to one output and subtracted from its mirror, so half as many iterations cover the whole transform. The inverse transform reuses the forward transform via the conjugate trick: it conjugates the input (negating each imaginary part), runs the forward transform, then conjugates again and divides by n . Because the inverse is expressed purely in terms of `transform`, the program needs only one transform kernel — a small but real demonstration of building the inverse operation out of the forward one rather than duplicating the recursion. The driver represents the polynomial `[1, 2, 3, 4]` as an array of complex literals `[1.0, 0.0]` and so on, prints the original, prints the forward transform, and prints the inverse — the inverse recovering the original coefficients, which is the program's correctness demonstration. Output formatting uses `#{sprintf '%.2f', $c->[0]}` interpolation to render each complex value to two decimal places. The file is a compact illustration of representing a missing type as an array, building an operator algebra over it, mixing recursion with C-style loops, and using the trigonometric builtins for the transform kernel.

Gaussian Elimination: Solving, Determinants, and Inversion

`gaussian_elimination_no_interop.stk` is the most extensive numerics program of the group, implementing partial-pivot row reduction to solve $Ax = b$, compute $\det A$, and invert square matrices — all in $O(n^3)$, all by hand. Matrices are arrayrefs of arrayrefs, and the program opens with a deep-copy helper because the elimination mutates its working matrix and must not corrupt the caller's input. The solver tracks the determinant as a running product of pivots, flipping its sign on each row swap, and selects the largest-magnitude pivot in each column for numerical stability:

```

for val $k (0:$n - 1) {
  var $pivot = $k
  var $max = abs($A->[$k]->[$k])
  for val $i ($k + 1:$n - 1) {
    val $v = abs($A->[$i]->[$k])
    if ($v > $max) {
      $max = $v
      $pivot = $i
    }
  }
  if ($max == 0) {
    return +{ ok => 0, x => [], det => 0 }
  }
}

```

A zero maximum pivot means the matrix is singular, so the function returns a record with `ok => 0` rather than dividing by zero — the program treats singularity as a first-class outcome. When a better pivot is found below the diagonal, the rows of both `$A` and the right-hand side vector `@bv` are swapped and the determinant's sign is negated. The forward-elimination inner loops then zero out the sub-diagonal entries, after which a back-substitution loop walks the rows in reverse to solve for each unknown:

```

var @x = (0) x $n
for val $i (0:$n - 1) {
  val $row = $n - 1 - $i
  var $s = $bv[$row]
  for val $j ($row + 1:$n - 1) {
    $s = $s - $A->[$row]->[$j] * $x[$j]
  }
  $x[$row] = $s / $A->[$row]->[$row]
}
+{ ok => 1, x => \@x, det => $det }

```

Inversion uses the augmented-matrix method that the header comment describes as reducing $[A \mid I]$ to $[I \mid A^{-1}]$. The `invert` function builds an $n \times 2n$ augmented matrix by appending an identity block to each row — `push @row, $i == $j ? 1.0 : 0.0` for the identity columns — then performs full Gauss-Jordan reduction: it normalises each pivot row by dividing through by the pivot, and eliminates the pivot column from every other row, skipping the pivot row itself with `next if $i == $k`. The inverse is read back out of the right-hand n columns. Supporting matrix-vector and matrix-matrix products round out the linear-algebra toolkit, the latter being the triple-nested loop with a zero-padded accumulator row `var @row = (0) x $m`.

What makes this file exemplary is the breadth of its verification. The driver defines four labelled cases as records — a 2×2, two 3×3s (one of which deliberately starts with a zero on the diagonal to force a pivot), and a 4×4 — each carrying its expected solution and determinant. The program then asserts seven independent invariants in sequence: that each solution matches its expected vector within `1e-6`, that each determinant matches, that the residual $A \cdot x$ reproduces `b`, that a deliberately singular matrix returns `ok == 0`, that $A \cdot A^{-1}$ equals the identity within tolerance for every case, that the inverse of $\begin{bmatrix} 7 & 0 \end{bmatrix}$ is $1/7$, and that the inverse of a 2×2 matches the closed-form adjugate formula. Float comparisons go through a small tolerance helper:

```

fn Gauss::approx($got, $want, $eps) {
  var $d = $got - $want
  $d = -$d if $d < 0
  $d < $eps
}

```

This is the right way to test floating-point numerics — never `==`, always within an epsilon — and the program's structure, where the algorithm and its seven-way property battery occupy a single self-checking file, models the discipline the language's example corpus consistently favours.

Common Threads

Across these six programs a consistent set of language idioms recurs. Two-dimensional data is always an arrayref of arrayrefs, allocated either by `[(0) x n]` row constructors or by pushing fresh rows in a loop, and indexed with the `->[ ]` arrow. Records — knapsack items, AST nodes, Fenwick trees, solver results, test cases — are hashrefs with string keys, constructed with `{ }` or the disambiguating `+{ }` and read with `->{ }`. Iteration overwhelmingly uses the inclusive range form `for val $i (0:$n - 1)`, with `rev` for backward passes and the C-style three-clause `for` reserved for non-unit strides as in the FFT split. The math surface stays small and explicit — `sqr`, `log`, `cos`, `sin`, `abs`, `int`, `rand`, `srand` — and complex numbers, when needed, are simply built out of two-element arrays with a hand-written operator algebra. Verification is pervasive: `die ... unless ...` guards, brute-force cross-checks, epsilon-based float comparison, and the `> aok / test_run` assertion pipeline turn each example into a self-contained regression test. The parallel primitives `pmap` and `pmap_reduce` appear where independent work is genuinely available — across knapsack instances and across gradient batches — with the documented closure-over-arrayref idiom making the worker boundary reliable. Taken together the chapter shows a language that keeps the algorithm in view: the loops, tables, recurrences, and bit-tricks are written out rather than abstracted away, which is exactly what makes these files useful as a reference for how numerical and combinatorial methods read in *stryke*.

Chapter 32 — Packages, FFI, and the Registry

Chapter 11 made the case that stryke needs no package manager to reach a function: the ten thousand standard-library primaries are barewords, compiled into the binary, present without an import. That claim is true and it is the right default, but it describes only the functions stryke ships. It does not describe *your* code once a project outgrows a single file, it does not describe a third-party library you did not write, and it does not describe the capabilities that cannot live in a portable binary because they need a C library underneath — talking to Postgres needs `libpq`, talking to Kafka needs `librdkafka`, reading Parquet needs the Arrow C++ stack. For those three cases — multi-file projects, external source libraries, and native extensions — stryke ships a full package manager, and it is the same `s` binary you already run. The design is specified in [docs/PACKAGE_REGISTRY.md](#); this chapter walks the parts that exist today and is explicit about the parts that are still ahead.

The reconciliation with the “no package manager” pitch is exact: the standard library is statically linked and needs no resolution step, so the common case of “is there a function for this?” never touches the package manager at all. The package manager is what you reach for when the answer is “not in the standard library” — and unlike Python or Node, where that moment is the *first* moment of every project, in stryke it is a comparatively rare one, because the standard library already covers the ground those ecosystems push into packages.

One binary, one mental model

There is no separate package-manager executable, no `cargo` to stryke’s `rustc`, no `npm` to its `node`. Every project task is a subcommand of `s`. The lifecycle commands are the ones you would expect: `s init` scaffolds the current directory as a project, `s new myapp` scaffolds `./myapp`, `s build` compiles, `s run` builds and runs the main binary, `s run myapp` runs a named binary, `s test` runs the test tree, `s bench` runs benchmarks, `s check` type-and-lint-checks without executing, `s fmt` formats, `s doc` generates documentation, and `s clean` clears local caches. The same binary that lexes, parses, compiles, JITs, runs the standard library, and hosts the fleet agent is also the build tool and the dependency manager. That is the same single-artifact commitment that governs the rest of the language, applied to tooling: one file to copy to a server, and that file is the compiler, the runtime, *and* the project tooling.

`s new myapp` lays down a conventional tree:

<code>myapp/stryke.toml</code>	manifest with <code>[package]</code> and <code>[bin]</code>
<code>myapp/main.stk</code>	entry point
<code>myapp/lib/</code>	library modules (reached by <code>use Foo::Bar</code>)
<code>myapp/t/</code>	test files (run with <code>s test</code>)
<code>myapp/benches/</code>	benchmark files (run with <code>s bench</code>)
<code>myapp/bin/</code>	additional executables (auto-discovered)
<code>myapp/examples/</code>	example programs
<code>myapp/.gitignore</code>	ignores <code>target/</code>

The `lib/` directory is the convention that matters: a package that has a `lib/` tree is a library, with no `[lib]` table needed to declare it. The directory is auto-discovered and published as-is, and consumers reach into it by namespace — `use Foo::Bar` resolves to `lib/Foo/Bar.stk`. There is no manifest bookkeeping that has to stay in sync with the file layout, because the file layout *is* the declaration.

The manifest: stryke.toml

The manifest is TOML, and the parser preserves table ordering on a round-trip. A representative one:

```
[package]
name = "myapp"
version = "0.1.0"
description = "My stryke application"
authors = ["user@example.com"]
license = "MIT"
repository = "https://github.com/user/myapp"
edition = "2026"           # language edition pin

[deps]
http = "1.0"               # semver range (registry - see roadmap)
local-lib = { path = "../my-local-lib" }
git-lib = { git = "https://github.com/user/lib", tag = "v1.0.0" }
stryke-parquet = { github = "MenkeTechnologies/stryke-parquet" }

[dev-deps]
test-utils = "1.0"        # present only for tests/benches

[groups.bench]            # arbitrary groups, beyond dev/prod
criterion = "0.5"

[features]
default = ["json"]
json = ["dep:json"]
full = ["json", "yaml"]

[scripts]                 # npm's one good idea, kept
test = "s test t/"
build = "s build --release"

[bin]
myapp = "main.stk"        # explicit entry point

[workspace]               # first-class from day one
members = ["crates/*"]

[workspace.deps]
http = "1.0"
```

Two manifest decisions are worth calling out as deliberate corrections of prior-art mistakes. Features are scoped per package, never unified across the workspace: a consumer turning on `yaml` for one dependency does not silently flip it on for every other package in the graph, which is cargo's most notorious footgun. And `[scripts]` is npm's one genuinely good idea — a task runner keyed to the project — kept without the rest of npm's model. The `[workspace]` table is present from the first release rather than bolted on later, with a `members = ["crates/*"]` glob and `{ workspace = true }` inheritance against `[workspace.deps]` so a multi-crate repo pins shared versions in one place under a single root lockfile.

Dependencies: path, git, and GitHub-release

A dependency takes one of several forms, and `s add` writes the right one for you from a single argument. `s add http@1.0.0` writes a version pin; `s add ../mylib`, `s add ../sibling`, `s add /abs/path`, and `s add ~/projects/mylib` all write path dependencies (the leading `./`, `../`, `/`, or `~/` is auto-detected, and `--path=DIR` overrides the positional guess); `s add github.com/OWNER/REPO` and `s add`

`github.com/OWNER/REPO@v1.2.3` write GitHub dependencies; `--dev` and `--group=NAME` route the entry to the dev or a named group instead of `[deps]`. `s remove` deletes it.

The three source forms behave differently on purpose. A **path dependency** loads straight from the filesystem, bypasses the store entirely, and reflects edits immediately on the next `s run` — this is how a workspace member or a library you are actively co-developing is consumed. A **git dependency** (`{ git = "...", tag|branch|rev = ... }`) clones into a `~/.stryke/git/` cache — shallow when a branch or tag is pinned, a full clone when a `rev` is — resolves to a concrete commit hash, and records `source = "git+<url>#<sha>"` in the lockfile, pinned as tightly as any registry dep. A **GitHub-release dependency** (`{ github = "OWNER/REPO", version = "..."`) is the one that distributes prebuilt binaries: `s install` downloads the release tarball built for the host triple, SHA-256-verifies it against a `.sha256` sidecar, and extracts into the store. The triple is auto-detected — `aarch64-apple-darwin`, `x86_64-apple-darwin`, `x86_64-unknown-linux-gnu`, `x86_64-pc-windows-msvc` — and overridable with `STRYKE_TARGET` for cross, musl, or exotic targets; without an explicit `version`, the resolver asks the GitHub releases API for the latest tag.

A property that holds across all of these: **install never runs package code**. Resolution parses manifests, builds the dependency graph, verifies integrity hashes, fetches missing tarballs, verifies again, extracts, and writes the lockfile — pure data movement plus hash verification. The first time any dependency's code executes is when *your* code imports it, not when `s install` finishes. That is the supply-chain property npm's install-time scripts famously throw away.

FFI: native extensions as cdylib packages

The GitHub-release path exists because of one specific kind of package: the native extension. A package that wants to call a C library — `libpq` for Postgres, `librdkafka` for Kafka, `libduckdb` for DuckDB, the Arrow stack for Parquet — cannot ship as portable `.stk` source, because reproducing the `cdylib` requires those platform libraries and the Rust toolchain at build time. So the canonical distribution is the prebuilt release artifact, and the manifest declares the binding with an `[ffi]` table:

```
[ffi]
lib-name = "stryke_parquet"
namespace = "Parquet"
exports = [
  "parquet__version",
  "parquet__inspect",
  "parquet__schema",
  # ...
]
```

The `lib-name` names the shipped `lib<name>.<ext>` inside the package, `namespace` is the stryke-side name the bindings hang under, and `exports` is the explicit allow-list of C symbols the runtime will register. That allow-list is load-bearing, not documentation: the loader registers **only** the symbols named in `exports`, so adding a new `pkg__*` entry point to the Rust side and forgetting to list it in `[ffi].exports` means the symbol is compiled, shipped, and invisible — the function simply will not exist at the stryke call site, and no surface test that only checks the binary will catch the omission. The manifest entry is the contract; the compiled symbol is not enough on its own.

The published catalog under `MenkeTechnologies/stryke-*` is built on this mechanism. The data and infrastructure integrations are FFI packages, each with its own namespace and symbol prefix — `stryke-parquet` exports `parquet__*` under `Parquet`, `stryke-redis` exports `redis__*` under `Redis`, `stryke-postgres` exports `pg__*` under `Postgres`, and the same pattern carries through `stryke-arrow`, `stryke-aws`, `stryke-azure`, `stryke-gcp`, `stryke-kafka`, `stryke-duckdb`, `stryke-mongo`, `stryke-mysql`, `stryke-k8s`, `stryke-docker`, `stryke-spark`, `stryke-grpc`, `stryke-polars`, `stryke-zmq`, `stryke-selenium`, and `stryke-office`. Not every package in the namespace is native, though: `stryke-utils` carries no `[ffi]` table at all — it is pure stryke, everything in `lib/*.stk`, resolved by the ordinary package loader with

no cdylib in sight. The split is exactly the one the design intends: pure-stryke packages ship as source and resolve like any library, while native packages ship as prebuilt cdylibs through GitHub releases. A git *source* clone that declares `[ffi]` is rejected fast, with a diagnostic pointing at the `{ github = "OWNER/REPO" }` rewrite, precisely because `s install` will not run `cargo build --release` inside a clone to manufacture a cdylib that the release already provides.

The store and the lockfile

Extracted packages live in a single global store at `~/.stryke/`, laid out as `store/` (extracted packages), `cache/` (downloaded tarballs), `git/` (git-dependency clones), `bin/` (global-tool launchers), and `index/`. Every `(name, version)` pair exists exactly once on disk per machine: two thousand projects depending on `json@2.1.0` share one extraction of `json@2.1.0`. There is no per-project `node_modules`, no symlink farm, and no hoisted phantom dependency you can import by accident — the lockfile is the index that maps a name and version to a store path, and that is the only path resolution consults.

`stryke.lock` is auto-generated, carries a format version and a resolution timestamp, and records each package with its source, an SHA-256 integrity hash, its enabled features, and its own resolved dependencies. It is sorted deterministically, which buys reproducibility with teeth: two `s installs` from the same lock file on different machines produce byte-identical store contents. The lockfile is the source of truth a project's `use` statements resolve against; you commit it, and a teammate's `s install` reconstructs your exact graph.

Module resolution: require and use

stryke offers two ways to pull in code, and they answer different questions. Path-based `require` `./lib/scanner.stk` resolves relative to the current file and is unaffected by the package manager — it works identically whether the file sits in your project, a workspace member, or an extracted store package, and it is the right tool for intra-project includes. Namespaced `use Foo::Bar` is the form for external dependencies, and it supports a Perl-style use-site version pin: `use Foo::Bar 2.13` demands a specific version at the import site.

Resolution walks a fixed order, and the order encodes a priority that is worth internalizing. A project-local `lib/Foo/Bar.stk` always wins first — live edits shadow any store version, so working on a library in place never fights the installed copy. A use-site pin (`use Foo::Bar 2.13`) loads that exact store version directly and errors hard if it is missing, never silently falling back. Inside a project, the name is looked up in `stryke.lock` and loaded from the locked version's store path. Outside a project — a standalone script with no `stryke.toml` ancestor — the resolver scans the store for every installed version (bridging the `stryke-foo@*` naming convention to the canonical `foo`), and picks the **highest** semver by numeric-tuple comparison, with release beating pre-release. Finally it falls back to `@INC` system paths. The governing rule throughout is strict version respect: whenever a pin cannot be satisfied by an existing extraction, the resolver returns a hard error rather than substituting a different version, because loading a different file under a pinned name is a correctness bug, not a convenience.

Build outputs: one native binary

`s build --release` is the capstone, and it is ahead-of-time native compilation in the literal sense. Your `main.stk`, every `lib/` module, every dependency from the lockfile, and the minimal stryke runtime all flow through Cranelift to native machine code and out to a single statically-linked executable — a real ELF, Mach-O, or PE binary that `file` reports as native and `objdump -d` disassembles as machine code. What is *not* in it is as important as what is: no interpreter, no JIT, no bytecode at runtime, no `.stk` source, and no external `.so/.dylib/.dll` unless you opted into dynamic linking. The default output paths are `target/release/<binary>` (optimized) and `target/debug/<binary>` (debug symbols, lighter optimization), and the target architecture is selectable for cross-compilation. The deployment story this produces is the

one the whole single-binary philosophy was aiming at: build once, copy one file, run anywhere with nothing else installed.

The single-script form of this has now landed and is worth distinguishing from the package-manager `--release build`. `stryke build SCRIPT` produces a standalone binary by embedding the script source as a compressed trailer in a copy of the interpreter — full language coverage, an interpreter carried along. `stryke build --native SCRIPT`, added in v0.17.33, is the real AOT path described in Chapter 20: it lowers the program to a fusevm chunk, compiles it through fusevm's Cranelift AOT to a relocatable object, and links it against the `libstryke.a` runtime staticlib into a native executable with no embedded interpreter. The catch is scope — `--native` covers only the self-contained subset (arithmetic, strings, scalars, arrays, hashes, `print/say/printf`) and rejects programs that use subs, closures, regex, the builtin surface, or block `map/grep/sort`, pointing those back at plain `stryke build`. That subset is exactly the frontier of the fusevm migration, and it widens with every phase of it.

What ships today, and what is ahead

Being honest about status is part of trusting the tool, and the design document tracks it precisely. Shipped today: the `stryke.toml` parser, the `~/.stryke` store and cache layout, `s install` for path dependencies, `s add` and `s remove` across every dependency form, `stryke.lock` generation with integrity hashes, lock-driven module resolution, git and GitHub-release dependencies (the latter being the FFI cdylib path the `stryke-*` catalog uses), workspaces with shared-dependency inheritance, global-tool installation (`s install -g`, `s uninstall -g`, `s list -g`, `s upgrade -g`, `s use -g`, `s gc -g`), and the operational commands `s update`, `s upgrade`, `s outdated`, `s vendor`, `s clean`, and `s run SCRIPT`. The CLI for `s publish` (dry-run), `s yank`, `s search`, and `s audit` exists as well.

Still ahead, and gated on the central registry being deployed: the registry protocol itself — meaning version-range dependencies like `http = "1.0"` that resolve against `registry.stryke.dev` are not yet live, while path, git, and GitHub dependencies are. The PubGrub semver resolver (the deterministic, backtracking algorithm uv and modern cargo use) and the parallel fetch/verify/extract pipeline are deferred until the registry exists to exercise them, as is registry-side feature activation and Sigstore signing of published artifacts. The shape is set and the local-only foundation is solid; the network tier is the next build-out. When the registry rule that anchors the ecosystem does land, it is the cargo one — published versions are immutable, you yank but never unpublish — so that a build that worked yesterday keeps working.

Chapter 33 – Objects, Classes, and Aspects

Chapter 8 introduced the record end of stryke's data model — `struct` for a named tuple of fields, `enum` for a closed set of variants, methods with an implicit `$self`. Those are the lightweight half. This chapter completes the object model with the heavyweight half: full classes with inheritance, roles, finality, operator overloading, and late static binding — and then a layer most object systems never grow, aspect-oriented advice that wraps existing functions from the outside without touching their source. The through-line is that stryke does not ask you to pick an object paradigm up front. You reach for the lightest construct the problem needs — a bare `struct` when you want fields and a couple of methods, a `class` when you want an inheritance hierarchy, a `role` when behavior is shared across unrelated types, and advice when the concern cuts across many functions at once.

From struct to class

A `struct` is the floor: fields, a positional or named constructor, methods with `$self`, and functional update via `->with`. A `class` is the same idea with a lineage. The `extends` keyword establishes a parent, methods are inherited and overridable, and the constructor is derived from the field declarations the same way a struct's is:

```
class Animal {
  name: Str
  fn speak { "... " }
  fn greeting { "Good day" }
}

class Dog extends Animal {
  fn speak { "Woof" }           # override
  fn bark { $self->name . " says woof" }
}

val $dog = Dog(name => "Rex")
p $dog->speak           # Woof           (overridden)
p $dog->greeting        # Good day       (inherited)
p $dog->bark            # Rex says woof
```

The field syntax — `name: Str, x: Int = 0` — carries an optional type annotation and an optional default, and the annotation is not decorative: a class field with a type participates in the same runtime type-checking that typed function parameters do, so constructing a `Dog` with a non-string `name` is an error at the constructor boundary rather than a mystery three methods later. `->with` is the functional-update primitive shared with structs: `$dog->with(name => "Fido")` returns a *new* instance with one field changed and the original untouched, which is the immutable-by-default posture of `val` extended to whole objects. There is no in-place setter idiom in the style; you produce a new value and rebind.

Finality, roles, and late static binding

Three modifiers refine the hierarchy. A `final class` cannot be extended and a `final fn` cannot be overridden — the enforcement is at definition time, so `class Bad extends Singleton` against a `final class`

`Singleton` is a compile error, not a runtime surprise. Finality is the counterweight to open inheritance: it lets a class author declare which parts of the contract are load-bearing and must not be altered by a subclass, the same guarantee Java's `final` and Kotlin's sealed hierarchies provide, expressed in the same one keyword.

Roles (spelled `trait`) carry behavior across types that do not share a parent. Where inheritance is a vertical “is-a” relationship, a role is a horizontal “can-do” one: a `Comparable` role or a `Serializable` role supplies methods that any class can mix in regardless of where it sits in the class tree. This is the composition-over-inheritance escape hatch — the answer to the diamond problem that single inheritance alone cannot express — and it keeps the hierarchy shallow because shared behavior does not force an artificial common ancestor.

Late static binding is the subtle one, and stryke spells it `static::`. When a method calls `static::identify()`, the call resolves against the *runtime* class of the object, not the class where the method text was written:

```
class Base {
  fn class_name { static::identify() }
  fn identify   { "Base" }
}
class Child extends Base {
  fn identify { "Child" }
}
Child()->class_name() # "Child" - resolved at runtime, not "Base"
```

Without late static binding, `class_name` written in `Base` would hard-bind `identify` to `Base::identify` and always answer `"Base"`, defeating the override. `static::` defers the lookup to the actual object's class, which is exactly what a template-method pattern needs: a parent that orchestrates and calls out to hooks the child fills in. It is the same mechanism PHP introduced under the same name and the same reason it earns its keep — a base class that must call the most-derived override of a method it names.

Operator overloading

A native class can define what the arithmetic and comparison operators mean on its instances by implementing specially-named methods. The set is fixed and complete: `op_add`, `op_sub`, `op_mul`, `op_div`, `op_mod`, `op_pow`, `op_concat` for the binary arithmetic and concatenation operators; `op_eq`, `op_ne`, `op_lt`, `op_gt`, `op_le`, `op_ge`, `op_spaceship`, `op_cmp` for comparison; and the unary `op_neg`, `op_bool`, `op_abs`, `op_numify`, plus `stringify` for string context.

```
class Vec2 {
  x: Int; y: Int
  fn op_add($other) { Vec2(x => $self->x + $other->x, y => $self->y + $other->y) }
  fn op_eq($other)  { $self->x == $other->x && $self->y == $other->y }
  fn stringify     { "(" . $self->x . ", " . $self->y . ")" }
}
val $v = Vec2(x => 1, y => 2) + Vec2(x => 3, y => 4)
p $v    # (4,6)
```

This is the same facility the sketch algebra of Chapter 16 and the set algebra of Chapter 8 use from the runtime side, now exposed to user classes: a `+` on two `Vec2` values dispatches to `op_add`, and because `op_add` returns a fresh `Vec2` the operator is functional in exactly the way the style prefers. `op_spaceship` deserves a special mention because defining it makes a class *sortable* — a class with a three-way comparison drops straight into `sort` with no comparator block, since the spaceship is the comparator `sort` reaches for. The pairing of `op_bool` and `stringify` is what lets an instance behave sanely in a conditional and in an interpolated string respectively, so a user class becomes indistinguishable from a builtin value at the two contexts where dynamic languages most often expose the seam.

Aspects: advice that wraps from the outside

Everything above modifies a type from within its own definition. Aspect-oriented advice modifies behavior from *outside*, and it is the feature that most distinguishes stryke's object story from a conventional one. The premise of AOP is that some concerns — logging, caching, timing, authorization — are not properties of any one function but cut across many, and scattering them into every function's body both duplicates the concern and tangles it with the function's real work. Advice lets you state the concern once and attach it to a *set* of functions named by a pattern.

There are three advice kinds. `before` runs ahead of the matched function and sees the call's name and arguments through `$INTERCEPT_NAME` and `@INTERCEPT_ARGS`. `after` runs following the call and sees the result and timing through `$INTERCEPT_RESULT`, `$INTERCEPT_MS`, and `$INTERCEPT_US`. `around` wraps the call entirely and must invoke `proceed()` to run the original:

```
before "fetch" { warn "calling fetch with @INTERCEPT_ARGS" }
after  "fetch" { warn "fetch returned $INTERCEPT_RESULT in ${INTERCEPT_MS}ms" }

around "expensive" {
  val $cached = cache_get($INTERCEPT_ARGS[0])
  return $cached if defined $cached
  val $r = proceed()
  cache_put($INTERCEPT_ARGS[0], $r)
  $r
}
```

The pointcut is a string with glob wildcards: `before "log_*` matches every function whose name starts with `log_`, and `before "*"` matches every user-defined call. `around` is AspectJ-style — the block's evaluated value *is* the call's return, so it can transform the result (`proceed() + 100`), forward it unchanged (`proceed()`), or replace it entirely by returning a value without ever calling `proceed`, which is precisely how the caching example above short-circuits a hit. Multiple `before` and `after` advices on the same name all fire in registration order; the *first* matching `around` wins and later `around` matches on that name are skipped, mirroring the intercept semantics of the zshrs shell this feature was adapted from.

Two invariants keep advice safe. A recursion guard means that calling an advised function from *inside* its own advice runs the original directly without re-firing the advice, so a `before "fetch"` that itself calls `fetch` does not spin into an infinite loop. And advice bodies are lowered to bytecode at compile time and dispatched through the same VM path that `map { }` and `grep { }` blocks use — the tree-walker is banned from the advice path, with a source-level test asserting its absence — which means name resolution inside an advice body behaves identically to name resolution anywhere else. The cost of that uniformity is one constraint borrowed from block lowering: an advice body must end in an expression, so a body that trails off in a bare `for/while/if` block or contains a literal `return` on its tail is rejected, and you rewrite it to end in a value. In v1, advice covers user-defined functions only; builtins like `print` and `pmap` are not interceptable.

Chapter 34 — Errors, Guards, and the Shape of Failure

The style guide’s section 15 is titled “no defensive code,” and read out of context it sounds reckless. It is the opposite. stryke’s error philosophy is that failure should be *loud and early* rather than *quiet and deferred*, and the way you achieve loud-and-early is by removing the defensive scaffolding — the null checks, the return-undef-and-hope, the caller-inspects-the-result dance — that a certain school of programming installs at every function boundary. That scaffolding does not prevent failures; it converts a failure that would have been a clean stack trace at the point of the mistake into a corrupted value that propagates silently until it detonates somewhere unrelated. This chapter is about the alternatives stryke gives you so that trusting the contracts is safe rather than naive.

die, eval, and \$@: the Perl core, kept

At the base is the inherited Perl mechanism, unchanged because it does not need changing. `die` raises; `eval { ... }` catches; `$@` holds the error, empty on success. The idiom is a guard at the top of a function and a trust everywhere below it:

```
fn Order::total($items) {
  die "empty order" unless @$items
  $items |> map { _->{price} * _->{qty} } |> sum
}

val $t = eval { Order::total(\@cart) }
warn "order failed: $@" if $@
```

The `die "..."` `if COND` guard is the workhorse. It is one line, it names the precondition, and it fails at the exact call where the precondition is violated — which is worth more than any amount of downstream `defined` checking, because the stack trace points at the cause instead of at a symptom. The scientific toolkit that ships with stryke leans on this hard: one single-file module carries 2,727 hand-crafted `die` invariants, one for every place a numerical routine has a precondition worth asserting. That is not defensive code in the pejorative sense; it is executable specification, each `die` a statement of what must be true for the result below it to mean anything.

`warn` is the non-fatal sibling: it writes to stderr, auto-appends a newline, and is catchable through `$SIG{__WARN__}` if you want to route diagnostics somewhere other than the terminal. The pairing — `die` for “this cannot proceed,” `warn` for “this is worth noting but survivable” — is the whole taxonomy, and the style is to reach for `die` at boundaries (user input, network responses, file contents) and to trust the builtins’ contracts everywhere inside.

try / catch / finally: structured handling

For code where the `eval/$@` pair reads too much like an idiom and not enough like intent, stryke ships a structured form: `try { } catch ($err) { } finally { }`. The `catch` block binds the raised value to a named variable, and `finally` runs on both the success and failure paths — the place for cleanup that must happen regardless:

```

try {
  val $conn = db_connect($url)
  $conn->query($sql)
}
catch ($err) {
  warn "query failed: $err"
  log_failure($err)
}
finally {
  metrics_flush()
}

```

The structured form and the `eval/$@` form are not two competing error systems; `try/catch` is sugar over the same raise-and-catch machinery, and the choice between them is a readability call. Reach for `eval { ... }` when you want the terse expression form and the error is handled inline (`val $x = eval { risky() } // $fallback`), and reach for `try/catch/finally` when the handling is substantial enough to want a named error, a distinct handler block, and a cleanup clause. The `finally` block is the one thing the `eval` form cannot express cleanly, so any code with a resource that must be released on both paths belongs in the structured form.

Timeouts, retries, and rate limits as control flow

Beyond raise-and-catch, `stryke` treats three operational failure modes — hanging, transient failure, and overload — as first-class control-flow constructs rather than library patterns you assemble by hand. `eval_timeout SECS { ... }` runs a block on a worker thread and applies a receive-timeout on the main thread, so a block that would hang forever is bounded and returns control after the deadline. This is the safe way to call anything whose latency you do not control — a network read, a subprocess, a regex on adversarial input — without threading a timeout parameter through every layer.

`retry` wraps a call in a backoff loop: `retry http_call times => 3, backoff => exponential` runs `http_call`, and on a raised error retries up to three times with exponentially growing pauses between attempts. It encodes the single most common answer to a transient failure — try again, but not immediately and not forever — as one line instead of the hand-rolled loop-counter-and-sleep it would otherwise be. `rate_limit(10, "1s") hit_api` caps how often a call may fire, ten times per second here, blocking the caller when the budget is spent; and `every "500ms" tick` runs a block on a fixed interval. Together these four — timeout, retry, rate-limit, interval — are the operational vocabulary of any program that talks to the outside world, and `stryke`'s stance is that they are common enough and error-prone enough to hand-write that they deserve to be constructs, on the same footing as `if` and `while`, rather than recipes each program reinvents.

Phase blocks: failure before main

One more piece of the failure story runs before your program's first ordinary statement. `stryke` honors Perl's compile-and-run phase blocks — `BEGIN`, `UNITCHECK`, `CHECK`, `INIT`, `END`, and the object-construction `BUILD` — which fire at defined points in the compile/run lifecycle. `BEGIN` runs as soon as it is parsed, before the rest of the file compiles, which is where a program validates that a required environment variable or a mandatory capability is present and `dies` at load time if it is not. `END` runs at shutdown regardless of how the program exits, the place for a flush or a final report. The value of these for error handling is that they move certain classes of failure to the earliest or latest possible moment: a missing prerequisite that would otherwise surface deep in a run instead aborts at `BEGIN`, before any partial work has been done, which is the loud-and-early principle applied to the program's own lifecycle.

Chapter 35 — The Interactive Loop: REPL, Debugger, and One-Liners

A scripting language is judged partly on how it behaves when you are not writing a program at all — when you are poking at data, checking a hypothesis, or reaching for it the way you reach for a calculator. Chapter 1 argued that stryke’s terseness is in service of the shell; this chapter is about the three interactive surfaces where that argument is cashed out: the read-eval-print loop, the one-liner harness that makes stryke a peer of AWK and `sed`, and the debugger for when a program misbehaves. All three share the same three-millisecond cold start, which is the property that makes them usable — an interactive tool you wait on is an interactive tool you stop using.

The REPL

Running `stryke` with no arguments drops you into a readline session, or you can ask for it explicitly with `stryke repl`, optionally pre-loading a library so its functions are in scope: `stryke repl --load lib.stk`. The session is a real editing environment, not a bare prompt. It carries line editing, persistent history in `~/.stryke/history` that survives across sessions, and tab completion that is context-aware in a way a naive completer is not: it completes keywords, the lexical variables currently in scope, defined function names, file paths, and — after a `->` on a blessed object — that object’s methods. The method completion is the tell that the REPL understands the live state of the session rather than matching against a static word list; it knows what `$dog` is and offers `speak` and `bark`.

The editing model is configurable. The REPL defaults to emacs keybindings — `Ctrl-A`, `Ctrl-E`, and the rest of the readline vocabulary — and switches to vi modal editing, with insert and normal modes and `h j k l` navigation, through a stanza in `~/.stryke/config.toml`:

```
[repl]
mode = "vi"    # or "emacs" (default)
```

For a single session without touching the config, `STRYKE_REPL_MODE=vi stryke` overrides it, which is the right knob for sanity-checking one mode against the other. One subtlety worth internalizing: when stdin is *not* a terminal — when you pipe a program into `stryke` — it is not read line-by-line as REPL input but slurped as a complete program and run, so `echo 'p 42' | stryke` executes rather than dropping into an interactive loop. The REPL is a TTY affordance; the pipe path is program execution.

The one-liner harness

The REPL is for exploration; the one-liner harness is for doing a job in a single shell invocation, and it is where stryke means to make AWK redundant. The `-e` flag is optional — if the first argument is not a file on disk and looks like code, stryke runs it — so `stryke 'p 42'` and `stryke -e 'p 42'` are the same, and you reach for the explicit `-e` mainly when combining it with the line-processing flags. Those flags are the Perl set, and they carry their Perl meaning exactly: `-n` wraps the program in a line-reading loop, `-p` does the same and auto-prints each record like `sed`, `-l` chomps the input record separator and sets the output one, `-a` auto-splits each line into `@F`, and `-F` sets the split pattern.

```

echo data | stryke -ne 'print uc $_'      # line loop, no auto-print
cat f.txt | stryke -pe 's/foo/bar/g'     # auto-print, sed-style
stryke -i -pe 's/foo/bar/g' file1 file2  # in-place edit, parallel across files
stryke -i.bak -pe 's/x/y/g' *.txt       # in-place with a .bak backup
echo a:b:c | stryke -aF: -ne 'print $F[1]' # auto-split on ':', print field 2

```

Two details lift this above being a Perl re-run. The in-place editor `-i` processes multiple files *in parallel* — the files are independent, so the work fans across cores for free, which a serial `sed -i` cannot do. And the whole apparatus sits on top of the standard library and parallel runtime of the preceding thirty-odd chapters, so a one-liner is not limited to text munging: `s -an 'freq(@F) |> dd'` computes a word-frequency table in twenty-two characters against the sixty-plus a portable AWK equivalent needs, because `freq` and `dd` are barewords and the field-split is already done. The `__DATA__` and `__END__` tokens close the loop for self-contained scripts: a line whose trimmed text is exactly `__DATA__` ends the program and exposes the trailing bytes through the `DATA` filehandle, so a script can carry its own test fixture or lookup table inline.

The debugger

When a program misbehaves rather than merely producing the wrong one-liner, stryke ships a TTY debugger in the shape Perl programmers already know: `stryke -d -- script.stk` drops into a `perl -d`-style REPL on stdin/stderr, where you step, inspect, and evaluate against the running program's state. For editor-driven debugging the same capability is exposed over the Debug Adapter Protocol, so an IDE can drive it through the standard wire: the DAP surface supports `setBreakpoints`, `setFunctionBreakpoints`, and `setExceptionBreakpoints`; the `stackTrace/scopes/variables` triad for inspecting frames, with `variables` recursive so every hash and array gets a drill-down handle; the stepping requests `continue`, `next`, `stepIn`, `stepOut`, `pause`; and `evaluate` for running an expression in the stopped frame. That the debugger speaks DAP rather than only its own console protocol is what lets the same engine back both the terminal `-d` session and a graphical debugger in an editor, one implementation serving two front-ends — the same single-source-of-truth discipline the glob qualifiers and the reflection hashes follow elsewhere.

Chapter 36 — Reflection: The Language That Describes Itself

Chapter 11 counted the standard library — ten thousand primaries, eleven thousand names — and noted in passing that `stryke` exposes its own metadata as queryable hashes. That was a footnote there; here it is the subject. The claim is unusual and worth stating flatly: `stryke`'s parser and dispatcher publish everything they know — every builtin, every keyword, every operator, every special variable, every alias, and a one-line description of each documented name — as ordinary Perl hashes you can look up, filter, and pipe over with the same operations you use on any other hash. There is no reflection *API* to learn, because reflection is not an API. It is data, in the data structures the language already gives you.

The hashes

There are eleven of them, living in the `%stryke::*` namespace, each with a short alias so a one-liner does not carry the full package path. The forward maps take a name to its category: `%stryke::builtins (%b)` maps each primary callable name to a category string like `"parallel"` or `"string"`; `%stryke::keywords (%k)` maps each language keyword to a category like `"control"`, `"decl"`, `"exception"`, or `"concurrency"`; `%stryke::operators (%o)` maps each symbolic operator to `"arith"`, `"compare"`, `"pipeline"`, and the rest; and `%stryke::special_vars (%v)` maps every special variable spelling — `$_`, `@ARGV`, `%ENV`, `$_X`, `__FILE__` — to its kind. Above them sits `%stryke::all (%all)`, the union of aliases, builtins, and keywords — every name `stryke` recognizes, which is what you count when you want the language's total surface. Two subsets slice the builtins by provenance: `%stryke::perl_comps (%pc)` is the Perl-5-core names, and `%stryke::extensions (%e)` is the `stryke`-only ones. `%stryke::aliases (%a)` resolves an alias to its canonical primary — `$a{tj}` is `"to_json"` — and `%stryke::descriptions (%d)` maps a name to its one-line LSP summary. Finally, two inverted indexes give constant-time reverse queries: `%stryke::categories (%c)` maps a category to the array of names in it, and `%stryke::primaries (%p)` maps a primary to the array of its aliases.

Every direct `$h{name}` lookup is $O(1)$, because these are real hashes, not a query layer pretending to be one:

```
stryke 'p $b{pmap}'           # "parallel"
stryke 'p $a{tj}'             # "to_json"      (alias → primary)
stryke 'p $all{tj}'           # "serialization" (alias resolved through %all)
stryke 'p $k{class}'          # "decl"
stryke 'p scalar keys %all'    # total callable spellings, one count
stryke '$c{parallel} |> e p'   # enumerate a whole category in O(1)
stryke '$p{to_json} |> e p'   # every alias of to_json
```

The two inverted indexes are what make the common “find names by property” queries cheap. Asking “what are all the parallel builtins?” is `$c{parallel}` — an $O(1)$ array fetch — rather than an $O(n)$ scan filtering `%b` by value. Asking “what does `to_json` also go by?” is `$p{to_json}`. Only the genuinely open-ended queries — “every documented op whose description mentions ‘parallel’” — pay the $O(n)$ filter, because there is no index for arbitrary substring search over descriptions, and that is the right trade: index the two frequent shapes, scan for the rare one.

Derived, never hand-maintained

The property that makes this trustworthy is that none of it is written by hand. The hashes are derived at compile time by `build.rs` from the actual source of truth: the category tags come from the section-commented groups in the dispatcher's own `is_perl5_core` and `stryke_extension_name` functions, the aliases come from the `try_builtin` match-arm names, and the descriptions come from the LSP's `doc_for_label_text`. There is no separate list to keep in sync and therefore no way for the reflection data to drift from the language — if a builtin is added to the dispatcher, it appears in `%b` automatically, categorized by the section comment it was filed under, and if it lacks a section comment it shows up categorized as `"uncategorized"`, which is not an error but a flag that a section header is missing. The counts the reference cites — 10,453 primaries, 11,229 total names — are `scalar keys %b` and `scalar keys %all` run against a specific build, not numbers a human typed into prose, which is why this book can quote them without hedging.

There are a few honest edges. `%descriptions` is deliberately sparse — not every builtin has an LSP summary yet — which turns `exists $d{$name}` into a useful predicate in its own right: “is this op documented?” And the short aliases (`%a`, `%b`, and the rest) are value copies of the long `%stryke::*` names, currently read-only in practice, so the copy never diverges from the original; the hash-sigil namespace being separate from scalars and subs is what lets `%a` and `%b` coexist with the `$a/$b` sort specials without collision.

Reflection as a workflow

The point of all this is not introspection for its own sake; it is that “is there a builtin for X?” becomes a query you run rather than a doc you search. A frequency table of the whole library — how many ops per category — is one line: `val %f; $f{$b{$_}}++ for keys %b; dd \%f`. Cataloguing the entire reflection surface, hash by hash with its size, is a short loop over the alias letters. And because the data is just hashes flowing through the same `|>` and `grep` and `sort` the rest of the language uses, discovery composes with everything else — `keys %d |> grep { $d{$_} =~ /parallel/i } |> sort |> p` finds every documented op that mentions parallelism, mixing reflection, regex, and a pipeline in a single expression. The same data backs the IDE's reflection tool window, a searchable tree of roughly twenty-five thousand entries across all the hashes, where a left-click renders the docs popup and a right-click offers copy-name — the graphical face of the exact same `%stryke::*` tables a one-liner queries, which is why the two can never show different answers.

Chapter 37 — The Fleet: Controller, Agents, and the Congregation

Chapter 5 fanned work across the cores of one machine. This chapter fans it across many. `stryke` ships a complete distributed-compute layer in the same single binary — no separate daemon to install, no cluster manager to stand up, no message broker in the middle. The same `s` you run a one-liner with can become a controller coordinating a fleet, or an agent taking orders, and a script can drop itself into either role. There are two faces to this: an interactive controller REPL built for human-driven load testing, and a scriptable twenty-eight-verb API built for programmatic scatter-gather. Both ride the same TCP-and-bincode transport underneath.

The controller REPL

`stryke controller` binds a listener — `0.0.0.0:9999` by default, tunable with `--port` and `--bind` — and agents started with `stryke agent` connect to it and register, reporting their cores, memory, and state. From the controller's prompt you drive the fleet. `status` lists the connected agents; `fire [SECS]` starts a stress test across all of them; `terminate` stops it; `shutdown` disconnects everyone and exits. The command that turns this from a load-tester into a distributed REPL is `eval`, and its shorthand `@`:

```
> @sub greet { "hello from " . $main::ENV{HOSTNAME} } greet()
[node-01/ok] hello from node-01
[node-02/ok] hello from node-02
> @$main::counter = 0
[node-01/ok] 0
[node-02/ok] 0
> @$main::counter += 5; $main::counter
[node-01/ok] 5
[node-02/ok] 5
> @die "boom"
[node-01/ERR] boom at -e line 1
[node-02/ERR] boom at -e line 1
```

Any REPL line beginning with `@` ships the rest as `stryke` source to every agent, which runs it against its own persistent VM. The persistence is the interesting part: because each agent holds a long-lived VM helper, `sub` definitions and `$main::` globals carry across `eval` calls — the fleet behaves like a set of coordinated `perl -de0` sessions, where the state you build in one command is there for the next. The controller fans the work out in two passes, writing every agent's EVAL frame before waiting on any reply and then collecting the results in sorted name order, so the wall-clock cost is the *slowest* agent's latency, not the sum — the fan-out is genuinely parallel, not a serial walk over the roster. Each result line is tagged with its agent and an ok/ERR marker, so a `die` on one node shows up as that node's error without derailing the others. The wire protocol carries a version number, bumped when the EVAL frame kinds were added, so a mismatched agent refuses the handshake rather than hanging on a frame it does not understand.

The two roles are also builtins, not only CLI subcommands: `controller(bind?, port?)` and `agent(addr?, name?)` let any `.stk` script drop into either mode and return the exit code as an integer. Wrapping one in `spawn { ... }` runs it in the background while the calling script continues, which is how a single process can bring up a controller and a local agent together for an integration test — no second machine required:

```
spawn {
  sleep 1 # let the controller bind
  agent("localhost:9999", "local-worker")
}
exit controller("127.0.0.1", 9999) # blocks on the REPL loop
```

The congregation: scriptable scatter-gather

The controller REPL is for a human at a prompt. When a script needs distributed compute — scatter work to N workers, gather the results, manage worker lifecycle — it uses the twenty-eight-verb API layered on the same controller infrastructure, themed, memorably, around a congregation. The minimum-viable shape is four calls:

```
val @workers = congregation(4) # fork 4 local agents, wait for them to
  ~ register
val $div      = pray "compute()", @workers # scatter EVAL frames in parallel, return a
  ~ "divination"
val %results = annex $div # block-and-gather, hash keyed by session id
excommunicate(@workers) # clean shutdown
```

`congregation($n)` forks N agent children on the local machine, auto-registers them, and hands back an array of handles. `pray` is the scatter verb: it ships a unit of work — a string of source, or a coderef whose body is deparsed and sent (closure captures are unsupported in v1, so the body must be self-contained) — to every worker in parallel and returns a handle to the pending results. `annex` is the gather: it blocks until the results are in and returns them as a hash keyed by session id. `excommunicate` shuts a subset down and drops them from the roster. The naming is a joke that also happens to be a good mnemonic — you assemble a congregation, you pray work out to it, you annex what comes back, and you excommunicate the workers when the service is over.

Around that core sits the rest of the taxonomy: `anoint` builds a congregation without taking over as the current controller, for scripts juggling several; `ordain` stands up a bare controller with no agents and registers a name in a shared “cathedral” registry so remote workers can find it by name via `profess`; `muster` enumerates the connected handles; `welcome($n, $timeout_ms)` blocks until N agents have joined; `bow` is a slave entering the receive loop; and `cathedral` and `apostatize` list and unregister named congregations. The whole apparatus is deliberately a superset of the controller REPL rather than a separate system — both are the same TCP-and-bincode transport, one exposed to a human typing @-lines and the other exposed to a script calling `pray`, so a mental model built on one transfers directly to the other.

The design conviction underneath this chapter is the same one that governs the standard library and the parallel primitives: a capability this useful should not require assembling a stack. Distributed compute in most languages means choosing a framework, running a coordinator, configuring a transport, and serializing your work by hand. `stryke` folds all of that into builtins in the one binary you already have, so scaling from one machine’s cores to a fleet’s is a change of verb — `pmap` to `pray` — not a change of platform.

Chapter 38 – Async, Generators, and Time

The final chapter of the operational movement gathers three facilities that share a theme — they all concern *when* a computation happens rather than *what* it computes. Asynchrony is about overlapping work that waits; generators are about producing values on demand instead of all at once; and the time library is about the one dimension every long-running or scheduled program has to reason over. Each has appeared in passing in earlier chapters; here they are treated as the coherent set they form.

Structured concurrency: `async`, `spawn`, `await`

Chapter 5's parallelism was data-parallel — the same operation over many elements, fanned across cores by `pmap` and its kin. This is the other axis: task-parallel concurrency, where a handful of *distinct* operations overlap because each spends time waiting. `async { ... }` and `spawn { ... }` both start a block running concurrently and return a handle; `await` blocks until a handle's value is ready:

```
val $data = async { "https://example.com/" |> fetch }
val $file = spawn { "big.csv" |> \&slurp }
print await($data), await($file)
```

The two blocks above run at the same time — the HTTP fetch and the file slurp overlap rather than queueing — and `await` collects each result when it is needed. This is structured concurrency in the precise sense: each concurrent block runs in its own interpreter context with the enclosing lexical scope *captured*, so there are no data races on the captured variables, and the lifetime of the concurrent work is bounded by the `await` that collects it rather than left dangling. It is the same no-shared-mutable-state discipline the parallel primitives enforce, applied to task concurrency, and it is why the shared-state primitives — `varsync` and `oursync` from Chapter 5 — exist for the cases where blocks genuinely must communicate: without them, captured scope is read-only across the boundary by construction.

Generators: values on demand

A generator is a function that produces a sequence lazily, yielding one value at a time and suspending between them, so the consumer pulls values as it needs them rather than receiving the whole sequence at once. `stryke` spells it `gen { ... yield ... }`, and the handle's `->next` returns a `[value, more]` pair — the value produced and a flag for whether the generator has more to give:

```
val $g = gen { yield $_ for 1:5 }
val $step = $g->next           # [1, true] - one value, more to come
```

The generator sits alongside the two other laziness mechanisms the book has already described, and the distinction between them is worth drawing because they solve adjacent problems. The `range()` builtin from Chapter 7 is a lazy *range* — a bidirectional iterator over a numeric span, `range(1, 5)` counting up and `range(5, 1)` counting down, yielding on demand so a billion-element span costs constant memory. The streaming iteration variants from Chapter 13 — `maps`, `greps`, `flat_maps` — are lazy *transformations*, chaining without materializing intermediate vectors. The generator is the general case of which those are specializations: an arbitrary computation that yields, so its sequence can be anything a function can compute step by step, not

only a range or a transform of an existing collection. Reach for `range` for spans, the streaming variants for pipeline stages, and `gen/yield` when the sequence has no closed form and must be produced by running code.

Timing: `timer` and `bench`

Two builtins measure elapsed time, and the split between them mirrors the eager/lazy and single/aggregate splits that recur throughout the language. `timer heavy_work` runs the block once and returns the wall-clock milliseconds it took — the answer to “how long did this take?” `bench heavy_work 1000` runs it many times and returns a distribution as a “min/mean/p99” string — the answer to “how long does this take?”, which is a different and more honest question because a single timing is noise and a percentile summary is signal. The `p99` in particular is the number that matters for anything user-facing: the mean hides the tail, and the tail is where the pain lives. That `bench` is a language builtin rather than a library import is of a piece with the rest of stryke’s stance — measurement is common enough to deserve a bareword, and having it always present means the reflex “is this actually faster?” is one word away rather than a dependency away.

Time itself: the `chrono` surface

For calendar time — timestamps, zones, formatting, arithmetic on dates — stryke exposes the Rust `chrono` and `chrono-tz` libraries as builtins. `datetime_utc` and `datetime_now_tz` produce the current instant; `datetime_from_epoch` and `datetime_parse_rfc3339` and `datetime_parse_local` parse; `datetime_strftime` and `datetime_format_tz` format; `datetime_add_seconds` does arithmetic; and the ergonomic `time_ago` and `elapsed` produce human-facing durations. Short aliases follow the same density principle as everything else — `utc`, `now`, `dte`, `dtf` — so a timestamp in a one-liner does not cost a long name.

The detail that ties this chapter back to the language’s core is that time is one of the types the colon range operator understands. Chapter 7 described `:` as polymorphic — “ints, chars, dates, IPs, romans” — and dates are a first-class member of that list, so a span of days is `$start:$end` in the same operator that gives you `1:10` and `'a':'z'`. This is not a coincidence of overloading; it is the same conviction expressed once more. A range is a range whether its endpoints are integers, characters, addresses, or dates, and a language that has decided ranges deserve a one-character operator has no reason to make the date case special. The polymorphism is the point: the operators, the reflection hashes, the glob qualifiers, the sketch algebra, and now the calendar all reflect a single design temperament — find the one general shape, give it the terse spelling, and let every specific case fall out of the general one.

Appendix A – The Style Guide in Full

Every other appendix describes what stryke *can* do; this one describes how it is *meant* to be written. The source is [docs/STYLE_GUIDE.md](#), and the principle that generates all of it is stated in its opening line: **min chars win**. stryke is the power-user language, so every design call defaults to the terser form — no newbie concessions, no optional verbosity. The guide is organized as DO/DON'T pairs, where the DON'T column is *valid* stryke (so old Perl ports keep parsing) but wrong on style for new code. The `--no-interop` flag rejects most of the DON'T column at parse time as a forcing function; treat its parser rejections as free QA, not as bugs.

What follows reproduces the ten hard rules verbatim, then walks the numbered sections that flesh them out.

A.1 – The ten hard rules

These are the load-bearing rules. The guide states it plainly: “Every other idiom is downstream of these. If a piece of stryke code violates any of them, it is wrong style, full stop.”

Rule 1. “`fn` not `sub`. `sub` is Perl 5 baggage and is rejected by `--no-interop`.”

`fn` is the keyword that opens stryke’s topic-variable protocol — `_`, `_N`, and the implicit-return machinery only exist inside an `fn`. `sub` parses for CPAN compatibility but does not unlock any of it. There is no situation in fresh stryke code where `sub` is correct.

Rule 2. “`p` not `print`, never `say`. `p` is the stryke verb, has the record separator built in, costs one character. `say` is rejected by `--no-interop`.”

`p` is print-with-newline in a single character. `print` requires you to manage `\n` yourself, and `say` — Perl’s own one-character-saving answer — is explicitly banned because stryke already solved the problem. One verb, no ceremony.

Rule 3. “Use `|>` / `~>` over nested calls. `f(g(h(x)))` reads inside-out; `x |> h |> g |> f` reads left-to-right and shows up at the call site without diving the cursor through three closing parens. **Always pipeline.**”

This is the single biggest readability lever in the language. Nested calls force the reader to find the innermost paren and unwind outward; a pipeline reads in execution order. The guide’s mechanical test (section 6b): if you typed two open parens in a row, `f(g(`, refactor to a pipeline.

Rule 4. “No semicolons at end of line. stryke statement-terminates on newline. Trailing `;` is noise that visually weights every line. The only place `;` belongs is **between statements on the same line** (rare, e.g. C-style `for` headers): `for ($i = 0; $i < 10; $i++)`.”

A semicolon per line is a character per line plus equal visual weight on every statement for no payoff. The sole survivor is the same-line statement separator, almost always a C-style `for` header. Note `--no-interop` does *not* flag trailing `;` (it is legal Perl 5), so this one is enforced by review, not the parser.

Rule 5. “Implicit closure parameters always. Never write `sub { my $x = shift; ... }` or `fn { my ($a, $b) = @_; ... }`. Use `_` for slot 0 and `_N` for slot N. Named params are only correct when the closure is a real top-level `fn` whose arity > 2 *and* the names carry meaning.”

The implicit-param family (`_`, `_0`, `_1`, `<`) is stryke’s calling card. Pulling args out by hand with `shift` or `my ($a, $b) = @_` re-introduces the Perl ceremony the topic system exists to delete. Named params survive only for genuinely-documenting arity-3+ top-level functions. The payoff compounds inside pipelines: every

`map/grep/sort` block and every thread-macro stage opens a fresh topic frame, so a five-stage chain that would carry five `my $x = shift` lines carries zero.

Rule 6. “Expression-bodied `fn` for one-liners. `fn double = _ * 2` over `fn double($a) { $a * 10 }` over `fn double { _ * 2 }`. No braces, no named param, implicit topic. The block form is for multi-statement bodies only.”

Three tiers of decreasing preference appear in the rule itself: expression-bodied with implicit topic, then named-param block, then brace-block with implicit topic. Anything that fits on a line should be `fn name = expr`. Braces signal “this body has more than one statement.”

Rule 7. “Implicit return — never `return` on the tail expression. The last expression in any `fn` body, arrow-block, or expression-position block IS the return value. `return` is **only** correct as an early-return guard (`return undef if $err`). Writing `return $x` at the bottom of a body is noise.”

The tail expression auto-returns — in function bodies, arrow-blocks, `if/else` arms, and `match` arms alike. `return` is reserved for guard clauses that exit early. A `return` on the last line both wastes characters and visually buries the value the function actually produces.

Rule 8. “No parens on `fn` calls unless precedence demands them. `stryke` calls take args list-operator-style. `p "hello", len @a, uc rev "abc"` — paren-less. Add parens **only** when the call is followed by an operator that would re-bind (`len(@a) > 1`), when nesting needs disambiguation, or when an empty arg list is the intent (`time()`).”

`stryke` is a list-operator language: args bind to the right of the function name without parens. The three legitimate paren cases are precedence (`len(@a) > 1`, where bare `len @a > 1` mis-parses as `len(@a > 1)`), disambiguation between adjacent list-ops, and explicit zero-arg intent (`time()`).

Rule 9. “No trailing `1` (or any sentinel truthy value) at the end of a library / module file. That’s Perl 5’s `package returns true` ritual... `stryke use/import` resolution does not consult the file’s tail expression, so the `1` is pure noise inherited from CPAN muscle memory. Library files end at the last `fn` (or `class` / `const`) definition. `--no-interop` rejects a bare-integer tail expression in module position.”

CPAN’s `1;` existed because `require` demanded a truthy last expression. `stryke`’s module resolution does not look at the tail, so the `1;` is dead weight carried over by reflex. Files end at their last definition.

Rule 10. “`val` for immutable, `var` for mutable — never bare `my` in new code. `stryke` flips Perl 5’s mutable-by-default `my` into Scala/Kotlin-style immutable-by-default: `val` marks the binding *frozen* at compile time (reassignment, push/pop, `$x[i] = $h{k}` =, `delete $h{k}` all error at compile time, not run time); `var` is the mutable counterpart. Both desugar to `my` in the AST — same scope, same bytecode, same slot allocation — so the cost is zero. Default to `val`; reach for `var` only when you actually mutate.”

This is the largest behavioral departure from Perl in the guide. `val` is a *compile-time* freeze: reassignment, `push/pop`, element assignment, and `delete` all error before the program runs. Because both desugar to `my`, the safety is free — same slot, same bytecode. `val` also takes a type annotation (`val $port : Int = 8080`), and both work in loop headers (`for val $x (LIST)`). Default to `val`; `var` is for the bindings you actually mutate. Bare `my` stays legal for CPAN ports but is rejected by `--no-interop`.

The guide bundles all ten into one canonical example. The right version:

```
fn shout = uc(_) . "!"
@lines |> maps { trim(_) |> shout } |> greps { len(_) > 0 } |> ep
```

The wrong version of the same logic — `sub`, `print`, nested `length(...)`, trailing `;`, `my $s = shift`, block body, explicit `return` — runs roughly 7× the characters and reads bottom-up because of the distance between the `for` header and the `print` it controls. Every one of the seven violations maps to a rule above.

A.2 – The numbered sections, in brief

The hard rules are the spine; the numbered sections supply the detail. A one-or-two-sentence summary of each follows.

Paren-less calls (0a). Function args bind to the right without parens — `p "hello", len @a, uc rev "abc"`. The three exceptions (precedence, disambiguation, explicit zero-arg) are the same three from rule 8; inside a pipeline, parens are almost never needed because `|>/~>` already supply the staging.

Output (1). `p` for stdout-with-newline, `warn` for stderr (auto-appends `\n`, catchable via `$SIG{__WARN__}`), `@xs |> ep` to print each element, `ddump $struct` for pretty-printing. Note `ep` is “each-print” and goes to `stdout`, not `stderr`.

String interpolation (1a). Always interpolate; never build strings with `.` concat when interpolation reaches. Four forms: `$name` (bare scalar), `${name}` (disambiguated scalar), `#{ EXPR }` (Ruby-style arbitrary expression — math, function calls, anything), and `@arr` (array, joined by `$`). `.` concat is correct only when joining two non-literal variables.

Topic variables (2). Four spellings for slot 0; pick the shortest that reads. Bare `_` over `$_`; `_1/_2` over the sort-only `$a/$b`; `_<` for the outer topic; `_[N]` for char-of-topic. `$_[N]` is the legacy Perl `@_` positional and is compat-only.

Function definition (3). Expression-bodied `fn name = expr` is the default; the block form is for multi-statement bodies, genuinely-documenting names, or internal control flow. Implicit return on the tail; `return` for early guards only. Never shadow a builtin — namespace under `Module::Sub::name`.

Ranges (4). `:` is the polymorphic, *eager* range operator — `1:10`, `'a':'z'`, `1:100:5` (third colon is the step). It works on ints, chars, dates, IPs, and romans. The `range()` builtin is the *lazy* counterpart — same sequence, yielded on demand — so the two are valid range forms with different semantics, not a right/wrong pair. `..` is the Perl spelling and is compat-only.

Length / count (5). `len` is the *one* verb for size — strings, arrays, hashes, sets, iterators. `length`, `scalar`, and `0 +` coercion are all Perl-isms rejected by `--no-interop`. Aliases `cnt/count` read better in some contexts (`pgrep { ... } |> cnt`).

Slicing (6a). `[N:M]` bracket-slicing replaces `substr`, for both strings and arrays. The semantic difference from Perl: stryke’s `[N:M]` is **inclusive on both ends** (so `$s[1:3]` is 3 chars at indices 1,2,3), and it clamps gracefully past the end. Open-ended forms (`$s[2:]`, `@a[:2]`, `$s[-3:]`) work directly.

Pipeline / threading (7). The operator family — covered in depth in Appendices B and C — with section 7.6’s selection table: `|>` for 1–2 stages, `~>` for 3+ no-I/O stages, `->>/~>>` for last-arg-is-data, `~s>` for uneven-latency I/O, `~p>` for CPU-heavy chunk-parallel work.

Iteration shorthand (8). `@xs |> e p /@xs |> ep` over `for` loops; streaming variants (`maps`, `greps`, `flat_maps`, `pmaps`, `pgreps`, `pflat_maps`) are the default when chaining, because they avoid materializing intermediate vecs. The array-returning forms (`pmap`, `pgrep`) are correct only when you genuinely need a vec.

File I/O (9). `c"path"` slurps as a quote-prefix; `spurt $path, $data` writes; `lines("path")` is a lazy line iterator; `glob("*.stk")` for globbing (`<>` is reserved for stdin).

Parallelism (10). `pmaps/pgreps` ride rayon work-stealing; `varsync/oursync` are the shared-state primitives (lockless reads, lock-on-write); `pchannel` and `barrier($n)` replace hand-rolled mpvc and latches.

Sort / reduce blocks (11). `$a/$b` are valid *only* inside sort/reduce comparator blocks. Everywhere else use the universal positional `_0/_1`. `--no-interop` rejects `$a/$b` outside comparators.

Implicit closure params (12). Three syntactic homes: plain `{ ... }` for `map/grep/sort/reduce` block-args and `|>` pipeline stages; `>{ ... }` arrow-blocks **only** inside `~>/~>>/~s>/~p>` thread macros (a parse error elsewhere — “Not a code reference”); and `fn { ... } / fn name = ...` for stored or named function values. The outer chain `_<` reaches the enclosing closure’s topic, one `<` per level.

Naming (18). `snake_case` for functions and variables, `Module::Sub::name` for namespaced functions, `SCREAMING_SNAKE` for true constants, `CamelCase` for classes/structs/enums, single letters only as block topic or loop index. Never name a function after a builtin; check with `s 'p $stryke::all{your_name} // "free"'`.

Multi-line expressions (13). `stryke` terminates statements at newline, Perl/shell-like, not at `;`. A pipeline that begins its next line with `|>` or `~>` auto-continues the previous statement; any other multi-line operator needs an explicit `\` continuation or it silently produces the wrong value. This is the inverse of rule 4 — the absence of `;` is what makes the continuation rule load-bearing.

Hash declarations (14). For hashes with more than three keys, one `key => value` pair per line, so block-editing (visual select, delete, reorder) stays cheap. Short hashes (≤ 3 keys) can stay on one line.

No defensive code (15). Trust the builtins' contracts — `val $sum = sum(@xs)` over `@xs ? sum(@xs) : 0; die "bad input" if $n < 0` over the check/return-undef/caller-checks dance. Validate at boundaries (user input, network) and nowhere else.

Test runner (16). `s t t/` and the `test()` builtin default to a pre-forked worker pool that forks-on-receive per test — roughly 5–7× faster than a shell `for` loop over `stryke $f`. Use `s t -j 1 t/foo.stk` to serialize when pinpointing a hang.

When to break the rules (19). Exactly three: porting CPAN code where line-by-line parity matters (mark with `# perl-port`), matching existing file convention mid-PR (queue the migration separately), and benchmark-verified hot paths (e.g. swapping `_<<` resolution for explicit lexical capture — profile first). Anywhere else, `stryke` idioms win.

Appendix B – Operator and Sigil Reference

stryke inherits Perl's sigil system and then layers a family of threading and slicing operators on top. The sigils carry their Perl meaning so CPAN ports keep parsing; everything built on top — the thread/pipe family, colon ranges, bracket-slicing, the topic operators — is where stryke departs. This appendix is the catalogue, with a when-to-reach-for-it note and a one-line example per entry.

B.1 – Sigils

The three sigils are Perl's, unchanged in meaning. They name the *kind* of container, and stryke's slicing and reference operators build on them.

Sigil	Names	Example
\$	scalar — a single value (number, string, ref, undef)	<code>val \$x = 1, \$s[1:3]</code>
@	array — an ordered list	<code>val @a = (10, 20, 30), @a[2:]</code>
%	hash — key/value pairs	<code>my %config = (host => "localhost", port => 8080)</code>

Reach for `$` whenever the binding holds exactly one thing — including a reference to an aggregate, since a ref *is* a single scalar. `@` is the container you iterate, slice, and pipe; `%` is the keyed store. The sigil is what the slicing and reference operators dispatch on: `$s[1:3]` slices a string, `@a[2:]` slices an array, and the leading-backslash forms below take a reference to whichever container the sigil names.

References take a leading backslash: `\$x`, `\@a`, `\%h`, `\&sub` (README §1122). Reach for a reference when you need to pass an aggregate without flattening it into the call's argument list — `getopts(\@args, ...)` hands `getopts` the array itself to mutate in place rather than a copy of its elements. `+{...}` forces a hash-reference constructor where a bare `{...}` would be ambiguous — used throughout the example corpus for anonymous hashrefs, e.g. `push @open, +{ ... }` in `a_star_no_interop.stk:31` and the config records in `adam_optimizer_no_interop.stk:107`. The `+` disambiguates the brace from a code block; reach for it any time you build an anonymous hashref in a position (after `push`, `return`, `=>`) where the parser would otherwise read `{` as opening a block.

B.2 – The thread / pipe family

This is stryke's signature operator group. Each member has its own execution semantics; they are not interchangeable. Appendix C drills into the threading subset; this is the at-a-glance catalogue.

Operator	Name	What it does	Real example
<code>\ ></code>	pipe-forward	Single stage; LHS becomes the first arg of the RHS callable. Zero-cost desugar.	<code>"hello" \ > uc \ > rev \ > p → OLLEH</code> (STYLE_GUIDE §7.1)
<code>~></code>	thread-first	Multi-stage macro; threaded value injected as first arg of each space-separated stage. Spellings: <code>~></code> , <code>thread</code> , <code>t</code> .	<code>~> " hello " tm uc rv \ > p → OLLEH</code> (STYLE_GUIDE §7.2)
<code>~>> / ->></code>	thread-last	Same syntax as <code>~></code> , value goes in as the last arg — right for (<code>block</code> , <code>list</code>)-shaped callees.	<code>~>> (1,2,3,4,5) take(3) p → (1,2,3)</code> (STYLE_GUIDE §7.3)
<code>~s></code>	streaming pipeline	Each stage runs as an independent worker thread over a bounded channel; items flow concurrently, nothing materialized. Output order non-deterministic; value is the count emitted.	<code>~s> [1,2,3,4,5] map { _ * 10 } map { _ + 1 }</code> (STYLE_GUIDE §7.4)
<code>~p></code>	chunk-parallel	Splits input into chunks, runs the whole pipeline per chunk via rayon, auto-merges.	<code>~p> @big map { _ * 2 } sum</code> (STYLE_GUIDE §7.5)
<code>\ \ ></code>	sequential boundary	Inside <code>~p>/~s></code> , switches back to sequential <code>~></code> mode on the auto-merged whole.	<code>~p> "abc" letters freq \ \ > values \ > sum → 3 (test_advanced_features_pin.stk:32)</code>
<code>\ then\ </code>	sequential boundary (English)	Identical to <code>\ \ ></code> ; the spelled-out equivalent.	<code>~p> \$huge letters freq \ then\ values \ > sum</code> (STYLE_GUIDE §7.5)

`|>` terminates an active `~>` macro and drops back to plain pipe-forward, which is why the threading examples end in `|> p` or `|> join ", " |> p`. The `\|\|> / \|then\|` boundary marker is the parallel-to-sequential switch: everything before it is chunk-local, everything after operates on the merged result. The auto-merger in `~p>` picks its strategy from the first chunk's type — hash-of-numbers merges key-wise with `+` (canonical histogram), numeric scalars `sum`, arrays `concat`, strings `concat` (STYLE_GUIDE §7.5).

Two `|>` behaviors are worth pinning down because they remove boilerplate inside pipeline stages. First, **blockless grep/filter** (README:1532): a string literal RHS tests `$_ eq EXPR`, a number tests `$_ == EXPR`, a regex tests `$_ =~ EXPR`, and anything else falls back to standard Perl grep semantics. This is why `files |> filter /[a-e]/` works without a block — the regex literal is recognized and bound as the per-element test (README:1248). Second, **coderef-in-block-position** (README:1533): wherever a `{ BLOCK }` is accepted — `grep`, `map`, `sort`, `first`, `any`, `all`, `none`, `take_while`, `drop_while`, `reject`, `partition`, `min_by`, `max_by` and their pipe-forward variants — a coderef-shaped expression works directly. The runtime checks whether the value is a code ref and calls it with the current element(s); otherwise truthiness drives the

filter. This eliminates the `{ $f->($_) }` wrapper: `@l |> grep $is_big` instead of `@l |> grep { $is_big->($_) }`. The coderef form is `|>-only`: `~>` whitespace-delimited stages can't disambiguate `~> @l grep $f` from two separate stages, so under threading you still write `{ $f($_) }` (README:1547).

Precedence matters when a `|>` shares a line with arithmetic or logic: `|>` binds **looser than** `||` but **tighter than** `?:` / **and/or/not** (README:1551). So `$x + 1 |> f` parses as `f($x + 1)` and `0 || 1 |> yes` parses as `yes(0 || 1)`. The RHS must be a callable — a bare literal or binary expression on the right is a parse error (`42 |> 1 + 2` is rejected, README:1551).

B.3 – Ranges and slicing

The colon does double duty as range and slice bound.

Form	Meaning	Source
<code>1:10</code>	inclusive range, ints	STYLE_GUIDE §4
<code>'a':'z'</code>	inclusive range, chars (type-inferred)	STYLE_GUIDE §4
<code>1:100:5</code>	range with step (third colon)	STYLE_GUIDE §4
<code>\$s[1:3]</code>	string slice, inclusive both ends → 3 chars	STYLE_GUIDE §6a
<code>\$s[-1]</code>	last char (negative index)	STYLE_GUIDE §6a
<code>\$s[2:]</code>	open-ended: index 2 to end	STYLE_GUIDE §6a
<code>@a[:2]</code>	open-ended: start through index 2	STYLE_GUIDE §6a

The colon range is polymorphic — the style guide notes it works for “ints, chars, dates, IPs, romans” — which is why it replaces Perl's `..` for numeric and string ranges. The third colon is the step: `1:100:5` walks by fives (STYLE_GUIDE §4). Reach for `:` for ordinary spans you intend to consume whole — loop headers (`for val $i (1:10)`), small slices, char sweeps (`'a':'z'`).

`:` is the **eager** range and `range()` is the **lazy** one — both valid, different semantics, not a right/wrong pair (STYLE_GUIDE §4). `:` materializes the whole list at once; `range(start, end[, step])` yields values on demand. Reach for `range()` whenever the span is large or unbounded or you only consume a prefix: `range(1, 1e9) |> take 3` is instant and constant-memory, where `(1:1e9) |> take 3` builds a billion-element list before the `take` ever runs (STYLE_GUIDE §4). The split is the same eager/lazy distinction that separates `pmap` (returns a vec) from `pmaps` (streams) elsewhere in the language.

For slicing, the colon carries one semantic that diverges from Perl: `[N:M]` is **inclusive on both ends**, so `$s[1:3]` is three chars at indices 1, 2, 3 — not a length-3 window. It also clamps past the end instead of warning (`$s[1:99]` on `"hello"` returns `"ello"`, STYLE_GUIDE §6a). Open-ended forms drop a bound: `$s[2:]` is index 2 to end, `@a[:2]` is start through index 2, `$s[-3:]` is the last three.

B.4 – Other operators

These are individual operators rather than a family.

x — list / string repeat. Perl's repetition operator, carried forward. Reach for it to build a fixed-size buffer or fill without a loop. `(0xff) x 6` builds a six-element array of `0xff` (README:2902). On a string it repeats the string; on a list it repeats the list. The parenthesization is load-bearing: the list form requires the LHS to be a parenthesized list, so `(0) x 8` is an eight-element zero array while `0 x 8` is the string `"00000000"`.

Chained comparisons. `stryke` supports Raku-style chained comparison: `1 < $x < 10` reads as the conjunction it looks like, rather than re-parsing as `(1 < $x) < 10` (README:1128). It is parse-time sugar — `1 < $x < 10` desugars to `(1 < $x) && ($x < 10)` — and works with every comparison operator (`<`, `<=`, `>`, `>=`, `lt`, `le`, `gt`, `ge`) and chains of any length (README:1214). Reach for it for range/bounds tests (`0 <= $i < len @a`) where the Perl spelling would need an explicit `&&`. This is a `stryke` extension over stock Perl 5, which has no chained-comparison form.

`\@ / \% / \$ / \&` — **references.** A leading backslash takes a reference to the container. `\@args` passes an array by reference (README:1029 `getopts(\@args, ...)`), `\%a / \%b` pass hashes (README:1235 `deep_merge(\%a, \%b)`), `\&slurp` takes a code reference (README:882 `"big.csv" |> \&slurp`).

`=~` and `s///r` — **regex binding.** `=~` binds a string to a match or substitution: `$s =~ /\d+/, $str =~ $pat` (dynamic), `$s =~ s/^\s+|\s+$//g` (trim, README:1698). The full flag set is `g/i/m/s/x/e/r` (README:1131). The `/r` flag is non-destructive — it returns the modified string instead of the match count — and `stryke` **auto-injects** it when `s///` or `tr///` appears as a pipe-forward RHS, so the modified string flows down the pipeline: `"hello world" |> s/world/perl/ |> p` prints `hello perl` (STYLE_GUIDE §7.1, README:1131).

`<=>` — **numeric spaceship.** Three-way numeric comparison returning `-1/0/1`, the canonical sort comparator. In `stryke` it pairs with positional topics: `sort { $_ <=> $_1 } @xs` (STYLE_GUIDE §11), or a stored comparator `fn ($a, $b) { $b <=> $a }` — equivalently `fn { $_0 <=> $_1 }` with positional aliases (README:1543). `cmp` is the string-comparison sibling. README:828 lists the underlying op as `op_spaceship` alongside `op_cmp`.

B.5 – The topic family

The implicit-topic operators are `stryke`'s calling card — the machinery that lets every closure and stage run param-free. All are documented in STYLE_GUIDE §2 and §12 and demonstrated in `examples/implicit_params.stk`. Reach for the bare `_` first; step up to `_N` only when a block reads two or more positional slots, and to `_<` only when an inner closure needs a value from the frame around it.

Form	Meaning
<code>_</code>	slot 0 — the bare topic; preferred over <code>\$_</code>
<code>_N</code>	slot N positional (<code>_0</code> , <code>_1</code> , <code>_2</code> , ...) — universal, replaces <code>\$a/\$b</code>
<code>_<</code>	outer topic — the topic of the enclosing closure; <code>_<<</code> two levels out, up to <code>_<<<<</code>
<code>_[N]</code>	char-of-topic at index N — alias for <code>!N!</code> (so <code>_[3]</code> over <code>substr(_, 3, 1)</code>)
<code>\$_[N]</code>	legacy Perl <code>@_</code> first-positional-arg — compat only; use <code>_N</code> in new code

The distinction between `_[N]` and `$_[N]` is a deliberate two-symbol split that the style guide flags explicitly: `_[N]` (no `$`) is character-of-topic, while `$_[N]` (with `$` and `[]`) is the old Perl positional-argument access kept for compatibility. The outer chain is demonstrated in `examples/implicit_params.stk` — `(1:3) |> map { _ + _< }` where the inner topic is the range element and `_<` reads the current outer topic. Positional slots take the same chevron chain: `_1<<` is slot 1 two frames out, one `<` per level (README:429, `examples/implicit_params.stk:42`, parser.rs `is_underscore_topic_slot`).

Appendix C – The Thread-Macro Cheat Sheet

The threading operators are the heart of idiomatic stryke — rule 3 (“always pipeline”) and section 6b (“no nested calls”) both funnel here. This appendix is the one-stop reference: a comparison table, the stage forms, the documented quirks, and a side-by-side of nested-versus-threaded. The throughline is that all five operators share one surface syntax — a value, then space-separated stages — and differ only in *where* the value is injected and *how* the stages execute, so picking between them is a question of injection point and concurrency, never of rewriting the chain.

C.1 – The five operators compared

Operator	Injection point	Execution	Value	Pick when (STYLE_GUIDE §7.6)
<code>\ ></code>	first arg	sequential, one stage at a time	last stage's result	1–2 stages, readable as-is
<code>~></code>	first arg	sequential, multi-stage macro	last stage's result	3+ stages, no I/O, per-stage char cost matters
<code>~>> / ->></code>	last arg	sequential, multi-stage macro	last stage's result	last-arg-is-the-data style (<code>map BLOCK LIST</code>)
<code>~s></code>	first arg (per stage)	streaming — each stage its own worker thread, bounded channels	count of items the last stage emitted	I/O-heavy stages, uneven latency, want pipeline parallelism
<code>~p></code>	first arg (per chunk)	chunk-parallel — whole pipeline per chunk via rayon, auto-merged	merged result	CPU-heavy on a big collection, want all-cores

Two things separate the parallel pair from the sequential trio. `~s>` runs *each stage* concurrently and produces items in **non-deterministic order** — its value is a count, not the items. `~p>` runs the *whole pipeline* on each chunk and merges; its auto-merger picks strategy from the first chunk's type (hash-of-numbers → key-wise `+`, scalars → sum, arrays → concat, strings → concat). Both have thread-last variants (`~s>>`, `~p>>`).

C.2 – Stage forms

Inside any `~>` / `~>>` / `~s>` / `~p>` macro, four stage notations are accepted (STYLE_GUIDE §7.7), and they mix freely:

```

~> $x uc # bare fn: x → uc(x)
~> $x div(_, 2) # placeholder: x → div(x, 2)
~> $x >{ _ * 2 + 1 } # arrow-block: x → (x*2+1)
~> @xs map { _ * 10 } sum # block-arg form: map { ... } @xs → sum(...)

```

- **Bare fn** — `uc`, `trim`, `sum`. The threaded value is supplied automatically at the injection point.
- **_placeholder** — `div(_, 2)`. The `_` marks where the threaded value lands and **overrides** the implicit position, so you can thread into any argument slot: `~> 10 add2(5, _) p` (README) puts the value second.
- **Arrow-block** `>{ ... }` — an anonymous-fn stage. This bracket form is a primitive of the thread-macro family; outside threading it is a parse error (“Not a code reference”, STYLE_GUIDE §12).
- **Block-arg** `name { ... }` — calling a list builtin like `map { ... }` or `grep { ... }` as a stage.

The style guide’s guidance on the last two: use `>{ }` for standalone closures, `name { ... }` when you are calling a builtin (STYLE_GUIDE §12).

C.3 — Known quirks

The threading macro has documented sharp edges. From the project’s auto-memory `reference_thread_macro_quirks.md`, four are pinned:

1. **~p> with +/|> boundary** — the parallel-to-sequential switch needs the explicit `|>` (or `|then|`) marker; stages that require the merged whole must sit after it, e.g. `~p> $huge letters freq |> values |> sum` (STYLE_GUIDE §7.5).
2. **Slice-arithmetic inside a threaded fn** — index arithmetic in a stage body interacts with the macro’s topic injection; extract to an intermediate when it bites.
3. **~p> with max/min/sum** — these reductions rely on the auto-merger recognizing numeric-scalar chunk results to sum/min/max across chunks.
4. **Greedy stage boundary at <** — the outer-chain `<` is greedy against stage parsing in some positions.

Two more from the corpus: nested `|>` maps that cross a function boundary can panic when a pipe-chain helper is itself called from an outer `|>` `maps` stage — inline or use native slicing (`reference_* / feedback_nested_maps_fn_boundary.md`). And multi-line threaded expressions auto-continue only when the *next* line starts with `|>` or `~>`; otherwise the newline terminates and you silently get the wrong value, so use `\` for mid-expression continuation (STYLE_GUIDE §13).

C.4 — Side by side: nested vs threaded

The README’s own comparison, using a real data pipeline (README:1561):

```

# Threaded - reads in execution order, one operator per stage
~> @data grep{ _ % 2 == 0 } map{ _ * _ } sort{ $1 <== $0 } |> join "," |> p
# 400,324,256,196,144,100,64,36,16,4

# Same pipeline with |> - same logic, one extra |> per stage
@data |> grep{ _ % 2 == 0 } |> map{ _ * _ } |> sort{ $1 <== $0 } |> join "," |> p

```

The `~>` form drops one operator-character per stage versus `|>` — which is exactly when section 7.2 says the macro pays for itself: “≥3 stages.” Now the inside-out form the style guide forbids:

```

# Nested - reads inside-out, cursor must dive through closing parens
first_word(uc(trim($x)))

# Threaded - reads top-to-bottom, left-to-right
$x |> trim |> uc |> first_word

```

A longer real chain from README:1569, mixing filters, maps, sort, and a head, with the macro terminating into a sequential tail at the first `|>`:

```
-> @nums grep{ _ % 3 == 0 } map{ _ * 2 } grep{ _ > 50 } sort{ $ _1 <=> $ _0 } |> head 5 |> join ", "  
~ |> p  
# 198,192,186,180,174
```

And arrow-block stages chained on a scalar (README:1571):

```
-> 100 >{ _ / 2 } >{ _ + 10 } >{ _ * 3 } p # 180
```

The rule that generates all of this is mechanical (STYLE_GUIDE §6b): two open parens in a row, `f(g(`, means refactor to a pipeline. The only stated exception is a constructor-style call where the arg is a literal struct, e.g. `Point(1, 2)`.

Appendix D — A Glossary of stryke Terms

Terms used throughout this book, alphabetized, each cross-linked to the appendix or source where it is specified in full.

--compat — The strict-Perl-5 mode flag. Disables all stryke extensions (`|>`, `struct`, `enum`, `match`, `pmap`, `#{expr}`, ...) so the interpreter behaves as stock Perl 5 (README:972). For parity testing only.

--no-interop — The “bot firewall” flag. Rejects every entry in the style guide’s DON’T column at parse time — `sub`, `say`, `print`, `length`, `scalar`, bare `my`, `$a/$b` outside comparators, trailing module `1` — as a forcing function for idiomatic code (STYLE_GUIDE §17). Used in CI for new stryke; never disabled, since its rejections are free QA. Note that topic vars (`_N`, `$_N`) and special vars (`@ARGV`, `%ENV`, ...) are always exempt regardless of mode (STYLE_GUIDE §17).

--strict / use strict — Opt-in Perl-style undefined-variable enforcement, *not* on by default in the CLI (that default is itself a Perl-ism stryke drops). Used only in modules being ported from CPAN. The IDE flips strict-vars on by default for inline diagnostics, while `stryke check` stays lenient (STYLE_GUIDE §17, README:3340).

Arrow-block (`>{ ... }`) — An anonymous-function stage form usable **only** inside `~/~>/~s>/~p>` thread macros. Outside threading it is a parse error: “Not a code reference” (STYLE_GUIDE §12).

Auto-merger — The `~p>` component that combines per-chunk results. It picks strategy from the first chunk’s type: hash-of-numbers merges key-wise with `+` (canonical histogram), numeric scalars sum, arrays concat, strings concat (STYLE_GUIDE §7.5).

Block-arg form — A thread-macro stage that calls a list builtin with a block, e.g. `map { ... }` or `grep { ... }`, as opposed to the standalone arrow-block `>{ ... }` (STYLE_GUIDE §12).

Chunk-parallel — See `~p>`: input split into chunks, the whole pipeline run on each chunk via rayon, results auto-merged (STYLE_GUIDE §7.5).

Cranelift JIT — The just-in-time backend. stryke compiles to a bytecode VM and JIT-compiles hot regions via Cranelift; the fusevm bridge lowers string-bearing segments through fusevm’s Cranelift block JIT (README:2040, :27).

fn — The stryke function keyword that opens the topic-variable protocol (`_`, `_N`, implicit return). Replaces Perl’s `sub`, which does not unlock the topic system and is rejected by `--no-interop` (STYLE_GUIDE rule 1, §3).

fusevm — The shared bytecode VM + Cranelift JIT used by both stryke and zshrs; stryke embeds inside zshrs through it. The fusevm bridge (`fusevm_bridge.rs`) is a Tier-0 JIT lowering string-bearing segments — compares, concat, length/ord/hex, substr, sprintf-with-literal-format — to fusevm’s Cranelift block JIT (README:2040). See **Cranelift JIT** and **Bytecode VM**.

Implicit param — A closure parameter accessed without declaration via the topic family (`_`, `_N`). Named for the rule that you “never write named params for one-liner closures” (STYLE_GUIDE rule 5). The mechanism: every closure and thread-macro stage opens a topic frame, so the argument is reachable as `_` (slot 0) or `_N` without a `shift` or `my (...)` = `@_` line. See **Topic / topic variable** and **Outer-chain** (STYLE_GUIDE §12, Appendix B.5).

Implicit return — The rule that the last expression in any `fn` body, arrow-block, or expression-position block is the return value. `return` is reserved for early-return guards (STYLE_GUIDE rule 7, §3).

Introspection hash / reflection hash — One of the `%stryke::*` registries exposing the language's own metadata as queryable hashes: `%stryke::builtins` (`%b`, primary name → category), `%stryke::all` (`%all`, every recognized name), `%stryke::keywords` (`%k`), `%stryke::operators` (`%o`), `%stryke::aliases` (`%a`), `%stryke::descriptions` (`%d`), and more (README:3410–3425). Used for self-documentation and the IDE reflection tool window.

len — The single stryke verb for length/count across strings, arrays, hashes, sets, and iterators. Replaces `length`, `scalar`, and `0 + coercion`, all rejected by `--no-interop`. Aliases `cnt/count` (STYLE_GUIDE §5).

match — Algebraic pattern match: `match (EXPR) { PATTERN [if EXPR] => EXPR, ... }`. Patterns cover regex, array, hash, wildcard, and literal forms; bindings are scoped per arm; matching over an enum is checked exhaustively (README:1125). Each arm's tail expression is the value of the match (implicit return, STYLE_GUIDE rule 7), so a `match` is an expression, not a statement.

NaN-boxing — The runtime value representation. `StrykeValue` is a NaN-boxed `u64`: immediates (`undef`, `i32`, raw `f64` bits) packed into the NaN space of a double, and tagged `Arc<HeapObject>` pointers for big ints, strings, arrays, hashes, refs, regexes, atomics, channels (README:100, :27).

Outer-chain (`_<`) — A topic reference that reaches the enclosing closure's topic; one `<` per level outward, up to `_<<<<<`. Positional slots take the same chain via `_N<M` (STYLE_GUIDE §12, `examples/implicit_params.stk`).

Pipe-forward (`|>`) — The single-stage pipe operator: LHS becomes the first argument of the RHS callable, zero-cost desugar. The day-to-day default for 1–2 stages (STYLE_GUIDE §7.1).

Primary — A canonical builtin name, as opposed to an alias. `%stryke::builtins` is “primaries-only — clean unique-op count, no keywords”; aliases resolve to their primary via `%stryke::aliases` (e.g. `tj` → `to_json`) (README:3410, 3417). The README cites 10,453 standard-library primaries.

pray — Dispatch work to a pool of persistent worker processes. Accepts a string of source or a coderef whose body is deparsed and shipped (closure captures unsupported in v1): `pray fn { 2 + _ }, @workers` or `pray "2 + 3", @workers` (README:2722). Paired with `annex` (block-and-gather results, keyed by session id) and `excommunicate` (clean shutdown) (README:2715).

Sketch algebra — Probabilistic data structures treated as first-class operands for `+` `|` `&` `^` `-`: Bloom union (`$bloom_a + $bloom_b`), HyperLogLog union (`$hll_a + $hll_b`), Count-Min counter sum (`$cms_a + $cms_b`), SpaceSaving top-k merge (`$topk_a + $topk_b`), t-digest centroid merge (`$td_a + $td_b`), Roaring set algebra (`$rb_a | $rb_b`, also `&` `^` `-`). The operators are functional — operands are never mutated, so a merge returns a fresh sketch (README:96, :2116). Everywhere else these are library function calls; here they are operator dispatch on the sketch value (README:96).

Streaming macro (`~s>`) — A pipeline where each stage runs as an independent worker thread connected by bounded channels; items flow concurrently and nothing intermediate is materialized. Output order is non-deterministic; the macro's value is the count of items emitted by the last stage (STYLE_GUIDE §7.4).

Streaming variant — The lazy, no-intermediate-vec form of an iteration builtin: `maps`, `greps`, `flat_maps`, `pmaps`, `pgreps`, `pflat_maps`. The default when chaining; the array-returning forms (`pmap`, `pgrep`) are correct only when a vec is genuinely needed (STYLE_GUIDE §8, §10).

struct / enum — Stryke's record and sum types. `struct Point { x => Float }` declares a named record; `enum Color { Red, Green }` declares unit variants and `enum Result { Ok => Int, Err => Str }` typed-data variants (README:552, :619). Enums are the natural subject of an exhaustive `match` — the compiler checks every variant is handled (README:1125). Both, with `class`, are stryke extensions disabled under `--compat`.

Thread-first (`~>`) — The multi-stage thread macro injecting the threaded value as the **first** argument of each space-separated stage. Spellings: `~>`, `thread`, `t`. Pays for itself at ≥3 stages (STYLE_GUIDE §7.2).

Thread-last (`~>>` / `~>>>`) — Same syntax as `~>` but the threaded value goes in as the **last** argument — the shape for (`block`, `list`) callees like `map/grep/reduce` (STYLE_GUIDE §7.3).

Topic / topic variable — The implicit subject of a closure or expression, spelled `_` (slot 0) or `_N` (slot N). Four equivalent spellings for slot 0 exist; pick the shortest that reads. `$_` is the Perl-ism, needed only inside string interpolation. `_[N]` is char-of-topic (distinct from the legacy `$_[N]` positional); `_<` reaches the enclosing frame's topic. See **Implicit param**, **Outer-chain**, and Appendix B.5 (STYLE_GUIDE §2, §12).

val / var — Immutable and mutable bindings. `val` is a *compile-time* freeze (reassignment, `push/pop`, element assignment, `delete` all error before run time); `var` is the mutable counterpart. Both desugar to `my` — same scope, same bytecode, zero cost. Default to `val`; bare `my` is compat-only (STYLE_GUIDE rule 10).

varsync / oursync — stryke's shared-state primitives for parallel code: lockless reads, lock-on-write. `varsync $count = 0` replaces a hand-rolled mutex (STYLE_GUIDE §10). `varsync` also fronts richer shared structures — `varsync $pq = heap { _ <=> _1 }` is a thread-shared priority queue (README:531).

Bytecode VM — stryke's execution model: source compiles to bytecode that runs on a VM, with Cranelift JIT for hot regions — not a tree-walking interpreter. Code paths to study live in `compiler.rs` and `vm.rs` (project memory `feedback_vm_not_interpreter.md`; README:27).

p / ep / warn / ddump — the output verbs. `p` is print-with-newline to stdout (one char, record separator built in); `ep` ("each-print") prints each element of a list to **stdout** (not stderr); `warn` writes to stderr, auto-appending `\n`, catchable via `$SIG{__WARN__}`; `ddump` is the built-in pretty-printer (STYLE_GUIDE §1).

Pipeline (the family) — collective name for `|>`, `~>`, `~>>/~>>`, `~s>`, `~p>`, with the `||>` / `|then|` boundary marker. The style guide calls it stryke's "universal-access protocol — every value, every callable, and every reflection-metadata access flows through" it without adapter code (STYLE_GUIDE §6b).